INDEX - Course Lecture Out-Line

Chapter: 1 Introduction to Data Analytics. Page No. 3 - 17

- Course Introduction
- Data Analytics Overview
- Dealing with Different Types of Data
- Data Visualization for Decision making
- Data Science, Data Analytics, and Machine Learning
- Data Science Methodology
- Data Analytics in Different Sectors
- Analytics Framework and Latest trends

Chapter: 2 Data Ingestion Page No. 18 - 56

- Power BI Data Ingestion and Modelling
- Reading from MySQL
- Reading from HDF5
- Reading from The Web
- Reading from APIs
- ETC
- Reading from Other Sources
- Summary and close

Chapter: 3 Data Cleansing Page No. 57 - 100

- we look at finding data and reading different file types.
- Getting and Cleaning Data! The primary goal is to introduce you to the most common data storage systems and the appropriate tools to extract data from web or from databases like MySQL.
- Introduction to Data Wrangling and Shaping
- Advanced Shaping
- Beyond Initial Shaping
- Advanced DAX
- Publishing and Testing Your Dataset
- Summarizing Data
- Creating New Variables

Chapter: 4 Data Modelling Page No. 101 - 141

- What is Data Modelling?
- Data Modelling sometimes needs Data Analysis
- Star Schema
- Snowflake Schema
- Create a Relationship
- One to many, many to many relationships

Chapter: 5 Data Visualization Page No. 142 - 204

- Overview of Power BI
- Advantages of using Power BI
- Chart Selection
- Getting Started
- Transforming Data
- Report Creation
- Different types of visualization charts
- Area Charts
- Line Charts
- Bar Charts
- Column Charts
- Combo Charts

Chapter: 6 Data Regression Page No. 205 - 244

- Types of Regression Models
- Simple Linear Regression Model
- Simple Linear Regression Equation
- Equation for the best straight line
- Graph of the best straight line
- Interpreting the Results
- Measures of Variation

Course Assignment Page No. 245 - 264

Assignment on Power BI report creation with Solution

Chapter: 1

Introduction to Data Analytics

Scope:

I. Course Introduction

II. Data Analytics Overview

III. Dealing with Different Types of Data

IV. Data Visualization for Decision making

V. Data Science, Data Analytics, and Machine Learning

VI. Data Science Methodology

VII. Data Analytics in Different Sectors

VIII. Analytics Framework and Latest trends

1.1Data Analytics:

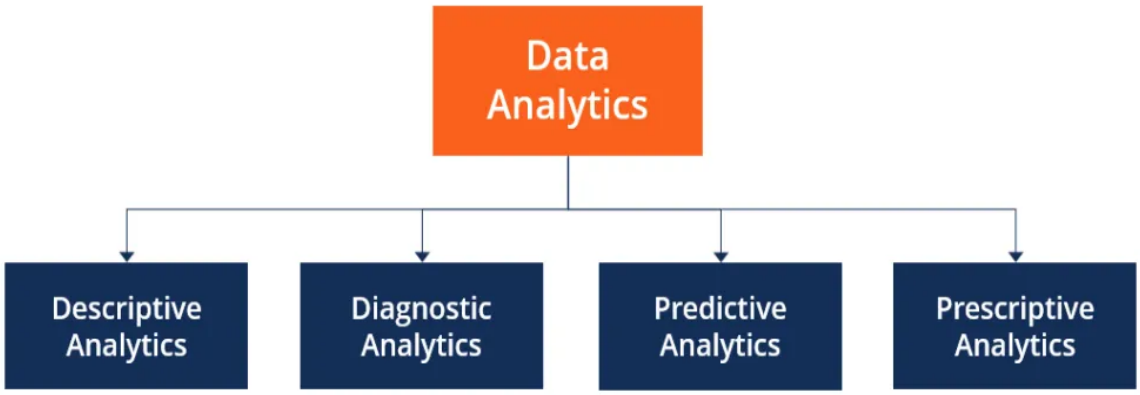
Data analytics is the science of analyzing raw datasets in order to derive a conclusion regarding the information they hold. It enables us to discover patterns in the raw data and draw valuable information from them. Data analytics processes and techniques may use applications incorporating machine learning algorithms, simulation, and automated systems. The systems and algorithms work on the unstructured data for human use. These findings are interpreted and used to help organizations understand their clients better, analyze their promotional campaigns, customize content, create content strategies, and develop products. Data analytics help organizations to maximize market efficiency and improve their earnings.

1.2Process of Data Analytics method:

Step 1: Determine the criteria for grouping the data
Step 2: Collecting the data
Step 3: Organizing the data
Step 4: Cleaning the data

1.3Data Analytics Types:

The following are the four fundamental types of data analytics:



1. **Descriptive Analytics** describes the happenings over time, such as whether the number of views increased or decreased and whether the current month's sales are better than the last one.

2. **Diagnostic Analytics** focuses on the reason for the occurrence of any event. It requires hypothesizing and involves a much diverse dataset. It examines data to answer questions, such as "Did the weather impact the selling of beer?" or "Did the new ad strategy affect sales?"

3. **Predictive Analytics** focuses on the events that are expected to occur in the immediate future. Predictive analytics tries to find answers to questions like, what happened to the sales in the last hot summer season? How many weather forecasts expect this year's hot summer?

4. **Prescriptive Analytics** indicates a plan of action. If the chance of a hot summer calculated as the average of the five weather models is above 58%, an evening shift can be added to the brewery, and an additional tank can be rented to maximize the production.

1.4 Role of Data Analytics:

As the process of analyzing raw data to find trends and answer questions, the definition of data analytics captures its broad scope of the field. However, it includes many techniques with many different goals.

The data analytics process has some components that can help a variety of initiatives. By combining these components, a successful data analytics initiative will provide a clear picture of where you are, where you have been and where you should go.

- Generally, this process begins with *descriptive analytics*. This is the process of describing historical trends in data. Descriptive analytics aims to answer the question "what happened?" This often involves measuring traditional indicators such as return on investment (ROI). The indicators used will be different for each industry. Descriptive analytics does not make predictions or directly inform decisions. It focuses on summarizing data in a meaningful and descriptive way.
- The next essential part of data analytics is *advanced analytics*. This part of data science takes advantage of advanced tools to extract data, make predictions and discover trends. These tools include classical statistics as well as machine learning. Machine learning technologies such as neural networks, natural language processing, sentiment analysis and more enable advanced analytics. This information provides new insight from data. Advanced analytics addresses "what if?" questions.
- The availability of machine learning techniques, massive data sets, and cheap computing power has enabled the use of these techniques in many industries. The collection of big data sets is instrumental in enabling these techniques. Big data analytics enables businesses to draw meaningful conclusions from complex and varied data sources, which has been made possible by advances in parallel processing and cheap computational power.
- Data analysts exist at the intersection of information technology, statistics and business. They combine these fields in order to help businesses and organizations succeed. The primary goal of a data analyst is to increase efficiency and improve performance by discovering patterns in data.

- The work of a data analyst involves working with data throughout the data analysis pipeline. This means working with data in various ways. The primary steps in the data

analytics process are data mining, data management, statistical analysis, and data presentation. The importance and balance of these steps depend on the data being used and the goal of the analysis.

- Data mining is an essential process for many data analytics tasks. This involves extracting data from unstructured data sources. These may include written text, large complex databases, or raw sensor data. The key steps in this process are to extract, transform, and load data (often called ETL.) These steps convert raw data into a useful and manageable format. This prepares data for storage and analysis. Data mining is generally the most time-intensive step in the data analysis pipeline.

- Data management or data warehousing is another key aspect of a data analyst's job. Data warehousing involves designing and implementing databases that allow easy access to the results of data mining. This step generally involves creating and managing SQL databases. Non-relational and NoSQL databases are becoming more common as well.

- Statistical analysis allows analysts to create insights from data. Both statistics and machine learning techniques are used to analyse data. Big data is used to create statistical models that reveal trends in data. These models can then be applied to new data to make predictions and inform decision making. Statistical programming languages such as R or Python (with pandas) are essential to this process. In addition, open source libraries and packages such as TensorFlow enable advanced analysis.

- The final step in most data analytics processes is data presentation. This step allows insights to be shared with stakeholders. Data visualization is often the most important tool in data presentation. Compelling visualizations can help tell the story in the data which may help executives and managers understand the importance of these insights.

1.5 Types of Data Analytics:

Data analytics is a broad field. There are four primary types of data analytics: descriptive, diagnostic, predictive and prescriptive analytics. Each type has a different goal and a different place in the data analysis process. These are also the primary data analytics applications in business.

- Descriptive analytics helps answer questions about what happened. These techniques summarize large datasets to describe outcomes to stakeholders. By developing key performance indicators (KPIs,) these strategies can help track successes or failures. Metrics such as return on investment (ROI) are used in many industries. Specialized metrics are developed to track performance in specific industries. This process requires the collection of relevant data, processing of the data, data analysis and data visualization. This process provides essential insight into past performance.
- Diagnostic analytics helps answer questions about why things happened. These techniques supplement more basic descriptive analytics. They take the findings from descriptive analytics and dig deeper to find the cause. The performance indicators are

further investigated to discover why they got better or worse. This generally occurs in three steps:

- Identify anomalies in the data. These may be unexpected changes in a metric or a particular market.
- Data that is related to these anomalies is collected.
- Statistical techniques are used to find relationships and trends that explain these anomalies.

- Predictive analytics helps answer questions about what will happen in the future. These techniques use historical data to identify trends and determine if they are likely to recur. Predictive analytical tools provide valuable insight into what may happen in the future and its techniques include a variety of statistical and machine learning techniques, such as: neural networks, decision trees, and regression.
- Prescriptive analytics helps answer questions about what should be done. By using insights from predictive analytics, data-driven decisions can be made. This allows businesses to make informed decisions in the face of uncertainty. Prescriptive analytics techniques rely on machine learning strategies that can find patterns in large datasets. By analyzing past decisions and events, the likelihood of different outcomes can be estimated.

These types of data analytics provide the insight that businesses need to make effective and efficient decisions. Used in combination they provide a well-rounded understanding of a company's needs and opportunities.

1.6 Benefits of Data Analytics:

1. Decision making improves
2. Marketing becomes more effective
3. Customer service improves
4. The efficiency of operations increases

1.7 Data Visualization:

Data visualization is the practice of translating information into a visual context, such as a map or graph, to make data easier for the human brain to understand and pull insights from. The main goal of data visualization is to make it easier to identify patterns, trends and outliers in large data sets. The term is often used interchangeably with others, including information graphics, information visualization and statistical graphics.

Data visualization is one of the steps of the data science process, which states that after data has been collected, processed and modelled, it must be visualized for conclusions to be made. Data visualization is also an element of the broader data presentation architecture (DPA) discipline, which aims to identify, locate, manipulate, format and deliver data in the most efficient way possible.

1.7.1 Why is data visualization important?

Data visualization provides a quick and effective way to communicate information in a universal manner using visual information. The practice can also help businesses identify which factors affect customer behaviour; pinpoint areas that need to be improved or need more attention; make data more memorable for stakeholders; understand when and where to place specific products; and predict sales volumes.

Other benefits of data visualization include the following:

- the ability to absorb information quickly, improve insights and make faster decisions.
- an increased understanding of the next steps that must be taken to improve the organization.
- an improved ability to maintain the audience's interest with information they can understand.
- an easy distribution of information that increases the opportunity to share insights with everyone involved.
- eliminate the need for data scientists since data is more accessible and understandable; and
- an increased ability to act on findings quickly and, therefore, achieve success with greater speed and less mistakes.

1.8 Data Visualization for Decision making

The average person makes a stunning 35,000 choices per day. People have been creating, adapting, and refining decision-making techniques long before big data made its entrance. Scientists have also been studying these techniques for decades, trying to understand how we can make good decisions, and how we can avoid bad decisions. Research shows that we make decisions by forming opinions and choosing actions through "mental processes which are influenced by biases, reason, emotions, and memories." While we work hard to develop rational, logical conclusions, there are still several factors that can limit our ability to make the right decisions.

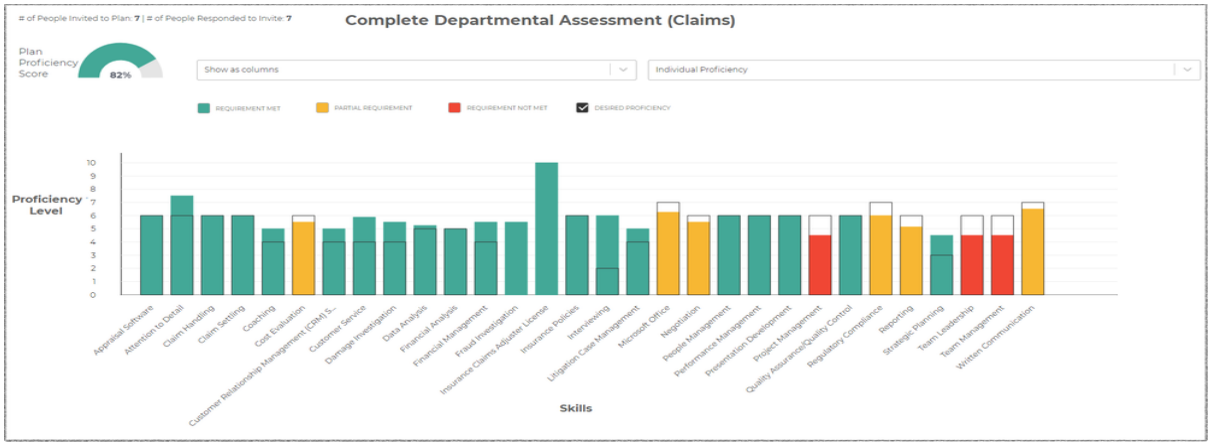
Here are four ways that data visualization can reduce these factors and improve your decision-making.

Increase Speed

Information travels at the speed of light in our always-connected world. Due to the fast pace of business, managers are often expected to make critical decisions in short periods. If you are not timely, an opportunity may pass by, or a small problem may grow exponentially worse overnight. Data visualization can help you draw actionable insights from massive amounts of

data in a short amount of time. Even a simple visualization, like a bar graph, can present valuable insights in seconds.

Take a look at the example below:



Data collected from a corporate technology assessment is organized into this colorful bar graph. By glancing at the chart, an IT manager could immediately recognize which skills need improvement. From there, the manager could decide to allocate resources towards training and recruiting in these skill areas. All within a few minutes.

Improve Accuracy

Business leaders must make timely and *informed* decisions. However, collecting and efficiently reviewing all of the numbers is not always a reasonable option. What happens when you don't have all of the facts before making a decision? You fill in the blanks with assumptions and biases.

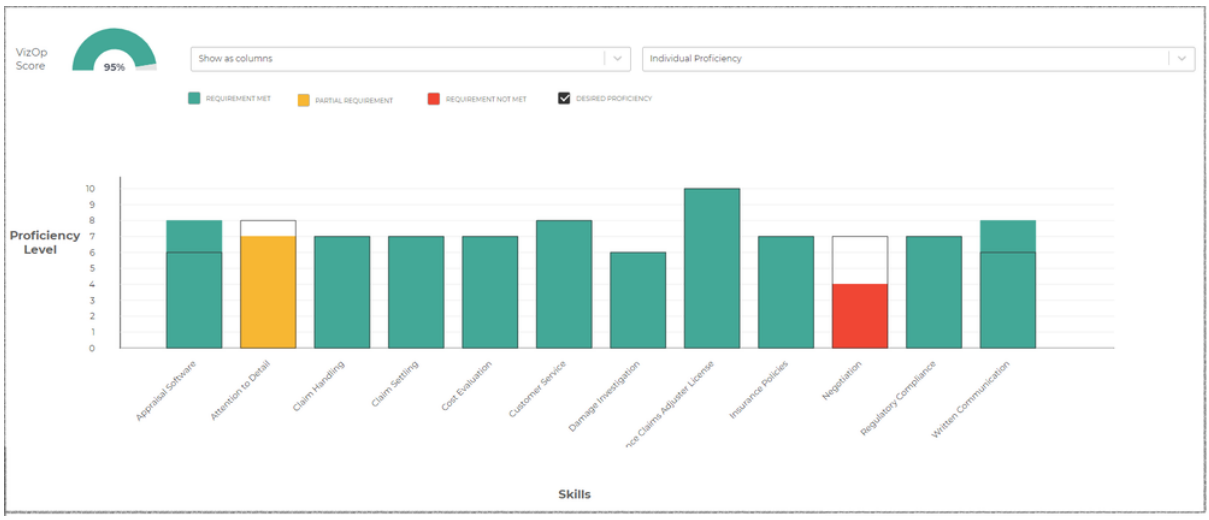
Simplify Communication

A decision is just words until it is carried out through people's actions. After you make a decision, you must effectively communicate your thoughts with the people who will carry out the subsequent steps. In the same way that data visualization simplifies data analysis, it can also streamline and objectify communication.

For example, a manager at an insurance company determines his lead adjuster needs to improve his negotiation skills (red column) and assigns him to training. The manager could just say this to his adjuster. However, now, the adjuster is left with several unanswered questions. Why does he need training? Where should his negotiation skills be right now? Does his manager dislike him?

While the decision may be apparent to the manager, it isn't communicated clearly to the adjuster. How can you expect people to take effective action when the message isn't clear in the first place?

Alternatively, the manager could use the graph below to clearly communicate why he is making this decision to the adjuster:



The chart clearly shows which skills do not meet the ideal proficiency levels and by how much those skills need to improve. By presenting his message in visual form, the manager can ensure the adjuster understands why he needs training and how he can gauge his progress. The visualization shifted the manager's message from unclear and subjective to concise and objective.

Empower Collaboration

Leaders should always seek the perspectives of others to enhance their decision-making process. Research shows that “being able to view the decision environment from multiple perspectives enhances the decision maker's ability to make better-informed choices.”

1.9 Data Science

Data science is an interdisciplinary field that uses scientific methods, processes, algorithms and systems to extract knowledge and insights from noisy, structured and unstructured data, and apply knowledge and actionable insights from data across a broad range of application domains. Data science is related to data mining, machine learning and big data.

Data science is a "concept to unify statistics, data analysis, informatics, and their related methods" in order to "understand and analyze actual phenomena" with data. It uses techniques and theories drawn from many fields within the context of mathematics, statistics, computer science, information science, and domain knowledge. However, data science is different from computer science and information science. Turing Award winner Jim Gray

imagined data science as a "fourth paradigm" of science (empirical, theoretical, computational, and now data-driven) and asserted that "everything about science is changing because of the impact of information technology" and the data deluge.

1.9.1 How Does Data Science Work?

Data science involves a plethora of disciplines and expertise areas to produce a holistic, thorough and refined look into raw data. Data scientists must be skilled in everything from

data engineering, math, statistics, advanced computing and visualizations to be able to effectively sift through muddled masses of information and communicate only the most vital bits that will help drive innovation and efficiency.

Data scientists also rely heavily on artificial intelligence, especially its subfields of machine learning and deep learning, to create models and make predictions using algorithms and other techniques.

Data science generally has a five-stage life cycle that consists of:

- Capture: Data acquisition, data entry, signal reception, data extraction
- Maintain: Data warehousing, data cleansing, data staging, data processing, data architecture
- Process: Data mining, clustering/classification, data Modelling, data summarization
- Communicate: Data reporting, data visualization, business intelligence, decision making
- Analyse: Exploratory/confirmatory, predictive analysis, regression, text mining, qualitative analysis

1.9.2 WHAT CAN DATA SCIENCE BE USED FOR?

- Anomaly detection (fraud, disease, crime, etc.)
- Automation and decision-making (background checks, credit worthiness, etc.)
- Classifications (in an email server, this could mean classifying emails as “important” or “junk”)
- Forecasting (sales, revenue and customer retention)
- Pattern detection (weather patterns, financial market patterns, etc.)
- Recognition (facial, voice, text, etc.)
- Recommendations (based on learned preferences, recommendation engines can refer you to movies, restaurants and books you may like)

1.10 Machine Learning:

Data analytics, also known as data analysis, is the process of cleaning, inspecting, modelling, and transforming data for finding valuable information, informing conclusions and enhancing the decision-making process. Data analytics focuses on generating valuable insights from the available data. Companies use data analytics to make better-informed decisions regarding various matters including marketing, production, etc. Data analytics helps you take raw data and extract helpful information from the same.

1.10.1 How machine learning works:

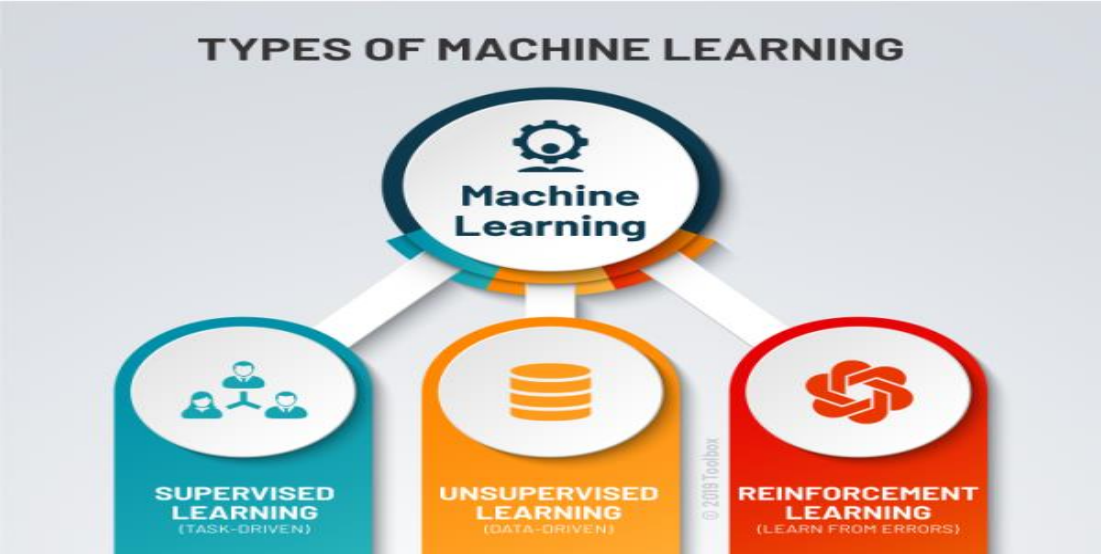
UC Berkeley (link resides outside IBM) breaks out the learning system of a machine learning algorithm into three main parts.

A Decision Process: In general, machine learning algorithms are used to make a prediction or classification. Based on some input data, which can be labelled or unlabeled, your algorithm will produce an estimate about a pattern in the data.

An Error Function: An error function serves to evaluate the prediction of the model. If there are known examples, an error function can make a comparison to assess the accuracy of the model.

A Model Optimization Process: If the model can fit better to the data points in the training set, then weights are adjusted to reduce the discrepancy between the known example and the model estimate. The algorithm will repeat this evaluate and optimize process, updating weights autonomously until a threshold of accuracy has been met.

1.10.2 Types of Machine Learning:



Supervised learning:

Supervised learning is one of the most basic types of machine learning. In this type, the machine learning algorithm is trained on labeled data. Even though the data needs to be

labeled accurately for this method to work, supervised learning is extremely powerful when used in the right circumstances.

In supervised learning, the ML algorithm is given a small training dataset to work with. This training dataset is a smaller part of the bigger dataset and serves to give the algorithm a basic idea of the problem, solution, and data points to be dealt with. The training dataset is also very similar to the final dataset in its characteristics and provides the algorithm with the labeled parameters required for the problem.

Unsupervised learning:

Unsupervised machine learning holds the advantage of being able to work with unlabeled data. This means that human labor is not required to make the dataset machine-readable, allowing much larger datasets to be worked on by the program.

In supervised learning, the labels allow the algorithm to find the exact nature of the relationship between any two data points. However, unsupervised learning does not have labels to work off of, resulting in the creation of hidden structures. Relationships between data points are perceived by the algorithm in an abstract manner, with no input required from human beings.

Reinforcement learning:

Reinforcement learning directly takes inspiration from how human beings learn from data in their lives. It features an algorithm that improves upon itself and learns from new situations using a trial-and-error method. Favorable outputs are encouraged, or 'reinforced', and non-favorable outputs are discouraged or 'punished'. Based on the psychological concept of conditioning, reinforcement learning works by putting the algorithm in a work environment with an interpreter and a reward system. In every iteration of the algorithm, the output result is given to the interpreter, which decides whether the outcome is favorable or not. In case of the program finding the correct solution, the interpreter reinforces the solution by providing a reward to the algorithm. If the outcome is not favorable, the algorithm is forced to reiterate until it finds a better result. In most cases, the reward system is directly tied to the effectiveness of the result. In typical reinforcement learning use-cases, such as finding the shortest route between two points on a map, the solution is not an absolute value. Instead, it takes on a score of effectiveness, expressed in a percentage value. The higher this percentage value is, the more reward is given to the algorithm. Thus, the program is trained to give the best possible solution for the best possible reward.

1.10.3 Real-world machine learning use cases:

Here are just a few examples of machine learning you might encounter every day:

- Speech recognition
- Customer service
- Computer vision

- Recommendation engines
- Automated stock trading

1.10.4 Challenges of machine learning:

As machine learning technology advances, it has certainly made our lives easier. However, implementing machine learning within businesses has also raised a number of ethical concerns surrounding AI technologies.

Some of these include:

- Technological singularity
- AI impact on jobs
- Privacy

- Bias and discrimination
- Accountability

1.10.5 Machine Learning vs. Deep Learning vs. Neural Networks:

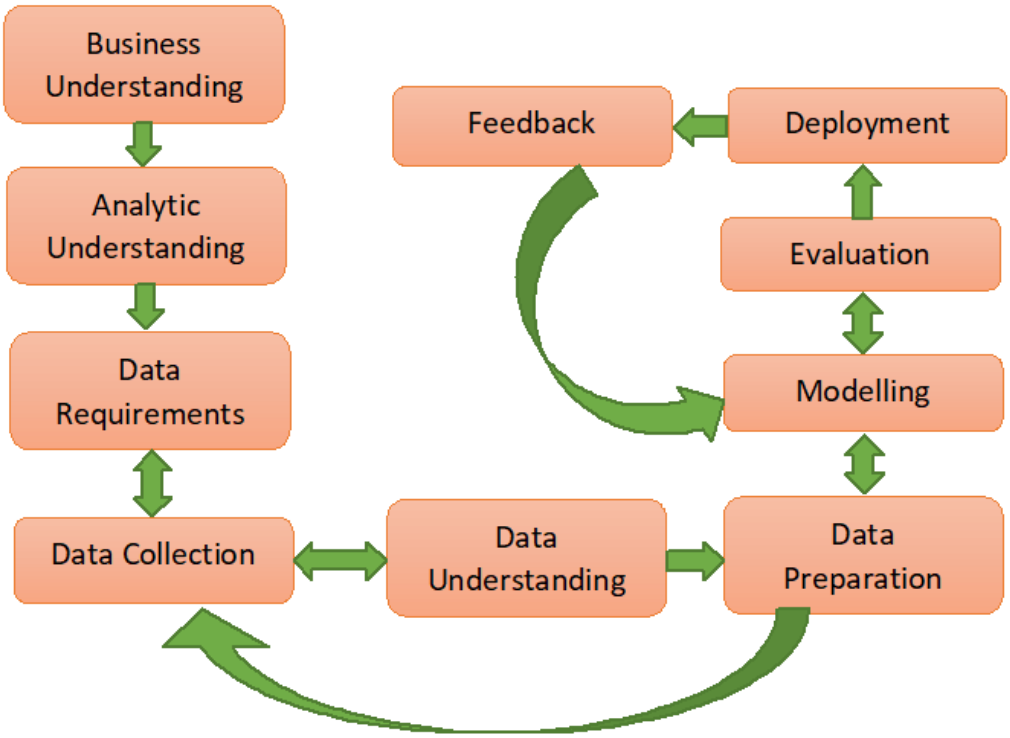
Since deep learning and machine learning tend to be used interchangeably, it's worth noting the nuances between the two. Machine learning, deep learning, and neural networks are all sub-fields of artificial intelligence. However, deep learning is actually a sub-field of machine learning, and neural networks is a sub-field of deep learning.

"Deep" machine learning can leverage labelled datasets, also known as supervised learning, to inform its algorithm, but it doesn't necessarily require a labelled dataset. It can ingest unstructured data in its raw form (e.g., text, images), and it can automatically determine the set of features which distinguish different categories of data from one another. Unlike machine learning, it doesn't require human intervention to process data, allowing us to scale machine learning in more interesting ways. Deep learning and neural networks are primarily credited with accelerating progress in areas, such as computer vision, natural language processing, and speech recognition.

Neural networks, or artificial neural networks (ANNs), are comprised of a node layer, containing an input layer, one or more hidden layers, and an output layer. Each node, or artificial neuron, connects to another and has an associated weight and threshold. If the output of any individual node is above the specified threshold value, that node is activated, sending data to the next layer of the network. Otherwise, no data is passed along to the next layer of the network. The "deep" in deep learning is just referring to the depth of layers in a neural network. A neural network that consists of more than three layers—which would be inclusive of the inputs and the output—can be considered a deep learning algorithm or a deep neural network. A neural network that only has two or three layers is just a basic neural network.

1.11 Data Science Methodology

Data Science Methodology indicates the routine for finding solutions to a specific problem. This is a cyclic process that undergoes a critic behaviour guiding business analysts and data scientists to act accordingly.



1. Business Understanding
2. Analytic Understanding
3. Data Requirements
4. Data Collection
5. Data Understanding
6. Data Preparation
7. Modelling
8. Evaluation
9. Deployment
10. Feedback

1.12 Data Analytics in Different Sectors:

The concept of data analytics has such potential that its application should not be restricted within the IT sectors only. The main factor of data analytics is that it provides a new way to gain insights into the challenges businesses face every day. Previously, no company could collect and analyze vast quantities of data. The necessity of analyzing such a long array of data is to analyze the condition of the company and predict the future strategies that would bring success. At present, a business generates all sorts of data – from machine data associated with the servers and networks to emails to customer purchasing trends. If the company aims to stay competitive and lean, they need to leverage that data to help move the business forward and using tools of data analytics is the easiest way to do that.



1. Retail
2. Medicine
3. Banking and Finance
4. Construction
5. Transportation
6. Communications, Media, and Entertainment
7. Education
8. Manufacturing and Natural Resources
9. Government
10. Energy and Utilities
11. Outsourcing Industry

1.13 Analytics Framework and Latest trends:



Trend 1: Smarter and Scalable Artificial Intelligence
Trend 2: Agile and Composed Data & Analytics
Trend 3: Hybrid Cloud Solutions and Cloud Computing
Trend 4: Data Fabric
Trend 5: Edge Computing For Faster Analysis
Trend 6: Augmented Analytics
Trend 7: The Death of Predefined Dashboards
Trend 8: XOps
Trend 9: Engineered Decision Intelligence
Trend 10: Data Visualization

Chapter: 2

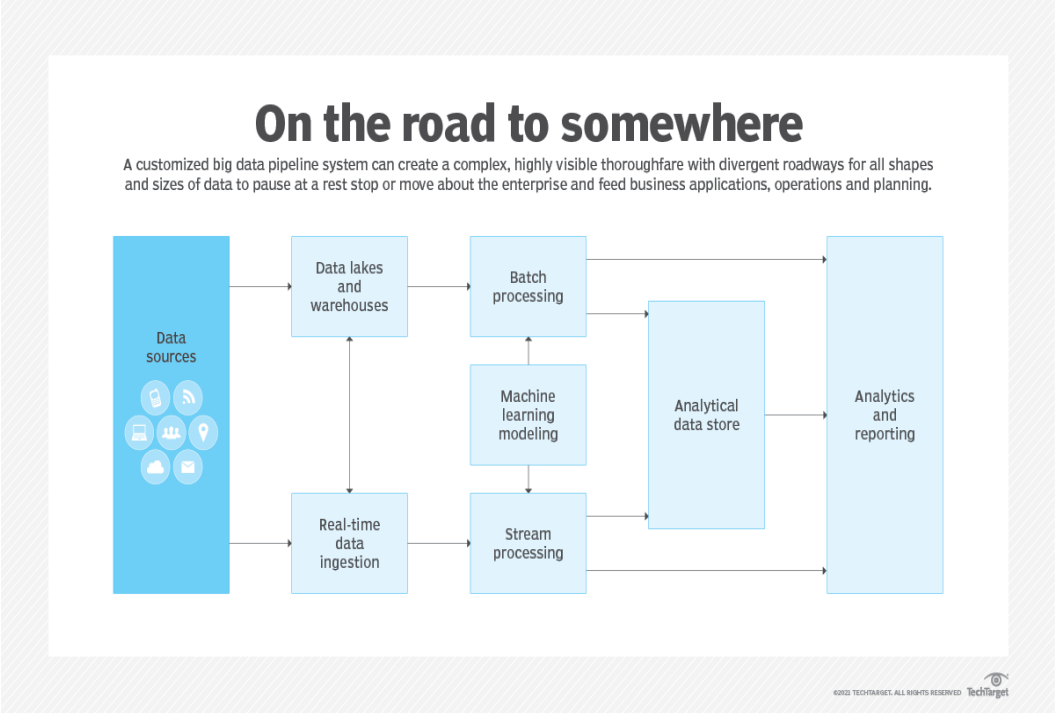
Data Ingestion

Scope:

- Power BI Data Ingestion and Modelling
- Reading from MySQL
- Reading from HDF5
- Reading from The Web
- Reading from APIs
- ETL
- Reading from Other Sources
- Summary and close

2.1 Data ingestion

It is the process of obtaining and importing data for immediate use or storage in a database. To ingest something is to take something in or absorb something. Data can be streamed in real time or ingested in batches. In real-time data ingestion, each data item is imported as the source emits it. When data is ingested in batches, data items are imported in discrete chunks at periodic intervals of time. The first step in an effective data ingestion process is to prioritize the data sources. Individual files must be validated and data items routed to the correct destinations. When numerous big data sources exist in diverse formats, the sources may number in the hundreds and the formats in the dozens. In this situation, it is challenging to ingest data at a reasonable speed and process it efficiently. To that end, vendors offer software to automate the process and tailor it to specific computing environments and applications. When data ingestion is automated, the software used may include data preparation capabilities. These features structure and organize data so it can be analyzed right away or later using business intelligence (BI) and business analytics programs.



Types of data ingestion

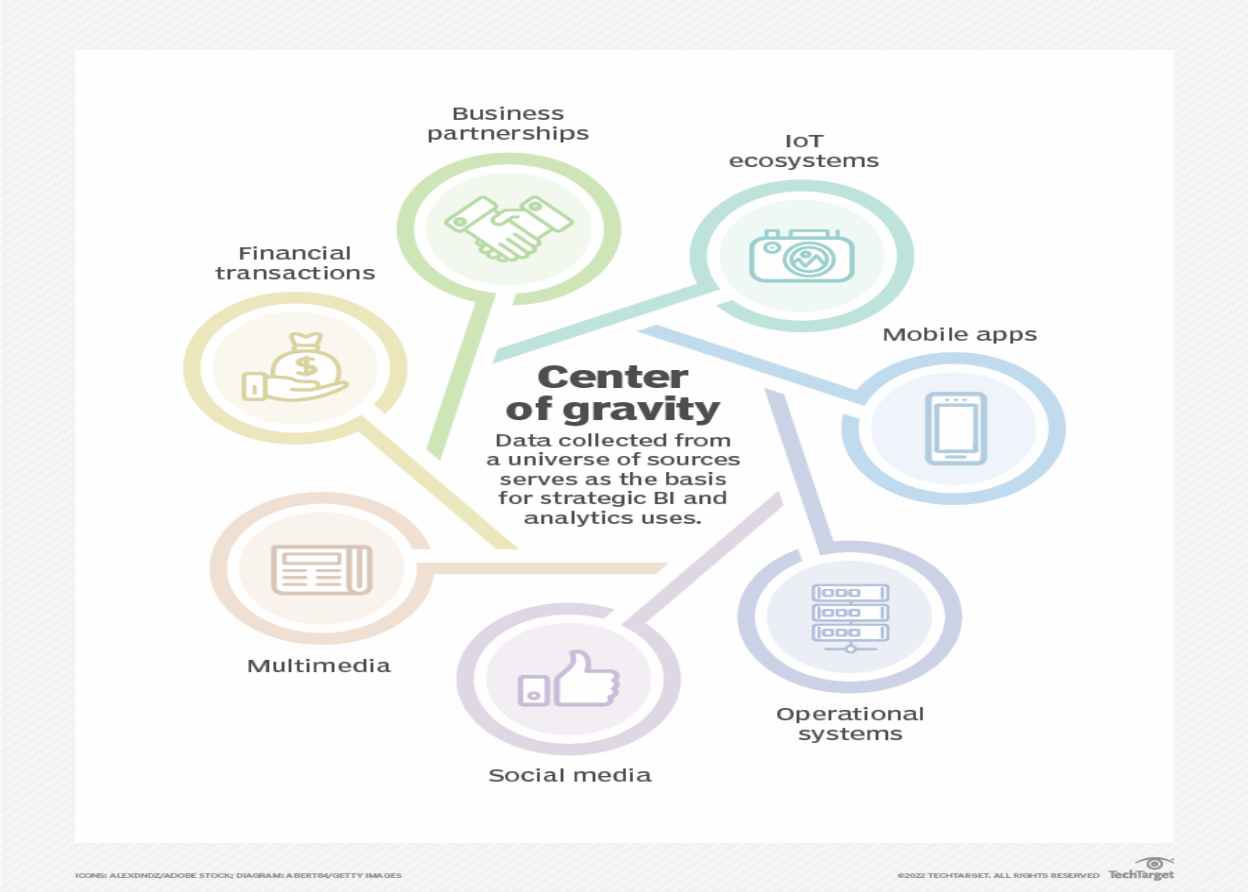
Batch processing. In batch processing, the ingestion layer collects data from sources incrementally and sends batches to the application or system where the data is to be used or stored. Data can be grouped based on a schedule or criteria, such as if certain conditions are

triggered. This approach is good for applications that don't require real-time data. It is typically less expensive.

Real-time processing. This type of data ingestion is also referred to as stream processing. Data is not grouped in any way in real-time processing. Instead, each piece of data is loaded as soon as it is recognized by the ingestion layer and is processed as an individual object. Applications that require real-time data should use this approach.

Micro batching. This is a type of batch processing that streaming systems like Apache Spark Streaming use. It divides data into groups, but ingests them in smaller increments that make it more suitable for applications that require streaming data.

The components of an organization's data strategy and its business requirements determine the data ingestion method it uses. Organizations choose their model and data ingestion tools in part based on the data sources they use and how quickly they will need access to the data for analysis.



Data ingestion tools and features

Data ingestion tools come with a range of capabilities and features, including the following:

- Extraction.
- Processing.

- Data types.
- Data flow tracking and visualization.
- Volume.
- Security and privacy.

Benefits of data ingestion

Data ingestion technology offers the following benefits to the data management process:

- Flexibility.
- Simplicity.
- Analytics.
- Application quality.
- Availability.
- Decision-making.

Challenges of data ingestion and big data sets

Data ingestion also poses challenges to the data analytics process, including the following:

- Scale.
- Security.
- Fragmentation and data integration.
- Data quality.
- Costs.

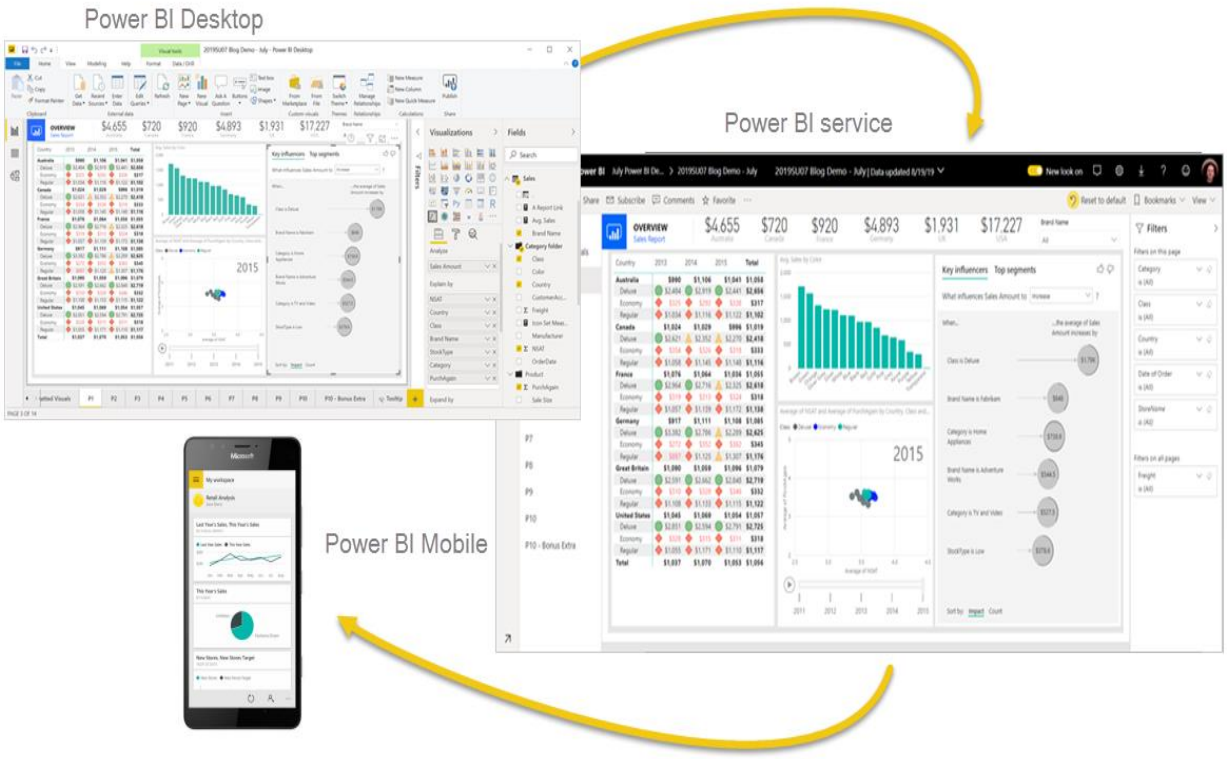
What is Power BI?

Power BI is a collection of software services, apps, and connectors that work together to turn your unrelated sources of data into coherent, visually immersive, and interactive insights. Your data may be an Excel spreadsheet, or a collection of cloud-based and on-premises hybrid data warehouses. Power BI lets you easily connect to your data sources, visualize and discover what's important, and share that with anyone or everyone you want.

The parts of Power BI

Power BI consists of several elements that all work together, starting with these three basics:

- A Windows desktop application called Power BI Desktop.
- An online SaaS (Software as a Service) service called the Power BI service.
- Power BI mobile apps for Windows, iOS, and Android devices.



These three elements—Power BI Desktop, the service, and the mobile apps—are designed to let you create, share, and consume business insights in the way that serves you and your role most effectively.

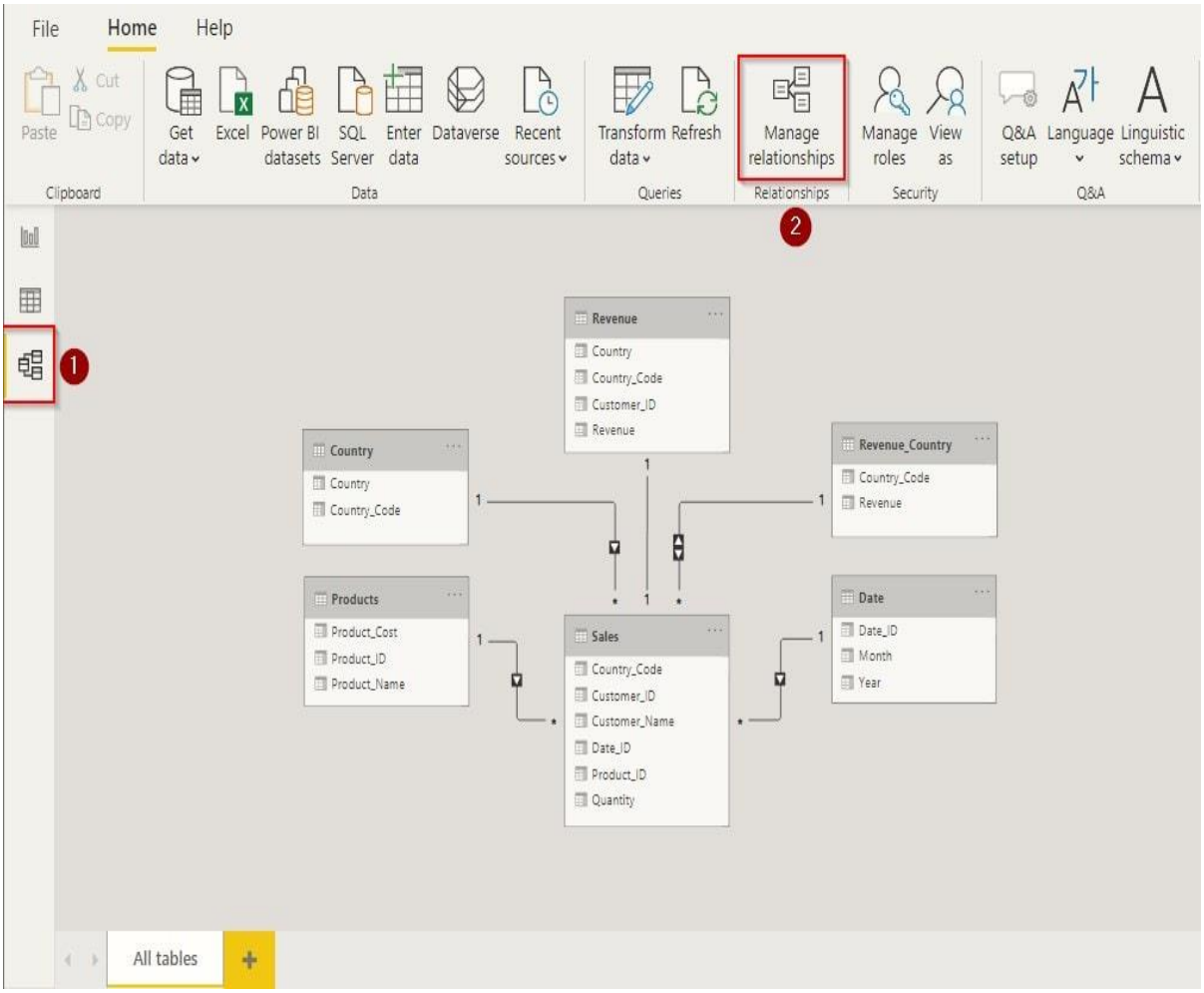
Beyond those three, Power BI also features two other elements:

- Power BI Report Builder, for creating paginated reports to share in the Power BI service. Read more about paginated reports later in this article.
- Power BI Report Server, an on-premises report server where you can publish your Power BI reports, after creating them in Power BI Desktop. Read more about Power BI Report Server later in this article.

Data Modelling on Power BI

Data Models in Power BI:

Power BI automatically detect all the possible relations between different sets of data. Sometimes we need to create a relation between the data manually.



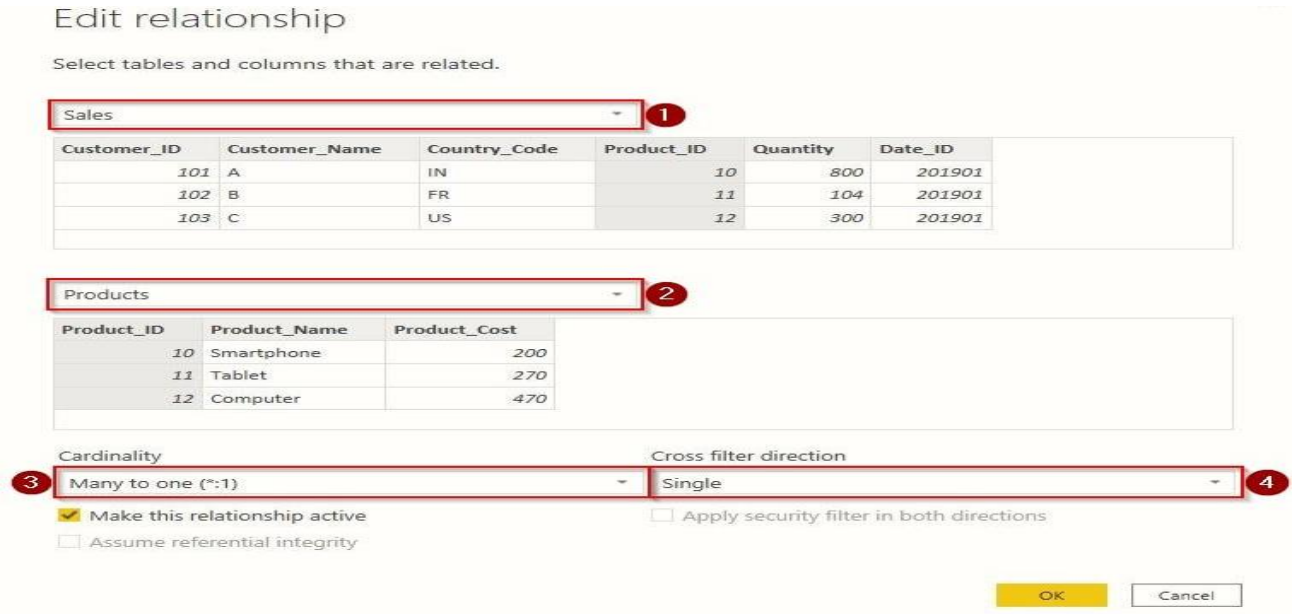
After the data import is complete, visit the 'Models' tab from the left side, as highlighted in the above image. Here you will see some automated relation created by Power BI. All the lines visible here in the 'Models' tab depicts the cardinality and direction of the relation from one table to another. You can create & modify this default relationship created by Power BI using the Manage Relationship tool given on the top menu as highlighted in the image.

Create and Manage Relationship



After clicking on 'Manage Relationship, 'a similar screen will appear, as shown in the above image. You can see all the active relations here from one table to another. All the things you can do with these relations are explained below:

- New – This option will help you create a new relationship between tables.
- Autodetect – Using this option, Power BI automatically detects the relationship between data present in tables.
- Edit – This option will help you to edit your data relationship.
- Delete – It deletes the selected relationship between the tables.

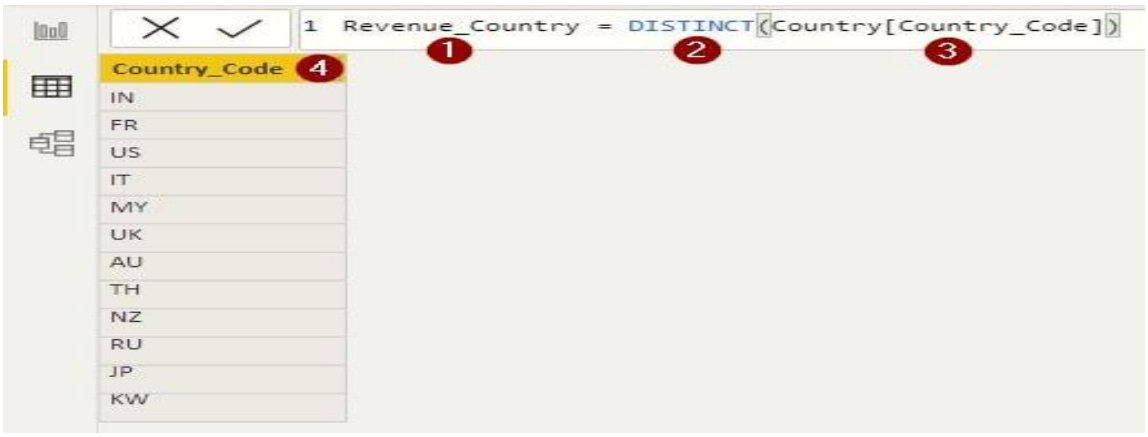


Now after clicking on '*New* 'or '*Edit* ', a similar screen will appear in front of you, as shown above. Here we will explain to you the purpose of all the options one by one.

Calculate and Measure Data

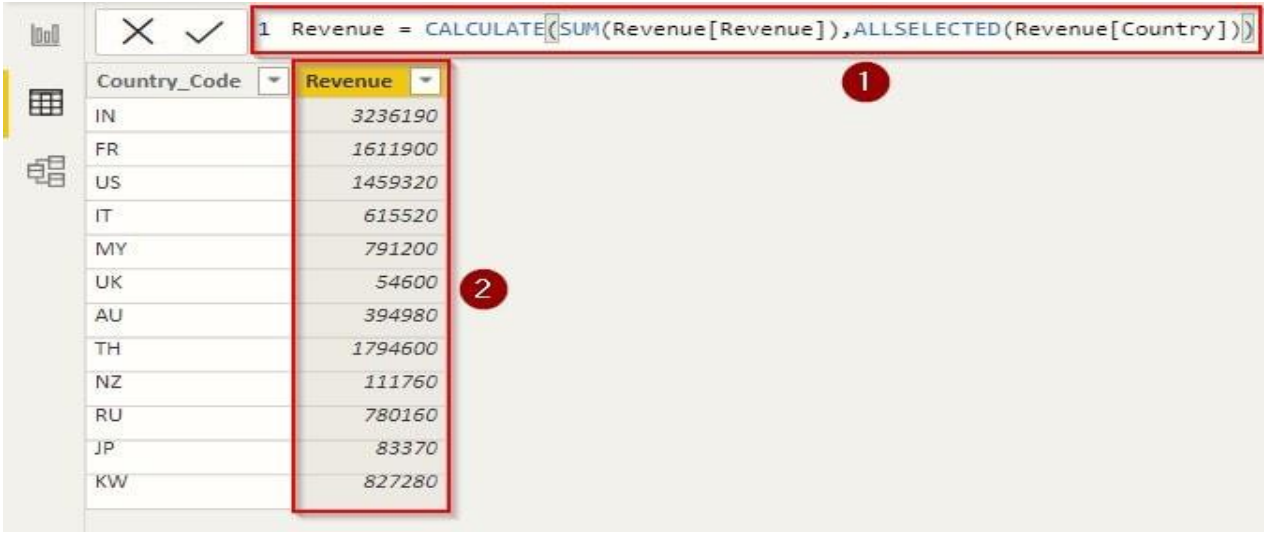
If you are familiar with Excel, you may have worked on the DAX (Data Analysis Expression) formula. If not, then no worries. I will explain in short and simple words. Like programming, DAX is a set of instructions used to calculate data from the tables. These expressions include commands for Addition, Multiplication, Average, Percentage and others with various filters.

Create Table



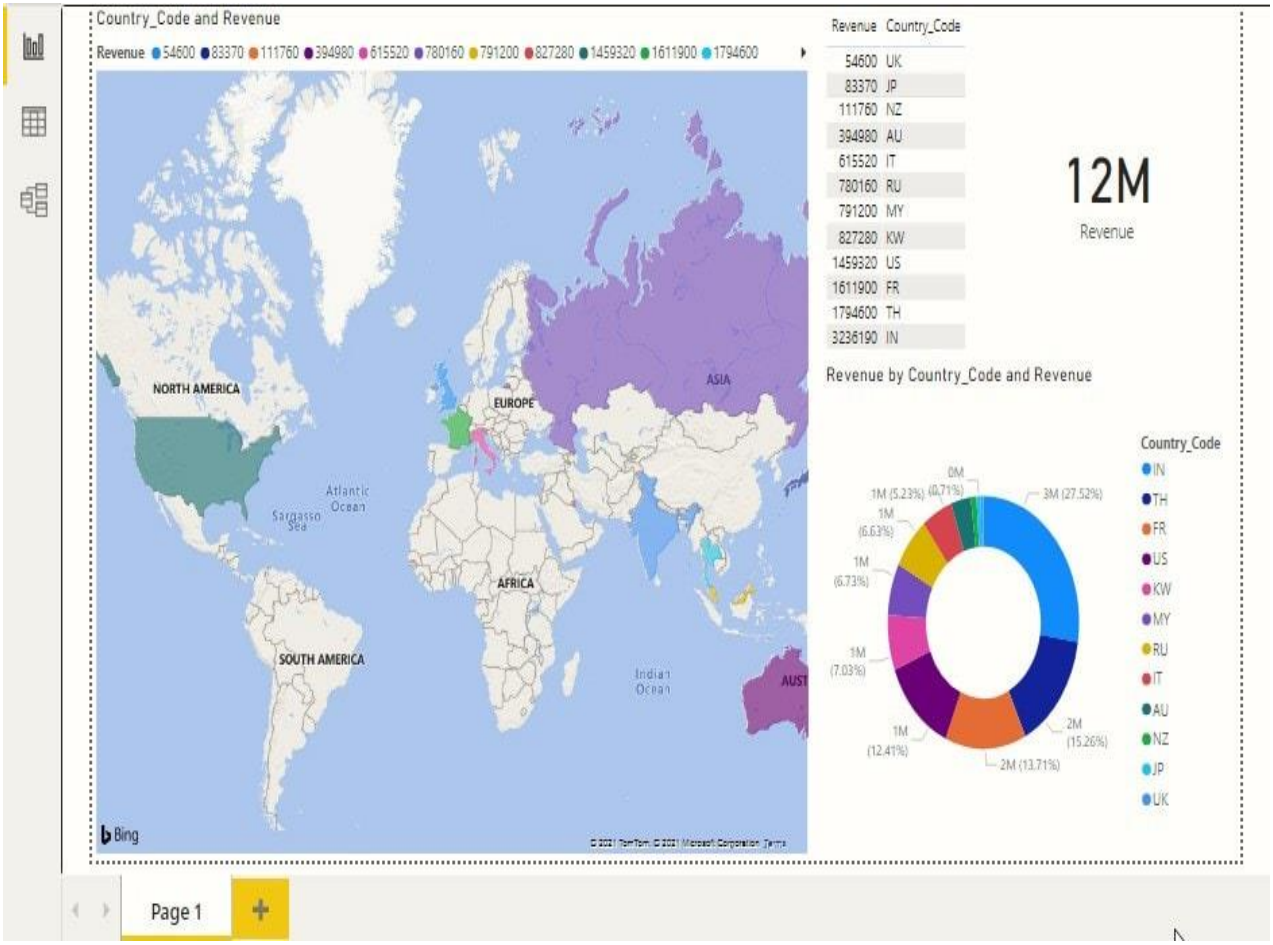
Create Column

Click on '*New Column* 'from the top menu to create a calculated column.



The DAX expression will calculate all the revenue from the table 'Revenue 'with filter as 'Country '. Without this expression, we might have spent hours calculating the individual revenue generated from the country.

Create Visualization

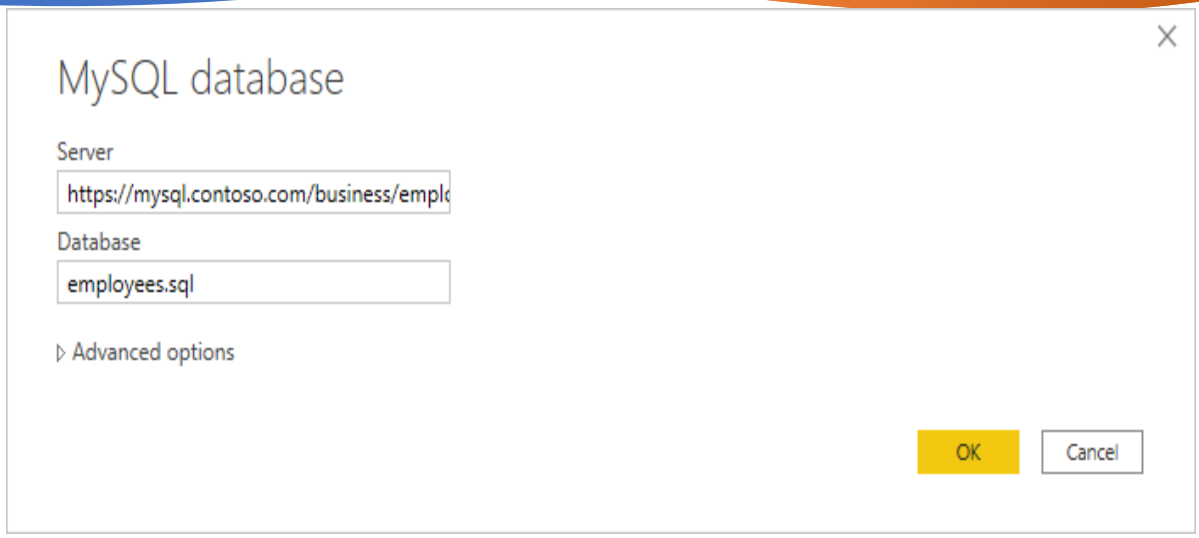


We only had sales, sales by country and product price in our imported data. Using the above methods, we created a separate table that contains the revenue generated from each country. Here we have shown the revenue from the country on the world map using different visualizations. Similarly, you can also create and manage data models in Power BI.

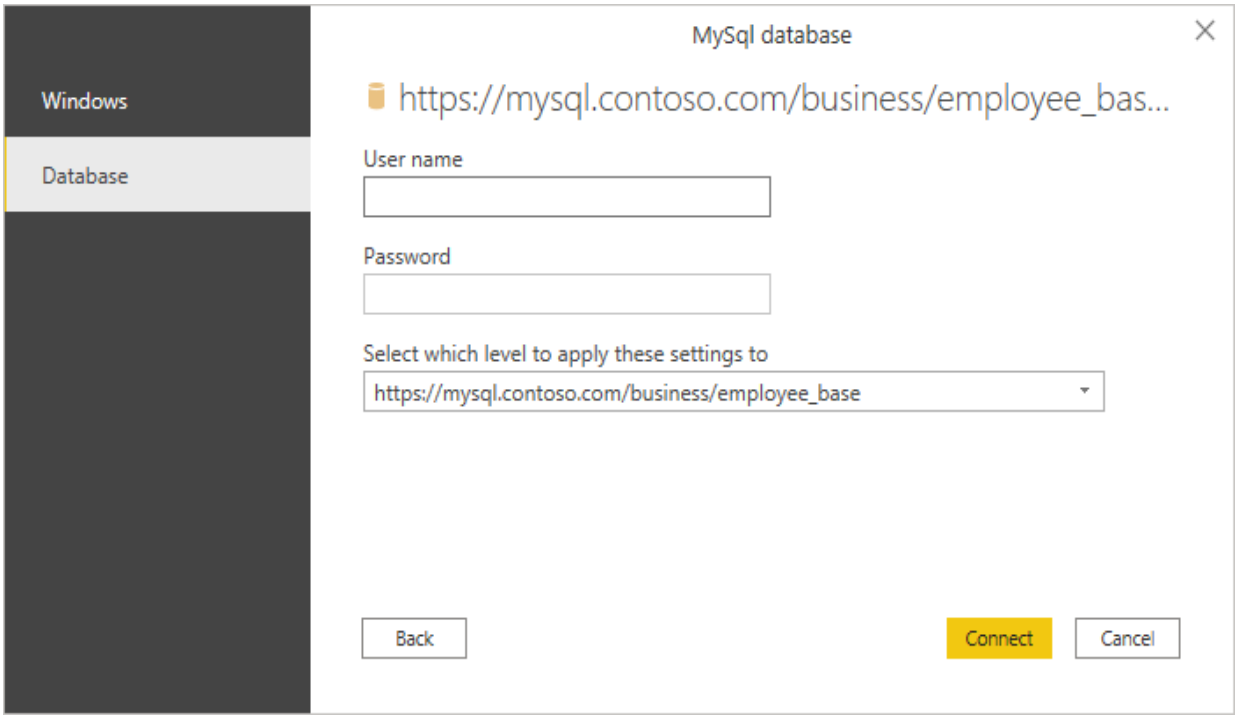
2.2 READING FROM MYSQL

To make the connection, take the following steps:

1. Select the **MySQL database** option in the connector selection.
2. In the **MySQL database** dialog, provide the name of the server and database.



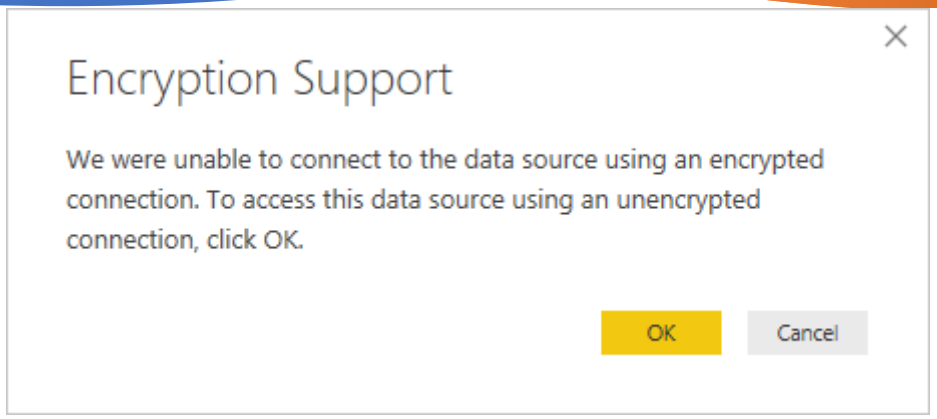
3. Select the **Database** authentication type and input your MySQL credentials in the **User name** and **Password** boxes.



4. Select the level to apply your credentials to.
5. Once you're done, select **OK**.

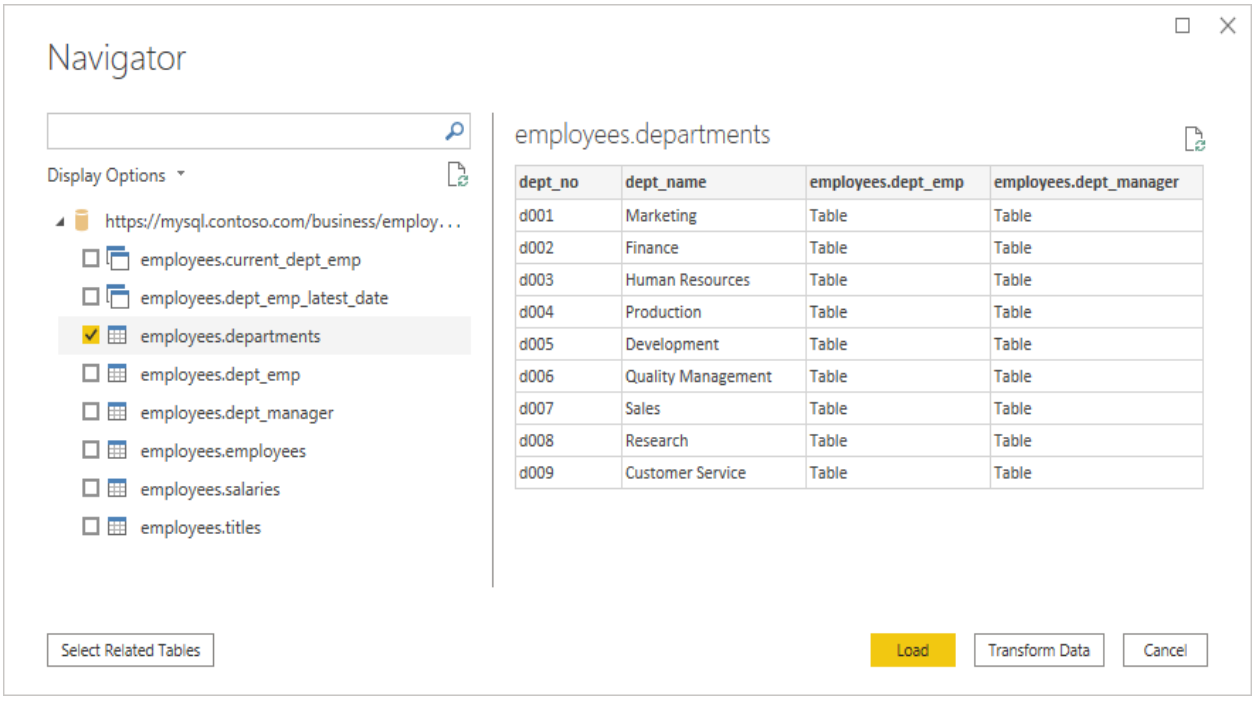
Note

If the connection is not encrypted, you'll be prompted with the following dialog.



Select **OK** to connect to the database by using an unencrypted connection, or follow the instructions to set up encrypted connections to SQL Server.

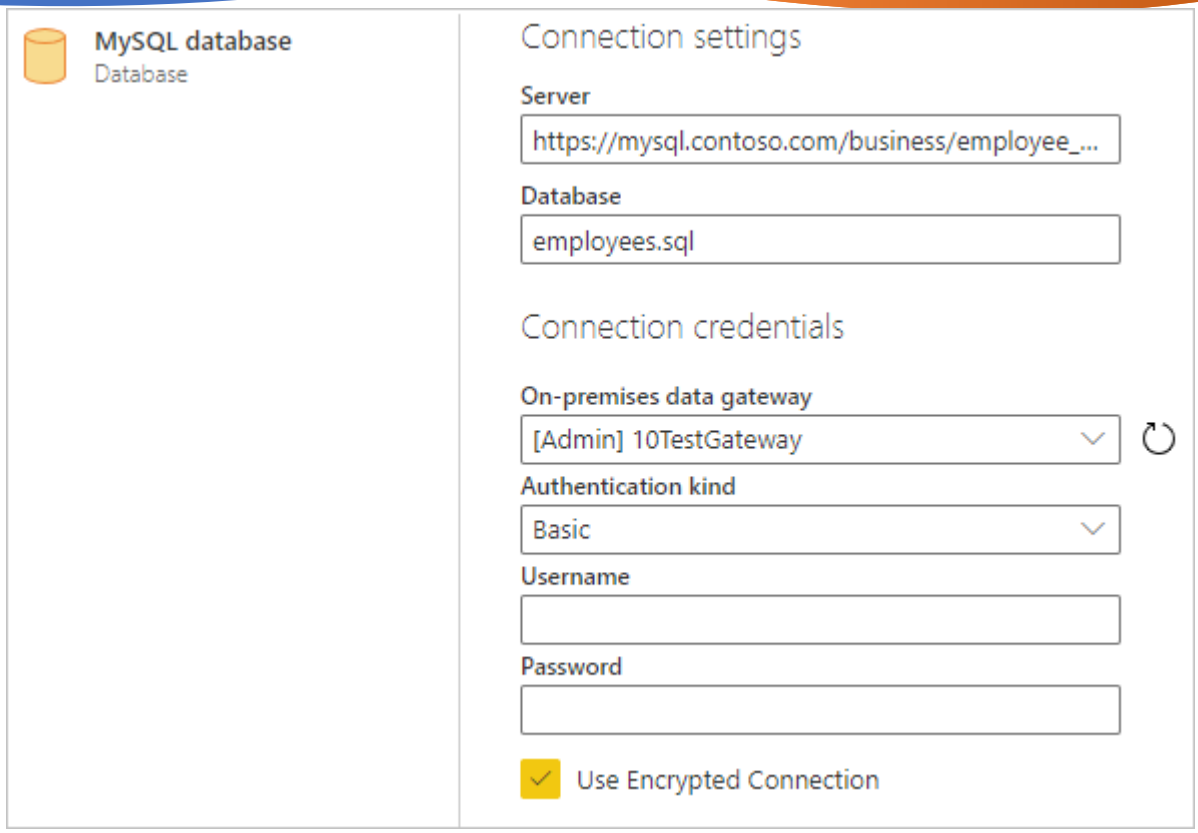
6. In **Navigator**, select the data you require, then either load or transform the data.



Connect to MySQL database from Power Query Online

To make the connection, take the following steps:

1. Select the **MySQL database** option in the connector selection.
2. In the **MySQL database** dialog, provide the name of the server and database.



3. If necessary, include the name of your on-premises data gateway.
4. Select the **Basic** authentication kind and input your MySQL credentials in the **Username** and **Password** boxes.
5. If your connection isn't encrypted, clear **Use Encrypted Connection**.
6. Select **Next** to connect to the database.
7. In **Navigator**, select the data you require, then select **Transform data** to transform the data in Power Query Editor.

2.3 READING FROM HDF5 FILES

HDF5 file stands for Hierarchical Data Format 5. It is an open-source file which comes in handy to store large amount of data. As the name suggests, it stores data in a hierarchical structure within a single file. So if we want to quickly access a particular part of the file rather than the whole file, we can easily do that using HDF5. This functionality is not seen in normal text files hence HDF5 is becoming seemingly popular in fact of being a new concept.

Let's now take a look at the most common problems that you may face if you do not clean the messy data (w.r.t to the above-mentioned types).

1. Data aggregation

Suppose you have null entries for a numeric column and you are calculating summary statistics (like mean, maximum, minimum values) on that column. The results will not get conveyed accurately in this case. Again consider the *Pima Indian Diabetes* dataset with the invalid zero entries. If you calculated summary statistics on the columns as mentioned before,

would you get the right results? Won't the results be erroneous? So, how to address this problem? There are several ways:

- Removing the entries containing missing/nullvalues(not recommended)

- Imputing the null entries with a numeric value(typically with mean or median of the respective column)

Let's now get hands-on with these problems and the second option for combating null values.

Consider the following PostgreSQL table named entries:

name text	weight_in_lbs double precision	age_in_years smallint
Christina	80.6	
Matthews		19
Gilbert	100.3	21

You can see two null entries in the above table. Suppose you want to the get the average weight value from this table and you executed the following query:

select avg(weight_in_lbs) as average_weight_in_lbs from entries;

You got 90.45 as the output. Is this correct? So, what can be done? Let's fill the null entry with this average value with the help of the COALESCE() function.
Let's fill the missing values first with COALESCE() (remember that COALESCE() does not change the values in the original table, it just returns a temporary view of the table with the values changed):

select *, COALESCE(weight_in_lbs, 90.45) as corrected_weights from entries;

You should get an output like:

name text	weight_in_lbs double precision	age_in_years smallint	corrected_weights double precision
Christina	80.6		80.6
Matthews		19	90.45
Gilbert	100.3	21	100.3

Now you can apply the AVG() again:

select avg(corrected_weights) from

(select *, COALESCE(weight_in_lbs, 90.45) as corrected_weights from entries) as subquery;

This is a much more accurate result than the earlier one. Let's now study another problem that can take place if you have mismatches in the column data-types.

2. Table Joins

Consider you are working with the following tables student_metadata and department_details:

id smallint	name text	dept_id smallint
1	Rick	100
2	Twinkle	100
3	Steve	101

student_metadata

dept_id text	dept_name text	hod text
100	Computer Science	Chris Manning
101	Economics	Steve Harris

department_details

You can see in the student_mtadata table, dept_id is of integer type, and inthe department_details table, it is present in text type. Now, suppose, you want to join these two tables and want to produce a report which will contain the following columns:

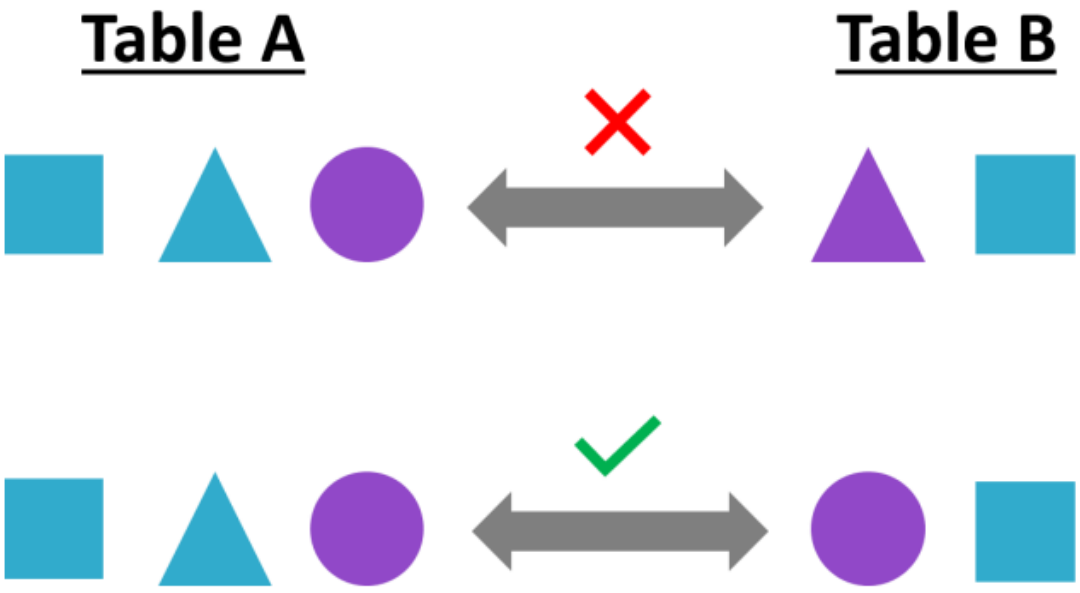
1. id
2. name
3. dept_name

To do this, you run this query:

```
select id, name, dept_name from
student_metadata s join department_details d


on s.dept_id = d.dept_id;
```

You will encounter this error, then:
ERROR: operator does not exist: smallint = text
Here's an amazing infographic which depicts this problem (from DataCamp's Reporting in SQL course):



This happening because the data-types are not getting matched while joining the two tables. Here, you can CAST the dept_id column in the department_details table to integer while joining the tables. Here's how to do that:

```
select id, name, dept_name from
student_metadata s join department_details d
on s.dept_id = cast(d.dept_id as smallint);
```

And you get your desired report:

id smallint	name text	dept_name text
1	Rick	Computer Science
2	Twinkle	Computer Science
3	Steve	Economics

Let's now discuss how strings can be present in messy forms, their problems and ways to deal with them.

Messy strings and cleaning them

String values are also very common. Let's start this section by looking at the values of a column dept_name (denoting department names) taken from a table named student_details:

id smallint	name text	dept_name text
1	Sayak	I.T
2	Hugo	Information Technology
2	Stephen	i.t
4	David	C.S.E
5	Andrew	C.S.E
6	Emily	E.C.E

String values like the above can cause a lot of unexpected problems. I.T, Information Technology, and i.t all mean the same department, i.e. *Information Technology* and suppose the specification document requires the values to be present as I.T only. Now, say, you want to count the number of students belonging to the department of I.T., and you run this query:

```
select dept_name, count(dept_name) as student_count
from student_details
group by dept_name;
```

And you get:

dept_name text	student_count bigint
Information Technology	1
i.t	1
C.S.E	2
I.T	1
E.C.E	1

Is this an accurate report? - No! So, how can you address this problem?
Let's first identify the problem in a more detailed way:

- You have Information Technology as a value which should be converted to I.T and
- You have i.t as another value which should be converted to I.T.

In the first case, you can REPLACE the value Information Technology to I.T, and in the second case, you convert the character to UPPER case. You can accomplish this in a single query though it is advised to address this kind of problems in a step-by-step fashion. Here's the query to address the problem:

```
select upper(replace(dept_name, 'Information Technology', 'I.T')) as dept_cleaned,
count(dept_name) as student_count
from student_details
group by dept_cleaned;
```

And the report:

dept_cleaned text	student_count bigint
C.S.E	2
I.T	3
E.C.E	1

Messy dates and cleaning them

Consider you are working with a table named employees which contains a column called birthdate but not in an appropriate date type. Now, you want to execute queries with dedicated date functions like DATE_PART(). You will not be able to do that until and unless you CAST the birthdate column to date type. Let's see this in action.
Consider the birthdates to be in the YYYY-MM-DD format.
Here's what the employees table looks like:

employee_id smallint	employee_name text	dept_name text	birthdate text
10	Ramesh	Retail	1990-11-08
15	Ricky	Data & Analyticss	1995-01-23
15	Richard	Customer Acquisition	1987-05-21

Now, you run the following query to extract the months from the birthdates:

```
select date_part('month', birthdate) from employees;
```

And you instantly get this error:
ERROR: function date_part(unknown, text) does not exist
Along with the error, you get a very good hint also:
HINT: No function matches the given name and argument types. You might need to add explicit type casts.
Let's follow the hint and CAST the birthdate to the appropriate date type and then apply DATE_PART():

select date_part('month', CAST(birthdate AS date)) as birthday_months from employees;

You should get the result:

birthday_months double precision
11
1
5

Let's now proceed to the pre-concluding section of this tutorial where you will study the effects of data duplications and how you can tackle them.

Data duplications: Causes, effects, and solutions

In this section, you will be studying some of the most common causes which lead to data duplications. You will also see their effects and some of the ways using which prevent them. Consider the following two tables band_details and some_festival_record:

band_id smallint	year smallint	performed smallint
12	2010	3
12	2011	2
12	2012	1
13	2010	2

band_details

band_name text	total_shows bigint	total_times_performed bigint
Band 2	25	2
Band 1	108	6

some_festival_record

2.4 READING DATA FROM THE WEB

- The urllib module allows you to download data from web servers. Typically, you will download web-pages written in HTML that were designed for a web-browser to render (draw on-screen) for a human to read. This allows you to download useful data from web-pages, such as the current temperature, sports scores, item prices from web stores, and anything else you can find on the web.
- However, before learning how to do this, you should understand that some websites put legal (and ethical) restrictions on the data they provide. In some cases, a website intends the data to be seen by humans, but do not want a program to download the data automatically.

ETHICAL WEB SCRAPING

- If you are designing a program that downloads data from a website, you should ask the website owner for written permission to retrieve the data automatically. In many cases where a website wants you to have access to their data, they will have a web-based API that makes downloading the data easier than reading it

from an HTML page. Typically, web-based API's are formatted using JSON or XML to make the data returned easier to parse for a computer. (Although a computer can parse HTML, it can be complicated and take more work that you would like.)

- For example, the website PaperBackSwap.com allows users to trade paperback books online. They have a web accessible search form that allows you to search for a book by title, author, or ISBN number to see if a member has that book posted for swapping. You could write a web scraper to check if a book you want is available. However, it would have to parse the HTML page that PaperBackSwap.com returns, looking for the data it wants.

To load data from a web site with Power Query Desktop:

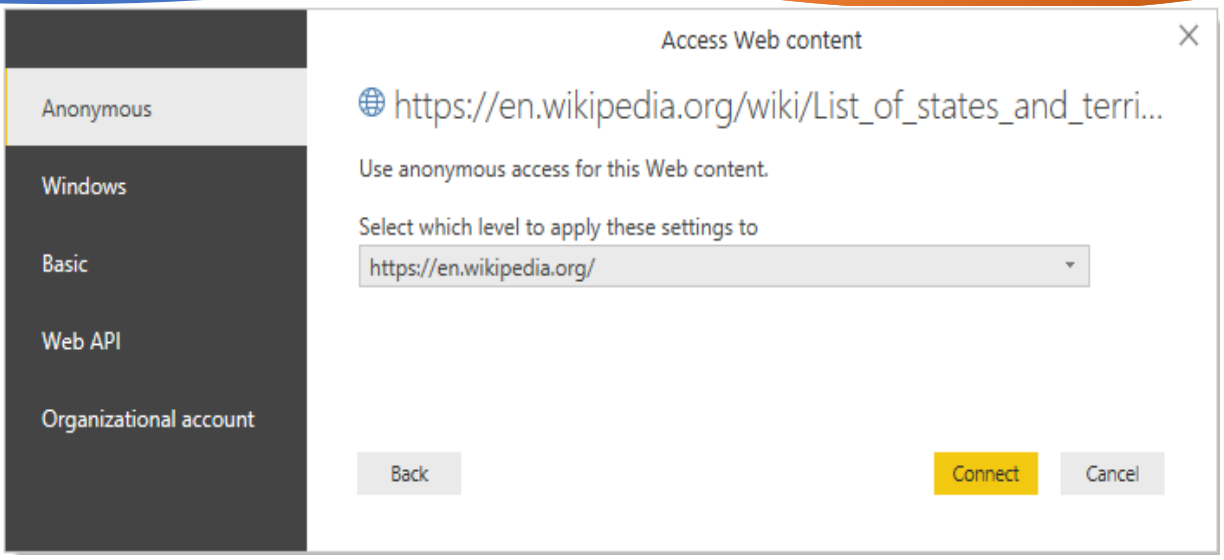
1. Select **Get Data > Web** in Power BI or **From Web** in the **Data** ribbon in Excel.
2. Choose the **Basic** button and enter a URL address in the text box. For example, enter https://en.wikipedia.org/wiki/List_of_states_and_territories_of_the_United_States. Then select **OK**.



If the URL address you enter is invalid, a warning icon will appear next to the **URL** textbox.

If you need to construct a more advanced URL before you connect to the website, go to **Load Web data using an advanced URL**.

3. Select the authentication method to use for this web site. In this example, select **Anonymous**. Then select the level to you want to apply these settings to—in this case, **https://en.wikipedia.org/**. Then select **Connect**.



The available authentication methods for this connector are:

- **Anonymous**: Select this authentication method if the web page doesn't require any credentials.
- **Windows**: Select this authentication method if the web page requires your Windows credentials.
- **Basic**: Select this authentication method if the web page requires a basic user name and password.
- **Web API**: Select this method if the web resource that you're connecting to uses an API Key for authentication purposes.
- **Organizational account**: Select this authentication method if the web page requires organizational account credentials.

Note

When uploading the report to the Power BI service, only the **anonymous**, **Windows** and **basic** authentication methods are available.

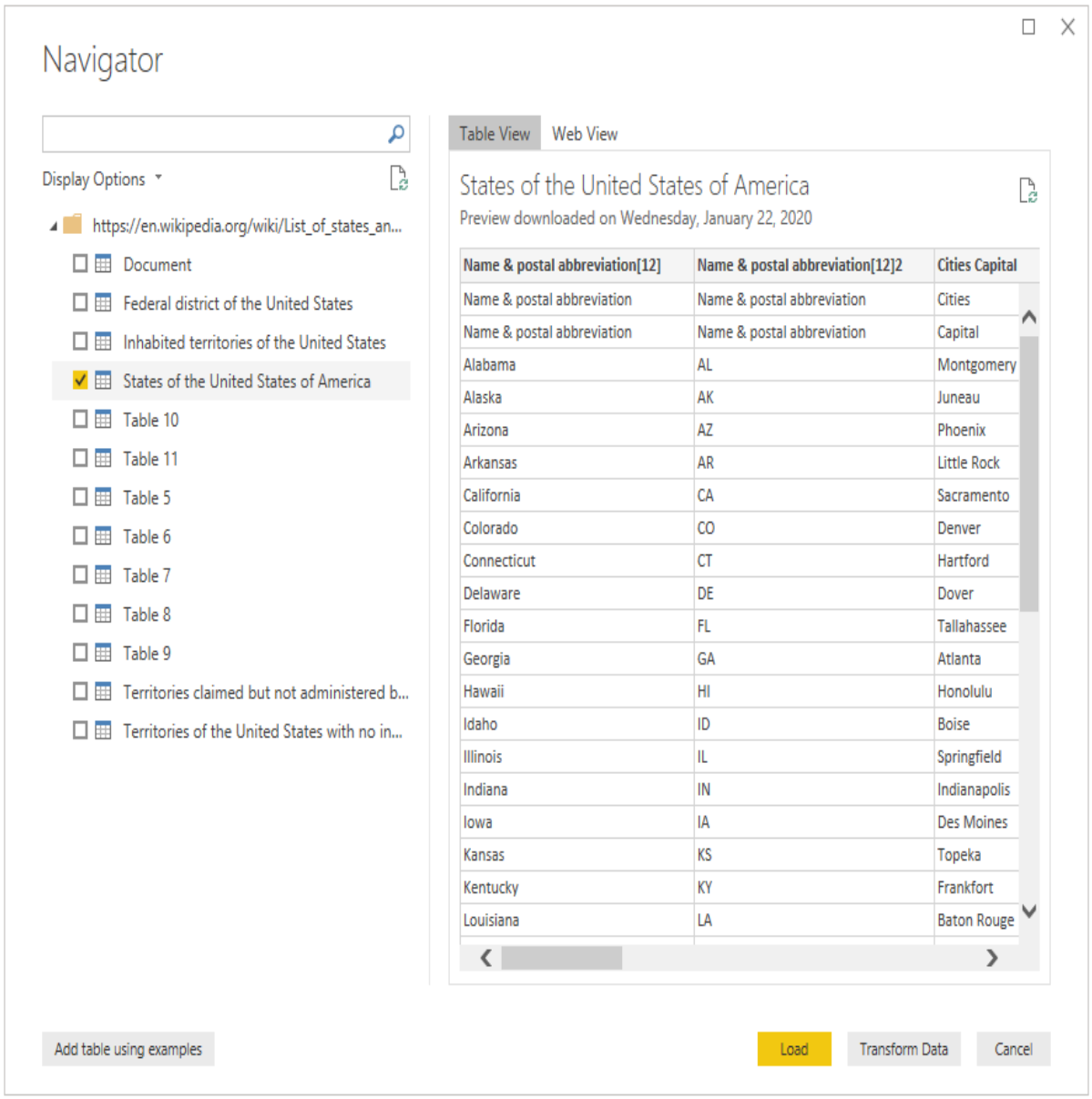
The level you select for the authentication method determines what part of a URL will have the authentication method applied to it. If you select the top-level web address, the authentication method you select here will be used for that URL address or any subaddress within that address. However, you might not want to set the top URL address to a specific authentication method because different subaddresses could require different authentication methods. For example, if you were accessing two separate folders of a single SharePoint site and wanted to use different Microsoft Accounts to access each one.

Once you've set the authentication method for a specific web site address, you won't need to select the authentication method for that URL address or any subaddress again. For example, if you select the https://en.wikipedia.org/ address in this dialog, any web page that begins with this address won't require that you select the authentication method again.

Note

If you need to change the authentication method later, go to **Changing the authentication method**.

4. From the **Navigator** dialog, you can select a table, then either transform the data in the Power Query editor by selecting **Transform Data**, or load the data by selecting **Load**.



The right side of the **Navigator** dialog displays the contents of the table you select to transform or load. If you're uncertain which table contains the data you're interested in, you can select the **Web View** tab. The web view lets you see the entire contents of the web page, and highlights each of the tables that have been detected on that site. You can select the check box above the highlighted table to obtain the data from that table.

On the lower left side of the **Navigator** dialog, you can also select the **Add table using examples** button. This selection presents an interactive window where you can preview the content of the web page and enter sample values of the data you want to extract.

2.5 READING FROM APIS

API Documentation

There's no way around navigating API documentation. For working with Outlook, you go here. If you are successful in following this tutorial and want to do more with the API, use that link for a reference.

Authorization

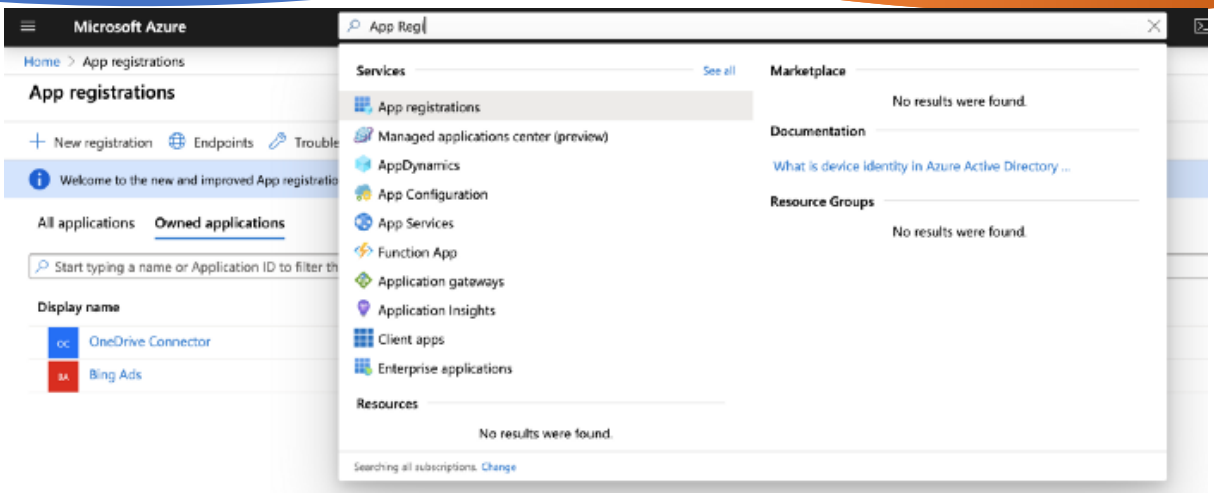
Most APIs, including the Microsoft Graph, require authorization before they'll give you an access token that will allow you to call the API. This is often the most difficult part for someone new to pulling data from APIs (it was for me).

Register your App

Follow the instructions here to register your app (these are pretty straightforward and use the Azure GUI) You'll need a school, work, or personal Microsoft account to register. If you use a work email, you'll need admin access to your Azure instance. If you're doing this for work, contact your IT department and they'll likely be able to register the app for you. Select 'web' as your app type and use a trusted URL for your redirect URI (one that only you have access to the underlying web data. I used *franklinsports.com* as only my team can access the web data for that site. If you were building an app, you would redirect to your app to a place where your app could pick up the code and authorize the user).

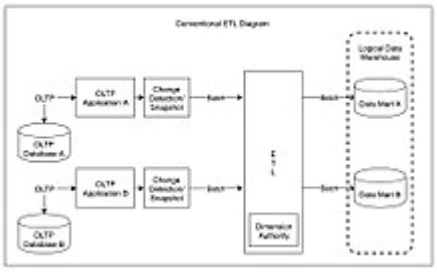
Enable Microsft Graph Permissions

Once your app is registered, go to the App Registration portal:



2.6 ETL (extract, transform, load)

In computing, extract, transform, load (ETL) is a three-phase process where data is first extracted then transformed (cleaned, sanitized, scrubbed) and finally loaded into an output data container. The data can be collated from one or more sources and it can also be outputted to one or more destinations. ETL processing is typically executed using software applications but it can be also be done manually by system operators. ETL software typically automates the entire process and can be run manually or on reoccurring schedules either as single jobs or aggregated into a batch of jobs.



Conventional ETL diagram

Extract

A properly designed ETL system extracts data from source systems and enforces data type and data validity standards and ensures it conforms structurally to the requirements of the output. Some ETL systems can also deliver data in a presentation-ready format so that application developers can build applications and end users can make decisions.

Data extraction involves extracting data from homogeneous or heterogeneous sources; data transformation processes data by data cleaning and transforming it into a proper storage format/structure for the purposes of querying and analysis; finally, data loading describes the insertion of data into the final target database such as an operational data store, a data mart, data lake or a data warehouse.

ETL processing involves extracting the data from the source system(s). In many cases, this represents the most important aspect of ETL, since extracting data correctly sets the stage for

the success of subsequent processes. Most data-warehousing projects combine data from different source systems. Each separate system may also use a different data organization and/or format. Common data-source formats include relational databases, XML, JSON and flat files, but may also include non-relational database structures such as Information Management System (IMS) or other data structures such as Virtual Storage Access Method (VSAM) or Indexed Sequential Access Method (ISAM), or even formats fetched from outside sources by means such as web spidering or screen-scraping. The streaming of the extracted data source and loading on-the-fly to the destination database is another way of performing ETL when no intermediate data storage is required.

An intrinsic part of the extraction involves data validation to confirm whether the data pulled from the sources has the correct/expected values in a given domain (such as a pattern/default or list of values). If the data fails the validation rules, it is rejected entirely or in part. The rejected data is ideally reported back to the source system for further analysis to identify and to rectify the incorrect records.

Transform

In the data transformation stage, a series of rules or functions are applied to the extracted data in order to prepare it for loading into the end target.

An important function of transformation is data cleansing, which aims to pass only "proper" data to the target. The challenge when different systems interact is in the relevant systems' interfacing and communicating. Character sets that may be available in one system may not be so in others.

Selecting only certain columns to load: (or selecting null columns not to load). For example, if the source data has three columns (aka "attributes"), roll_no, age, and salary, then the selection may take only roll_no and salary. Or, the selection mechanism may ignore all those records where salary is not present (salary = null).

Translating coded values: (e.g., if the source system codes male as "1" and female as "2", but the warehouse codes male as "M" and female as "F")

Encoding free-form values: (e.g., mapping "Male" to "M")

Deriving a new calculated value: (e.g., sale_amount = qty * unit_price)

Sorting or ordering the data based on a list of columns to improve search performance

Joining data from multiple sources (e.g., lookup, merge) and deduplicating the data

Aggregating (for example, rollup — summarizing multiple rows of data — total sales for each store, and for each region, etc.)

Generating surrogate-key values

Transposing or pivoting (turning multiple columns into multiple rows or vice versa)

Splitting a column into multiple columns (e.g., converting a comma-separated list, specified as a string in one column, into individual values in different columns)

Disaggregating repeating columns

Looking up and validating the relevant data from tables or referential files

Applying any form of data validation; failed validation may result in a full rejection of the data, partial rejection, or no rejection at all, and thus none, some, or all of the data is handed over to the next step depending on the rule design and exception handling; many of the above transformations may result in exceptions, e.g., when a code translation parses an unknown code in the extracted data

Load

The load phase loads the data into the end target, which can be any data store including a simple delimited flat file or a data warehouse. Depending on the requirements of the organization, this process varies widely. Some data warehouses may overwrite existing information with cumulative information; updating extracted data is frequently done on a daily, weekly, or monthly basis. Other data warehouses (or even other parts of the same data warehouse) may add new data in a historical form at regular intervals — for example, hourly. To understand this, consider a data warehouse that is required to maintain sales records of the last year. This data warehouse overwrites any data older than a year with newer data. However, the entry of data for any one-year window is made in a historical manner. The timing and scope to replace or append are strategic design choices dependent on the time available and the business needs. More complex systems can maintain a history and audit trail of all changes to the data loaded in the data warehouse. As the load phase interacts with a database, the constraints defined in the database schema — as well as in triggers activated upon data load — apply (for example, uniqueness, referential integrity, mandatory fields), which also contribute to the overall data quality performance of the ETL process.

For example, a financial institution might have information on a customer in several departments and each department might have that customer's information listed in a different way. The membership department might list the customer by name, whereas the accounting department might list the customer by number. ETL can bundle all of these data elements and consolidate them into a uniform presentation, such as for storing in a database or data warehouse.

Another way that companies use ETL is to move information to another application permanently. For instance, the new application might use another database vendor and most likely a very different database schema. ETL can be used to transform the data into a format suitable for the new application to use.

An example would be an Expense and Cost Recovery System (ECRS) such as used by accountancies, consultancies, and legal firms. The data usually ends up in the time and billing system, although some businesses may also utilize the raw data for employee productivity reports to Human Resources (personnel dept.) or equipment usage reports to Facilities Management.

Real-life ETL cycle

The typical real-life ETL cycle consists of the following execution steps:

- Cycle initiation
- Build reference data
- Extract (from sources)

- Validate
- Transform (clean, apply business rules, check for data integrity, create aggregates or disaggregates)
- Stage (load into staging tables, if used)
- Audit reports (for example, on compliance with business rules. Also, in case of failure, helps to diagnose/repair)
- Publish (to target tables)
- Archive

Challenges

ETL processes can involve considerable complexity, and significant operational problems can occur with improperly designed ETL systems.

The range of data values or data quality in an operational system may exceed the expectations of designers at the time validation and transformation rules are specified. Data profiling of a source during data analysis can identify the data conditions that must be managed by transform rules specifications, leading to an amendment of validation rules explicitly and implicitly implemented in the ETL process.

Data warehouses are typically assembled from a variety of data sources with different formats and purposes. As such, ETL is a key process to bring all the data together in a standard, homogeneous environment.

Design analysis should establish the scalability of an ETL system across the lifetime of its usage — including understanding the volumes of data that must be processed within service level agreements. The time available to extract from source systems may change, which may mean the same amount of data may have to be processed in less time. Some ETL systems have to scale to process terabytes of data to update data warehouses with tens of terabytes of data. Increasing volumes of data may require designs that can scale from daily batch to multiple-day micro batch to integration with message queues or real-time change-data-capture for continuous transformation and update.

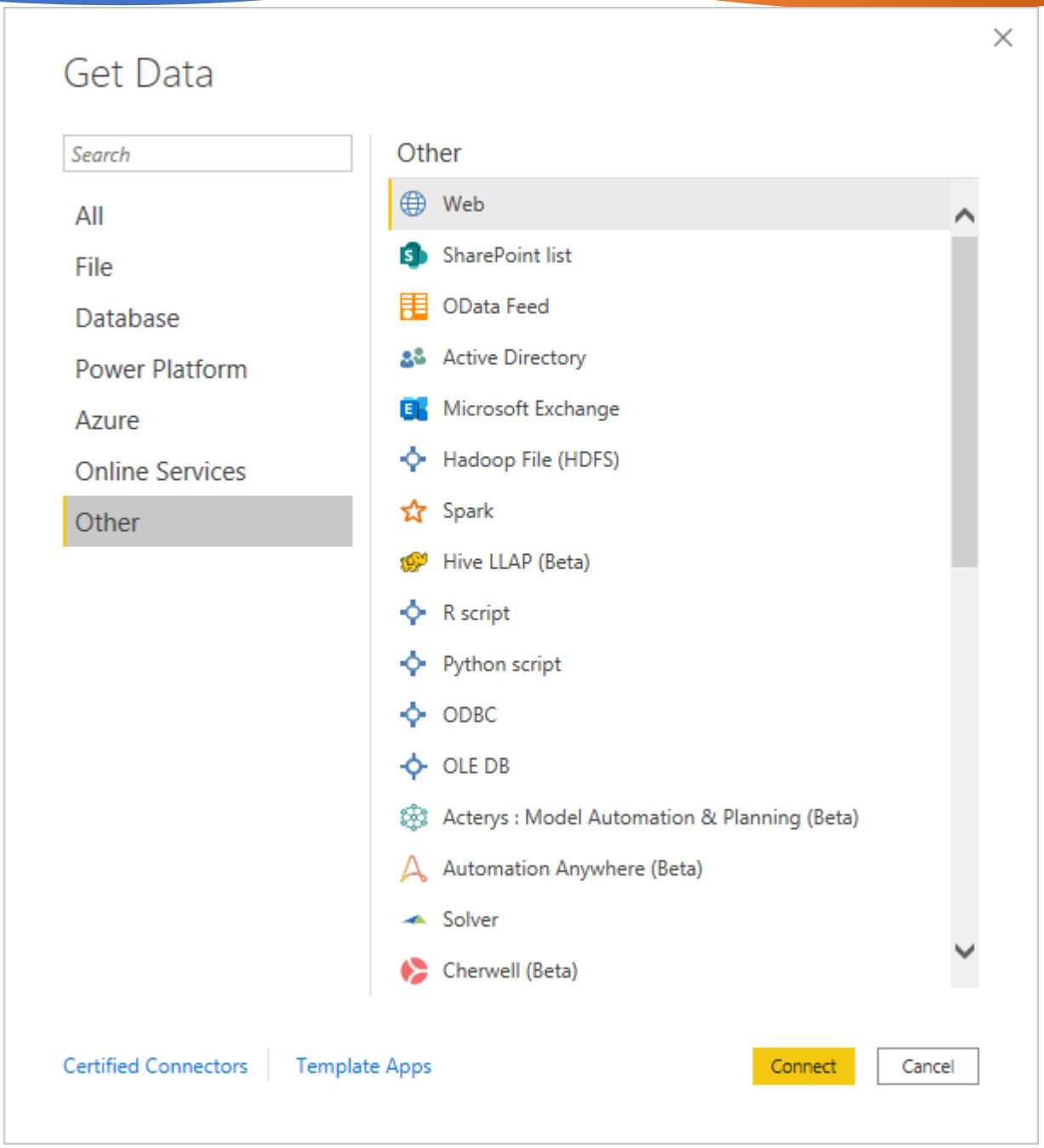
2.7 Reading Data From Other Sources

The **Other** category provides the following data connections:

- Web
- SharePoint list
- OData Feed
- Active Directory
- Microsoft Exchange
- Hadoop File (HDFS)
- Spark
- Hive LLAP
- R script
- Python script
- ODBC
- OLE DB

- Acterys : Model Automation & Planning (Beta)
- Amazon OpenSearch Service (Beta)
- Anaplan Connector Autodesk Construction Cloud (Beta)
- Solver
- BitSight Security Ratings
- BQE Core
- Bloomberg Data and Analytics
- Cherwell (Beta)
- Cognite Data Fusion
- Delta Sharing
- Eduframe (Beta)
- EQuIS (Beta)
- FactSet RMS (Beta)
- FHIR
- Google Sheets (Beta)
- Information Grid (Beta)
- Jamf Pro (Beta)
- Kognitwin
- MicroStrategy for Power BI
- OpenSearch Project (Beta)
- Paxata
- QubolePresto (Beta)
- Roamler (Beta)
- SIS-CC SDMX (Beta)
- Shortcuts Business Insights (Beta)
- SingleStore Direct Query Connector 1.0 (Beta)
- Siteimprove
- Socialbakers Metrics 1.1.0 (Beta)
- Starburst Enterprise
- SumTotal
- SurveyMonkey (Beta)
- Microsoft Teams Personal Analytics (Beta)
- Tenforce (Smart)List
- Usercube (Beta)
- Vena
- Vessel Insight
- Zucchetti HR Infinity (Beta)
- Blank Query

The following image shows the **Get Data** window for **Other**.

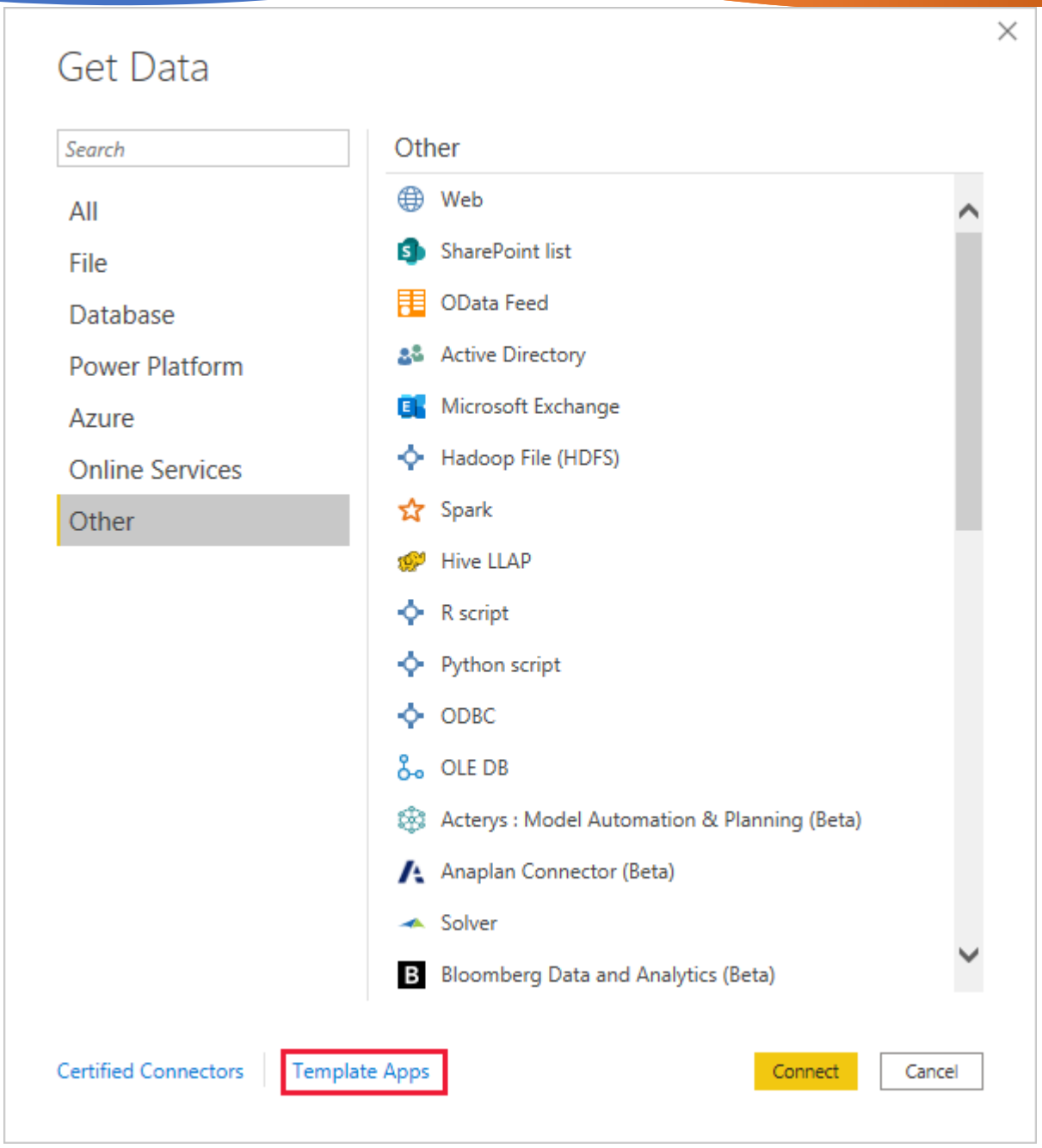


Note

At this time, it's not possible to connect to custom data sources secured using Azure Active Directory.

Template apps

You can find template apps for your organization by selecting the **Template Apps** link near the bottom of the **Get Data** window.



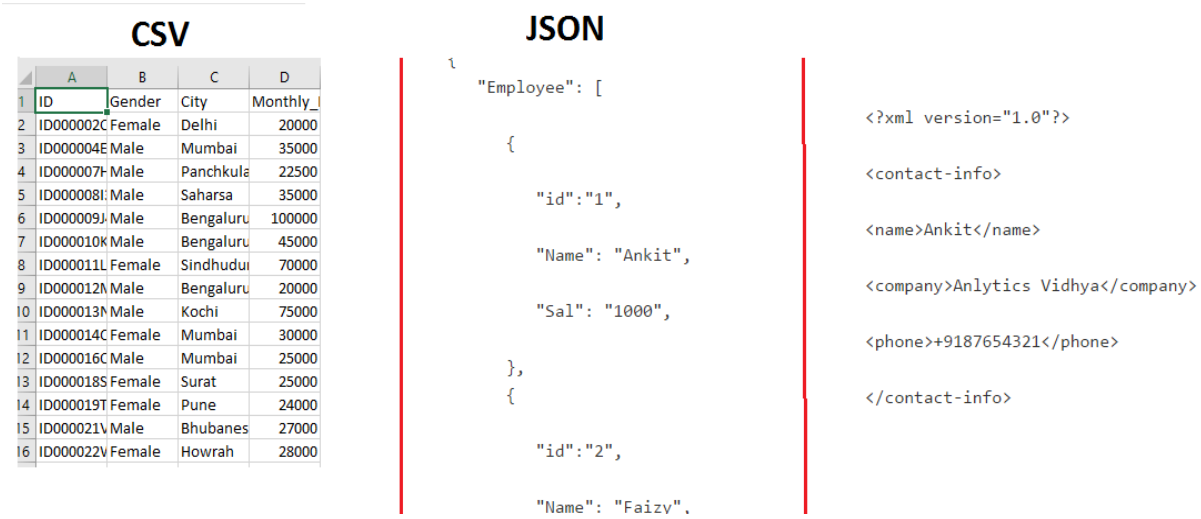
Available Template Apps may vary based on your organization.

READING DIFFERENT FILE TYPES.

1. What is a file format?

A file format is a standard way in which information is encoded for storage in a file. First, the file format specifies whether the file is a binary or ASCII file. Second, it shows how the information is organized. For example, comma-separated values (CSV) file format stores tabular data in plain text.

To identify a file format, you can usually look at the file extension to get an idea. For example, a file saved with name “Data” in “CSV” format will appear as “Data.csv”. By noticing “.csv” extension we can clearly identify that it is a “CSV” file and data is stored in a tabular format.



	A	B	C	D
1	ID	Gender	City	Monthly_I
2	ID000002C	Female	Delhi	20000
3	ID000004E	Male	Mumbai	35000
4	ID000007H	Male	Panchkula	22500
5	ID000008I	Male	Saharsa	35000
6	ID000009J	Male	Bengaluru	100000
7	ID000010K	Male	Bengaluru	45000
8	ID000011L	Female	Sindhudur	70000
9	ID000012M	Male	Bengaluru	20000
10	ID000013N	Male	Kochi	75000
11	ID000014C	Female	Mumbai	30000
12	ID000016C	Male	Mumbai	25000
13	ID000018S	Female	Surat	25000
14	ID000019T	Female	Pune	24000
15	ID000021V	Male	Bhubanes	27000
16	ID000022V	Female	Howrah	28000

```
{
   "Employee": [
      {
         "id":"1",
         "Name": "Ankit",
         "Sal": "1000",
      },
      {
         "id":"2",
         "Name": "Faizy",
```

2. Why should we understand different file formats?

Usually, the files you will come across will depend on the application you are building. For example, in an image processing system, you need image files as input and output. So you will mostly see files in jpeg, gif or png format.

As a data scientist, you need to understand the underlying structure of various file formats, their advantages and dis-advantages. Unless you understand the underlying structure of the data, you will not be able to explore it. Also, at times you need to make decisions about how to store data.

Comma-separated values

Comma-separated values file format falls under spreadsheet file format.

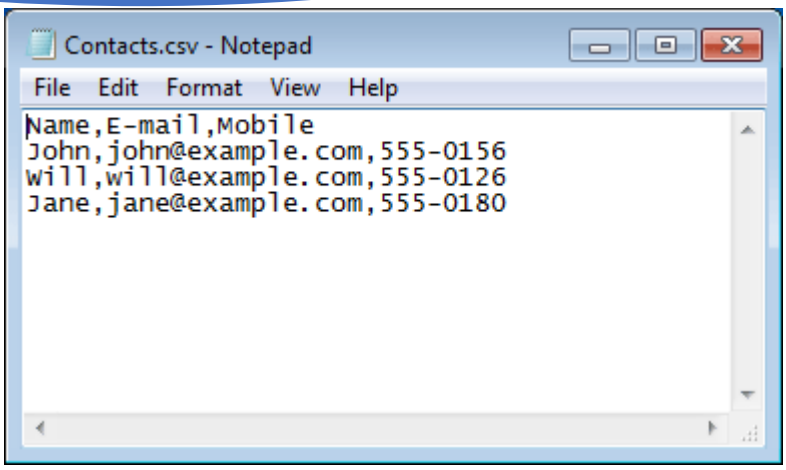
What is Spreadsheet File Format?

In spreadsheet file format, data is stored in cells. Each cell is organized in rows and columns. A column in the spreadsheet file can have different types. For example, a column can be of string type, a date type or an integer type. Some of the most popular spreadsheet file formats are Comma Separated Values (CSV), Microsoft Excel Spreadsheet (xlsx) and Microsoft Excel Open XML Spreadsheet (xlsx).

Each line in CSV file represents an observation or commonly called a record. Each record may contain one or more fields which are separated by a comma.

Sometimes you may come across files where fields are not separated by using a comma, but they are separated using tab. This file format is known as TSV (Tab Separated Values) file format.

The below image shows a CSV file which is opened in Notepad.

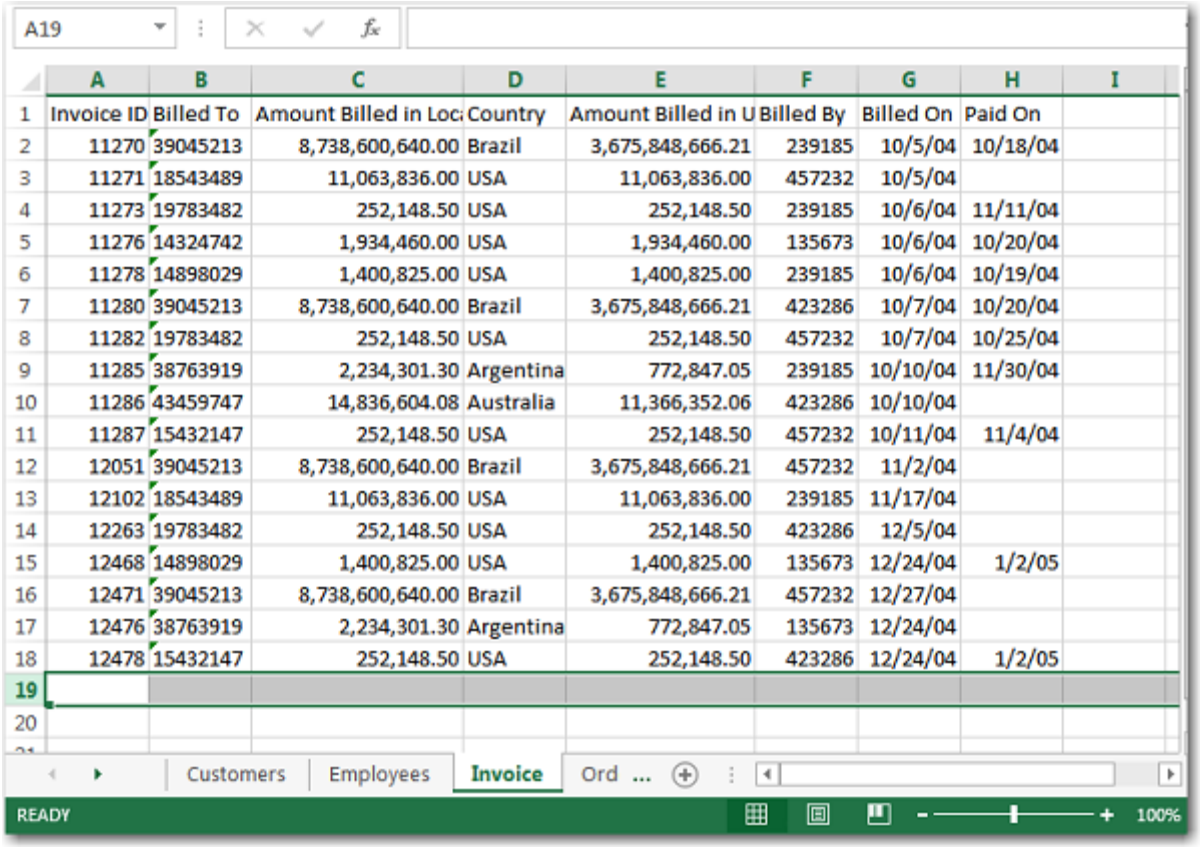


XLSX files

XLSX is a Microsoft Excel Open XML file format. It also comes under the Spreadsheet file format. It is an XML-based file format created by Microsoft Excel. The XLSX format was introduced with Microsoft Office 2007.

In XLSX data is organized under the cells and columns in a sheet. Each XLSX file may contain one or more sheets. So a workbook can contain multiple sheets.

The below image shows a “xlsx” file which is opened in Microsoft Excel.



A19

	A	B	C	D	E	F	G	H	I
1	Invoice ID	Billed To	Amount Billed in Loc	Country	Amount Billed in U	Billed By	Billed On	Paid On	
2	11270	39045213	8,738,600,640.00	Brazil	3,675,848,666.21	239185	10/5/04	10/18/04	
3	11271	18543489	11,063,836.00	USA	11,063,836.00	457232	10/5/04		
4	11273	19783482	252,148.50	USA	252,148.50	239185	10/6/04	11/11/04	
5	11276	14324742	1,934,460.00	USA	1,934,460.00	135673	10/6/04	10/20/04	
6	11278	14898029	1,400,825.00	USA	1,400,825.00	239185	10/6/04	10/19/04	
7	11280	39045213	8,738,600,640.00	Brazil	3,675,848,666.21	423286	10/7/04	10/20/04	
8	11282	19783482	252,148.50	USA	252,148.50	457232	10/7/04	10/25/04	
9	11285	38763919	2,234,301.30	Argentina	772,847.05	239185	10/10/04	11/30/04	
10	11286	43459747	14,836,604.08	Australia	11,366,352.06	423286	10/10/04		
11	11287	15432147	252,148.50	USA	252,148.50	457232	10/11/04	11/4/04	
12	12051	39045213	8,738,600,640.00	Brazil	3,675,848,666.21	457232	11/2/04		
13	12102	18543489	11,063,836.00	USA	11,063,836.00	239185	11/17/04		
14	12263	19783482	252,148.50	USA	252,148.50	423286	12/5/04		
15	12468	14898029	1,400,825.00	USA	1,400,825.00	135673	12/24/04	1/2/05	
16	12471	39045213	8,738,600,640.00	Brazil	3,675,848,666.21	457232	12/27/04		
17	12476	38763919	2,234,301.30	Argentina	772,847.05	135673	12/24/04		
18	12478	15432147	252,148.50	USA	252,148.50	423286	12/24/04	1/2/05	
19									
20									

Customers | Employees | Invoice | Ord ...

READY 100%

In above image, you can see that there are multiple sheets present (bottom left) in this file, which are Customers, Employees, Invoice, Order. The image shows the data of only one sheet – "Invoice".

ZIP files

ZIP format is an archive file format.

What is Archive File format?
In Archive file format, you create a file that contains multiple files along with metadata. An archive file format is used to collect multiple data files together into a single file. This is done for simply compressing the files to use less storage space.

There are many popular computer data archive format for creating archive files. Zip, RAR and Tar being the most popular archive file format for compressing the data.

So, a ZIP file format is a lossless compression format, which means that if you compress the multiple files using ZIP format you can fully recover the data after decompressing the ZIP file. ZIP file format uses many compression algorithms for compressing the documents. You can easily identify a ZIP file by the .zip extension.

Plain Text (txt) file format

In Plain Text file format, everything is written in plain text. Usually, this text is in unstructured form and there is no meta-data associated with it. The txt file format can easily be read by any program. But interpreting this is very difficult by a computer program.

Let's take a simple example of a text File.

The following example shows **text file data** that contain **text**:

```
"world is full of idiots"
```

JSON file format

JavaScript Object Notation(JSON) is a text-based open standard designed for exchanging the data over web. JSON format is used for transmitting structured data over the web. The JSON file format can be easily read in any programming language because it is language-independent data format.

Let's take an example of a JSON file

The following example shows how a typical JSON file stores information of employees.

```
{
  "Employee": [

    {

      "id":"1",

      "Name": "Ankit",

      "Sal": "1000",

    },
    {

      "id":"2",

      "Name": "Faizy",

      "Sal": "2000",

    }
  ]
}
```

XML file format

XML is also known as Extensible Markup Language. As the name suggests, it is a markup language. It has certain rules for encoding data. XML file format is a human-readable and machine-readable file format. XML is a self-descriptive language designed for sending information over the internet. XML is very similar to HTML, but has some differences. For example, XML does not use predefined tags as HTML.

Let's take the simple example of XML File format.

The following example shows an xml document that contains the information of an employee.

```
<?xml version="1.0"?>
<contact-info>

<name>Ankit</name>

<company>AnlyticsVidhya</company>

<phone>+9187654321</phone>

</contact-info>
```

The "<?xml version="1.0"?>" is a XML declaration at the start of the file (it is optional). In this deceleration, *version* specifies the XML version and *encoding* specifies the character encoding used in the document. <contact-info> is a tag in this document. Each XML-tag needs to be closed.

HTML files

HTML stands for Hyper Text Markup Language. It is the standard markup language which is used for creating Web pages. HTML is used to describe structure of web pages using markup. HTML tags are same as XML but these are predefined. You can easily identify HTML document subsection on basis of tags such as <head> represent the heading of HTML document. <p> "paragraph" paragraph in HTML. HTML is not case sensitive.

The following example shows an HTML document.

```
<!DOCTYPE html>
<html>
<head>
<title>Page Title</title>
</head>
<body><h1>My First Heading</h1>
<p>My first paragraph.</p></body>
</html>
```

Each tag in HTML is enclosed under the angular bracket(<>). The <!DOCTYPE html> tag defines that document is in HTML format. <html> is the root tag of this document. The <head> element contains heading part of this document. The <title>, <body>, <h1>, <p> represent the title, body, heading and paragraph respectively in the HTML document.

Image files

Image files are probably the most fascinating file format used in data science. Any computer vision application is based on image processing. So it is necessary to know different image file formats.

Usual image files are 3-Dimensional, having RGB values. But, they can also be 2-Dimensional (grayscale) or 4-Dimensional (having intensity) – an Image consisting of pixels and meta-data associated with it.

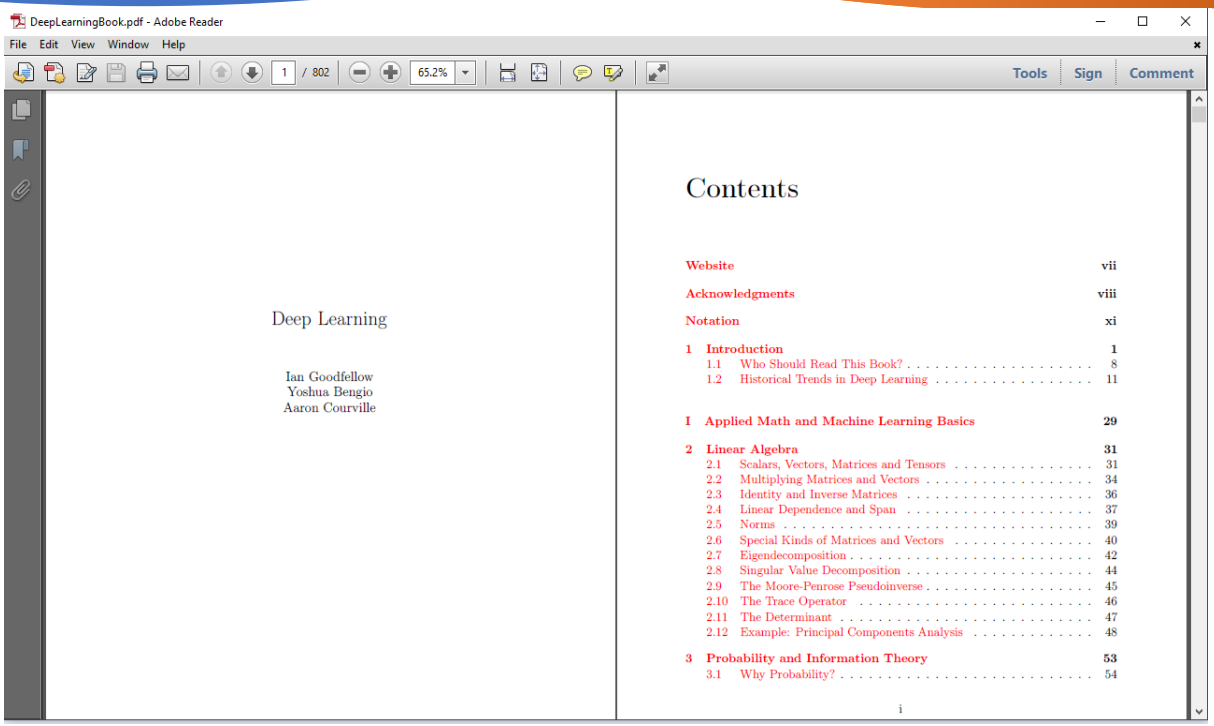
Each image consists of one or more frames of pixels. And each frame is made up of two-dimensional array of pixel values. Pixel values can be of any intensity. Meta-data associated with an image, can be an image type (.png) or pixel dimensions.



PDF file format

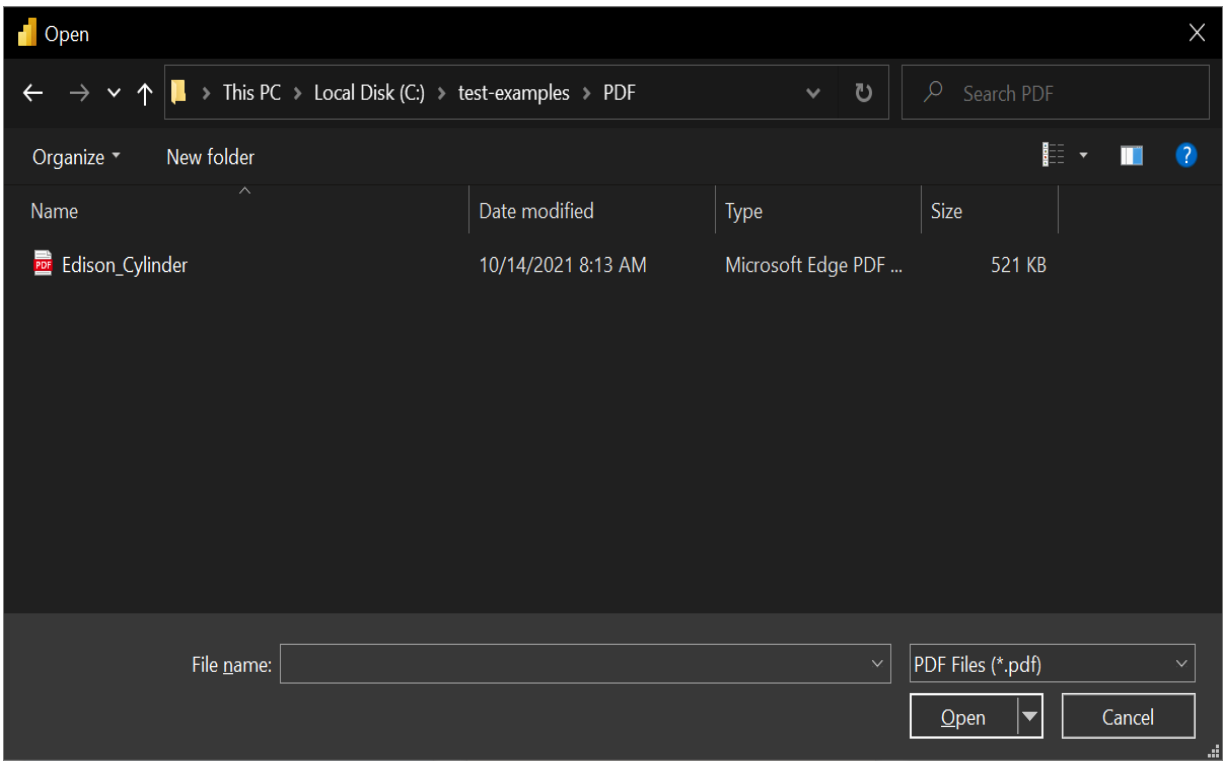
PDF (Portable Document Format) is an incredibly useful format used for interpretation and display of text documents along with incorporated graphics. A special feature of a PDF file is that it can be secured by a password.

Here's an example of a pdf file.



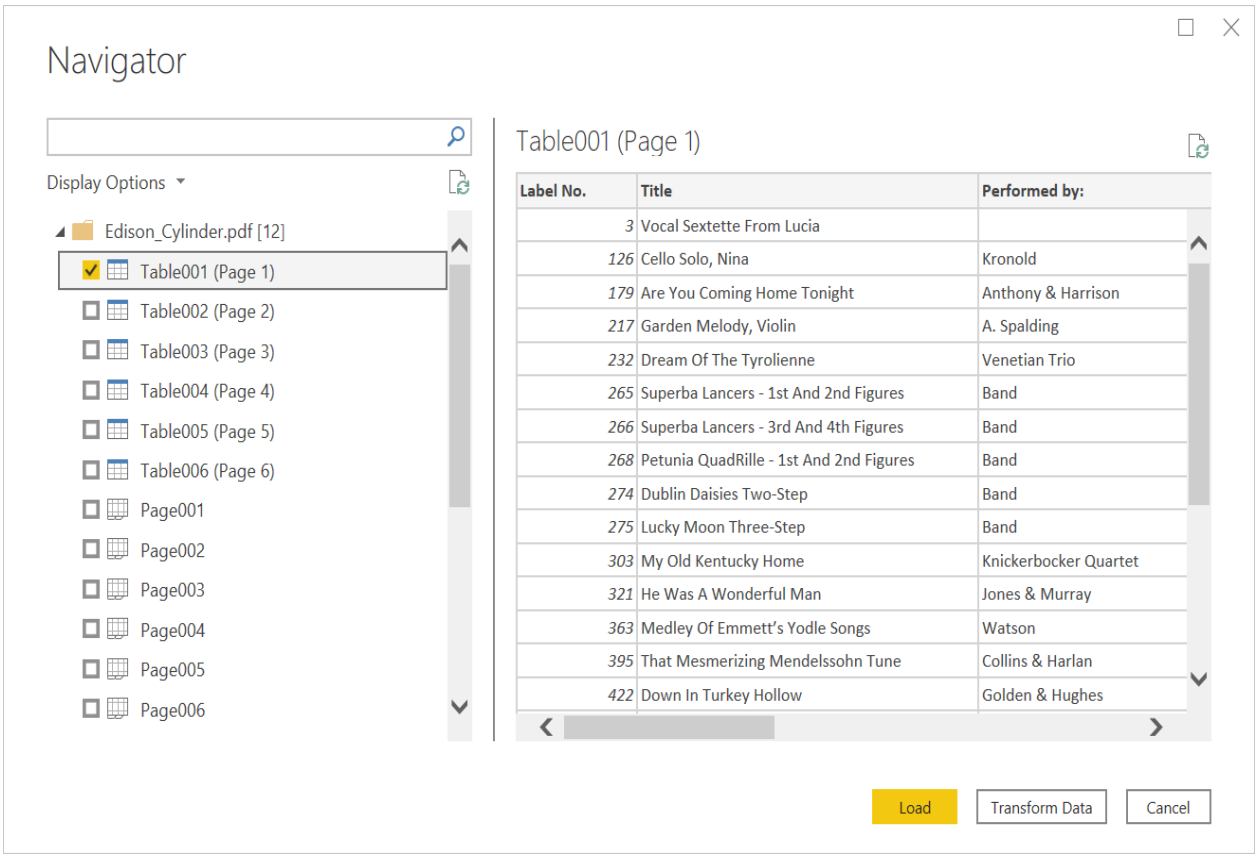
To make the connection from Power Query Desktop:

1. Select the **PDF** option in the connector selection.
2. Browse for and select the PDF file you want to load. Then select **Open**.



If the PDF file is online, use the Web connector to connect to the file.

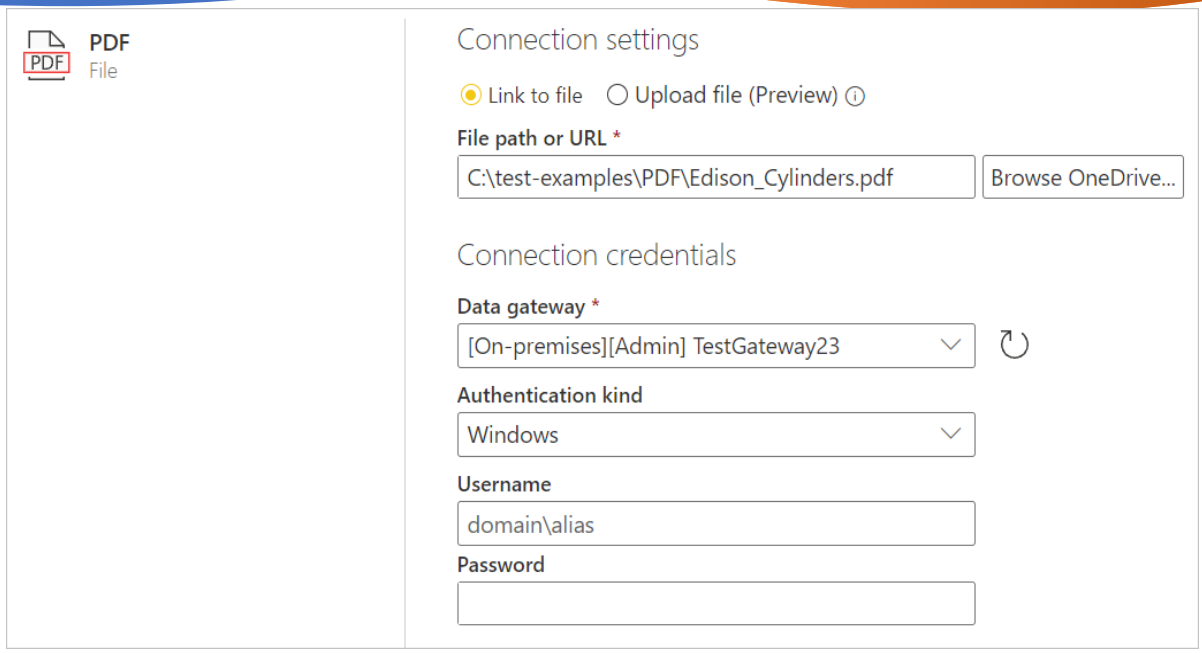
3. In **Navigator**, select the file information you want, then either select **Load** to load the data or **Transform Data** to continue transforming the data in Power Query Editor.



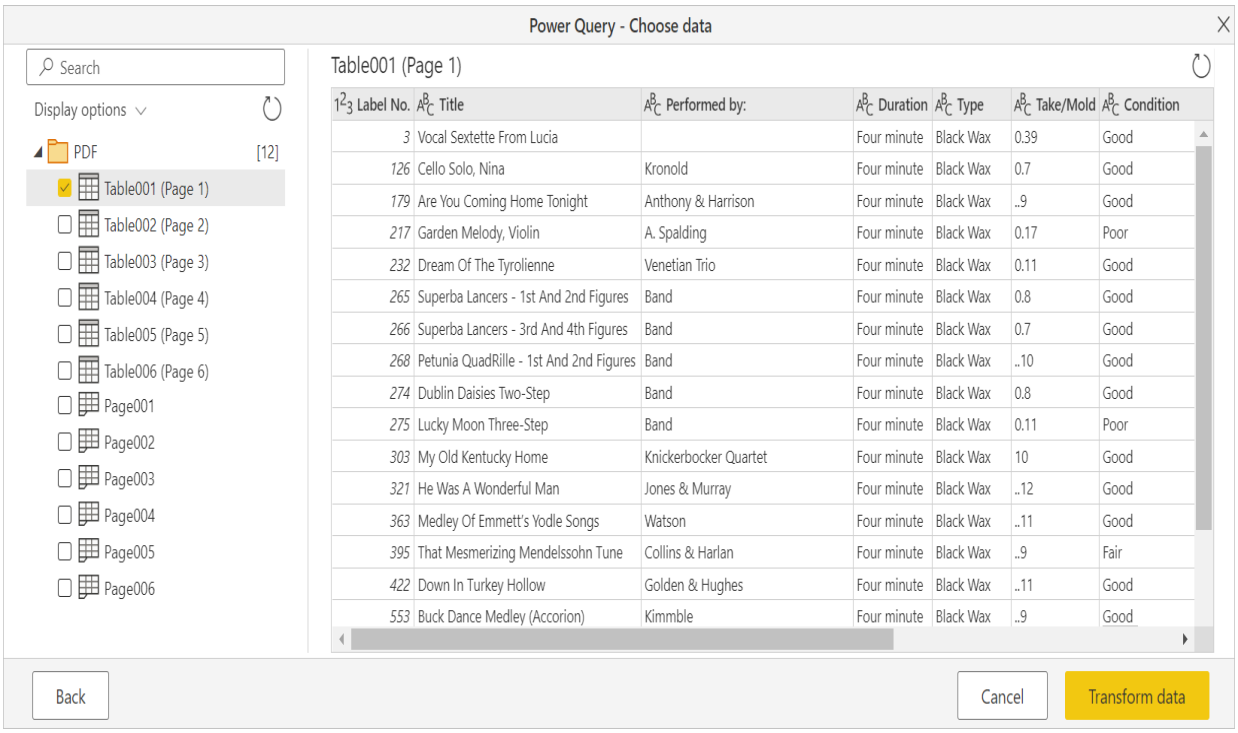
Connect to a PDF file from Power Query Online

To make the connection from Power Query Online:

1. Select the **PDF** option in the connector selection.
2. In the PDF dialog box that appears, either provide the file path or the URL to the location of the PDF file. If you're loading a local file, you can also select **Upload file (Preview)** to browse to the local file or drag and drop the file.



3. If necessary, select an on-premises data gateway to access the PDF file.
4. If this is the first time you've accessed this PDF file, select the authentication kind and sign in to your account (if needed).
5. In **Navigator**, select the file information you want, and then select **Transform Data** to continue transforming the data in Power Query Editor.



Assignment

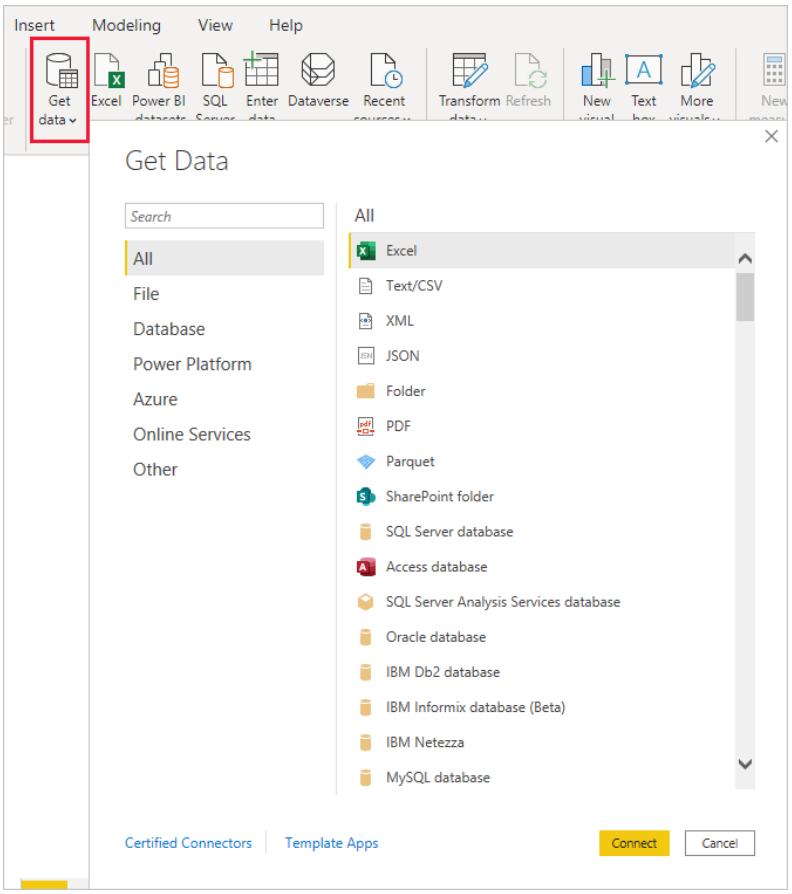
Connect excel data source with Power BI. (Get free sample database from online)

Solution :

Click on the "**Get data**" button to see the most common data types menu.

The "**Get Data**" dialog box categorizes the data types in the following way.

- **File:** Excel, CSV, XML, JSON, etc.
- **Database:** SQL Server, Oracle, MySQL, etc.
- **Power Platform:** Power BI datasets, Power BI dataflows, Dataverse, etc.
- **Azure:** MS Azure SQL Database, MS Azure Marketplace, etc.
- **Online Services:** Google Analytics, Zendesk, Asana, GitHub, Marketo, etc.
- **Other:** Web, R script, Python script, Spark, Google Sheets, etc.



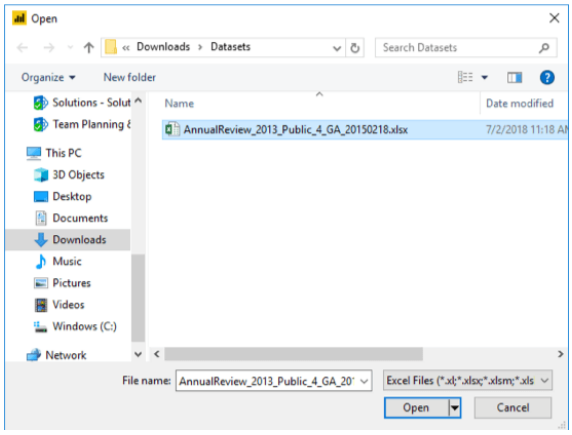
Get Data

Power BI Desktop and Power Query allow users to automate the process of Power BI Data Ingestion. Let's discuss an example to understand this better. Before proceeding, make sure you've Power BI Desktop installed on your system. For the purposes of this demonstration,

the General Aviation 2013 Excel dataset file (available for public use from Data.gov) has been used. Download the Excel file and save it in a local folder.

Follow the below-mentioned steps to load data into Power BI.

- Launch Power BI Desktop and click on the "**Get Data**" button from the Power BI toolbar. Now, choose the Excel connector and click on "**Connect**".
- Locate the General Aviation Excel file you just downloaded and click on "**Open**".



Chapter: 3

Data Cleansing

Scope:

- we look at finding data and reading different file types.
- Getting and Cleaning Data! The primary goal is to introduce you to the most common data storage systems and the appropriate tools to extract data from web or from databases like MySQL.
- Introduction to Data Wrangling and Shaping
- Advanced Shaping
- Beyond Initial Shaping
- Advanced DAX
- Publishing and Testing Your Dataset
- Summarizing Data
- Creating New Variables

DATA CLEANSING OVERVIEW

- Data cleaning is the process of fixing or removing incorrect, corrupted, incorrectly formatted, duplicate, or incomplete data within a dataset.
- When combining multiple data sources, there are many opportunities for data to be duplicated or mislabeled. If data is incorrect, outcomes and algorithms are unreliable, even though they may look correct.
- There is no one absolute way to prescribe the exact steps in the data cleaning process because the processes will vary from dataset to dataset. But it is crucial to establish a template for your data cleaning process, so you know you are doing it the right way every time.

What is the difference between data cleaning and data transformation?

Data cleaning is the process that removes data that does not belong in your dataset. Data transformation is the process of converting data from one format or structure into another. Transformation processes can also be referred to as data wrangling, or data munging, transforming and mapping data from one "raw" data form into another format for warehousing and analyzing. This article focuses on the processes of cleaning that data.

How do you clean data?

While the techniques used for data cleaning may vary according to the types of data your company stores, you can follow these basic steps to map out a framework for your organization.

Step 1: Remove duplicate or irrelevant observations

Remove unwanted observations from your dataset, including duplicate observations or irrelevant observations. Duplicate observations will happen most often during data collection. When you combine data sets from multiple places, scrape data, or receive data from clients or multiple departments, there are opportunities to create duplicate data. De-duplication is one of the largest areas to be considered in this process. Irrelevant observations are when you notice observations that do not fit into the specific problem you are trying to analyze. For example, if you want to analyze data regarding millennial customers, but your dataset includes older generations, you might remove those irrelevant observations. This can make analysis more efficient and minimize distraction from your primary target—as well as creating a more manageable and more performant dataset.

Step 2: Fix structural errors

Structural errors are when you measure or transfer data and notice strange naming conventions, typos, or incorrect capitalization. These inconsistencies can cause mislabeled

categories or classes. For example, you may find “N/A” and “Not Applicable” both appear, but they should be analyzed as the same category.

Step 3: Filter unwanted outliers

Often, there will be one-off observations where, at a glance, they do not appear to fit within the data you are analyzing. If you have a legitimate reason to remove an outlier, like improper data-entry, doing so will help the performance of the data you are working with. However, sometimes it is the appearance of an outlier that will prove a theory you are working on. Remember: just because an outlier exists, doesn’t mean it is incorrect. This step is needed to determine the validity of that number. If an outlier proves to be irrelevant for analysis or is a mistake, consider removing it.

Step 4: Handle missing data

You can’t ignore missing data because many algorithms will not accept missing values. There are a couple of ways to deal with missing data. Neither is optimal, but both can be considered.

- As a first option, you can drop observations that have missing values, but doing this will drop or lose information, so be mindful of this before you remove it.
- As a second option, you can input missing values based on other observations; again, there is an opportunity to lose integrity of the data because you may be operating from assumptions and not actual observations.
- As a third option, you might alter the way the data is used to effectively navigate null values.

Step 5: Validate and QA

At the end of the data cleaning process, you should be able to answer these questions as a part of basic validation:

- Does the data make sense?
- Does the data follow the appropriate rules for its field?
- Does it prove or disprove your working theory, or bring any insight to light?
- Can you find trends in the data to help you form your next theory?
- If not, is that because of a data quality issue?

Components of quality data

Determining the quality of data requires an examination of its characteristics, then weighing those characteristics according to what is most important to your organization and the application(s) for which they will be used.

Characteristics of quality data

- Accuracy. Ensure your data is close to the true values.
- Completeness. The degree to which all required data is known.
- Consistency. Ensure your data is consistent within the same dataset and/or across multiple data sets.

- Uniformity. The degree to which the data is specified using the same unit of measure.

Benefits of data cleaning

Having clean data will ultimately increase overall productivity and allow for the highest quality information in your decision-making. Benefits include:

- Removal of errors when multiple sources of data are at play.
- Fewer errors make for happier clients and less-frustrated employees.
- Ability to map the different functions and what your data is intended to do.
- Monitoring errors and better reporting to see where errors are coming from, making it easier to fix incorrect or corrupt data for future applications.
- Using tools for data cleaning will make for more efficient business practices and quicker decision-making.

3.1 WE LOOK AT FINDING DATA AND READING DIFFERENT FILE TYPES

Data Types

Data types generally fall into five categories:

1. Observational
   - Can't be recaptured, recreated or replaced
   - Examples: Sensor readings, sensory (human) observations, survey results
2. Experimental
   - Data collected under controlled conditions, in laboratory
   - Should be reproducible, but can be expensive
   - Examples: gene sequences, chromatograms, spectroscopy, microscopy
3. Derived or compiled
   - Reproducible, but can be very expensive
   - Examples: text and data mining, derived variables, compiled database, 3D models
4. Simulation
   - Results from using a model to study the behaviour and performance of an actual or theoretical system
   - Models and metadata, where the input can be more important than output data
   - Examples: climate models, economic models, biogeochemical models
5. Reference or canonical
   - Static or organic collection [peer-reviewed] datasets, most probably published and/or curated.
   - Examples: gene sequence databanks, chemical structures, census data, spatial data portals.

   Data Formats

Research data comes in many varied formats: text, numeric, multimedia, models, software languages, discipline specific (e.g. crystallographic information file (CIF) in chemistry), and instrument specific.

Formats more likely to be accessible in the future are:
- Non-proprietary
- Open, documented standards
- In common usage by the research community
- Using standard character encodings (ASCII, UTF-8)
- Uncompressed (desirable, space permitting

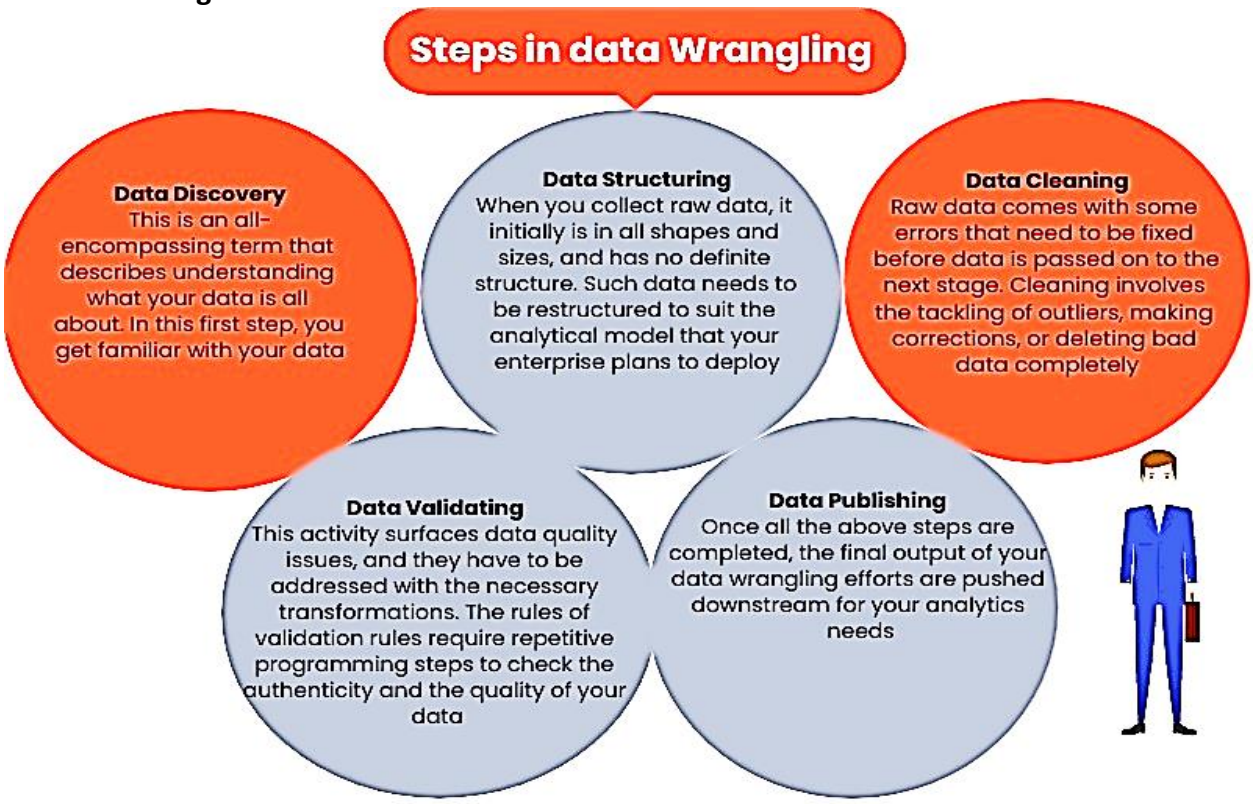
Geospatial data vector and raster data	ESRI Shapefile (essential: .shp, .shx, .dbf ; optional: .prj, .sbx, .sbn) geo-referenced TIFF (.tif, .tfw) CAD data (.dwg) tabular GIS attribute data	ESRI Geodatabase format (.mdb) MapInfo Interchange Format (.mif) for vector data
Qualitative data textual	• eXtensible Mark-up Language (XML) text according to an appropriate Document Type Definition (DTD) or schema (.xml) • Rich Text Format (.rtf) • plain text data, UTF-8 (Unicode; .txt)	• plain text data, ASCII (.txt) • Hypertext Mark-up Language (HTML) (.html) • widely-used proprietary formats, e.g. MS Word (.doc/.docx) • LaTeX(.tex)
Digital image data	TIFF version 6 uncompressed (.tif)	• JPEG (.jpeg, .jpg) • TIFF (other versions; .tif, .tiff) • JPEG 2000 (.jp2) • Adobe Portable Document Format (PDF/A, PDF) (.pdf)
Digital audio data	➢ Free Lossless Audio Codec (FLAC) (.flac) ➢ Waveform Audio Format (WAV) (.wav) ➢ MPEG-1 Audio Layer 3 (.mp3) - spoken word audio only	• MPEG-1 Audio Layer 3 (.mp3) • Audio Interchange File Format (AIFF) (.aif)
Digital video data	• MPEG-4 High Profile (.mp4) • motion JPEG 2000 (.jp2)	JPEG 2000 (.mj2)
Documentation & Scripts	• Rich Text Format (.rtf) • Open Document Text (.odt) • HTML (.htm, .html)	• plain text (.txt) • widely-used proprietary formats, e.g. MS Word (.doc/.docx) or MS Excel (.xls/ .xlsx) • XML marked-up text (.xml) according to an appropriate DTD or schema, e.g. XHMTL 1.0 • PDF/A or PDF (.pdf)

3.3 Data wrangling

Data wrangling is the practice of converting and then plotting data from one "raw" form into another. The aim is to make it ready for downstream analytics. Often in charge of this is a data wrangler or a team of "mangers".

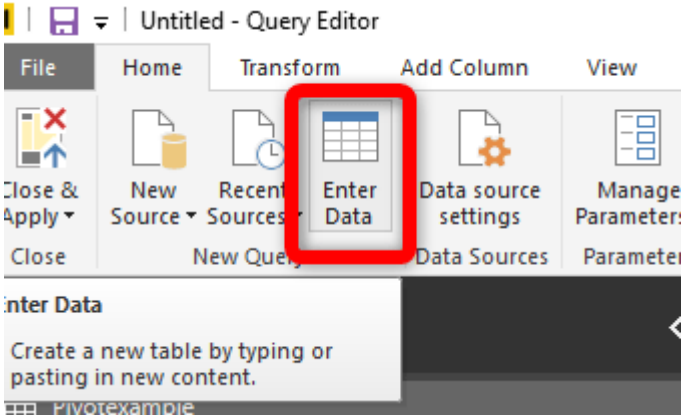
Steps in Data Wrangling:

1. **Data Discovery**.
2. **Data Structuring.**
3. **Data Cleaning.**
4. **Data Enriching.**
5. **Data Validating**.
6. **Data Publishing**.



Data Wrangling in Power BI

To pivot data using Power Query, we need to enter data using "Enter Data Manually" option.

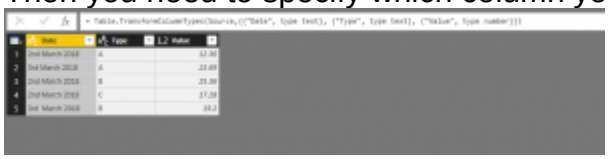


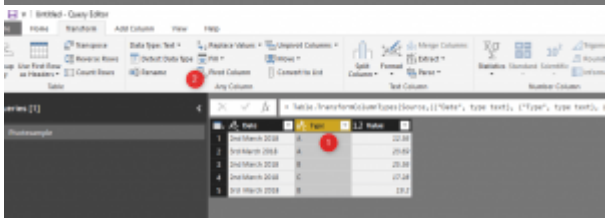
By clicking on the “Enter Data”, you able to put data manually in Power Query.

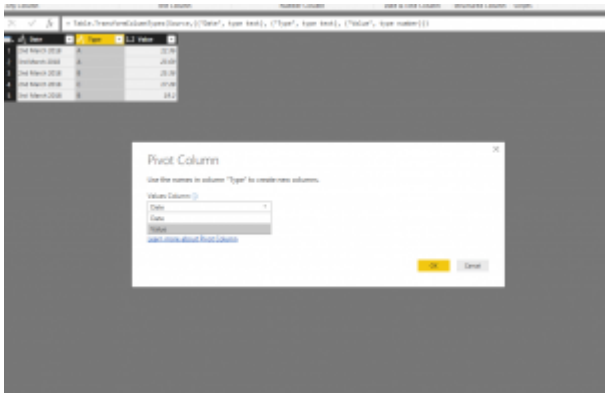


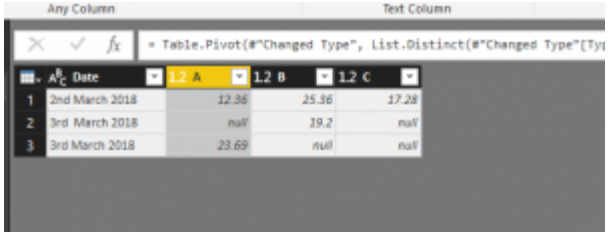
We have the same data as we have in Python. Then we need to just click on the type column, Then click on the “Pivot Colum” in Transformation tab. The result of pivoting is shown in below picture.

Then you need to specify which column you are going to put a value and type.





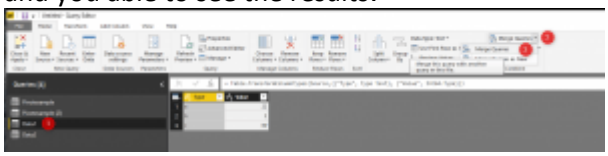


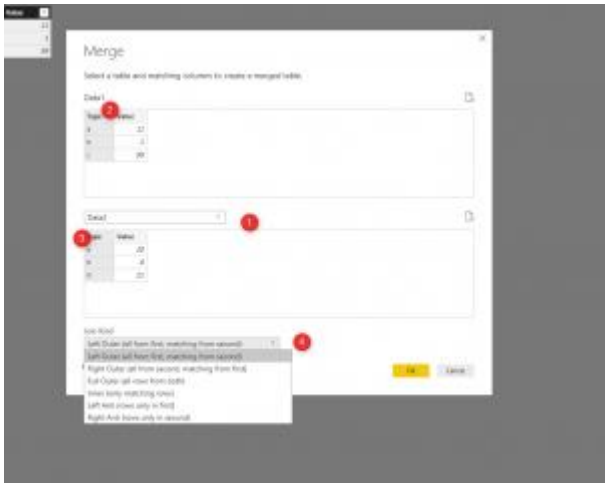


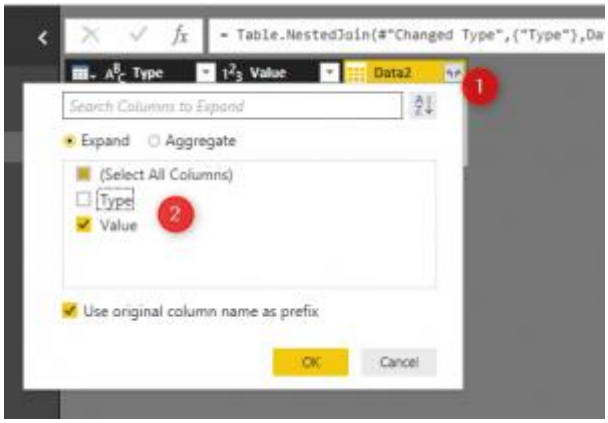
Merging DataSet

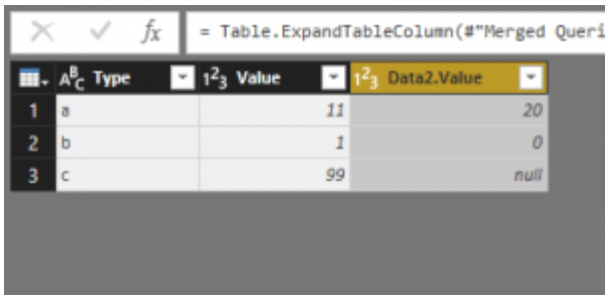
in Power Query. I have created two different datasets “Data1” and “Data2”. After creating a dataset, you need to click on the “Merge” column to merge “Data1” with “Data2” .

In merge window, we able to specify which column we are going to merge based on and what type of merge we are interested. After merging, you need to specify what columns you want to show to users. Please follow steps explained in Figure 6-19. Then the output will be there and you able to see the results.





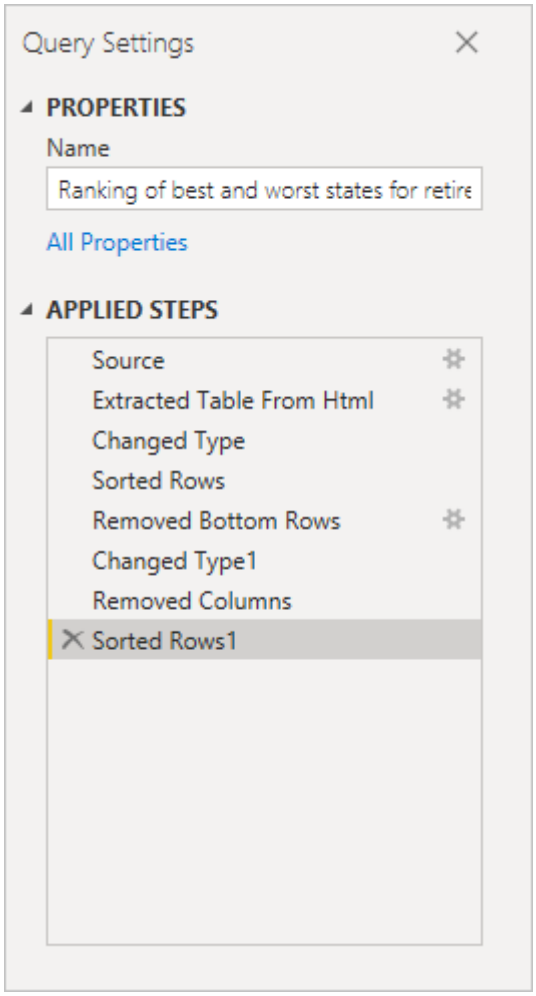




Data Shaping

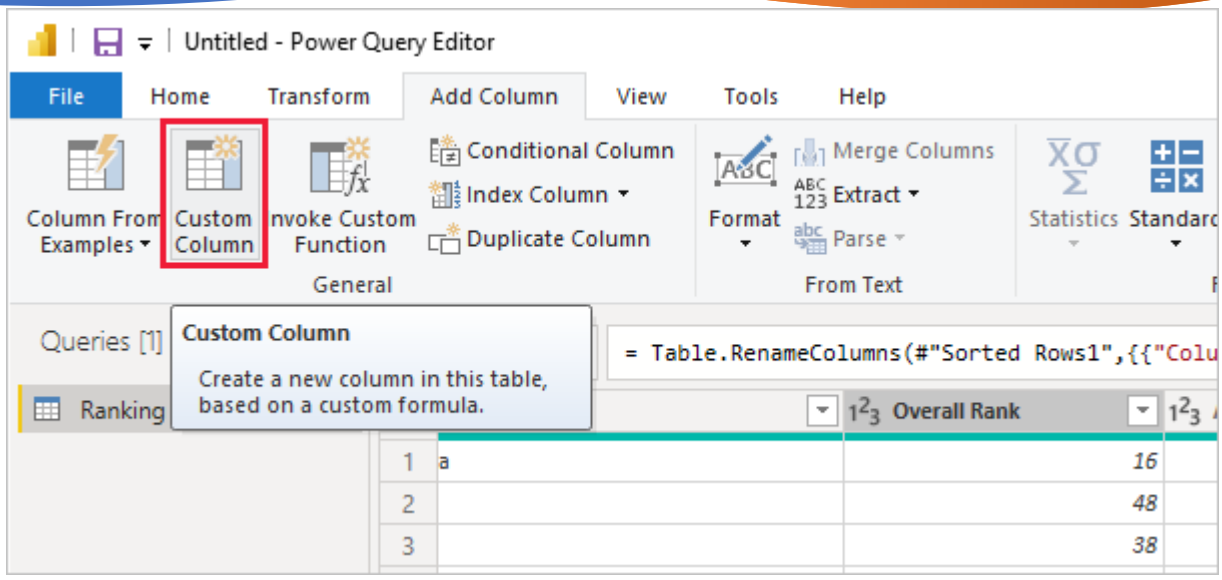
When you shape data in Power Query Editor, you provide step-by-step instructions for Power Query Editor to carry out for you to adjust the data as it loads and presents it. The original data source isn't affected; only this particular view of the data is adjusted, or *shaped*.

The steps you specify (such as rename a table, transform a data type, or delete a column) are recorded by Power Query Editor. Each time this query connects to the data source, Power Query Editor carries out those steps so that the data is always shaped the way you specify. This process occurs whenever you use Power Query Editor, or for anyone who uses your shared query, such as on the Power BI service. Those steps are captured, sequentially, in the **Query Settings** pane, under **Applied Steps**. We'll go through each of those steps in the next few paragraphs.



From Getting Started with Power BI Desktop, let's use the retirement data, which we found by connecting to a web data source, to shape that data to fit our needs. We'll add a custom column to calculate rank based on all data being equal factors, and compare this column to the existing column, **Rank**.

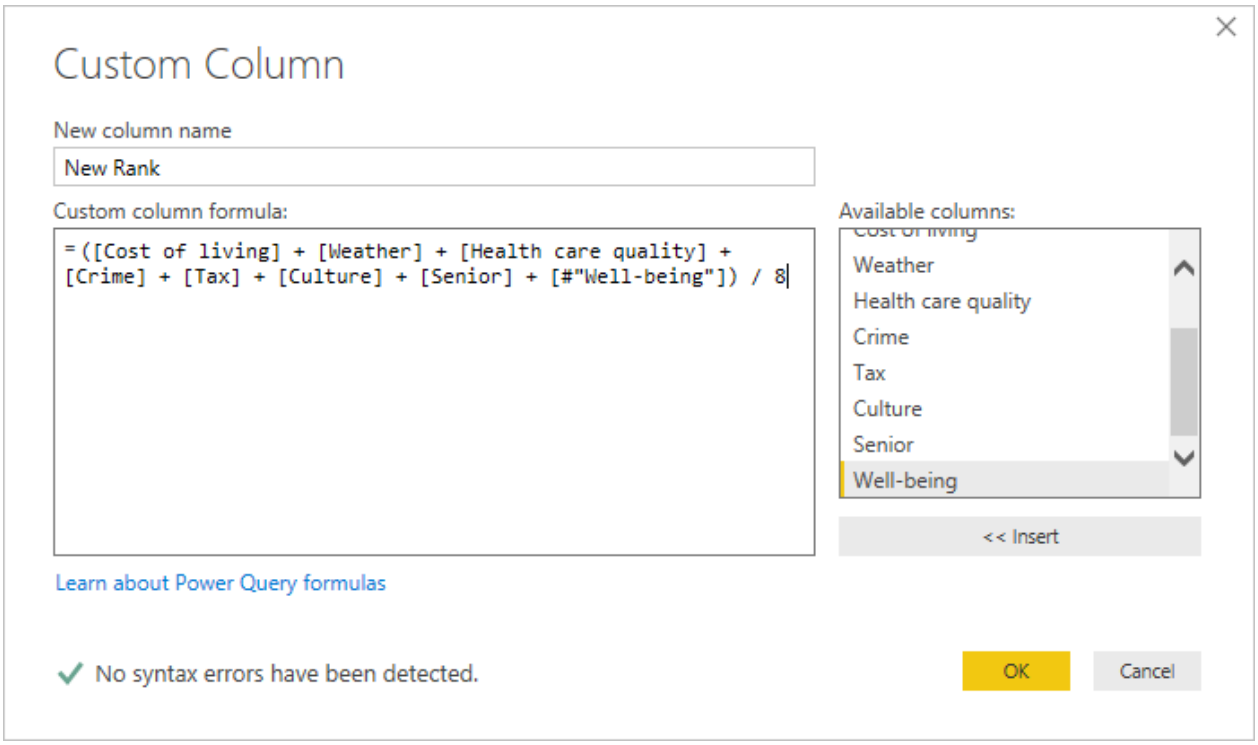
1. From the **Add Column** ribbon, select **Custom Column**, which lets you add a custom column.



2. In the **Custom Column** window, in **New column name**, enter *New Rank*.
   In **Custom column formula**, enter the following data:

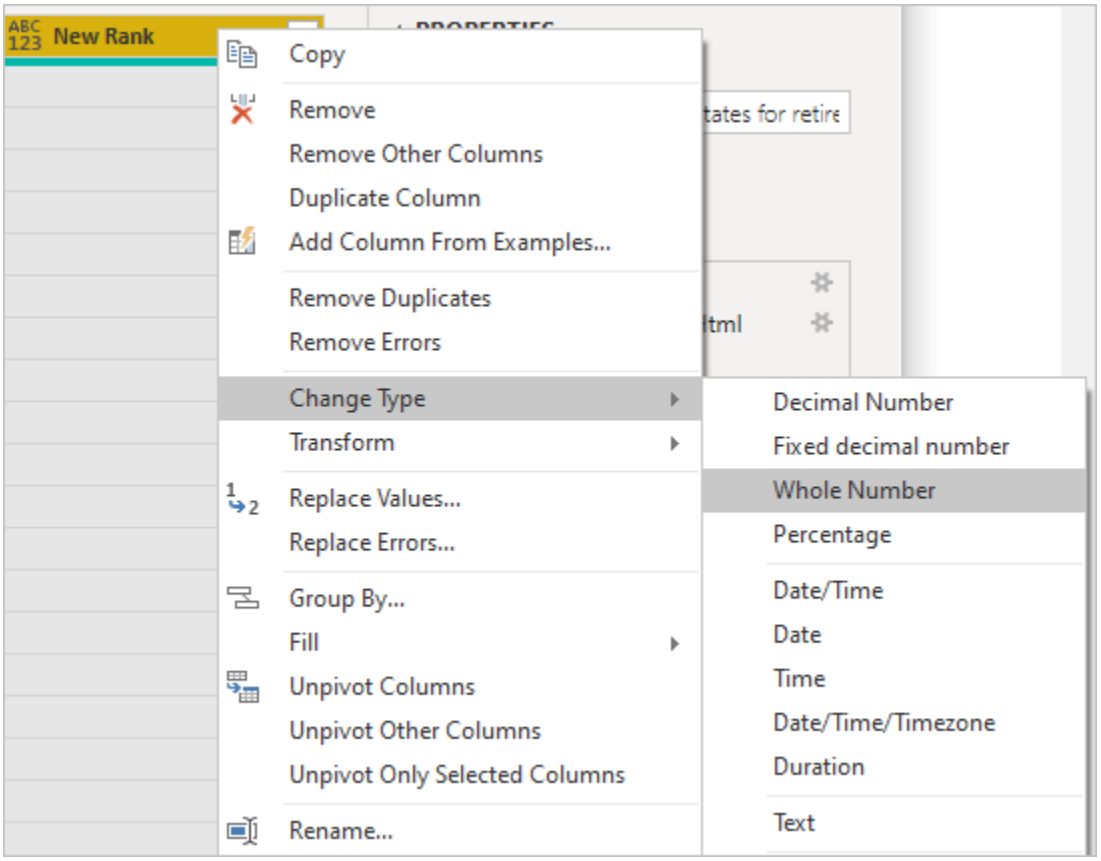
([Cost of living] + [Weather] + [Health care quality] + [Crime] + [Tax] + [Culture] + [Senior] + [#"Well-being"]) / 8

1. Make sure the status message is *No syntax errors have been detected*, and select **OK**.

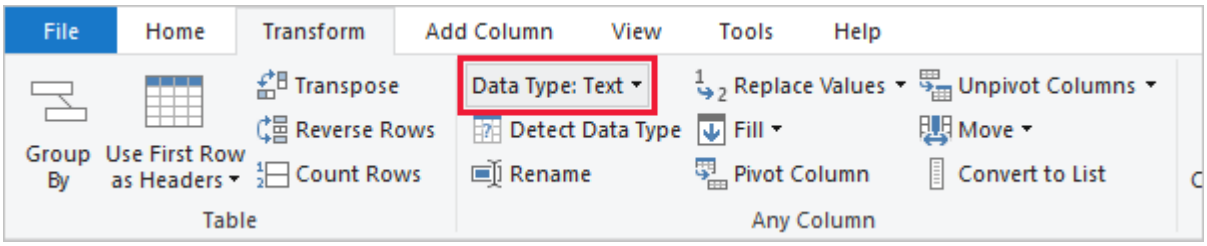


2. To keep column data consistent, transform the new column values to whole numbers. To change them, right-click the column header, and then select **Change Type** > **Whole Number**.

   If you need to choose more than one column, select a column, hold down **SHIFT**, select additional adjacent columns, and then right-click a column header. You can also use the **CTRL** key to choose non-adjacent columns.



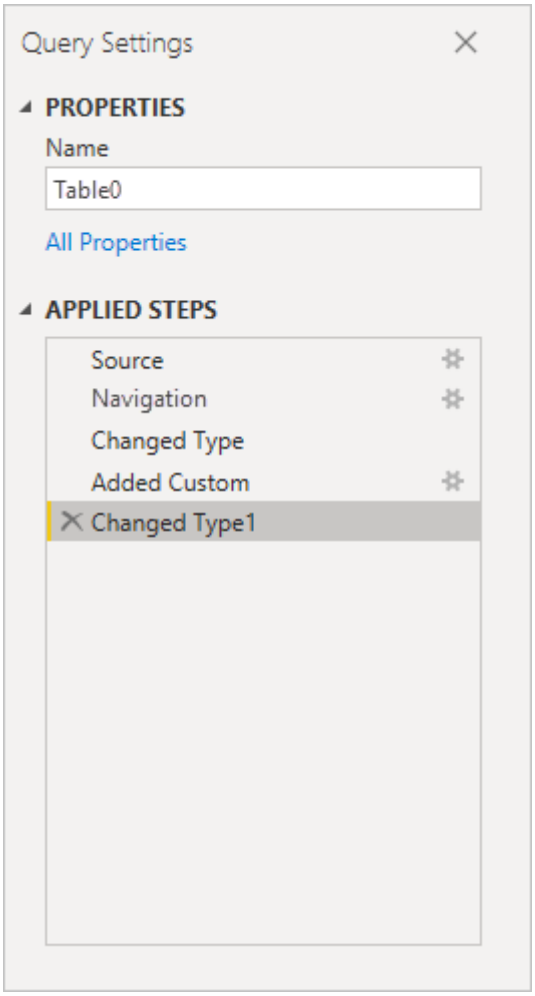
3. To *transform* column data types, in which you transform the current data type to another, select **Data Type Text** from the **Transform** ribbon.



4. In **Query Settings**, the **Applied Steps** list reflects any shaping steps applied to the data. To remove a step from the shaping process, select the **X** to the left of the step.

   In the following image, the **Applied Steps** list reflects the added steps so far:

- **Source**: Connecting to the website.
- **Extracted Table from Html**: Selecting the table.
- **Changed Type**: Changing text-based number columns from *Text* to *Whole Number*.
- **Added Custom**: Adding a custom column.
- **Changed Type1**: The last applied step.



3.4 Advance shaping

Adjust data

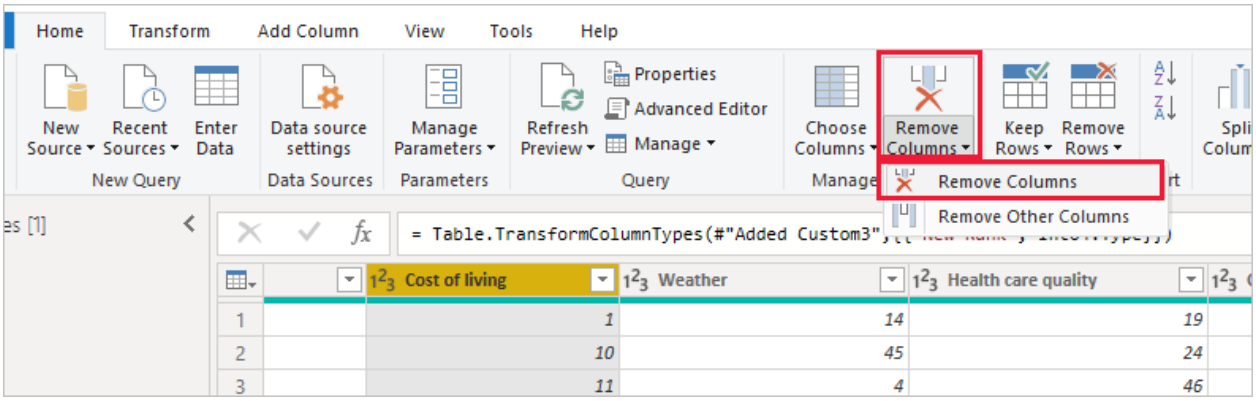
Before we can work with this query, we need to make a few changes to adjust its data:

- Adjust the rankings by removing a column.
  We've decided **Cost of living** is a non-factor in our results. After removing this column, we find that the data remains unchanged.
- Fix a few errors.

Because we removed a column, we need to readjust our calculations in the **New Rank** column, which involves changing a formula.

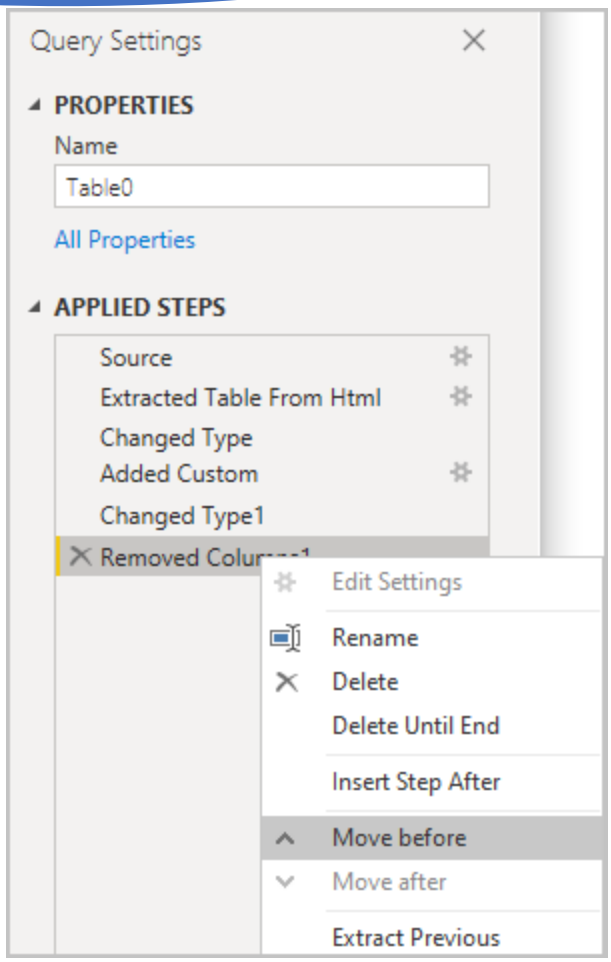
- Sort the data.
  Sort the data based on the **New Rank** and **Rank** columns.
- Replace the data.
  We'll highlight how to replace a specific value and the need of inserting an **Applied Step**.
- Change the table name.
  Because **Table 0** isn't a useful descriptor for the table, we'll change its name.

1. To remove the **Cost of living** column, select the column, choose the **Home** tab from the ribbon, and then select **Remove Columns**.



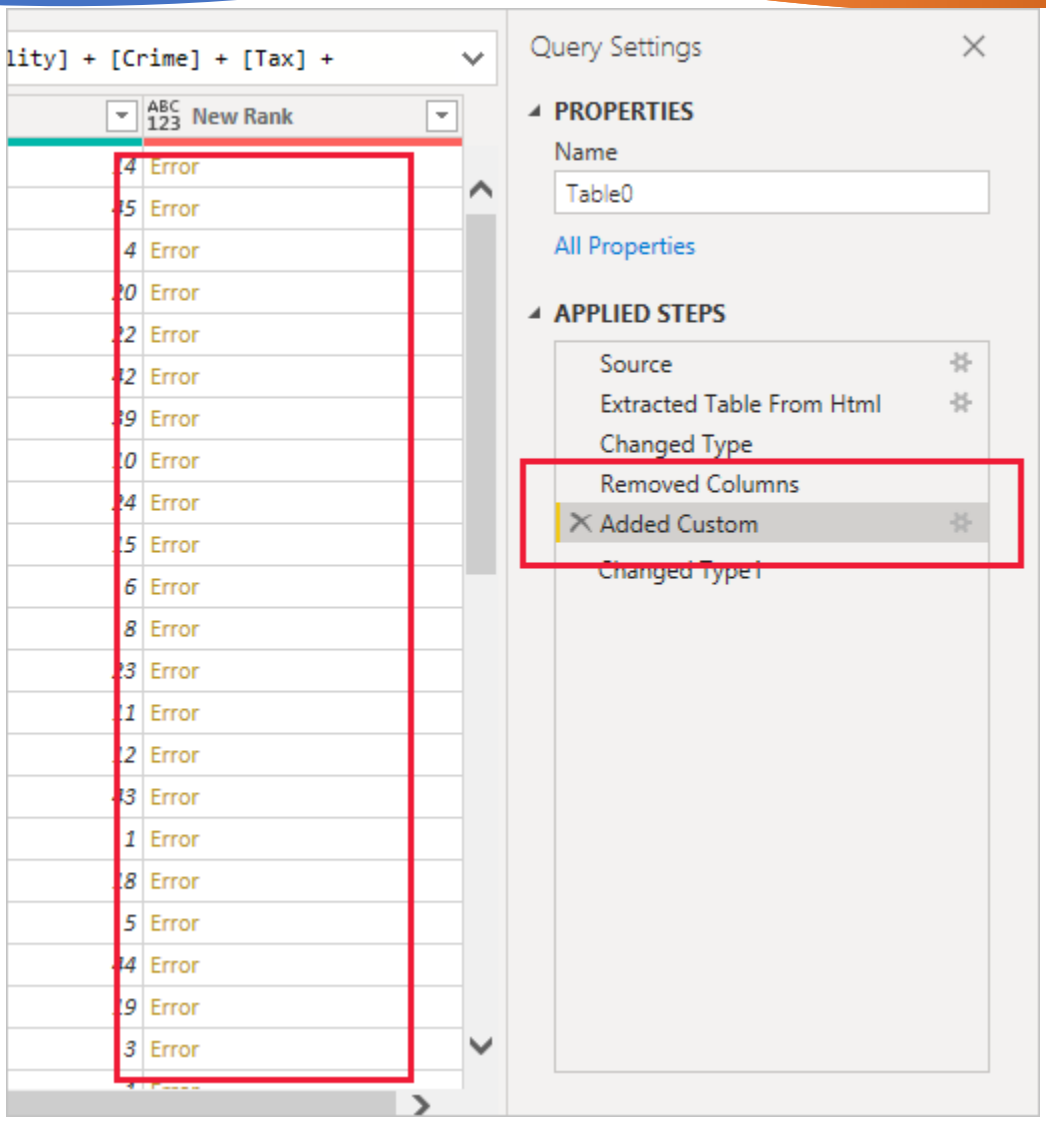
Notice the **New Rank** values haven't changed, due to the ordering of the steps. Because Power Query Editor records the steps sequentially, yet independently, of each other, you can move each **Applied Step** up or down in the sequence.

2. Right-click a step. Power Query Editor provides a menu that lets you do the following tasks:
   - **Rename**; Rename the step.
   - **Delete**: Delete the step.
   - **Delete Until End**: Remove the current step, and all subsequent steps.
   - **Move before**: Move the step up in the list.
   - **Move after**: Move the step down in the list.
3. Move up the last step, **Removed Columns**, to just above the **Added Custom** step.



4. Select the **Added Custom** step.

   Notice the data now shows *Error*, which we'll need to address.



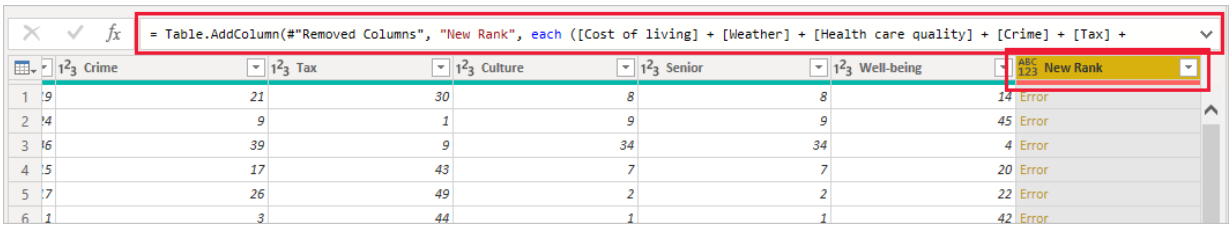
There are a few ways to get more information about each error. If you select the cell without clicking on the word *Error*, Power Query Editor displays the error information.



If you select the word *Error* directly, Power Query Editor creates an **Applied Step** in the **Query Settings** pane and displays information about the error.

5. Because we don't need to display information about the errors, select **Cancel**.

6. To fix the errors, select the **New Rank** column, then display the column's data formula by selecting the **Formula Bar** checkbox from the **View** tab.



7. Remove the *Cost of living* parameter and decrement the divisor, by changing the formula as follows:

```
Table.AddColumn(#"Removed Columns", "New Rank", each ([Weather] + [Health care quality] + [Crime] + [Tax] + [Culture] + [Senior] + [#"Well-being"]) / 7)
```

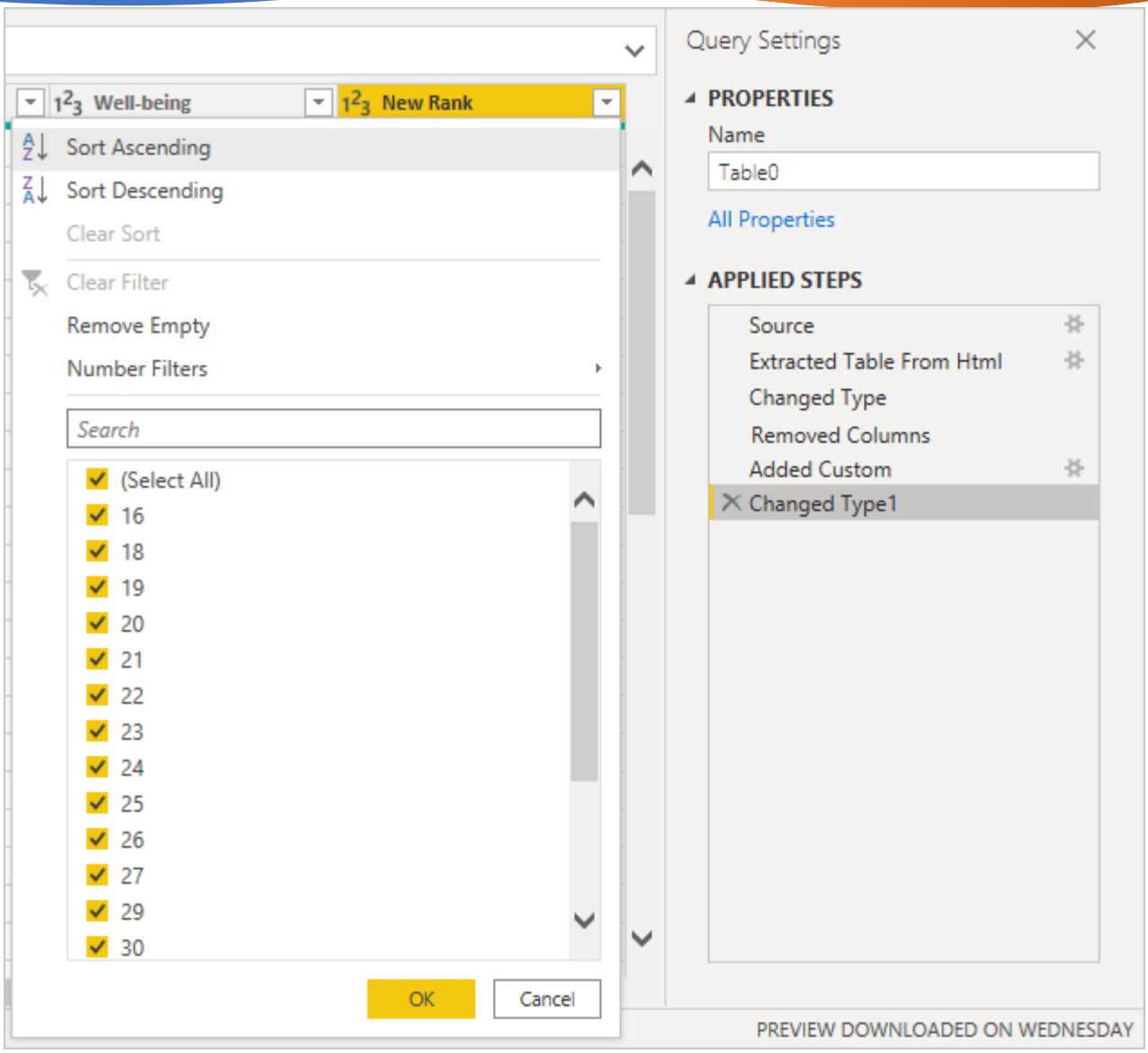
8. Select the green checkmark to the left of the formula box or press **Enter**.

Power Query Editor replaces the data with the revised values and the **Added Custom** step completes with no errors.

Note

You can also select **Remove Errors**, by using the ribbon or the right-click menu, which removes any rows that have errors. However, we didn't want to do so in this tutorial because we wanted to preserve the data in the table.

9. Sort the data based on the **New Rank** column. First, select the last applied step, **Changed Type1** to display the most recent data. Then, select the drop-down located next to the **New Rank** column header and select **Sort Ascending**.



The data is now sorted according to **New Rank**. However, if you look at the **Rank** column, you'll notice the data isn't sorted properly in cases where the **New Rank** value is a tie. We'll fix it in the next step.

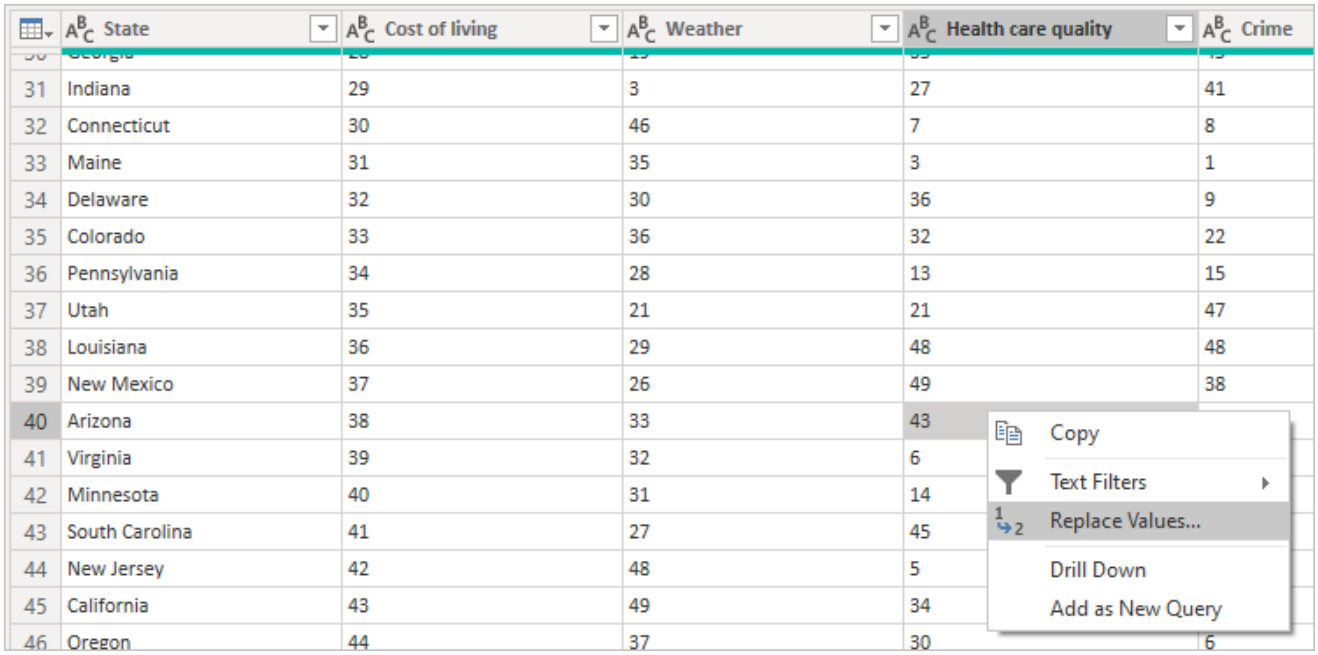
10. To fix the data sorting issue, select the **New Rank** column and change the formula in the **Formula Bar** to the following formula:

```
= Table.Sort(#"Changed Type1",{{"New Rank", Order.Ascending},{"Rank", Order.Ascending}})
```

11. Select the green checkmark to the left of the formula box or press **Enter**.

The rows are now ordered in accordance with both **New Rank** and **Rank**. In addition, you can select an **Applied Step** anywhere in the list, and continue shaping the data at that point in the sequence. Power Query Editor automatically inserts a new step directly after the currently selected **Applied Step**.

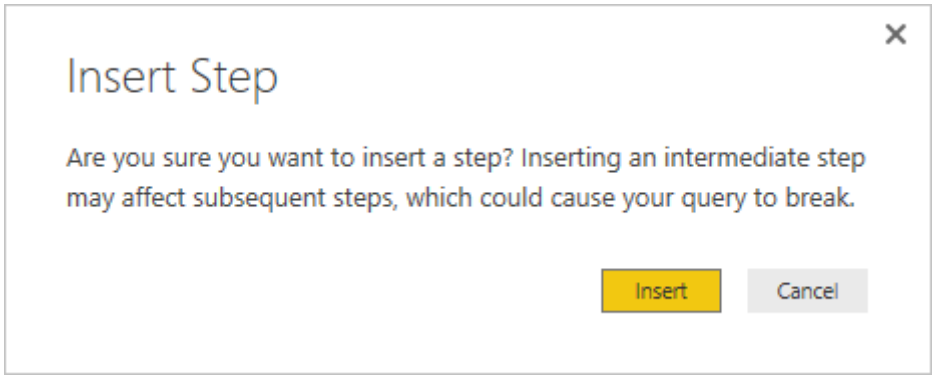
12. In **Applied Step**, select the step preceding the custom column, which is the **Removed Columns** step. Here we'll replace the value of the **Weather** ranking in Arizona. Right-click the appropriate cell that contains Arizona's **Weather** ranking, and then select **Replace Values**. Note which **Applied Step** is currently selected.



	State	Cost of living	Weather	Health care quality	Crime
31	Indiana	29	3	27	41
32	Connecticut	30	46	7	8
33	Maine	31	35	3	1
34	Delaware	32	30	36	9
35	Colorado	33	36	32	22
36	Pennsylvania	34	28	13	15
37	Utah	35	21	21	47
38	Louisiana	36	29	48	48
39	New Mexico	37	26	49	38
40	Arizona	38	33	43	
41	Virginia	39	32	6	
42	Minnesota	40	31	14	
43	South Carolina	41	27	45	
44	New Jersey	42	48	5	
45	California	43	49	34	
46	Oregon	44	37	30	6

13. Select **Insert**.

Because we're inserting a step, Power Query Editor warns us about the danger of doing so; subsequent steps could cause the query to break.



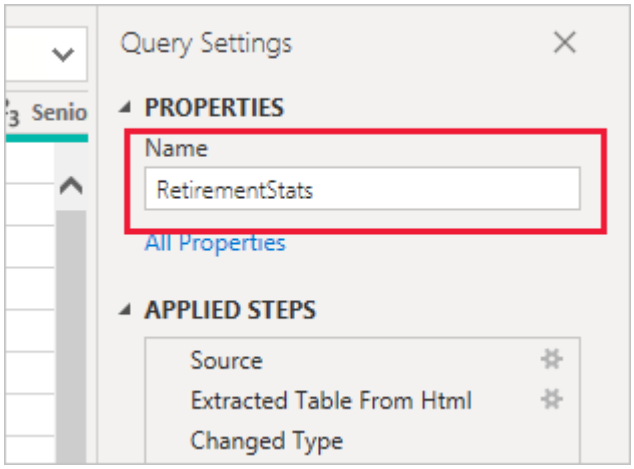
14. Change the data value to *51*.

Power Query Editor replaces the data for Arizona. When you create a new **Applied Step**, Power Query Editor names it based on the action; in this case, **Replaced Value**. If you have more than one step with the same name in your query, Power Query Editor adds a number (in sequence) to each subsequent **Applied Step** to differentiate between them.

15. Select the last **Applied Step**, **Sorted Rows**.

Notice the data has changed regarding Arizona's new ranking. This change occurs because we inserted the **Replaced Value** step in the correct location, before the **Added Custom** step.

16. Lastly, we want to change the name of that table to something descriptive. In the **Query Settings** pane, under **Properties**, enter the new name of the table, and then select **Enter**. Name this table *RetirementStats*.



When we start creating reports, it's useful to have descriptive table names, especially when we connect to multiple data sources, which are listed in the **Fields** pane of the **Report** view.

We've now shaped our data to the extent we need to. Next let's connect to another data source, and combine data.

3.5 Beyond Initial Shaping

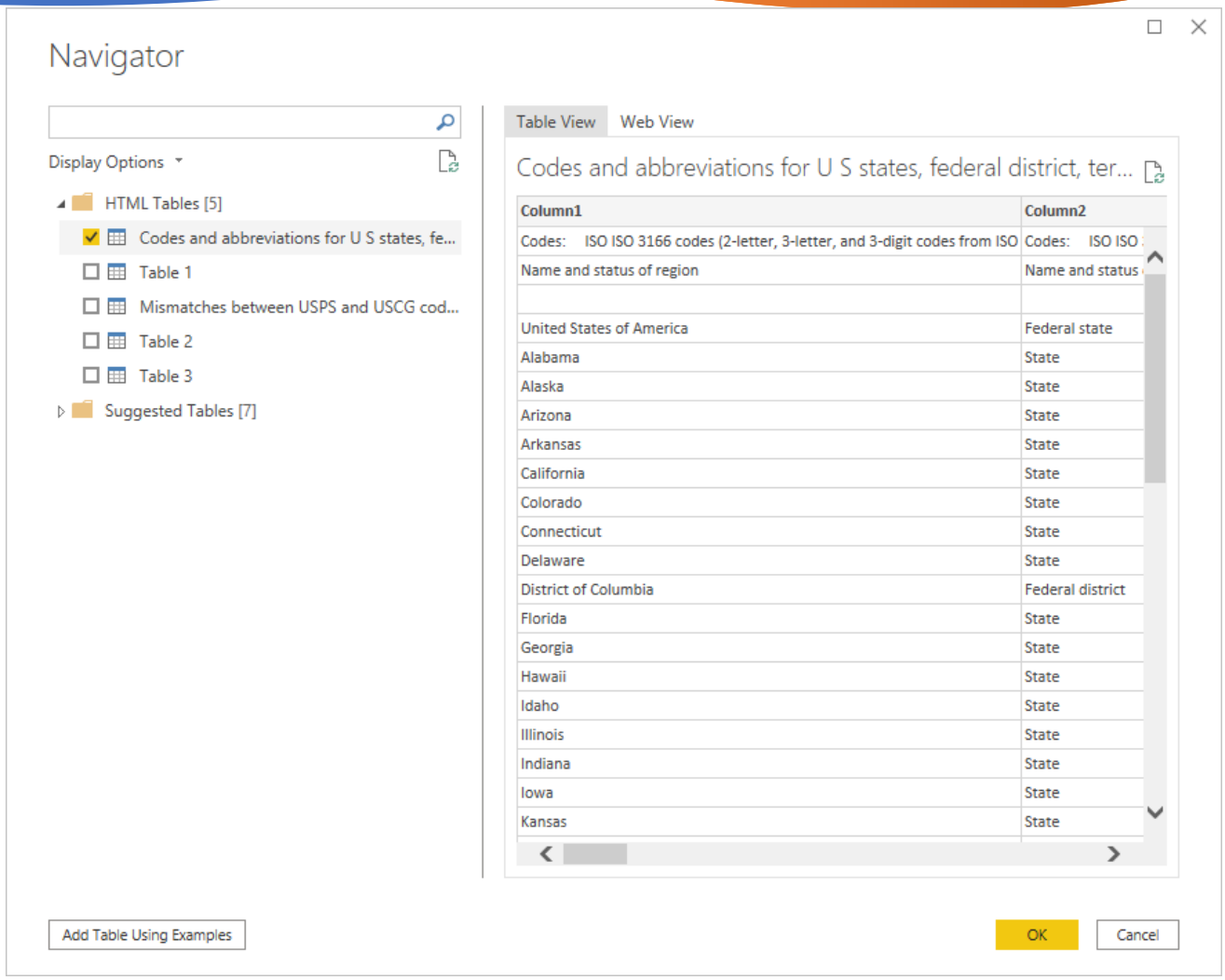
Combine data

The data about various states is interesting, and will be useful for building additional analysis efforts and queries. But there's one problem: most data out there uses a two-letter abbreviation for state codes, not the full name of the state. We need a way to associate state names with their abbreviations.

We're in luck; there's another public data source that does just that, but it needs a fair amount of shaping before we can connect it to our retirement table. TO shape the data, follow these steps:

1. From the **Home** ribbon in Power Query Editor, select **New Source > Web**.
2. Enter the address of the website for state abbreviations, *https://en.wikipedia.org/wiki/List_of_U.S._state_abbreviations*, and then select **Connect**.

   The Navigator displays the content of the website.



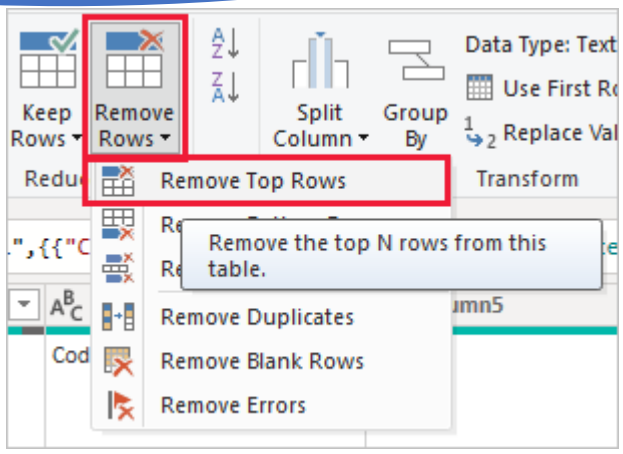
3. Select **Codes and abbreviations**.

Tip

It will take quite a bit of shaping to pare this table's data down to what we want. Is there a faster or easier way to accomplish the steps below? Yes, we could create a *relationship* between the two tables, and shape the data based on that relationship. The following steps are still good to learn for working with tables; however, relationships can help you quickly use data from multiple tables.

To get the data into shape, follow these steps:

1. Remove the top row. Because it's a result of the way that the web page's table was created, we don't need it. From the **Home** ribbon, select **Remove Rows > Remove Top Rows**.

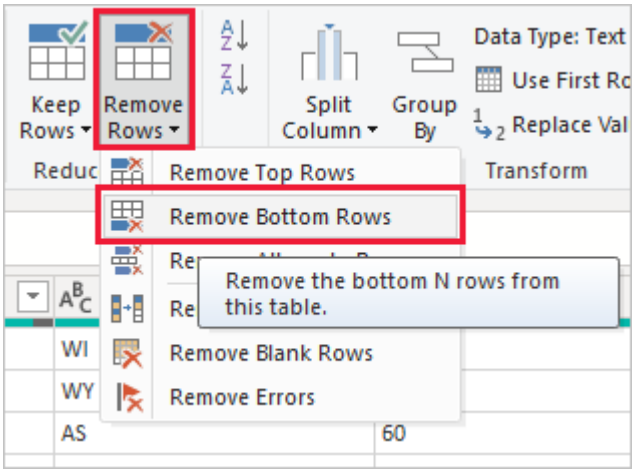


The **Remove Top Rows** window appears, letting you specify how many rows you want to remove.

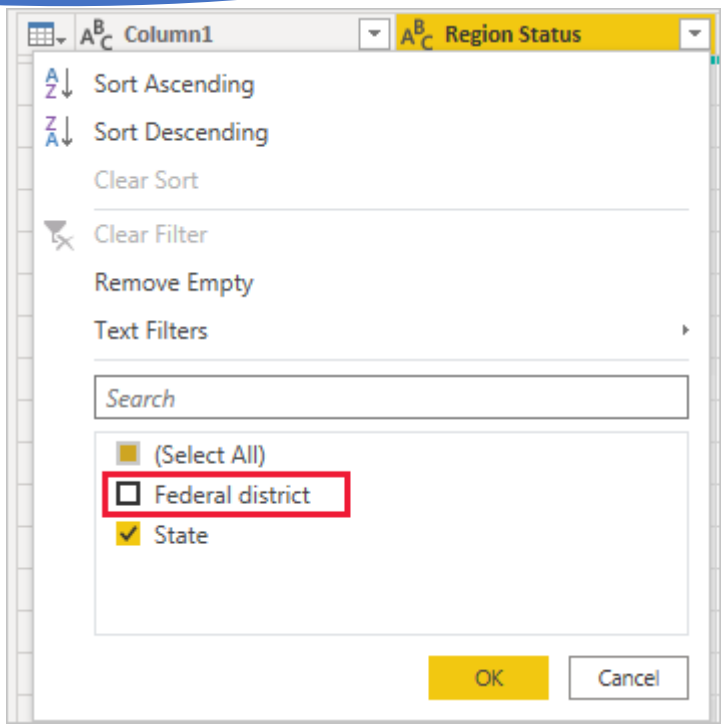
Note

If Power BI accidentally imports the table headers as a row in your data table, you can select **Use First Row As Headers** from the **Home** tab, or from the **Transform** tab in the ribbon, to fix your table.

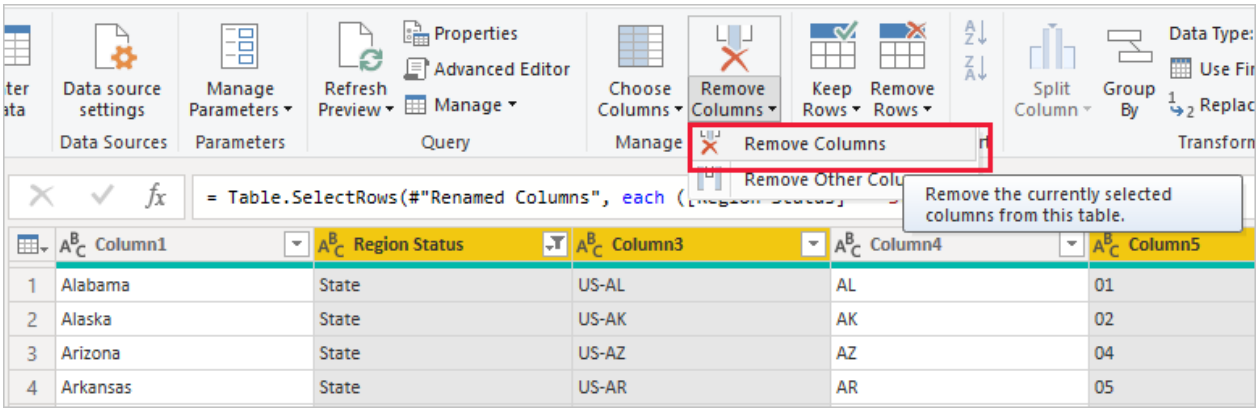
2. Remove the bottom 26 rows. These rows are U.S. territories, which we don't need to include. From the **Home** ribbon, select **Remove Rows > Remove Bottom Rows**.



3. Because the RetirementStats table doesn't have information for Washington DC, we need to filter it from our list. Select the **Region Status** drop-down, then clear the checkbox beside **Federal district**.



4. Remove a few unneeded columns. Because we need only the mapping of each state to its official two-letter abbreviation, we can remove several columns. First select a column, then hold down the **CTRL** key and select each of the other columns to be removed. From the **Home** tab on the ribbon, select **Remove Columns > Remove Columns**.



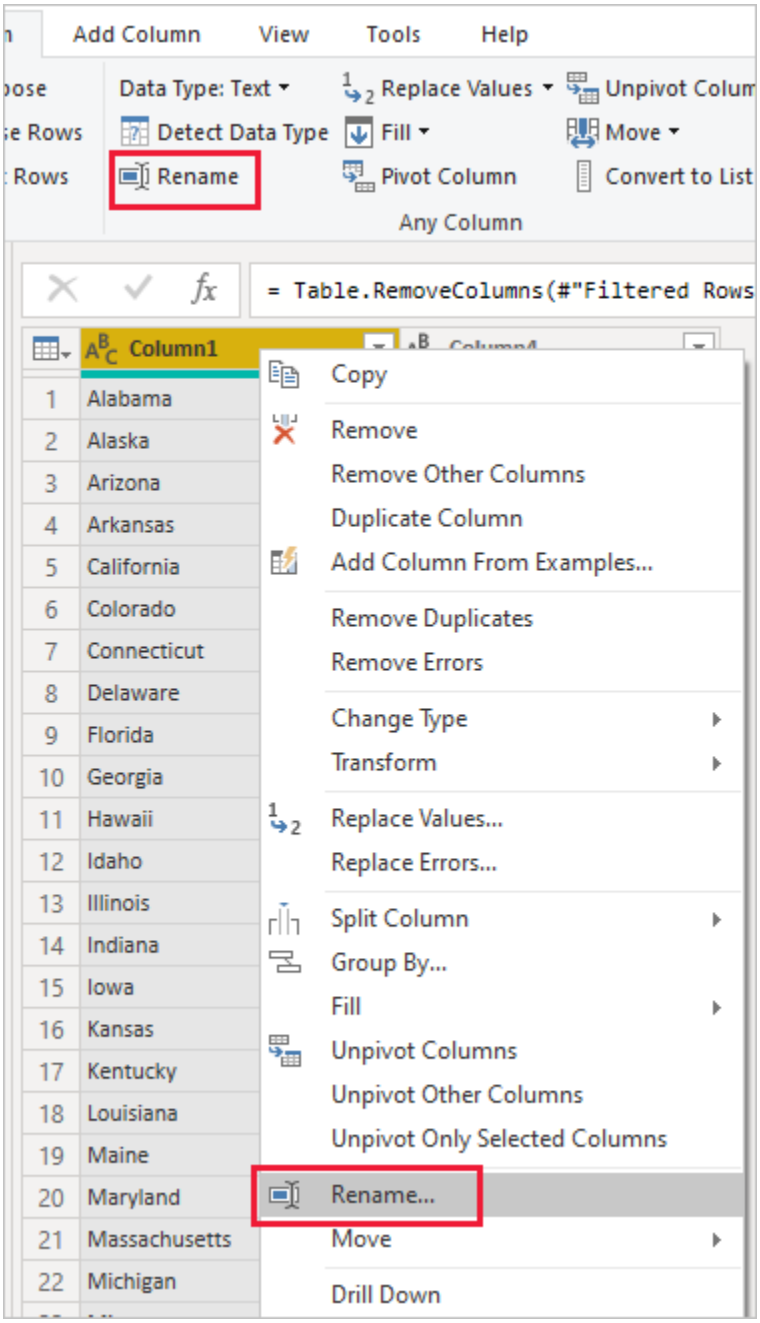
Note

This is a good time to point out that the *sequence* of applied steps in Power Query Editor is important, and can affect how the data is shaped. It's also important to consider how one step may impact another subsequent step; if you remove a step from the Applied Steps, subsequent steps may not behave as originally intended, because of the impact of the query's sequence of steps.

Note

When you resize the Power Query Editor window to make the width smaller, some ribbon items are condensed to make the best use of visible space. When you increase the width of the Power Query Editor window, the ribbon items expand to make the most use of the increased ribbon area.

5. Rename the columns and the table. There are a few ways to rename a column: First, select the column, then either select **Rename** from the **Transform** tab on the ribbon, or right-click and select **Rename**. The following image has arrows pointing to both options; you only need to choose one.



6. Rename the columns to *State Name* and *State Code*. To rename the table, enter the **Name** in the **Query Settings** pane. Name this table *StateCodes*.

3.6 DAX overview

Data Analysis Expressions (DAX) is a formula expression language used in Analysis Services, Power BI, and Power Pivot in Excel. DAX formulas include functions, operators, and values to perform advanced calculations and queries on data in related tables and columns in tabular data models.

DAX function

Aggregation functions - These functions calculate a (scalar) value such as count, sum, average, minimum, or maximum for all rows in a column or table as defined by the expression.

Date and time functions - These functions in DAX are similar to date and time functions in Microsoft Excel. However, DAX functions are based on the datetime data types used by Microsoft SQL Server.

Filter functions - These functions help you return specific data types, look up values in related tables, and filter by related values. Lookup functions work by using tables and relationships between them. Filtering functions let you manipulate data context to create dynamic calculations.

Financial functions - These functions are used in formulas that perform financial calculations, such as net present value and rate of return.

Information functions - These functions look at a table or column provided as an argument to another function and returns whether the value matches the expected type. For example, the ISERROR function returns TRUE if the value you reference contains an error.

Logical functions - These functions return information about values in an expression. For example, the TRUE function lets you know whether an expression that you are evaluating returns a TRUE value.

Math and Trig functions - Mathematical functions in DAX are similar to Excel's mathematical and trigonometric functions. However, there are some differences in the numeric data types used by DAX functions.

Other functions - These functions perform unique actions that cannot be defined by any of the categories most other functions belong to.

Parent and Child functions - These functions help users manage data that is presented as a parent/child hierarchy in their data models.

Relationship functions - These functions are for managing and utilizing relationships between tables. For example, you can specify a particular relationship to be used in a calculation.

Statistical functions - These functions calculate values related to statistical distributions and probability, such as standard deviation and number of permutations.

Table manipulation functions - These functions return a table or manipulate existing tables.

Text functions - With these functions, you can return part of a string, search for text within a string, or concatenate string values. Additional functions are for controlling the formats for dates, times, and numbers.

Time intelligence functions - These functions help you create calculations that use built-in knowledge about calendars and dates. By using time and date ranges in combination with aggregations or calculations, you can build meaningful comparisons across comparable time periods for sales, inventory, and so on.

DAX Statements

DEFINE	(Keyword) Defines entities that exist only for the duration of a DAX query.
EVALUATE	(Keyword) A statement required to execute a DAX query.
ORDER BY	(Keyword) Defines one or more expressions used to sort results of a DAX query.
VAR	(Keyword) Stores the result of an expression as a named variable, which can then be passed as an argument to other measure expressions.

DAX Calculations

DAX formulas are used in measures, calculated columns, calculated tables, and row-level security.

Measures

Measures are dynamic calculation formulas where the results change depending on context. Measures are used in reporting that support combining and filtering model data by using multiple attributes such as a Power BI report or Excel PivotTable or PivotChart. Measures are created by using the DAX formula bar in the model designer.

A formula in a measure can use standard aggregation functions automatically created by using the Autosome feature, such as COUNT or SUM, or you can define your own formula by using the DAX formula bar. Named measures can be passed as an argument to other measures.

When you define a formula for a measure in the formula bar, a Tooltip feature shows a preview of what the results would be for the total in the current context, but otherwise the results are not immediately output anywhere. The reason you cannot see the (filtered) results of the calculation immediately is because the result of a measure cannot be determined without context. To evaluate a measure requires a reporting client application that can provide the context needed to retrieve the data relevant to each cell and then evaluate the

expression for each cell. That client might be an Excel PivotTable or PivotChart, a Power BI report, or a table expression in a DAX query in SQL Server Management Studio (SSMS).

Regardless of the client, a separate query is run for each cell in the results. That is to say, each combination of row and column headers in a PivotTable, or each selection of slicers and filters in a Power BI report, generates a different subset of data over which the measure is calculated.

For example, using this very simple measure formula:

DAXCopy

Total Sales = SUM ([Sales Amount])

When a user places the Total Sales measure in a report, and then places the Product Category column from a Product table into Filters, the sum of Sales Amount is calculated and displayed for each product category.

Unlike calculated columns, the syntax for a measure includes the measure's name preceding the formula. In the example just provided, the name Total Sales appears preceding the formula. After you've created a measure, the name and its definition appear in the reporting client application Fields list and depending on perspectives and roles is available to all users of the model.

Calculated columns

A calculated column is a column that you add to an existing table (in the model designer) and then create a DAX formula that defines the column's values. When a calculated column contains a valid DAX formula, values are calculated for each row as soon as the formula is entered. Values are then stored in the in-memory data model.

For example, in a Date table, when the formula is entered into the formula bar:

DAXCopy

= [Calendar Year] & " Q" & [Calendar Quarter]

A value for each row in the table is calculated by taking values from the Calendar Year column (in the same Date table), adding a space and the capital letter Q, and then adding the values from the Calendar Quarter column (in the same Date table). The result for each row in the calculated column is calculated immediately and appears, for example, as 2017 Q1. Column values are only recalculated if the table or any related table is processed (refresh) or the model is unloaded from memory and then reloaded, like when closing and reopening a Power BI Desktop file.

Calculated tables

A calculated table is a computed object, based on a formula expression, derived from all or part of other tables in the same model. Instead of querying and loading values into your new table's columns from a data source, a DAX formula defines the table's values.

Calculated tables can be helpful in a role-playing dimension. An example is the Date table, as OrderDate, ShipDate, or DueDate, depending on the foreign key relationship. By creating a calculated table for ShipDate explicitly, you get a standalone table that is available for queries, as fully operable as any other table. Calculated tables are also useful when configuring a filtered rowset, or a subset or superset of columns from other existing tables. This allows you to keep the original table intact while creating variations of that table to support specific scenarios.

Calculated tables support relationships with other tables. The columns in your calculated table have data types, formatting, and can belong to a data category. Calculated tables can be named and surfaced or hidden just like any other table. Calculated tables are re-calculated if any of the tables it pulls data from are refreshed or updated.

Row-level security

With row-level security, a DAX formula must evaluate to a Boolean TRUE/FALSE condition, defining which rows can be returned by the results of a query by members of a particular role.

For example, for members of the Sales role, the Customers table with the following DAX formula:

DAXCo= Customers[Country] = "USA"

Members of the Sales role will only be able to view data for customers in the USA, and aggregates, such as SUM are returned only for customers in the USA. Row-level security is not available in Power Pivot in Excel.

When defining row-level secuirty by using DAX formula, you are creating an allowed row set. This does not deny access to other rows; rather, they are simply not returned as part of the allowed row set. Other roles can allow access to the rows excluded by the DAX formula. If a user is a member of another role, and that role's row-level security allows access to that particular row set, the user can view data for that row.

Row-level security formulas apply to the specified rows as well as related rows. When a table has multiple relationships, filters apply security for the relationship that is active. Row-level security formulas will be intersected with other formulas defined for related tables.

Advanced DAX

The Advanced DAX course is designed to build on the fundamentals taught in the Introduction to DAX course. This course focuses on the advanced and difficult to comprehend DAX concepts like evaluation context, security, iterator functions and advanced DAX functions.

3.7 Publishing Data Set:

A training data set is a data set of examples used during the learning process and is used to fit the parameters (e.g., weights) of, for example, a classifier. For classification tasks, a supervised learning algorithm looks at the training data set to determine, or learn, the optimal combinations of variables that will generate a good predictive model. The goal is to produce a trained (fitted) model that generalizes well to new, unknown data. The fitted model is evaluated using "new" examples from the held-out datasets (validation and test datasets) to estimate the model's accuracy in classifying new data. To reduce the risk of issues such as over-fitting, the examples in the validation and test datasets should not be used to train the model. Most approaches that search through training data for empirical relationships tend to over fit the data, meaning that they can identify and exploit apparent relationships in the training data that do not hold in general.

3.8 SUMMARIZING DATA

Why do we summarize? We summarize data to "simplify" the data and quickly identify what looks "normal" and what looks odd. The distribution of a variable shows what values the variable takes and how often the variable takes these values.

The two most useful ways of describing the distribution of data are:

- The typical: This describes the center–or middle–of the data. This way of describing the center is also called a "measure of central tendency".
- The spread of the values around the center: This describes how densely the data is distributed around the center. This is also called a "measure of dispersion".

These two ways of describing the data are also referred to as descriptive statistics.

1. The Center: What is Typical? (Central Tendencies)

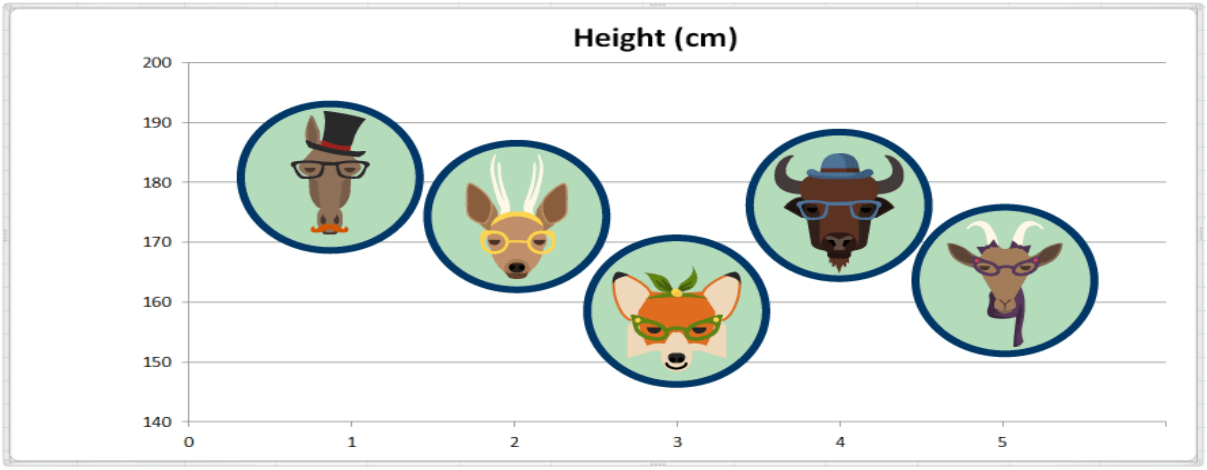
The three common ways of looking at the center are average (also called mean), mode and median. All three summarize a distribution of the data by describing the typical value of a variable (average), the most frequently repeated number (mode), or the number in the middle of all the other numbers in a data set (median). In this module, we are going to focus on the average. The average is the most appropriate way to measure the center for interval/continuous data (e.g, numbers of registered voters). To calculate the average, we add up all the numbers for a variable and then divide by how many numbers there are. Put another way the average (mean) is the sum divided by the count.

Simple Example

In the example dataset below, we have information about the names of some animals. We also have measurements of the height of each animal. The dataset has two variables – name and height – and five observations. Here is the dataset:

	A	B
1	**Name**	**Height (cm)**
2	Harry the Horse	181
3	Dana the Deer	175
4	Fran the Fox	159
5	Bob the Buffalo	177
6	Gracie the Goat	165

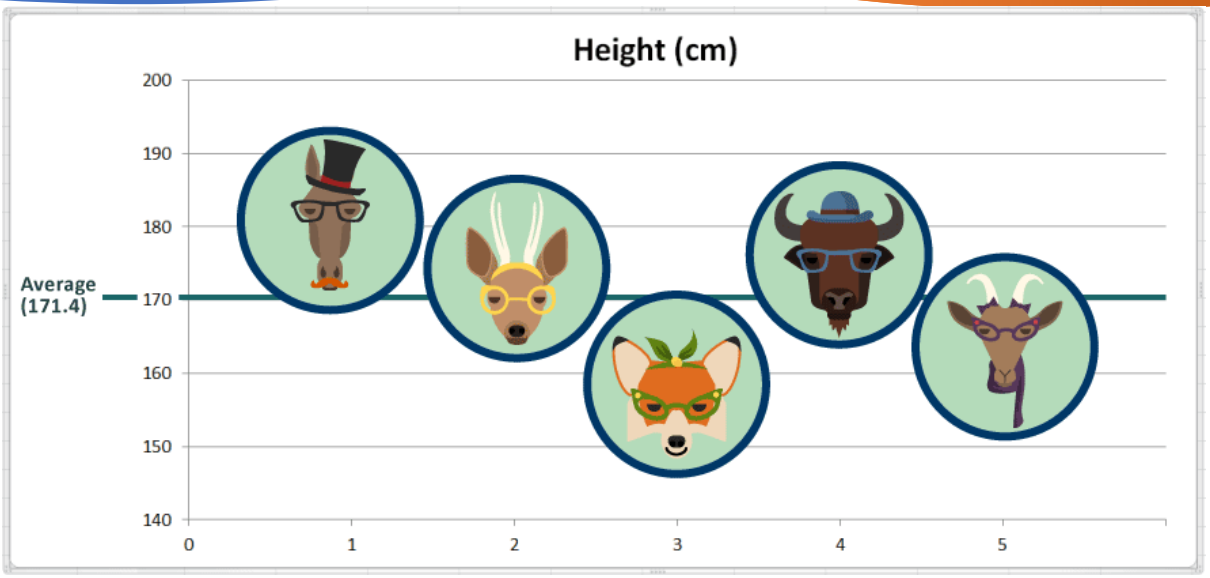
Here we've made a quick chart that plots the height of each animal:



To calculate the Average height (in cm) we sum up all the values and divide by the total count of observations:

Average height = (181 + 175 + 159 + 177 + 165) ÷ 5 = 857 ÷ 5 = 171.4

The average value for height is 171.4 centimeters. Here we have added a reference line marking the average on our chart so we can see how that looks:



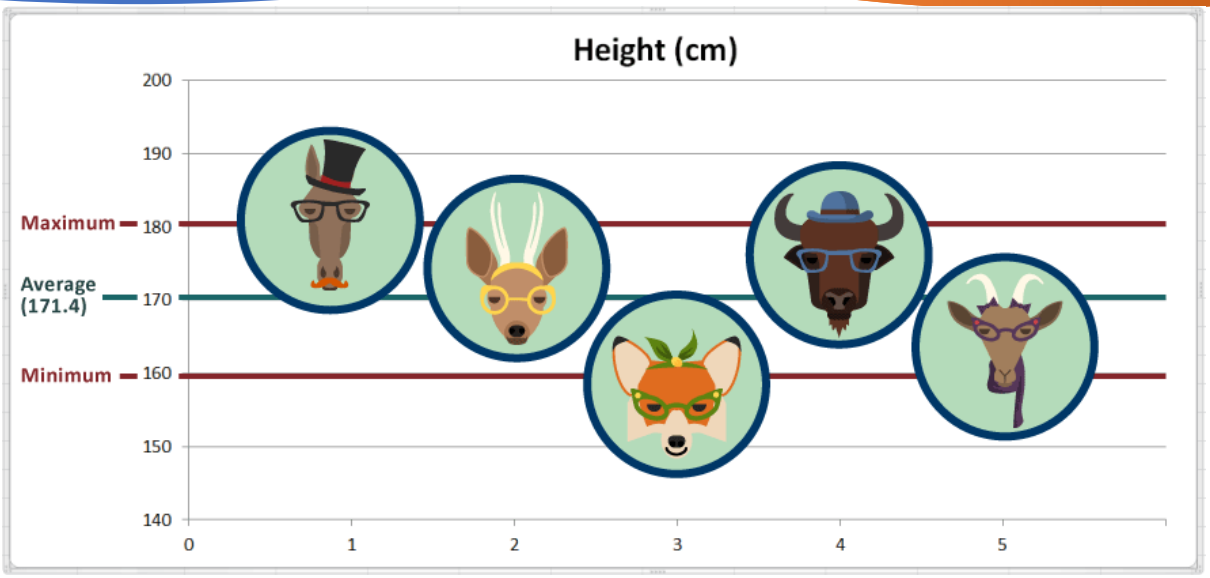
2. The Spread: How is the data distributed around the center? (Measures of Dispersion)

Looking at the spread of the distribution of data tells us about the amount of variation, or diversity, within the data. The three measures of the spread of the data are the range, the standard deviation, and the variance.

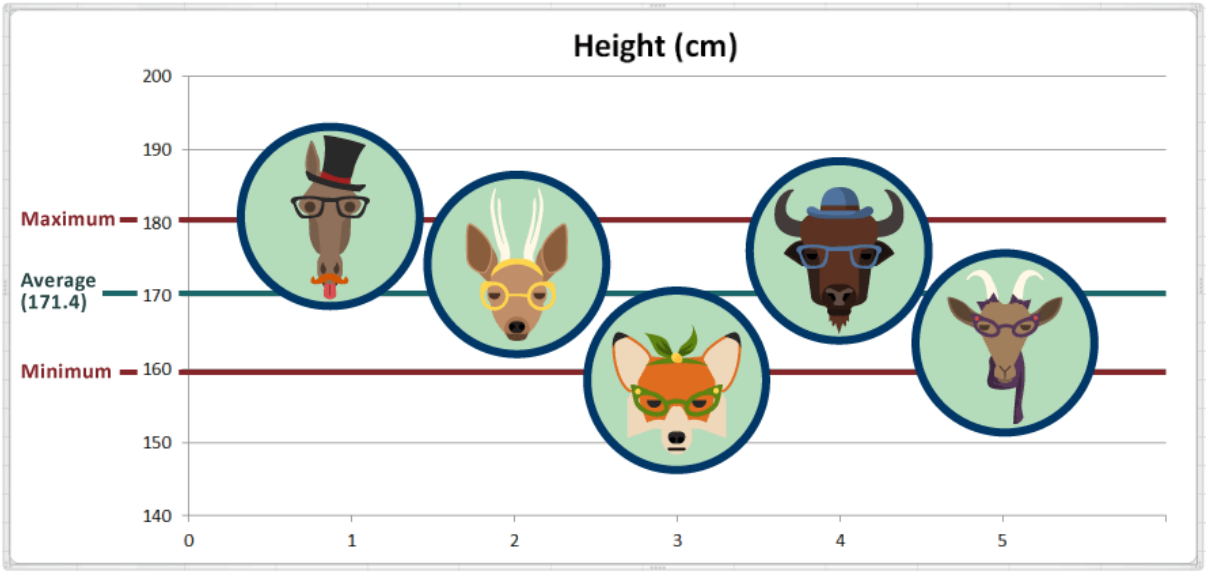
Range

This is the difference between the largest and the smallest values. It is the distance between the extremes. To calculate the range, we take the maximum value and subtract the minimum value.

In our height dataset, what is the largest value (maximum)? 181 cm
In the same example, what is the smallest value (minimum)? 159 cm
The range in our small dataset of heights is 181 - 159 = 22 cm
Here we added some reference lines on the chart to indicate the maximum and minimum:



In practical terms, the animal with the maximum value is the tallest, and the animal with the minimum value is the shortest. So Harry the Horse is the tallest, and Fran the Fox is the shortest.



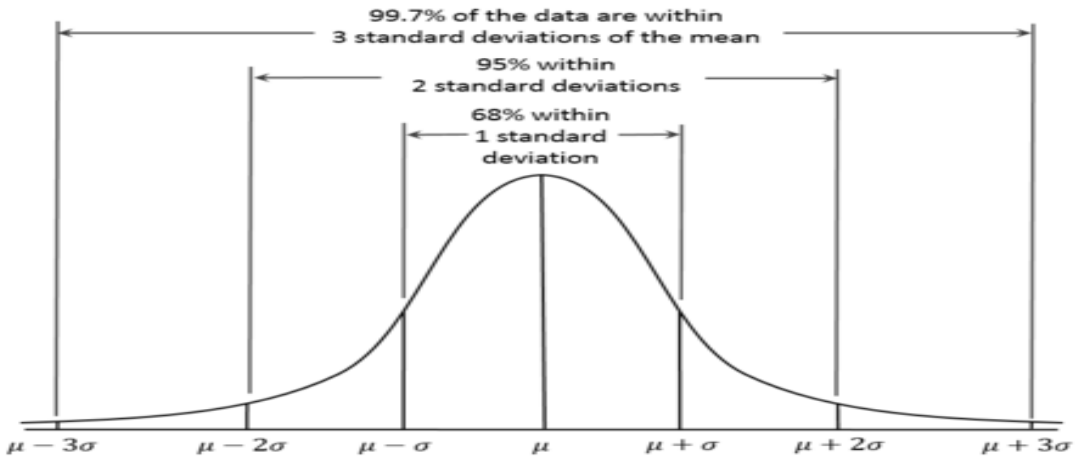
While the range gives us the endpoints (i.e., extremes), it does not tell us anything about how tightly or loosely the data are distributed between those two endpoints. We also do not know whether more of the data is closer to the average, the maximum or the minimum. From our chart, it looks like just over half of the animals are tall (i.e., above the average height).

Two other related measures of dispersion–the variance and the standard deviation–can help us answer these questions. They provide a numerical summary of how much the data are scattered.

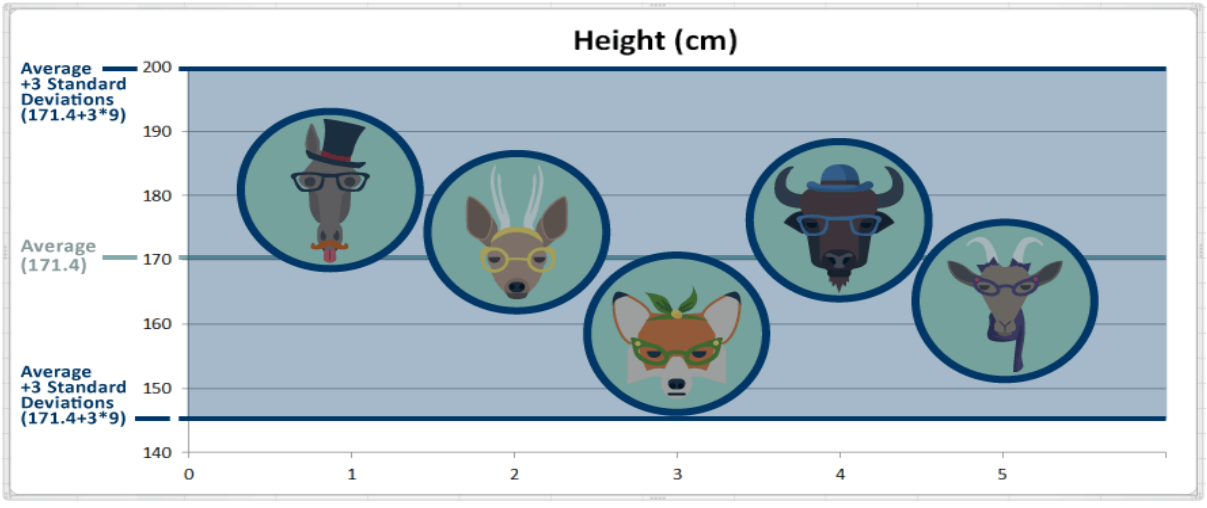
Standard Deviation

The standard deviation provides us with a standard way of knowing what is normal[2] given the average. A really helpful attribute of the standard deviation is that it is expressed in the same units as the data itself. The standard deviation is like an "index of variability," because it is proportional to the scatter of the data. The standard deviation is larger for more diverse distributions (i.e., the data are widely scattered). The standard deviation is smaller for less diverse distributions (i.e., the data are clustered together).

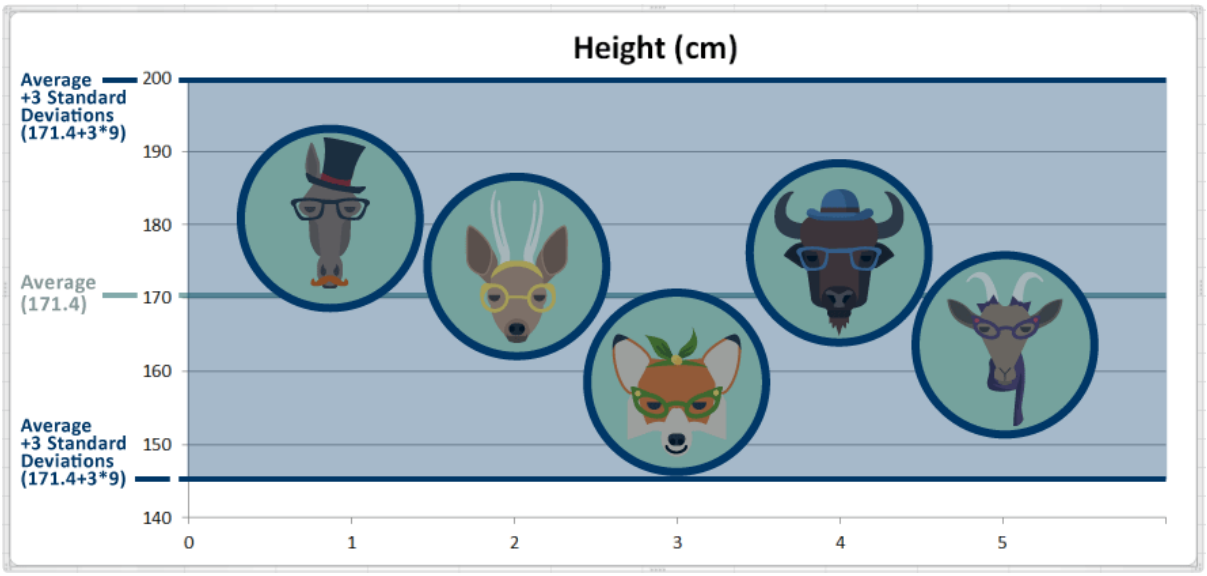
The standard deviation is very useful for understanding the spread of a variable. For most "normally" distributed data, generally almost all of the values will be within three standard deviations of the average. In statistics, this is sometimes referred to as the 68-95-99.7 rule. About 68.27% of the values lie within 1 standard deviation of the average (mean). Similarly, approximately 95.45% of the values lie within 2 standard deviations of the mean. Nearly all (99.73%) of the values lie within 3 standard deviations of the mean.



In the sample animal heights dataset, we've calculated the standard deviation for heights. It is 9.1 cm[3]. On the chart we have shaded the area to show what data is within three standard deviations (9.1 x 3) of the average. Any data within this range is "normal."



The standard deviation gives us a standardized way of knowing what is normal, what is extra-large or what is extra small. We know that Fran the Fox is short. When we consider the standard deviation and that nearly all (99.73%) of all values are generally within 3 standard deviations, we can conclude that Fran is short but not abnormally short.



3.10 Creating New Variables

What is the DAX Variable?

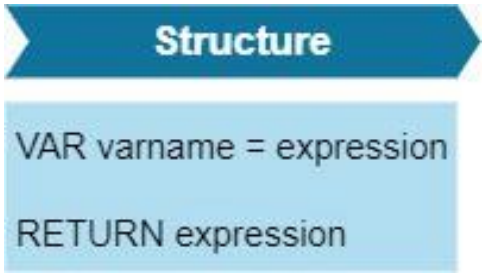
In DAX calculation, we can use variables to make the calculations easier to understand. When you are writing any complex or **nested** expression using DAX functions, variables can help to break these **complex** calculations into **smaller**, more useful sections.

Why we need DAX variable



You can use variables in any form of DAX calculations and it includes calculated measures, columns and tables. A type of variable is nothing but the calculated object type in DAX

Structure of Variable



1. To close a declared variable scope, there **must** be a **RETURN** statement.

VAR varA=2
RETURN varA * 2

2. You can declare **multiple** variables with the **same** layer of **scope** and a **single** RETURN statement.

VAR varA=2

VAR varB=varA+2

RETURN varB*3

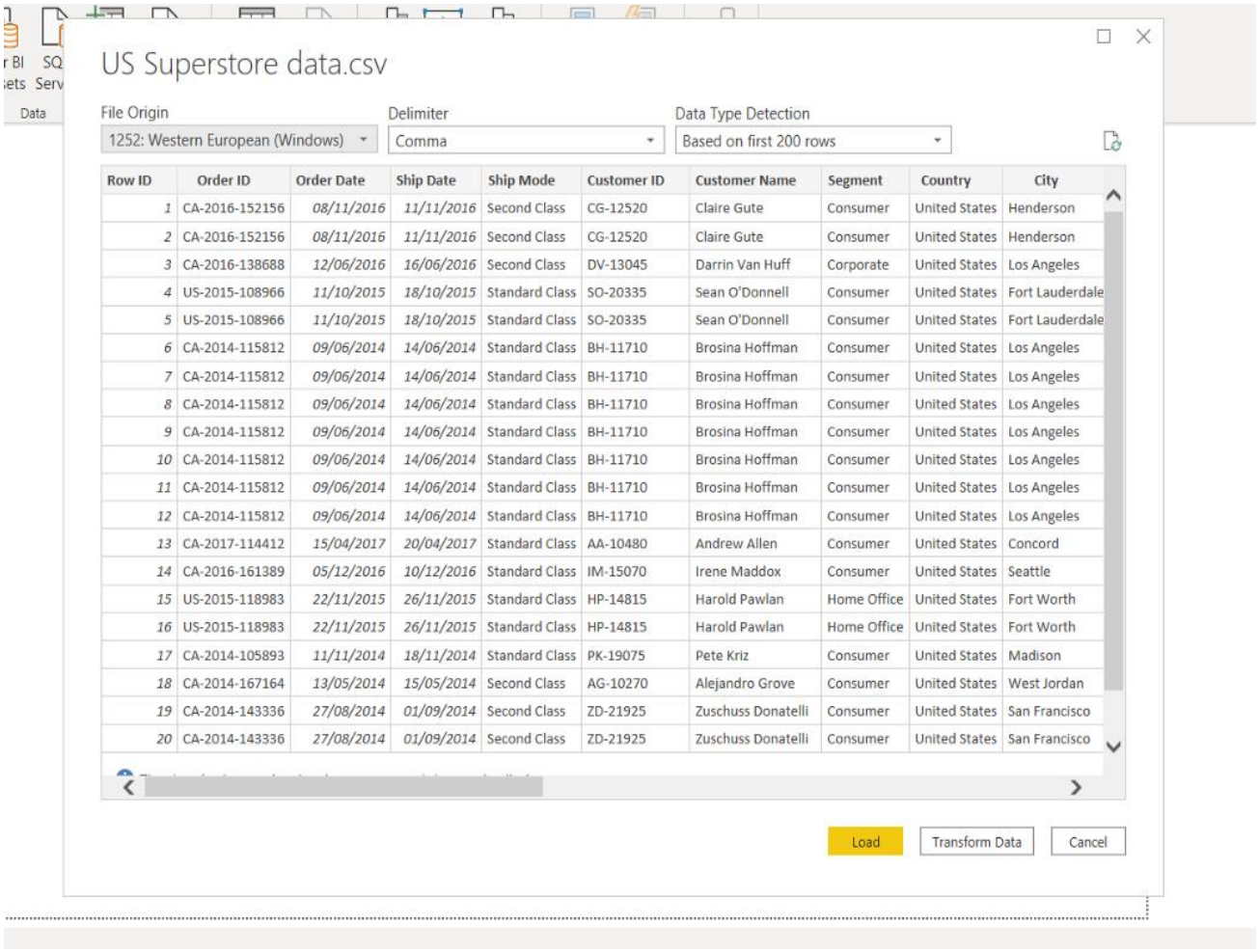
3. Variables can be assigned once and **cannot be reassigned**. Give an error if you like below

VAR varA=2

varA=varA+2

Get Data
For this case study, I consider the US Superstore dataset from Kaggle.

- Let's start with the **Get Data** option under the **Home** tab. As this is a CSV file, select the **Text/CSV** option from the drop-down list
- Select the file named **US Superstore data.csv**
- After selecting the file, data will be displayed in the below format



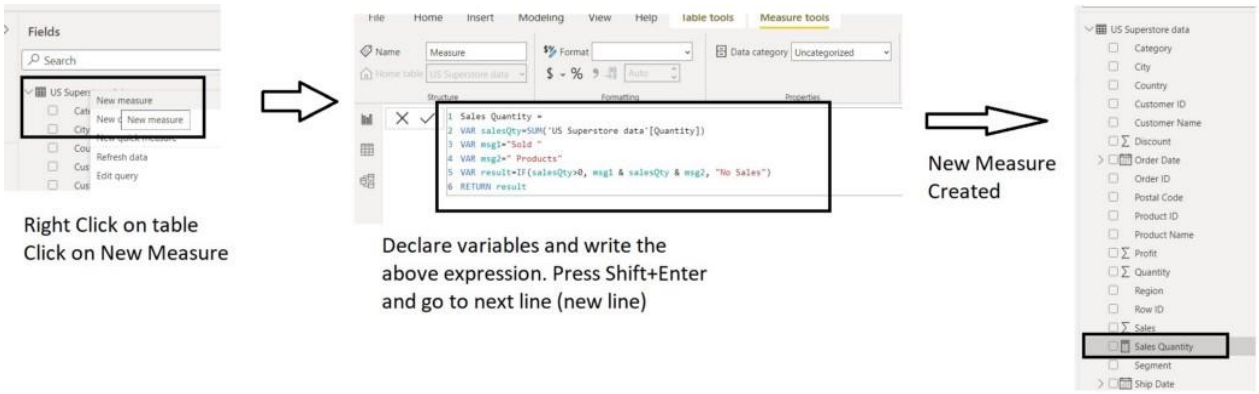
Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Country	City
1	CA-2016-152156	08/11/2016	11/11/2016	Second Class	CG-12520	Claire Gute	Consumer	United States	Henderson
2	CA-2016-152156	08/11/2016	11/11/2016	Second Class	CG-12520	Claire Gute	Consumer	United States	Henderson
3	CA-2016-138688	12/06/2016	16/06/2016	Second Class	DV-13045	Darrin Van Huff	Corporate	United States	Los Angeles
4	US-2015-108966	11/10/2015	18/10/2015	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States	Fort Lauderdale
5	US-2015-108966	11/10/2015	18/10/2015	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States	Fort Lauderdale
6	CA-2014-115812	09/06/2014	14/06/2014	Standard Class	BH-11710	Brosina Hoffman	Consumer	United States	Los Angeles
7	CA-2014-115812	09/06/2014	14/06/2014	Standard Class	BH-11710	Brosina Hoffman	Consumer	United States	Los Angeles
8	CA-2014-115812	09/06/2014	14/06/2014	Standard Class	BH-11710	Brosina Hoffman	Consumer	United States	Los Angeles
9	CA-2014-115812	09/06/2014	14/06/2014	Standard Class	BH-11710	Brosina Hoffman	Consumer	United States	Los Angeles
10	CA-2014-115812	09/06/2014	14/06/2014	Standard Class	BH-11710	Brosina Hoffman	Consumer	United States	Los Angeles
11	CA-2014-115812	09/06/2014	14/06/2014	Standard Class	BH-11710	Brosina Hoffman	Consumer	United States	Los Angeles
12	CA-2014-115812	09/06/2014	14/06/2014	Standard Class	BH-11710	Brosina Hoffman	Consumer	United States	Los Angeles
13	CA-2017-114412	15/04/2017	20/04/2017	Standard Class	AA-10480	Andrew Allen	Consumer	United States	Concord
14	CA-2016-161389	05/12/2016	10/12/2016	Standard Class	IM-15070	Irene Maddox	Consumer	United States	Seattle
15	US-2015-118983	22/11/2015	26/11/2015	Standard Class	HP-14815	Harold Pawlan	Home Office	United States	Fort Worth
16	US-2015-118983	22/11/2015	26/11/2015	Standard Class	HP-14815	Harold Pawlan	Home Office	United States	Fort Worth
17	CA-2014-105893	11/11/2014	18/11/2014	Standard Class	PK-19075	Pete Kriz	Consumer	United States	Madison
18	CA-2014-167164	13/05/2014	15/05/2014	Second Class	AG-10270	Alejandro Grove	Consumer	United States	West Jordan
19	CA-2014-143336	27/08/2014	01/09/2014	Second Class	ZD-21925	Zuschuss Donatelli	Consumer	United States	San Francisco
20	CA-2014-143336	27/08/2014	01/09/2014	Second Class	ZD-21925	Zuschuss Donatelli	Consumer	United States	San Francisco

- Click on Load and save data.

VARIABLES WITH TEXT

Variables can store numeric data as well as text. Using this feature, you can create a dynamic text-based measure that can be varied depending on filter selection. Let's see one example.

1. Create one text-based calculated measure (Sales Quantity) using variable
2. There are 4 variables. one for the direct measure, two for hard-coded messages and the final result variable where you are using IF clause condition and concatenate operator. Here you are using the result as a return statement.
3. To make the next line or new line, press Shift + Enter.



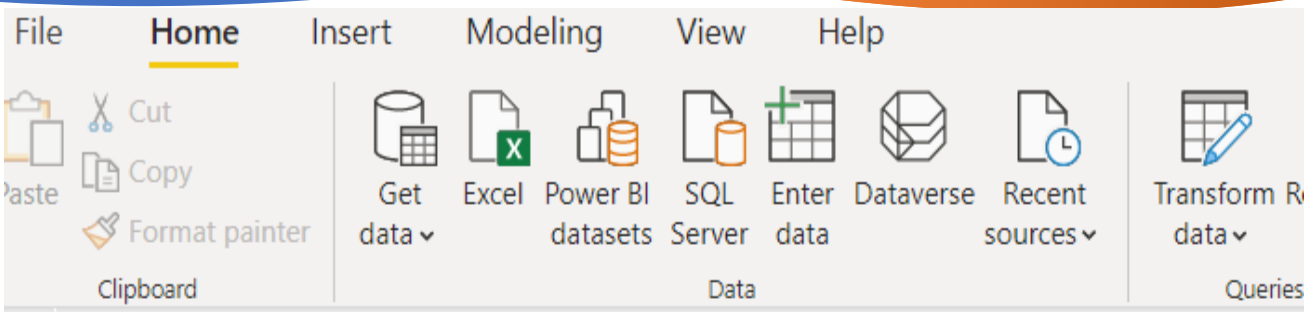
4 . Your first measure using variables has been ready for presentation.

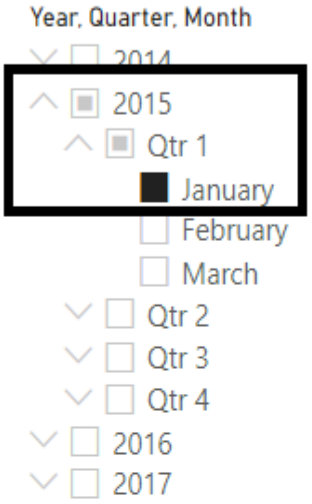
5. Create one Slicer and one table to test your newly created measure.

6. For Slicer, add Order Date Hierarchy in the Field section.

7. For Table visual, add Order Date Hierarchy and Sales Quantity in the Values section.

8. Display report with some filter condition like 2015 as Year, Quarter as Qtr1, Month as January and observe the report result.





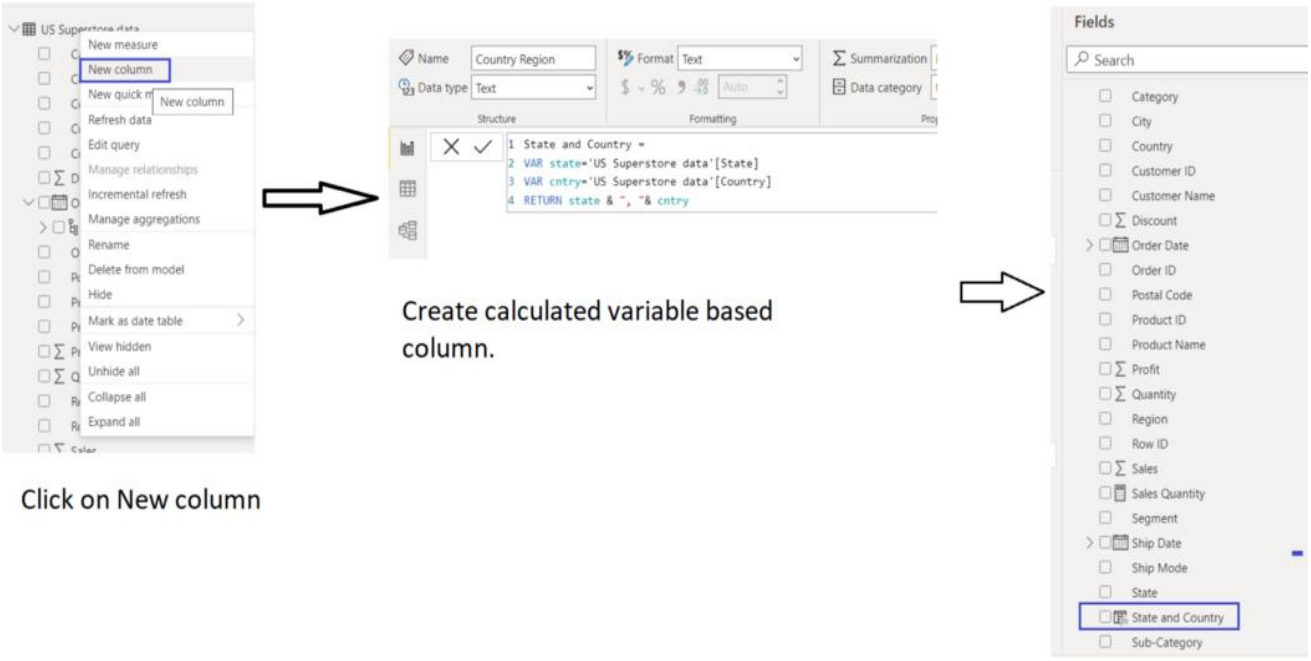
Year	Quarter	Month	Day	Sales Quantity
2015	Qtr 1	January	1	No Sales
2015	Qtr 1	January	2	Sold 29 Products
2015	Qtr 1	January	3	Sold 19 Products
2015	Qtr 1	January	4	Sold 22 Products
2015	Qtr 1	January	5	Sold 20 Products
2015	Qtr 1	January	6	Sold 7 Products
2015	Qtr 1	January	7	No Sales
2015	Qtr 1	January	8	No Sales
2015	Qtr 1	January	9	Sold 13 Products
2015	Qtr 1	January	10	Sold 4 Products
2015	Qtr 1	January	11	No Sales
2015	Qtr 1	January	12	Sold 16 Products
2015	Qtr 1	January	13	Sold 18 Products
2015	Qtr 1	January	14	No Sales
2015	Qtr 1	January	15	No Sales
2015	Qtr 1	January	16	No Sales
2015	Qtr 1	January	17	Sold 17 Products
2015	Qtr 1	January	18	No Sales
2015	Qtr 1	January	19	Sold 5 Products
2015	Qtr 1	January	20	No Sales
2015	Qtr 1	January	21	No Sales
2015	Qtr 1	January	22	No Sales
2015	Qtr 1	January	23	Sold 5 Products
2015	Qtr 1	January	24	Sold 5 Products
2015	Qtr 1	January	25	No Sales
Total				**Sold 236 Products**

VARIABLES IN CALCULATED COLUMNS

When you are creating variables inside calculated columns, variables automatically get access to values in other columns from the same row.

It can be useful to display text-based data in some visuals.

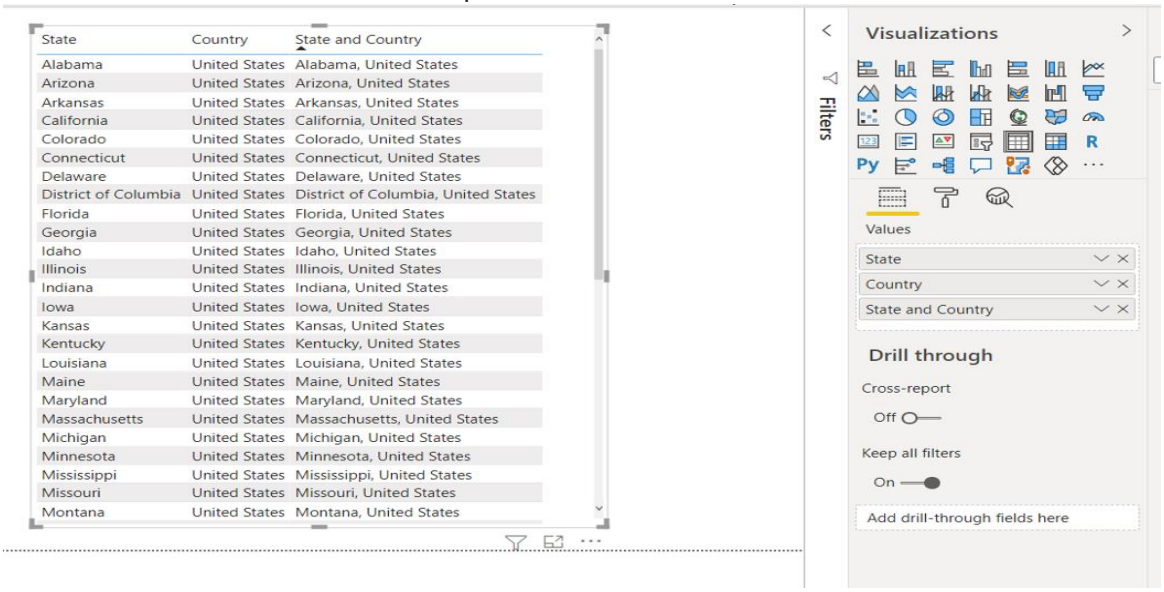
1. Create your first calculated variable based column.
2. Please follow the below steps to create that.



Click on New column

Create calculated variable based column.

3. Create a Table visual to see the output.



State	Country	State and Country
Alabama	United States	Alabama, United States
Arizona	United States	Arizona, United States
Arkansas	United States	Arkansas, United States
California	United States	California, United States
Colorado	United States	Colorado, United States
Connecticut	United States	Connecticut, United States
Delaware	United States	Delaware, United States
District of Columbia	United States	District of Columbia, United States
Florida	United States	Florida, United States
Georgia	United States	Georgia, United States
Idaho	United States	Idaho, United States
Illinois	United States	Illinois, United States
Indiana	United States	Indiana, United States
Iowa	United States	Iowa, United States
Kansas	United States	Kansas, United States
Kentucky	United States	Kentucky, United States
Louisiana	United States	Louisiana, United States
Maine	United States	Maine, United States
Maryland	United States	Maryland, United States
Massachusetts	United States	Massachusetts, United States
Michigan	United States	Michigan, United States
Minnesota	United States	Minnesota, United States
Mississippi	United States	Mississippi, United States
Missouri	United States	Missouri, United States
Montana	United States	Montana, United States

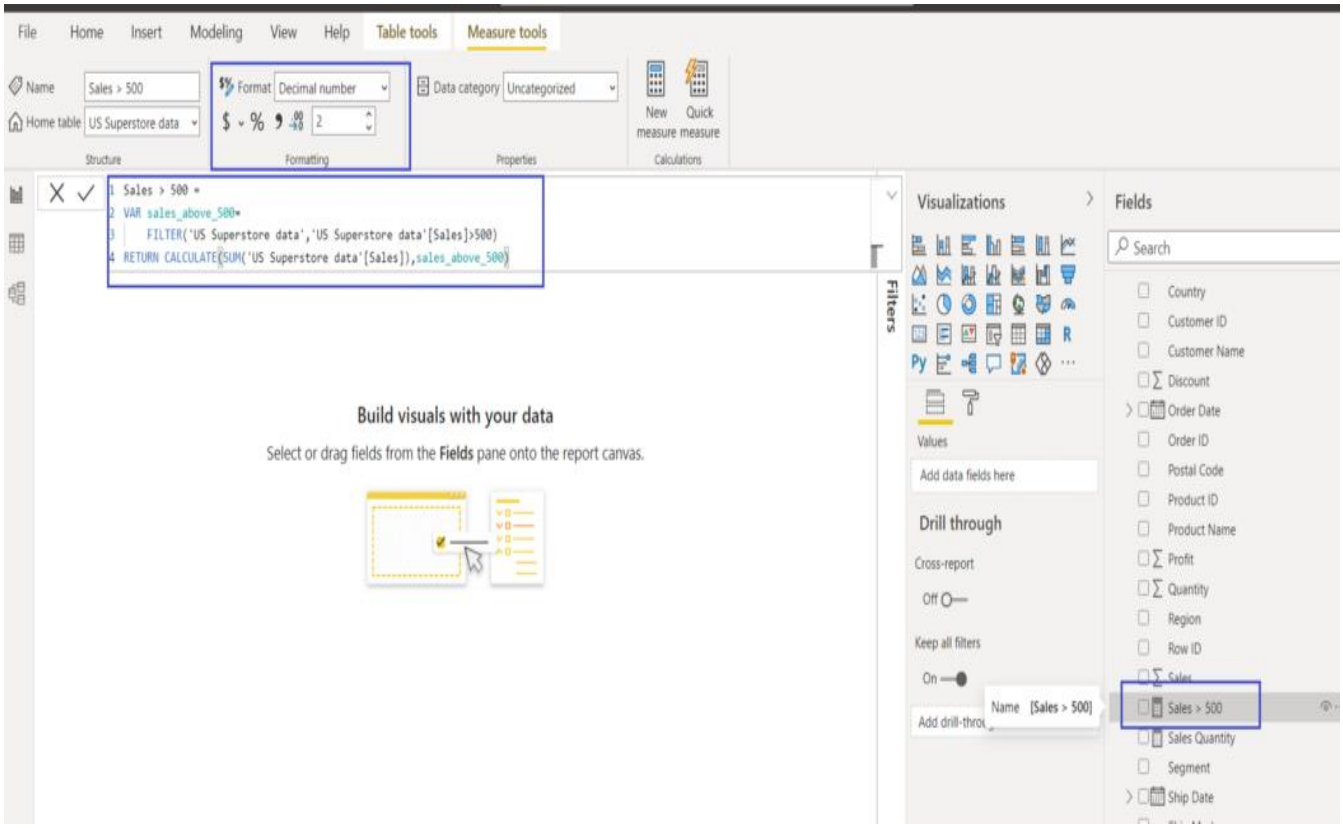
VARIABLES IN CALCULATED MEASURES

To make a calculated measure faster, you can use variables.

However, variables in calculated measures do not relate with individual rows that mean it cannot be assigned as a column-based value. This is the key difference with variables in calculated columns.

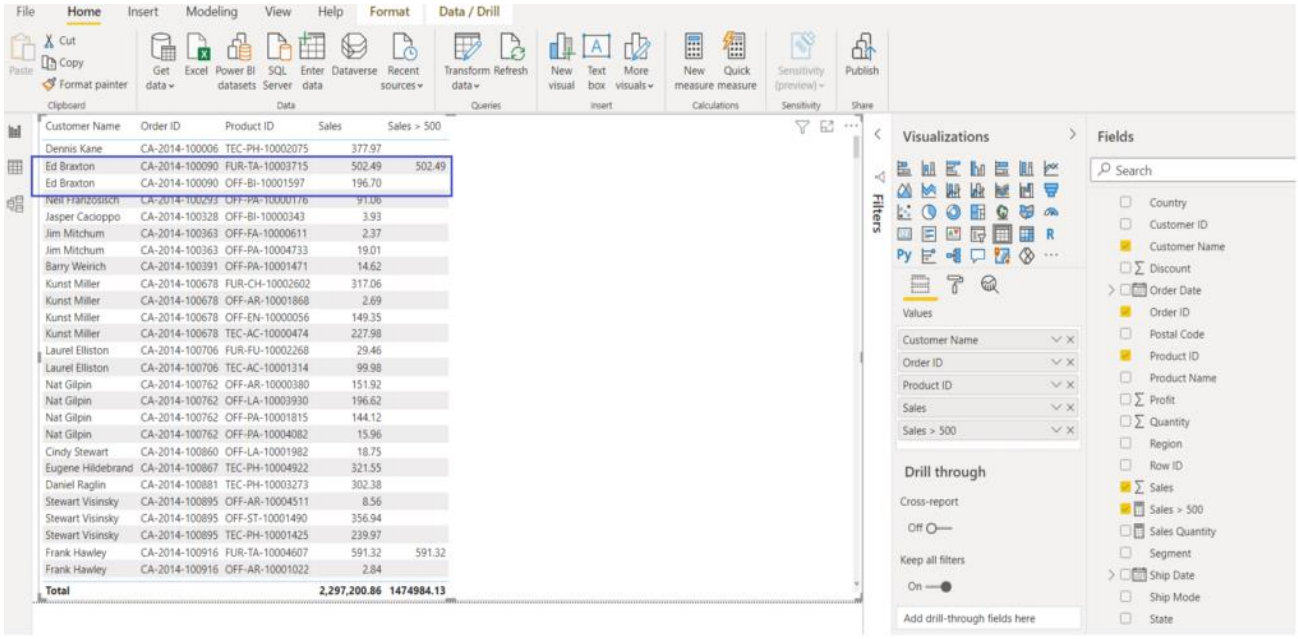
Let us see some example.

1. Create one calculated measure Sales > 500 with variable.
2. Change the number Formatting from the Whole Number to a Decimal Number with 2 decimal places.



3. Create your report with Customer Name, Order ID, Product ID, Sales and Sales > 500.

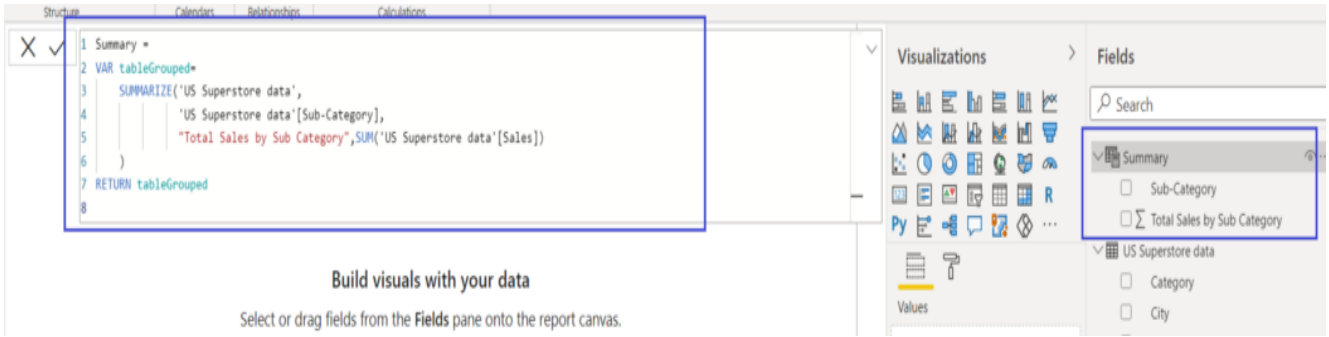
4. Now analyse the report. For example, Ed Braxton placed one order for two products. For one product sales value is above 500 and another one is below 500.



VARIABLES IN CALCULATED TABLES

Variables in the calculated table are useful when you are creating any summary or aggregated table from the existing table. It helps to improve the performance of the report page.

1. Create one calculated table “Summary” using a variable.



2. Now create two reports and compare the results whether your calculated table is working as expected or not.

Report from Summary Table (Calculated Table using variable)

Sub-Category	Total Sales by Sub Category
Phones	330,007.05
Chairs	328,449.10
Storage	223,843.61
Tables	206,965.53
Binders	203,412.73
Machines	189,238.63
Accessories	167,380.32
Copiers	149,528.03
Bookcases	114,880.00
Appliances	107,532.16
Furnishings	91,705.16
Paper	78,479.21
Supplies	46,673.54
Art	27,118.79
Envelopes	16,476.40
Labels	12,486.31
Fasteners	3,024.28
Total	**2,297,200.86**

Report from Original Table

Sub-Category	Sales
Phones	330,007.05
Chairs	328,449.10
Storage	223,843.61
Tables	206,965.53
Binders	203,412.73
Machines	189,238.63
Accessories	167,380.32
Copiers	149,528.03
Bookcases	114,880.00
Appliances	107,532.16
Furnishings	91,705.16
Paper	78,479.21
Supplies	46,673.54
Art	27,118.79
Envelopes	16,476.40
Labels	12,486.31
Fasteners	3,024.28
Total	**2,297,200.86**

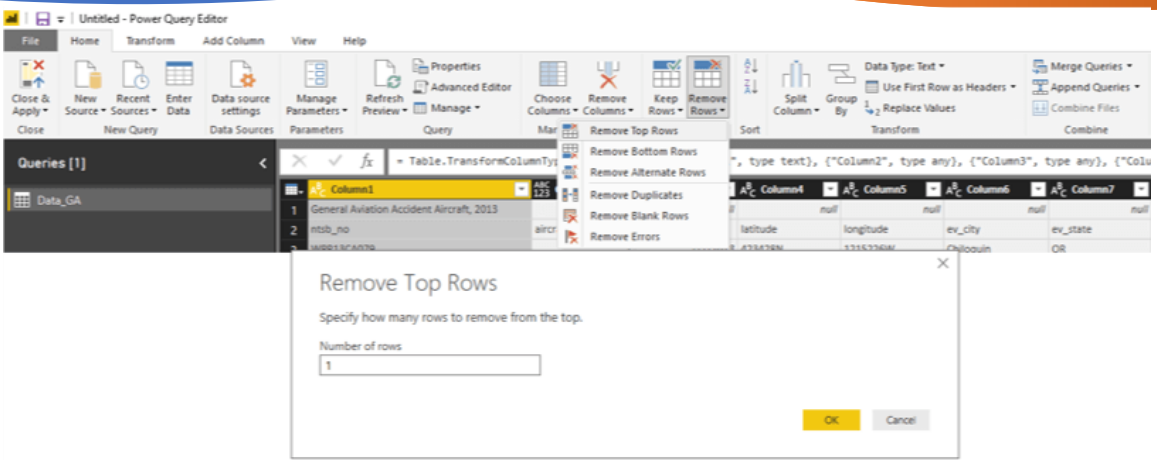
Assignment:

Connect external data source (excel) with Power BI, remove redundant columns and set appropriate column headers.

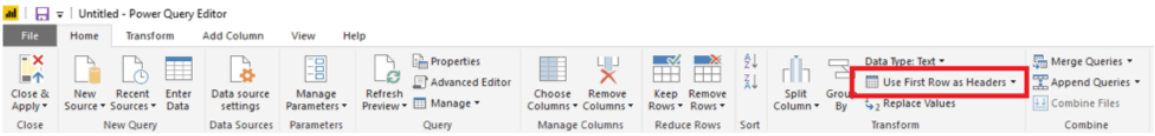
Solution:

For Data source connection please Refer to page 35

the Column Headers are actually in the second row, hence, the first row needs to be removed. Click on "**Remove Rows**" from the Power Query Editor toolbar. Then, click on "**Remove Top Rows**" and enter "**1**" in the "**Number of rows**" dialog box to remove the first row.



After removing that row, click on the "**Use First Row as Headers**" option to use the new first row as the Column Headers.

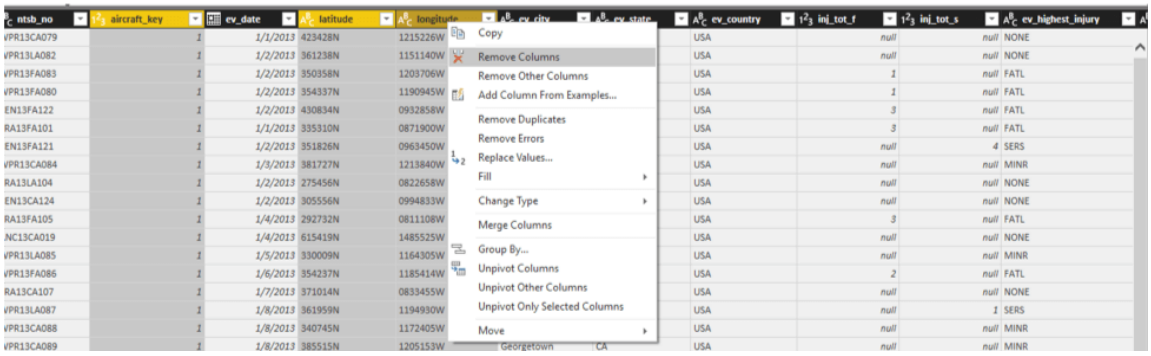


Now, the table is displaying the correct Column Headers.



Remove Redundant Columns

Here, you can remove all the columns that are not required for your analysis. Select the columns that you wish to remove while pressing the CTRL-key. Then, right-click and click on "**Remove Columns**" to eliminate columns.



Chapter: 4

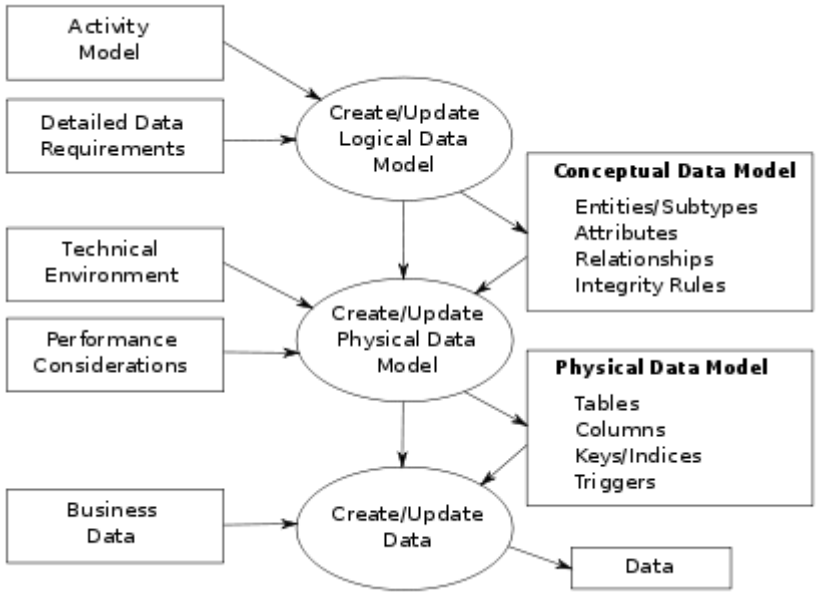
Data Modelling

Scope:

- What is Data Modelling?
- Data Modelling sometimes needs Data Analysis
- Star Schema
- Snowflake Schema
- Create a Relationship
- One to many, many to many relationships

4.1 Data Modelling:

Data Modelling is the process of analysing the data objects and their relationship to the other objects. It is used to analyse the data requirements that are required for the business processes. The data models are created for the data to be stored in a database. The Data Model's main focus is on what data is needed and how we have to organize data rather than what operations we have to perform.



The figure illustrates the way data models are developed and used today. A conceptual data model is developed based on the data requirements for the application that is being developed, perhaps in the context of an activity model. The data model will normally consist of entity types, attributes, relationships, integrity rules, and the definitions of those objects. This is then used as the start point for interface or database design.

Data modelling techniques and methodologies are used to model data in a standard, consistent, predictable manner in order to manage it as a resource. The use of data modelling standards is strongly recommended for all projects requiring a standard means of defining and analysing data within an organization, e.g., using data modelling:

- to assist business analysts, programmers, testers, manual writers, IT package selectors, engineers, managers, related organizations and clients to understand and use an agreed upon semi-formal model that encompasses the concepts of the organization and how they relate to one another
- to manage data as a resource
- to integrate information systems
- to design databases/data warehouses (aka data repositories)

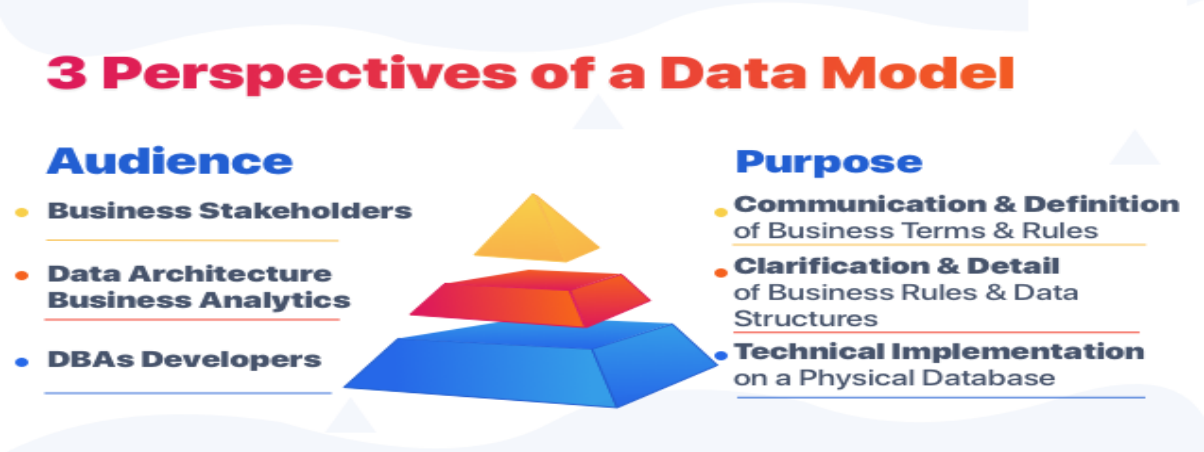
4.2 Uses of Data Modelling Tools

Data Modelling is a process to formulate data in an information system in a structured format. Listed below are certain practical uses of the related tools in any sector or industry.

- Data Modelling helps create a robust design with a data model that can show an organization's entire data on the same platform.
- The data model makes sure that all the data objects required by the database are represented or not.
- The database at the logical, physical, and conceptual levels can be designed with the help data model.
- The relation tables, foreign keys, and primary keys can be defined with the data model's help.
- Data Modelling Tools help in the improvement of data quality.
- Data Model gives the clear picture of business requirements.
- Redundant data and missing data can be identified with the help of data models.
- In data models, all the important data is accurately represented. The chances of incorrect results and faulty reports decreased as the data model reduces data omission.
- The data models create a visual representation of the data. With the help of it, the data analysis gets improved. We get the data picture, which can then be used by developers to create a physical database.
- Better consistency can be qualified with the help of a data model across all the projects.
- The data model is quite a time consuming, but it makes the maintenance cheaper and faster.

4.3 Three Perspectives of a Data Model

Data Modelling helps to create a conceptual model and create the relationship between the items. The basic data modelling techniques involve dealing with three perspectives of a data model.



1. Conceptual Model

The conceptual data model is a view of the data that is required to help business processes. It also keeps track of business events and keeps related performance measures. The conceptual model defines what the system contains. This type of Data Modelling focuses on finding the data used in a business rather than processing flow. The main purpose of this data model is to organize, define business rules and concepts. For example, it helps business people to view any data like market data, customer data, and purchase data.

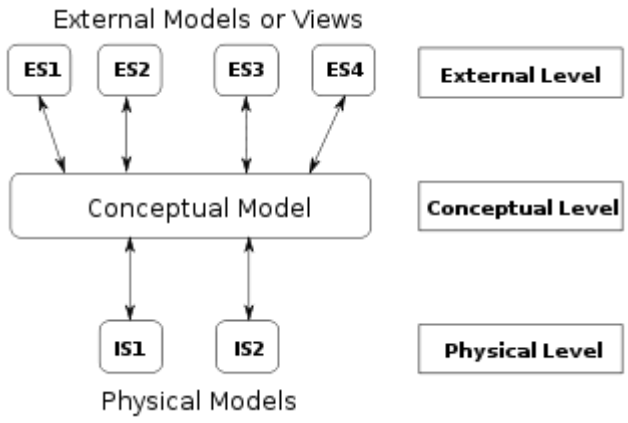
2. Logical Model

In the logical data model, the map of rules and data structures includes the data required, such as tables, columns, etc. Data architects and Business Analysts create the Logical Model. We can use the logical model to transform it into a database. This type of Data Modelling is always present in the root package object. This data model helps to form the base for the physical model. In this model, there is no secondary or primary key is defined.

3. Physical Data Model

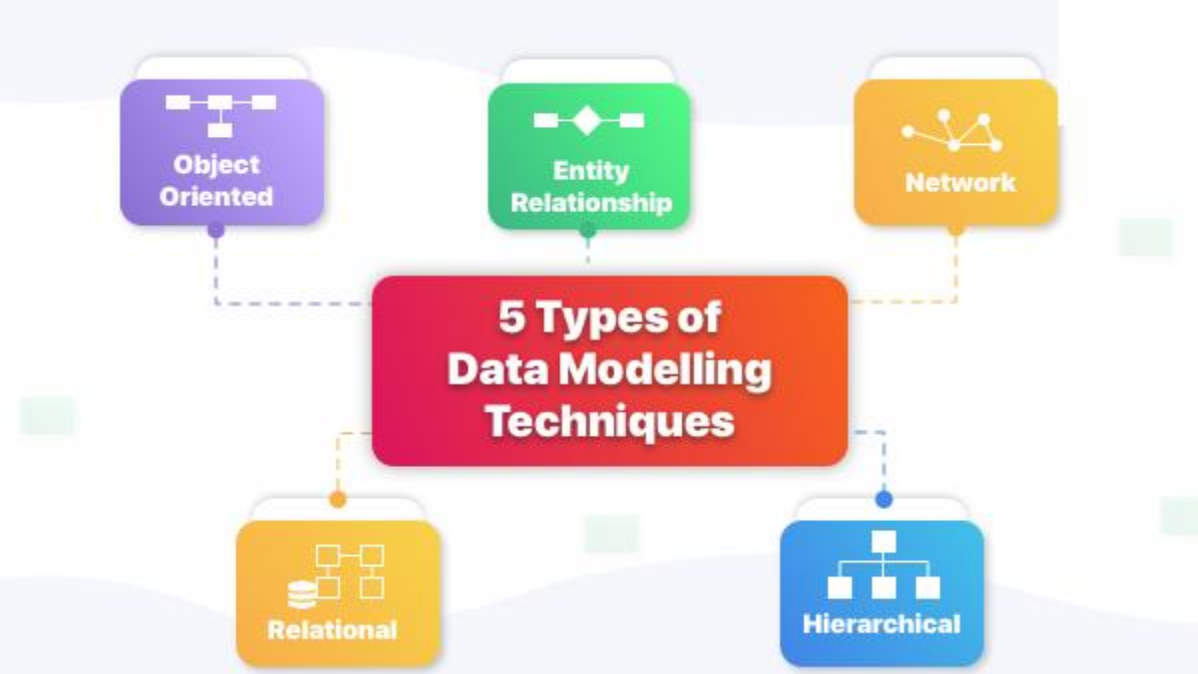
In a physical data model, the implementation is described using a specific database system. It defines all the components and services that are required to build a database. It is created by using the database language and queries. The physical data model represents each table, column, constraints like primary key, foreign key, NOT NULL, etc. The main work of the physical data model is to create a database. This model is created by the Database Administrator (DBA) and developers. This type of Data Modelling gives us the abstraction of the databases and helps to create the schema. This model describes the particular implementation of the data model. The physical data model helps us to have database column keys, constraints, and RDBMS features.

According to ANSI, this approach allows the three perspectives to be relatively independent of each other. Storage technology can change without affecting either the logical or the conceptual schema. The table/column structure can change without (necessarily) affecting the conceptual schema. In each case, of course, the structures must remain consistent across all schemas of the same data model.



The ANSI/SPARC three level architecture. This shows that a data model can be an external model (or view), a conceptual model, or a physical model. This is not the only way to look at data models, but it is a useful way, particularly when comparing models.

4.4Data Modelling Techniques



Below given are 5 different types of techniques used to organize the data:

1. Hierarchical Technique

The hierarchical model is a tree-like structure. There is one root node, or we can say one parent node and the other child nodes are sorted in a particular order. But, the hierarchical model is very rarely used now. This model can be used for real-world model relationships.

2. Object-oriented Model

The object-oriented approach is the creation of objects that contains stored values. The object-oriented model communicates while supporting data abstraction, inheritance, and encapsulation.

3. Network Technique

The network model provides us with a flexible way of representing objects and relationships between these entities. It has a feature known as a schema representing the data in the form

of a graph. An object is represented inside a node and the relation between them as an edge, enabling them to maintain multiple parent and child records in a generalized manner.

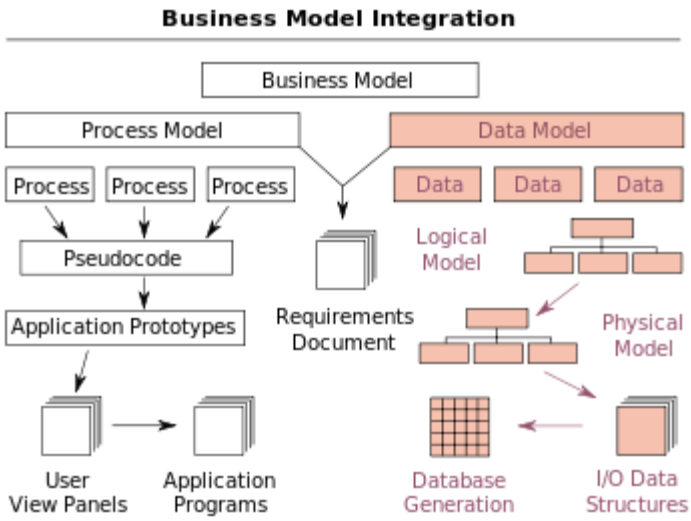
4. Entity-relationship Model

ER model (Entity-relationship model) is a high-level relational model which is used to define data elements and relationship for the entities in a system. This conceptual design provides a better view of the data that helps us easy to understand. In this model, the entire database is represented in a diagram called an entity-relationship diagram, consisting of Entities, Attributes, and Relationships.

5. Relational Technique

Relational is used to describe the different relationships between the entities. And there are different sets of relations between the entities such as one to one, one to many, many to one, and many to many.

4.5 **Business process integration**



The process of designing a database involves producing the previously described three types of schemas - conceptual, logical, and physical. The database design documented in these schemas are converted through a Data Definition Language, which can then be used to generate a database. A fully attributed data model contains detailed attributes (descriptions) for every entity within it. The term "database design" can describe many different parts of the design of an overall database system. Principally, and most correctly, it can be thought of as the logical design of the base data structures used to store the data. In the relational model these are the tables and views. In an object database the entities and relationships map directly to object classes and named relationships. However, the term "database design" could also be used to apply to the overall process of designing, not just the base data structures, but also the forms and queries used as part of the overall database application within the Database Management System or DBMS.

4.6 Modelling methodologies

Data models represent information areas of interest. While there are many ways to create data models, according to Len Silverston (1997) only two Modelling methodologies stand out, top-down and bottom-up:

- Bottom-up models or View Integration models are often the result of a reengineering effort. They usually start with existing data structures forms, fields on application screens, or reports. These models are usually physical, application-specific, and incomplete from an enterprise perspective. They may not promote data sharing, especially if they are built without reference to other parts of the organization.
- Top-down logical data models, on the other hand, are created in an abstract way by getting information from people who know the subject area. A system may not implement all the entities in a logical model, but the model serves as a reference point or template.

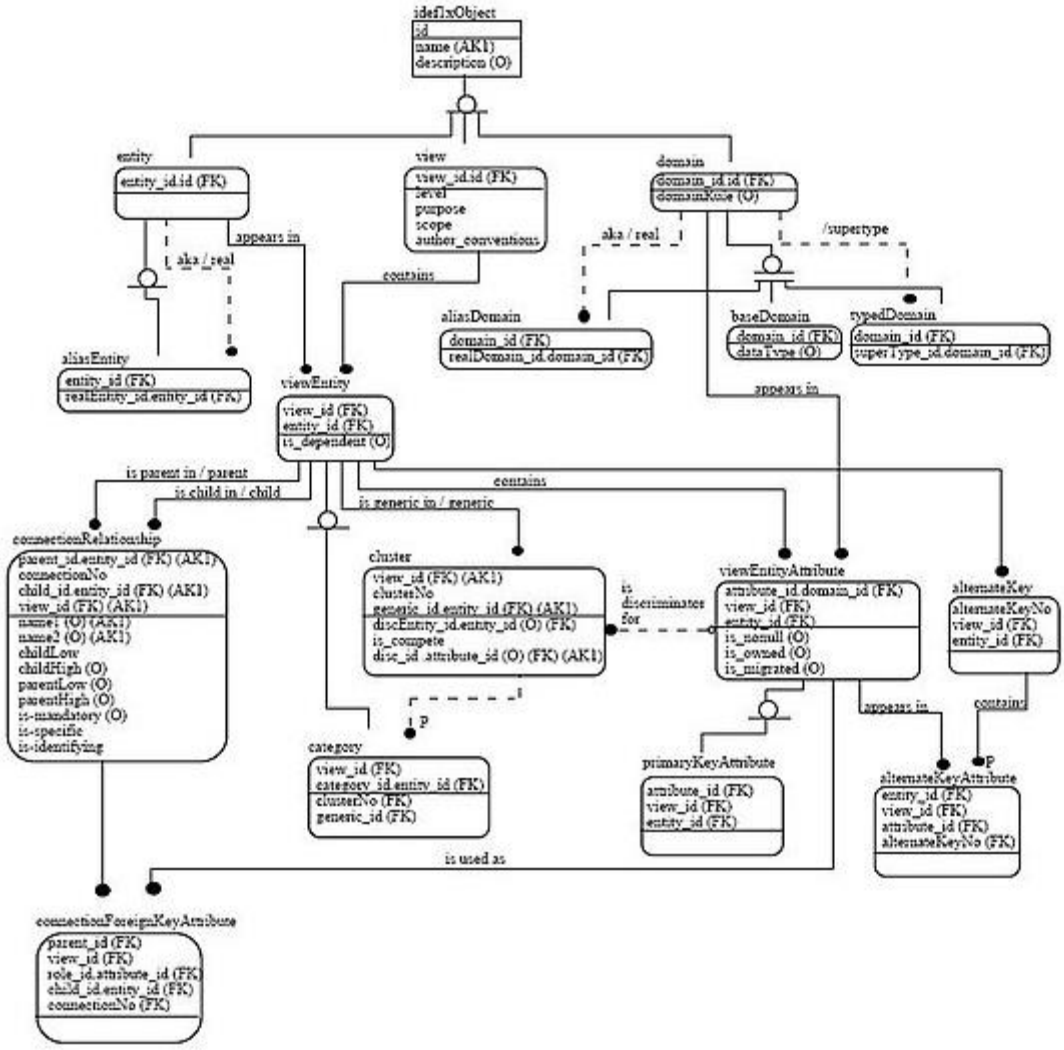
4.7 Entity–relationship Model

There are several notations for data Modelling. The actual model is frequently called "entity–relationship model", because it depicts data in terms of the entities and relationships described in the data. An entity–relationship model (ERM) is an abstract conceptual representation of structured data. Entity–relationship Modelling is a relational schema database Modelling method, used in software engineering to produce a type of conceptual data model (or semantic data model) of a system, often a relational database, and its requirements in a top-down fashion.

These models are being used in the first stage of information system design during the requirements analysis to describe information needs or the type of information that is to be stored in a database. The data Modelling technique can be used to describe any ontology (i.e. an overview and classifications of used terms and their relationships) for a certain universe of discourse i.e. area of interest.

Several techniques have been developed for the design of data models. While these methodologies guide data modelers in their work, two different people using the same methodology will often come up with very different results. Most notable are:

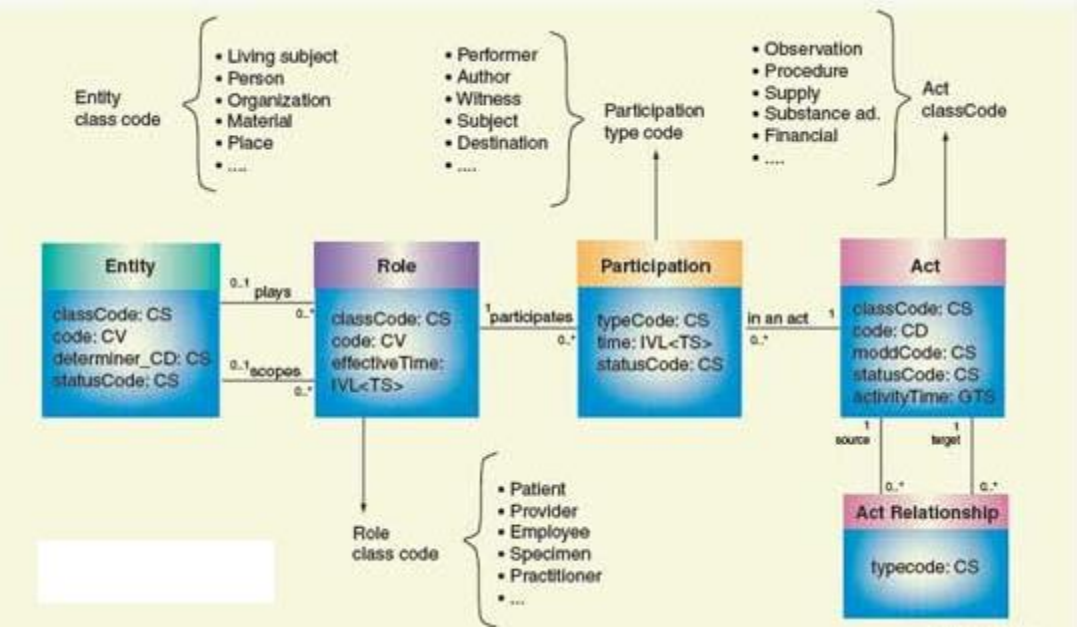
- Bachman diagrams
- Barker's notation
- Chen's notation
- Data Vault Modelling
- Extended Backus–Naur form
- IDEF1X
- Object-relational mapping
- Object-Role Modelling and Fully Communication Oriented Information Modelling
- Relational Model
- Relational Model/Tasmania



Example of an IDEF1X entity–relationship diagrams

4.8 Generic data Modelling

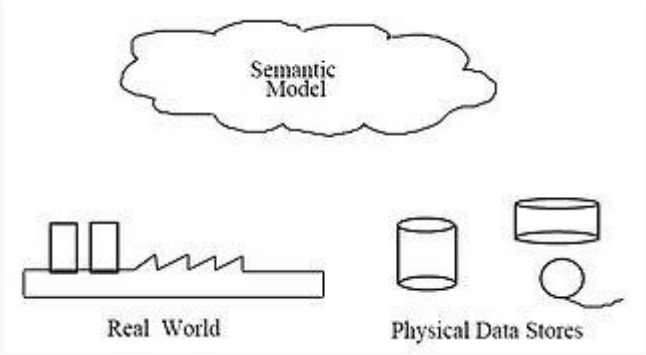
Generic data models are generalizations of conventional data models. They define standardized general relation types, together with the kinds of things that may be related by such a relation type. The definition of generic data model is similar to the definition of a natural language. For example, a generic data model may define relation types such as a 'classification relation', being a binary relation between an individual thing and a kind of thing (a class) and a 'part-whole relation', being a binary relation between two things, one with the role of part, the other with the role of whole, regardless the kind of things that are related.



Given an extensible list of classes, this allows the classification of any individual thing and to specify part-whole relations for any individual object. By standardization of an extensible list of relation types, a generic data model enables the expression of an unlimited number of kinds of facts and will approach the capabilities of natural languages. Conventional data models, on the other hand, have a fixed and limited domain scope, because the instantiation (usage) of such a model only allows expressions of kinds of facts that are predefined in the model.

4.9 Semantic data Modelling

The logical data structure of a DBMS, whether hierarchical, network, or relational, cannot totally satisfy the requirements for a conceptual definition of data because it is limited in scope and biased toward the implementation strategy employed by the DBMS. That is unless the semantic data model is implemented in the database on purpose, a choice which may slightly impact performance but generally vastly improves productivity.



Semantic data models.

Therefore, the need to define data from a conceptual view has led to the development of semantic data Modelling techniques. That is, techniques to define the meaning of data within the context of its interrelationships with other data. As illustrated in the figure the real world, in terms of resources, ideas, events, etc., are symbolically defined within physical data stores. A semantic data model is an abstraction which defines how the stored symbols relate to the real world. Thus, the model must be a true representation of the real world.

A semantic data model can be used to serve many purposes, such as:

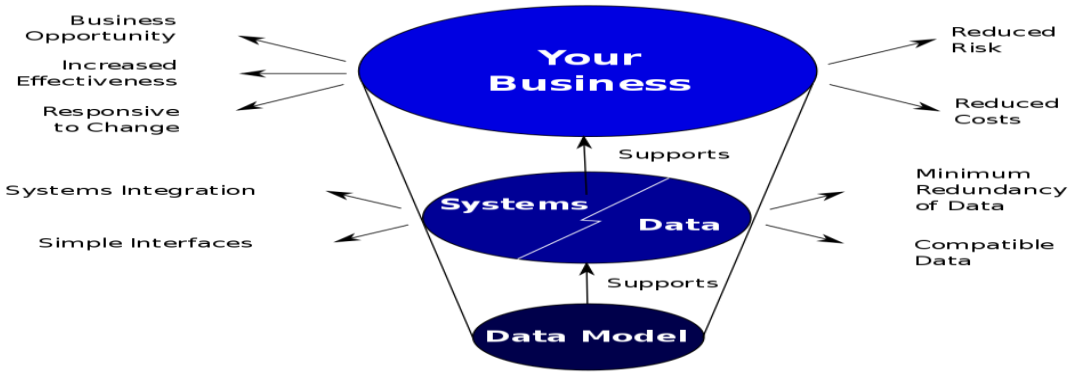
- planning of data resources
- building of shareable databases
- evaluation of vendor software
- integration of existing databases

4.10 What does a Data Modeller do?

- They create an entity relationship diagram to visualise relationships between key business concepts.
- They create a conceptual-level data dictionary to communicate data requirements that are important to business stakeholders.
- They create a data map to resolve potential data issues for a data migration or integration project.
- A data modeller would not necessarily query or manipulate data or become involved in designing or implementing databases or data repositories.

4.11 How data models deliver benefit

Data models provide a framework for data to be used within information systems by providing specific definition and format. If a data model is used consistently across systems then compatibility of data can be achieved. If the same data structures are used to store and access data then different applications can share data seamlessly. The results of this are indicated in the diagram. However, systems and interfaces are often expensive to build, operate, and maintain. They may also constrain the business rather than support it. This may occur when the quality of the data models implemented in systems and interfaces is poor.



4.12 Some common problems found in data models are:

- Business rules, specific to how things are done in a particular place, are often fixed in the structure of a data model. This means that small changes in the way business is conducted lead to large changes in computer systems and interfaces. So, business rules need to be implemented in a flexible way that does not result in complicated dependencies, rather the data model should be flexible enough so that changes in the business can be implemented within the data model in a relatively quick and efficient way.

- Entity types are often not identified, or are identified incorrectly. This can lead to replication of data, data structure and functionality, together with the attendant costs of that duplication in development and maintenance. Therefore, data definitions should be made as explicit and easy to understand as possible to minimize misinterpretation and duplication.

- Data models for different systems are arbitrarily different. The result of this is that complex interfaces are required between systems that share data. These interfaces can account for between 25-70% of the cost of current systems. Required interfaces should be considered inherently while designing a data model, as a data model on its own would not be usable without interfaces within different systems.

- Data cannot be shared electronically with customers and suppliers, because the structure and meaning of data has not been standardised. To obtain optimal value from an implemented data model, it is very important to define standards that will ensure that data models will both meet business needs and be consistent.

4.13 Data Modelling sometimes needs Data Analysis

BA's often need to analyse data as part of making data Modelling decisions, and this means that data Modelling can include some amount of data analysis. A lot can be accomplished with very basic technical skills, such as the ability to run simple database queries. This is why you may see a technical skill like SQL in a business analyst job description.

Many BA's succeed without knowing these more technical skills, instead, they rely on their ability to collaborate with technical professionals and other knowledgeable stakeholders to ensure the data is understood well enough to make the right modelling decisions.

The non-technical BA can also evaluate sample data, interview stakeholders to discover possible data-related issues, review current state database models, and analyse exception reports.

While data analysis skills are valuable for the business analyst, they are not essential. However, data modelling falls squarely within the business analyst's domain.

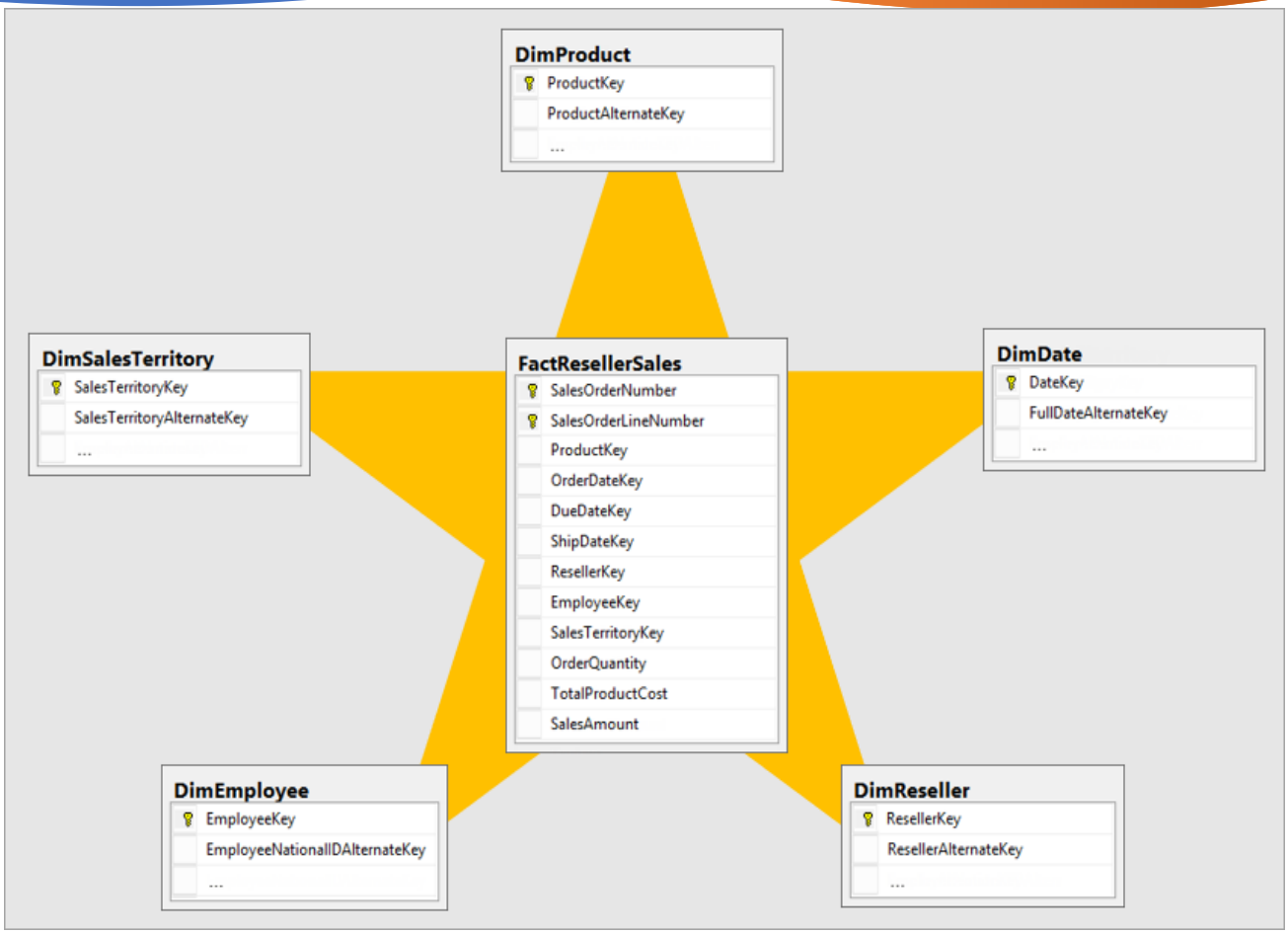
4.14 Star schema overview

Star schema is a mature modeling approach widely adopted by relational data warehouses. It requires modelers to classify their model tables as either *dimension* or *fact*.

Dimension tables describe business entities—the *things* you model. Entities can include products, people, places, and concepts including time itself. The most consistent table you'll find in a star schema is a date dimension table. A dimension table contains a key column (or columns) that acts as a unique identifier, and descriptive columns.

Fact tables store observations or events, and can be sales orders, stock balances, exchange rates, temperatures, etc. A fact table contains dimension key columns that relate to dimension tables, and numeric measure columns. The dimension key columns determine the *dimensionality* of a fact table, while the dimension key values determine the *granularity* of a fact table. For example, consider a fact table designed to store sale targets that has two-dimension key columns **Date** and **ProductKey**. It's easy to understand that the table has two dimensions. The granularity, however, can't be determined without considering the dimension key values. In this example, consider that the values stored in the **Date** column are the first day of each month. In this case, the granularity is at month-product level.

Generally, dimension tables contain a relatively small number of rows. Fact tables, on the other hand, can contain a very large number of rows and continue to grow over time.



Normalization vs. denormalization

To understand some star schema concepts described in this article, it's important to know two terms: normalization and denormalization.

Normalization is the term used to describe data that's stored in a way that reduces repetitious data. Consider a table of products that has a unique key value column, like the product key, and additional columns describing product characteristics, including product name, category, color, and size. A sales table is considered normalized when it stores only keys, like the product key. In the following image, notice that only the **ProductKey** column records the product.

	SalesOrderNumber	OrderDate	ProductKey	ResellerKey	SalesAmount
1	SO69561	2020-05-31	594	546	226.00
2	SO69560	2020-05-30	513	100	218.45
3	SO69560	2020-05-30	594	100	113.00
4	SO69539	2020-05-28	243	529	858.90
5	SO69539	2020-05-28	378	529	1466.01
6	SO69541	2020-05-28	594	661	113.00
7	SO69542	2020-05-28	243	317	1717.80
8	SO69544	2020-05-28	243	666	3435.60
9	SO69545	2020-05-28	378	436	5864.04
10	SO69532	2020-05-27	594	312	113.00
11	SO69532	2020-05-27	513	312	436.90
12	SO69533	2020-05-27	594	476	226.00

If, however, the sales table stores product details beyond the key, it's considered *denormalized*. In the following image, notice that the **ProductKey** and other product-related columns record the product.

	SalesOrderNumber	OrderDate	ProductKey	Product	Category	Color	Size	ResellerKey	SalesAmount
1	SO69561	2020-05-31	594	Mountain-500 Silver, 48	Bikes	Silver	48	546	226.00
2	SO69560	2020-05-30	513	ML Mountain Frame-W - Silver, 46	Components	Silver	46	100	218.45
3	SO69560	2020-05-30	594	Mountain-500 Silver, 48	Bikes	Silver	48	100	113.00
4	SO69539	2020-05-28	243	HL Road Frame - Red, 44	Components	Red	44	529	858.90
5	SO69539	2020-05-28	378	Road-250 Black, 52	Bikes	Black	52	529	1466.01
6	SO69541	2020-05-28	594	Mountain-500 Silver, 48	Bikes	Silver	48	661	113.00
7	SO69542	2020-05-28	243	HL Road Frame - Red, 44	Components	Red	44	317	1717.80
8	SO69544	2020-05-28	243	HL Road Frame - Red, 44	Components	Red	44	666	3435.60
9	SO69545	2020-05-28	378	Road-250 Black, 52	Bikes	Black	52	436	5864.04
10	SO69532	2020-05-27	594	Mountain-500 Silver, 48	Bikes	Silver	48	312	113.00
11	SO69532	2020-05-27	513	ML Mountain Frame-W - Silver, 46	Components	Silver	46	312	436.90
12	SO69533	2020-05-27	594	Mountain-500 Silver, 48	Bikes	Silver	48	476	226.00

When you source data from an export file or data extract, it's likely that it represents a denormalized set of data. In this case, use Power Query to transform and shape the source data into multiple normalized tables.

As described in this article, you should strive to develop optimized Power BI data models with tables that represent normalized fact and dimension data. However, there's one exception where a snowflake dimension should be denormalized to produce a single model table.

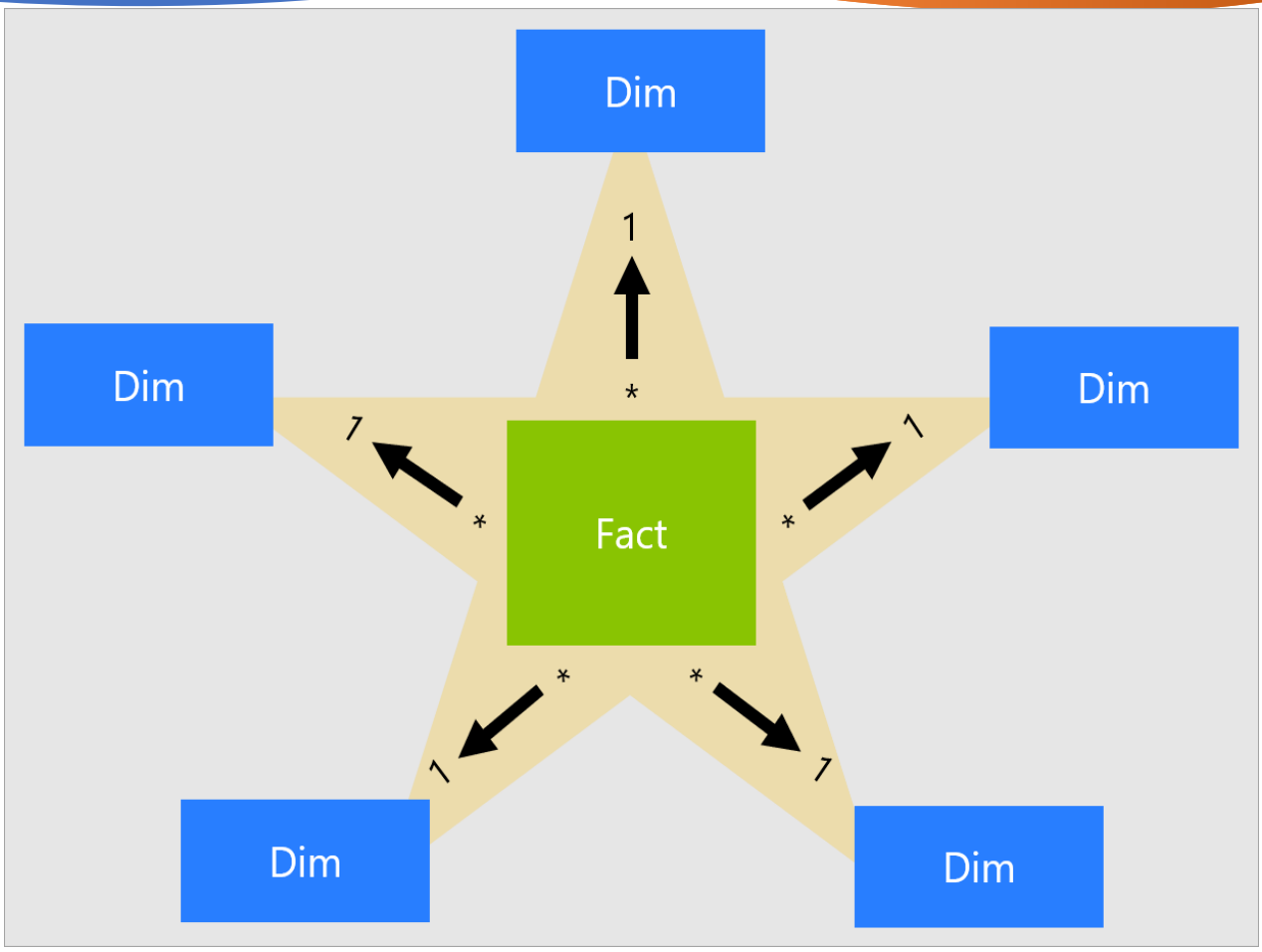
Star schema relevance to Power BI models

Star schema design and many related concepts introduced in this article are highly relevant to developing Power BI models that are optimized for performance and usability.

Consider that each Power BI report visual generates a query that is sent to the Power BI model (which the Power BI service calls a dataset). These queries are used to filter, group, and summarize model data. A well-designed model, then, is one that provides tables for filtering and grouping, and tables for summarizing. This design fits well with star schema principles:

- Dimension tables support *filtering* and *grouping*
- Fact tables support *summarization*

There's no table property that modelers set to configure the table type as dimension or fact. It's in fact determined by the model relationships. A model relationship establishes a filter propagation path between two tables, and it's the **Cardinality** property of the relationship that determines the table type. A common relationship cardinality is *one-to-many* or its inverse *many-to-one*. The "one" side is always a dimension-type table while the "many" side is always a fact-type table.



A well-structured model design should include tables that are either dimension-type tables or fact-type tables. Avoid mixing the two types together for a single table. We also recommend that you should strive to deliver the right number of tables with the right relationships in place. It's also important that fact-type tables always load data at a consistent grain.

Lastly, it's important to understand that optimal model design is part science and part art. Sometimes you can break with good guidance when it makes sense to do so.

There are many additional concepts related to star schema design that can be applied to a Power BI model. These concepts include:

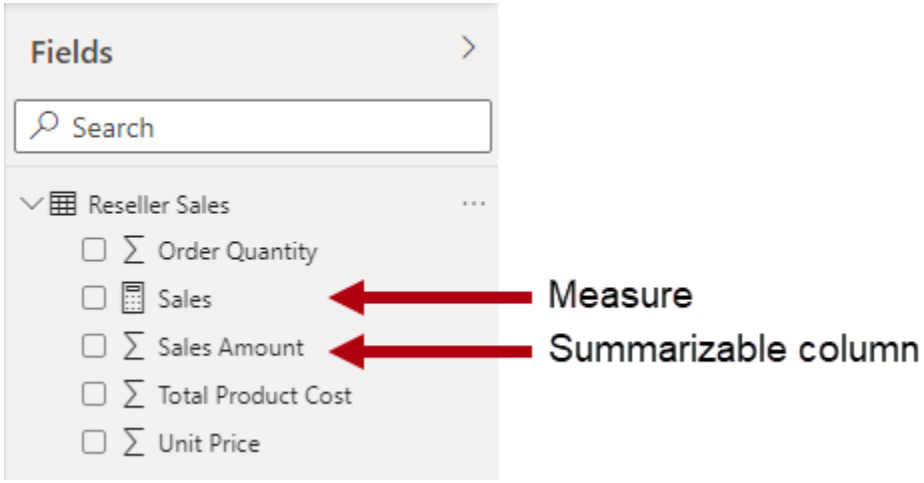
- Measures
- Surrogate keys
- Snowflake dimensions
- Role-playing dimensions
- Slowly changing dimensions
- Junk dimensions
- Degenerate dimensions
- Factless fact tables

Measures

In star schema design, a **measure** is a fact table column that stores values to be summarized.

In a Power BI model, a **measure** has a different—but similar—definition. It's a formula written in Data Analysis Expressions (DAX) that achieves summarization. Measure expressions often leverage DAX aggregation functions like SUM, MIN, MAX, AVERAGE, etc. to produce a scalar value result at query time (values are never stored in the model). Measure expression can range from simple column aggregations to more sophisticated formulas that override filter context and/or relationship propagation. For more information, read the DAX Basics in Power BI Desktop article.

It's important to understand that Power BI models support a second method for achieving summarization. Any column—and typically numeric columns—can be summarized by a report visual or Q&A. These columns are referred to as *implicit measures*. They offer a convenience for you as a model developer, as in many instances you do not need to create measures. For example, the Adventure Works reseller sales **Sales Amount** column could be summarized in numerous ways (sum, count, average, median, min, max, etc.), without the need to create a measure for each possible aggregation type.



However, there are three compelling reasons for you to create measures, even for simple column-level summarizations:

- When you know your report authors will query the model by using **Multidimensional Expressions (MDX)**, the model must include *explicit measures*. **Explicit measures are defined by using DAX**. This design approach is highly relevant when a Power BI dataset is queried by using MDX, because MDX can't achieve summarization of column values. Notably, MDX will be used when performing Analyse in Excel, because PivotTables issue MDX queries.
  Note: *MDX can be used to query against SSAS Multi-dimensional models, while DAX is used for SSAS Tabular models. If you use Power BI for your visualization needs, it can connect to SSAS Multi-dimensional models even though Power BI primarily uses Tabular models.*

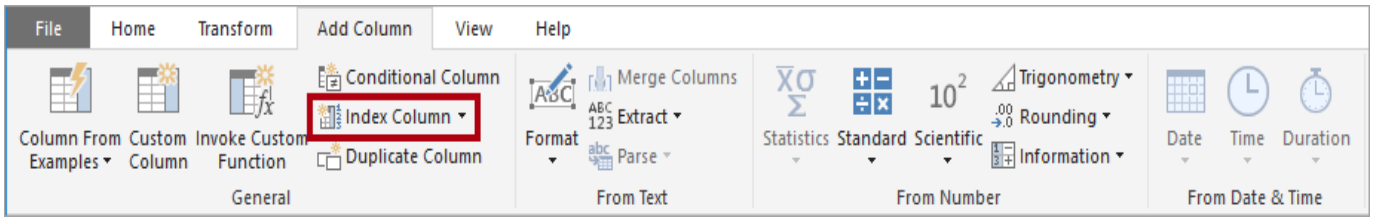
- When you know your report authors will create Power BI paginated reports using the MDX query designer, the model must include explicit measures. Only the MDX query designer supports server aggregates. So, if report authors need to have measures evaluated by Power BI (instead of by the paginated report engine), they must use the MDX query designer.
- When you need to ensure that your report authors can only summarize columns in specific ways. For example, the reseller sales **Unit Price** column (which represents a per unit rate) can be summarized, but only by using specific aggregation functions. It should never be summed, but it's appropriate to summarize by using other aggregation functions like min, max, average, etc. In this instance, the modeler can hide the **Unit Price** column, and create measures for all appropriate aggregation functions.

This design approach works well for reports authored in the Power BI service and for Q&A. However, Power BI Desktop live connections allow report authors to show hidden fields in the **Fields** pane, which can result in circumventing this design approach.

Surrogate keys

A **surrogate key** is a unique identifier that you add to a table to support star schema modeling. By definition, it's not defined or stored in the source data. Commonly, surrogate keys are added to relational data warehouse dimension tables to provide a unique identifier for each dimension table row.

Power BI model relationships are based on a single unique column in one table, which propagates filters to a single column in a different table. When a dimension-type table in your model doesn't include a single unique column, you must add a unique identifier to become the "one" side of a relationship. In Power BI Desktop, you can easily achieve this requirement by creating a Power Query index column.



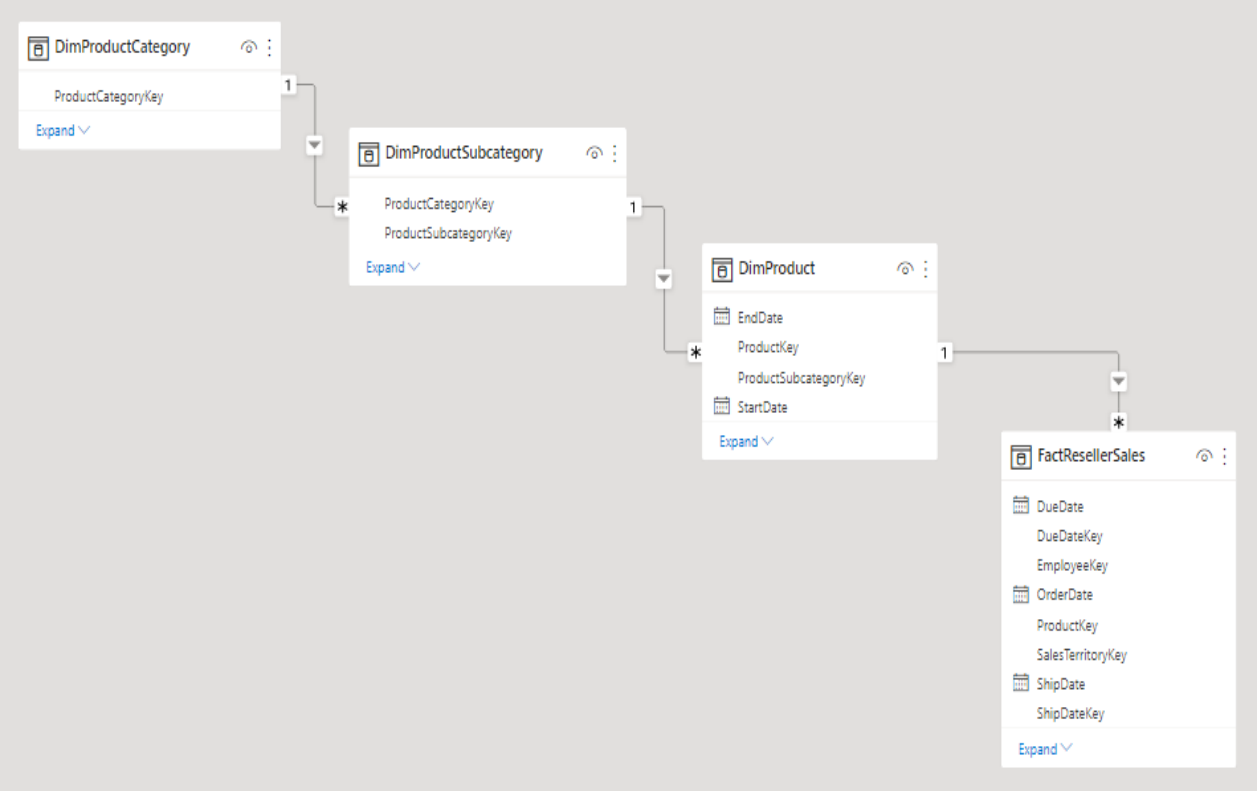
You must merge this query with the "many"-side query so that you can add the index column to it also. When you load these queries to the model, you can then create a one-to-many relationship between the model tables.

Snowflake dimensions

A **snowflake dimension** is a set of normalized tables for a single business entity. For example, Adventure Works classifies products by category and subcategory. Products are assigned to subcategories, and subcategories are in turn assigned to categories. In the Adventure Works relational data warehouse, the product dimension is normalized and

stored in three related tables: **DimProductCategory**, **DimProductSubcategory**, and **DimProduct**.

If you use your imagination, you can picture the normalized tables positioned outwards from the fact table, forming a snowflake design.



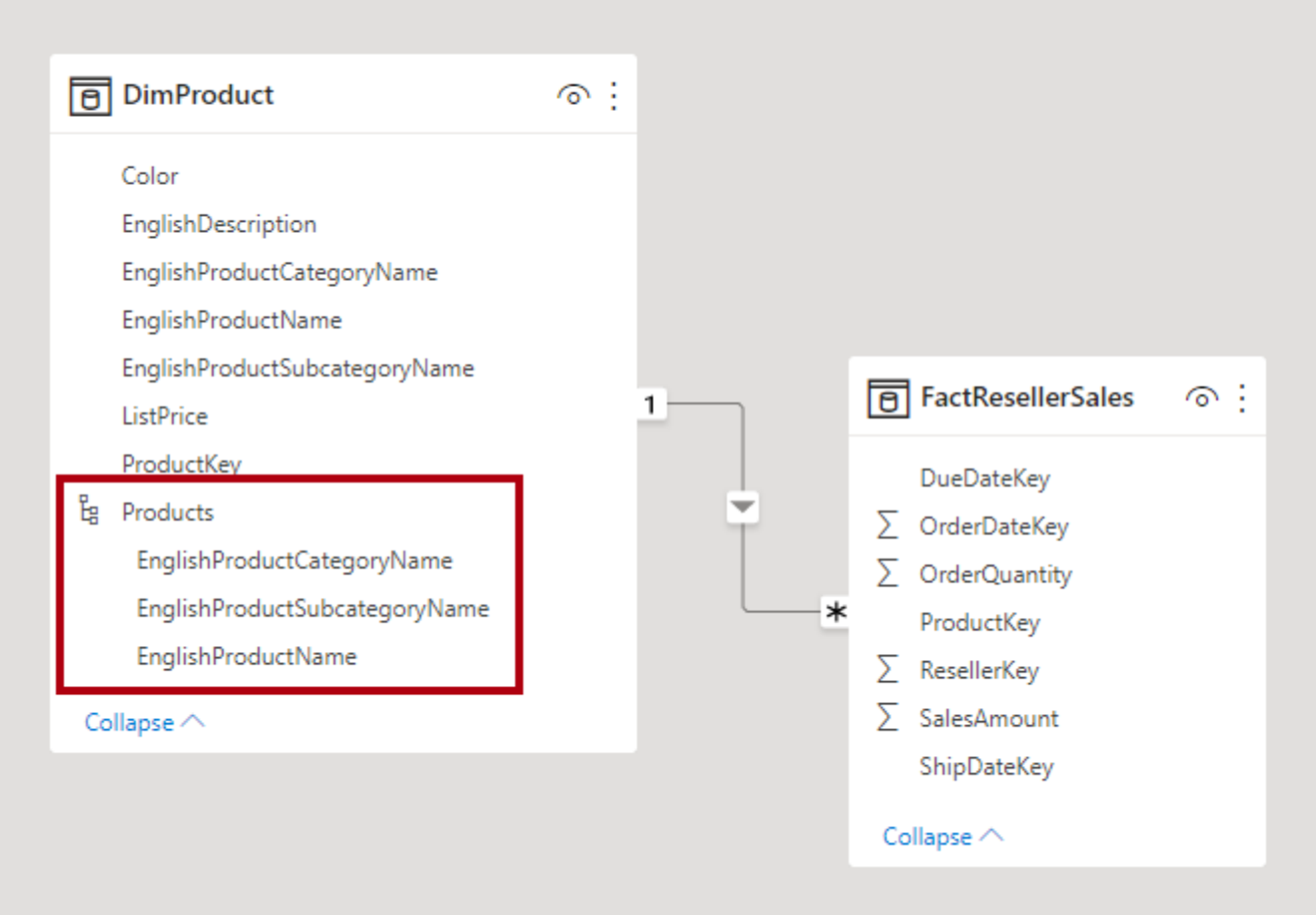
In Power BI Desktop, you can choose to mimic a snowflake dimension design (perhaps because your source data does) or integrate (denormalize) the source tables into a single model table. Generally, the benefits of a single model table outweigh the benefits of multiple model tables. The most optimal decision can depend on the volumes of data and the usability requirements for the model.

When you choose to mimic a snowflake dimension design:

- Power BI loads more tables, which is less efficient from storage and performance perspectives. These tables must include columns to support model relationships, and it can result in a larger model size.
- Longer relationship filter propagation chains will need to be traversed, which will likely be less efficient than filters applied to a single table.
- The **Fields** pane presents more model tables to report authors, which can result in a less intuitive experience, especially when snowflake dimension tables contain just one or two columns.
- It's not possible to create a hierarchy that spans the tables.

When you choose to integrate into a single model table, you can also define a hierarchy that encompasses the highest and lowest grain of the dimension. Possibly, the storage of

redundant denormalized data can result in increased model storage size, particularly for very large dimension tables.



Slowly changing dimensions

A **slowly changing dimension** (SCD) is one that appropriately manages change of dimension members over time. It applies when business entity values change over time, and in an ad hoc manner. A good example of a *slowly* changing dimension is a customer dimension, specifically its contact detail columns like email address and phone number. In contrast, some dimensions are considered to be *rapidly* changing when a dimension attribute changes often, like a stock's market price. The common design approach in these instances is to store rapidly changing attribute values in a fact table measure.

Star schema design theory refers to two common SCD types: Type 1 and Type 2. A dimension-type table could be Type 1 or Type 2, or support both types simultaneously for different columns.

Type 1 SCD

A **Type 1 SCD** always reflects the latest values, and when changes in source data are detected, the dimension table data is overwritten. This design approach is common for columns that store supplementary values, like the email address or phone number of a customer. When a customer email address or phone number changes, the dimension table

updates the customer row with the new values. It's as if the customer always had this contact information.

A non-incremental refresh of a Power BI model dimension-type table achieves the result of a Type 1 SCD. It refreshes the table data to ensure the latest values are loaded.

Type 2 SCD

A **Type 2 SCD** supports versioning of dimension members. If the source system doesn't store versions, then it's usually the data warehouse load process that detects changes, and appropriately manages the change in a dimension table. In this case, the dimension table must use a surrogate key to provide a unique reference to a *version* of the dimension member. It also includes columns that define the date range validity of the version (for example, **StartDate** and **EndDate**) and possibly a flag column (for example, **IsCurrent**) to easily filter by current dimension members.

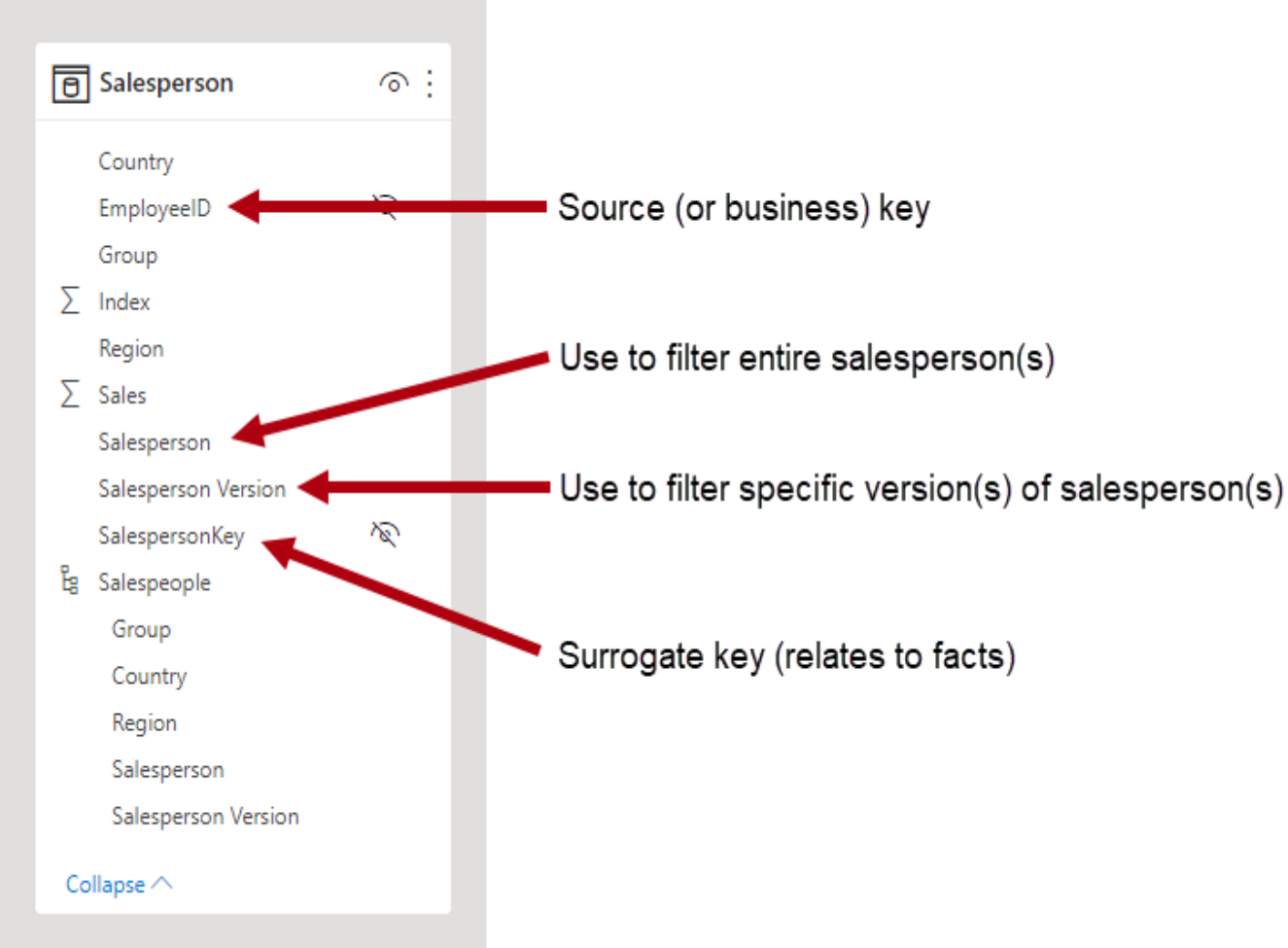
For example, Adventure Works assigns salespeople to a sales region. When a salesperson relocates region, a new version of the salesperson must be created to ensure that historical facts remain associated with the former region. To support accurate historic analysis of sales by salesperson, the dimension table must store versions of salespeople and their associated region(s). The table should also include start and end date values to define the time validity. Current versions may define an empty end date (or 12/31/9999), which indicates that the row is the current version. The table must also define a surrogate key because the business key (in this instance, employee ID) won't be unique.

It's important to understand that when the source data doesn't store versions, you must use an intermediate system (like a data warehouse) to detect and store changes. The table load process must preserve existing data and detect changes. When a change is detected, the table load process must expire the current version. It records these changes by updating the **EndDate** value and inserting a new version with the **StartDate** value commencing from the previous **EndDate** value. Also, related facts must use a time-based lookup to retrieve the dimension key value relevant to the fact date. A Power BI model using Power Query can't produce this result. It can, however, load data from a pre-loaded SCD Type 2 dimension table.

The Power BI model should support querying historical data for a member, regardless of change, and for a version of the member, which represents a particular state of the member in time. In the context of Adventure Works, this design enables you to query the salesperson regardless of assigned sales region, or for a particular version of the salesperson.

To achieve this requirement, the Power BI model dimension-type table must include a column for filtering the salesperson, and a different column for filtering a specific version of the salesperson. It's important that the version column provides a non-ambiguous description, like "Michael Blythe (12/15/2008-06/26/2019)" or "Michael Blythe (current)". It's also important to educate report authors and consumers about the basics of SCD Type 2, and how to achieve appropriate report designs by applying correct filters.

It's also a good design practice to include a hierarchy that allows visuals to drill down to the version level.



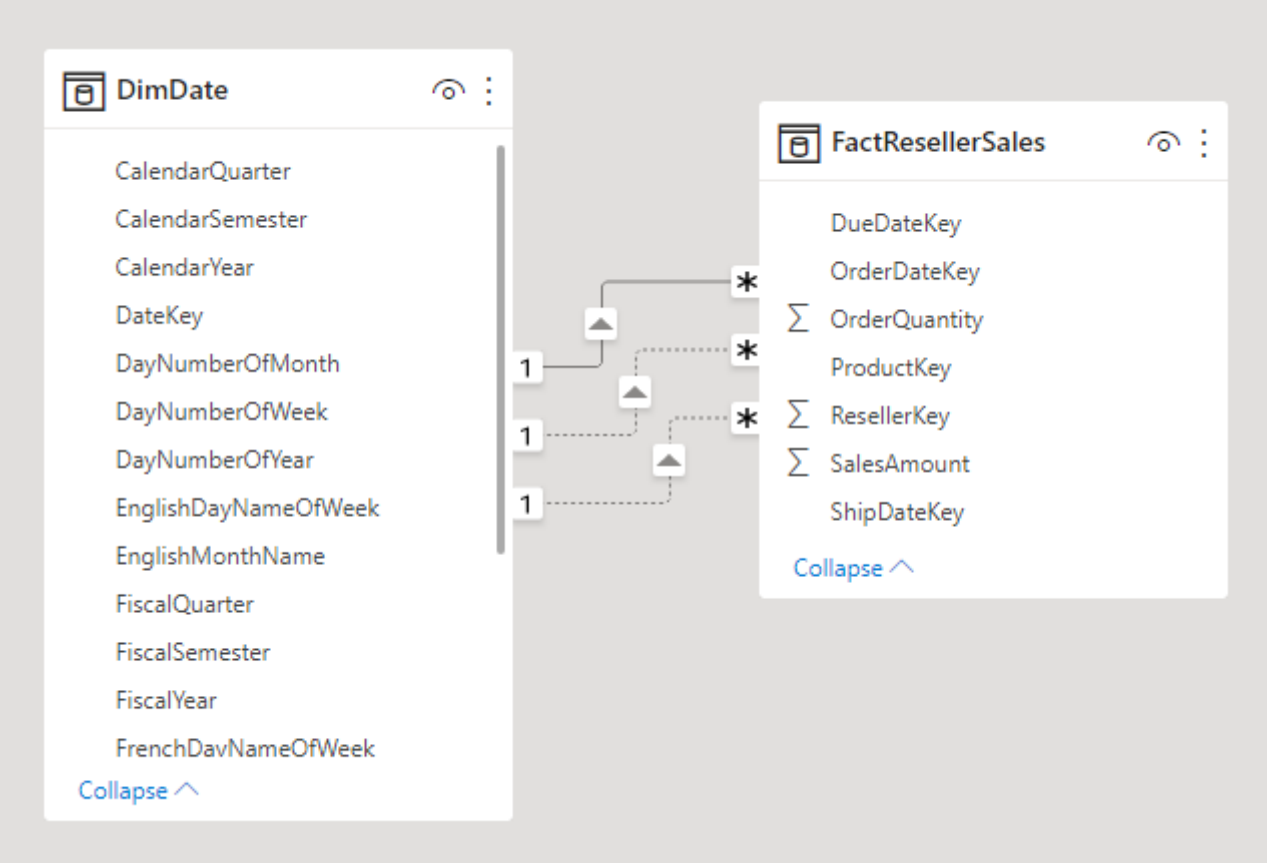
Group	Country	Region	Salesperson	Salesperson Version	Sales
North America	United States	Northeast	David Campbell	David Campbell (12/15/2008-06/26/2019)	$52,307.11
				David Campbell (06/27/2019-Current)	$119,807.56
				Total	**$172,114.67**
			Pamela Ansman-Wolfe	Pamela Ansman-Wolfe (03/03/2007-Current)	$222,087.01
				Total	**$222,087.01**
			Tete Mensa-Annan	Tete Mensa-Annan (01/09/2005-Current)	$23,098.4
				Total	**$23,098.4**
			Total		**$417,300.08**
		Total			**$417,300.08**
	Total				**$417,300.08**
Total					**$417,300.08**

Role-playing dimensions

A **role-playing dimension** is a dimension that can filter related facts differently. For example, at Adventure Works, the date dimension table has three relationships to the reseller sales facts. The same dimension table can be used to filter the facts by order date, ship date, or delivery date.

In a data warehouse, the accepted design approach is to define a single date dimension table. At query time, the "role" of the date dimension is established by which fact column you use to join the tables. For example, when you analyze sales by order date, the table join relates to the reseller sales order date column.

In a Power BI model, this design can be imitated by creating multiple relationships between two tables. In the Adventure Works example, the date and reseller sales tables would have three relationships. While this design is possible, it's important to understand that there can only be one active relationship between two Power BI model tables. All remaining relationships must be set to inactive. Having a single active relationship means there is a default filter propagation from date to reseller sales. In this instance, the active relationship is set to the most common filter that is used by reports, which at Adventure Works is the order date relationship.

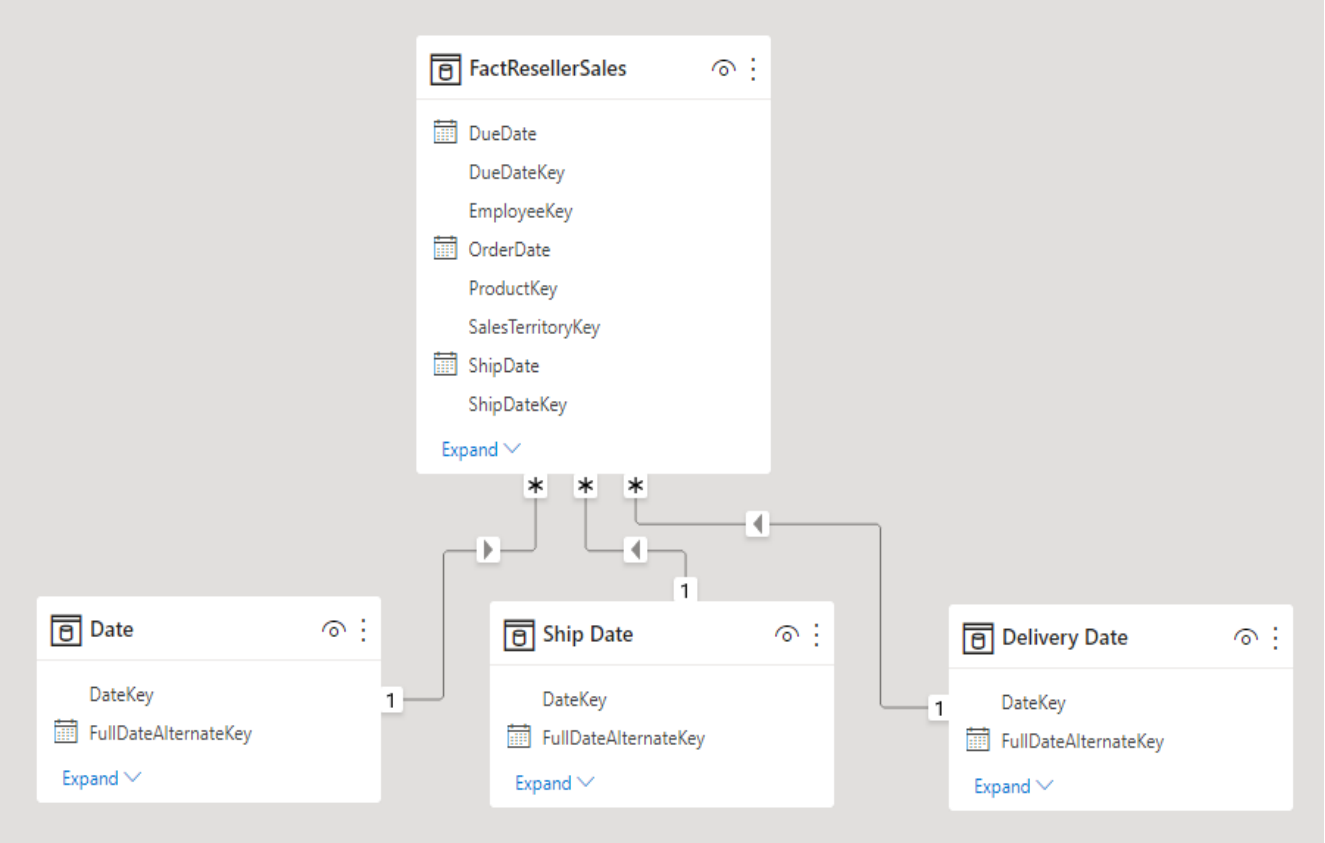


The only way to use an inactive relationship is to define a DAX expression that uses the USERELATIONSHIP function. In our example, the model developer must create measures to enable analysis of reseller sales by ship date and delivery date. This work can be tedious, especially when the reseller table defines many measures. It also creates **Fields** pane clutter, with an overabundance of measures. There are other limitations, too:

- When report authors rely on summarizing columns, rather than defining measures, they can't achieve summarization for the inactive relationships without writing a report-level measure. Report-level measures can only be defined when authoring reports in Power BI Desktop.

- With only one active relationship path between date and reseller sales, it's not possible to simultaneously filter reseller sales by different types of dates. For example, you can't produce a visual that plots order date sales by shipped sales.

To overcome these limitations, a common Power BI modeling technique is to create a dimension-type table for each role-playing instance. You typically create the additional dimension tables as calculated tables, using DAX. Using calculated tables, the model can contain a **Date** table, a **Ship Date** table and a **Delivery Date** table, each with a single and active relationship to their respective reseller sales table columns.



This design approach doesn't require you to define multiple measures for different date roles, and it allows simultaneous filtering by different date roles. A minor price to pay, however, with this design approach is that there will be duplication of the date dimension table resulting in an increased model storage size. As dimension-type tables typically store fewer rows relative to fact-type tables, it is rarely a concern.

Observe the following good design practices when you create model dimension-type tables for each role:

- Ensure that the column names are self-describing. While it's possible to have a **Year** column in all date tables (column names are unique within their table), it's not self-describing by default visual titles. Consider renaming columns in each dimension role table, so that the **Ship Date** table has a year column named **Ship Year**, etc.
- When relevant, ensure that table descriptions provide feedback to report authors (through **Fields** pane tooltips) about how filter propagation is configured. This clarity is

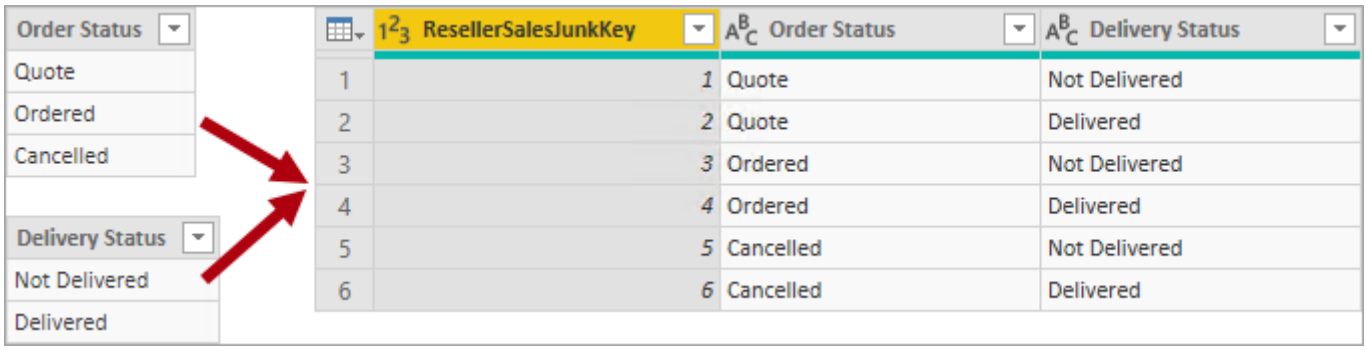
important when the model contains a generically named table, like **Date**, which is used to filter many fact-type tables. In the case that this table has, for example, an active relationship to the reseller sales order date column, consider providing a table description like "Filters reseller sales by order date".

Junk dimensions

A **junk dimension** is useful when there are many dimensions, especially consisting of few attributes (perhaps one), and when these attributes have few values. Good candidates include order status columns, or customer demographic columns (gender, age group, etc.).

The design objective of a junk dimension is to consolidate many "small" dimensions into a single dimension to both reduce the model storage size and also reduce **Fields** pane clutter by surfacing fewer model tables.

A junk dimension table is typically the Cartesian product of all dimension attribute members, with a surrogate key column. The surrogate key provides a unique reference to each row in the table. You can build the dimension in a data warehouse, or by using Power Query to create a query that performs full outer query joins, then adds a surrogate key (index column).



Order Status
Quote
Ordered
Cancelled

Delivery Status
Not Delivered
Delivered

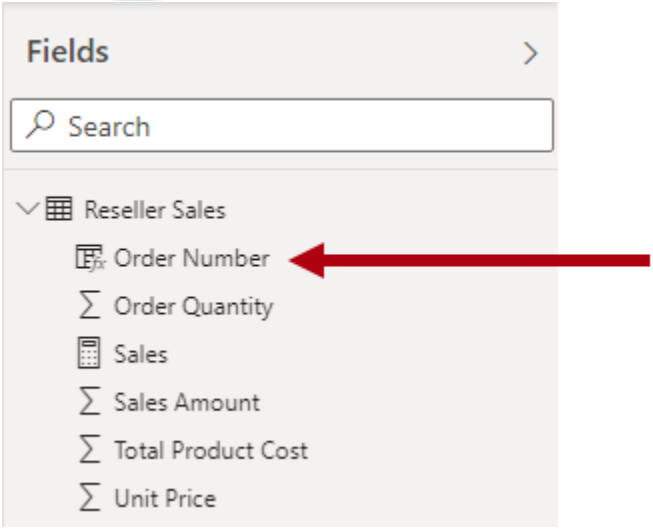
	ResellerSalesJunkKey	Order Status	Delivery Status
1	1	Quote	Not Delivered
2	2	Quote	Delivered
3	3	Ordered	Not Delivered
4	4	Ordered	Delivered
5	5	Cancelled	Not Delivered
6	6	Cancelled	Delivered

You load this query to the model as a dimension-type table. You also need to merge this query with the fact query, so the index column is loaded to the model to support the creation of a "one-to-many" model relationship.

Degenerate dimensions

A **degenerate dimension** refers to an attribute of the fact table that is required for filtering. At Adventure Works, the reseller sales order number is a good example. In this case, it doesn't make good model design sense to create an independent table consisting of just this one column, because it would increase the model storage size and result in **Fields** pane clutter.

In the Power BI model, it can be appropriate to add the sales order number column to the fact-type table to allow filtering or grouping by sales order number. It is an exception to the formerly introduced rule that you should not mix table types (generally, model tables should be either dimension-type or fact-type).



However, if the Adventure Works resellers sales table has order number *and* order line number columns, and they're required for filtering, a degenerate dimension table would be a good design. For more information, see One-to-one relationship guidance (Degenerate dimensions).

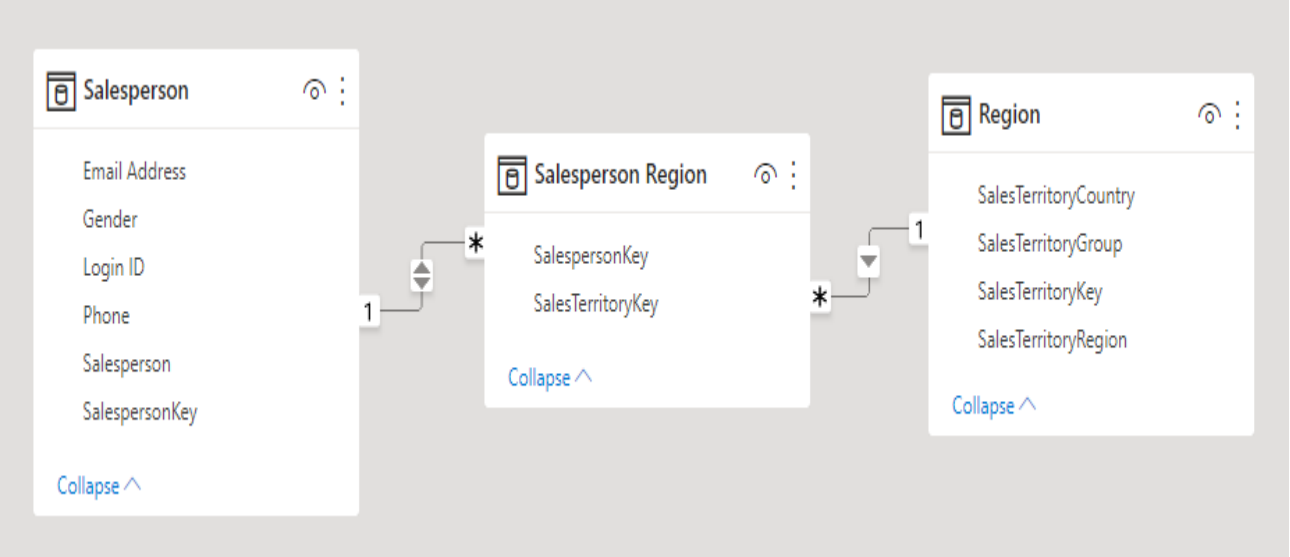
Factless fact tables

A **factless fact** table doesn't include any measure columns. It contains only dimension keys.

A factless fact table could store observations defined by dimension keys. For example, at a particular date and time, a particular customer logged into your web site. You could define a measure to count the rows of the factless fact table to perform analysis of when and how many customers have logged in.

A more compelling use of a factless fact table is to store relationships between dimensions, and it's the Power BI model design approach we recommend defining many-to-many dimension relationships. In a many-to-many dimension relationship design, the factless fact table is referred to as a *bridging table*.

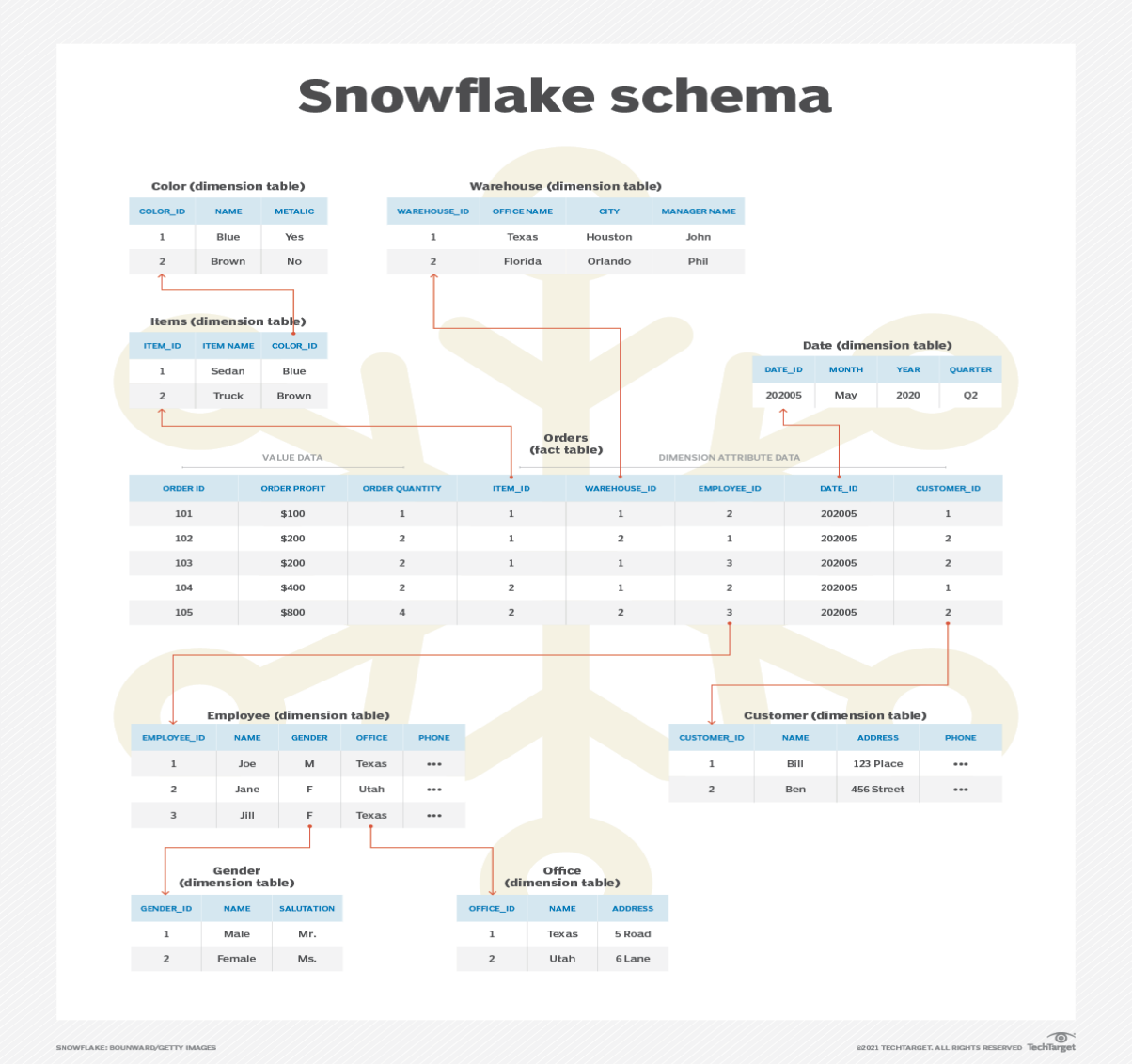
For example, consider that salespeople can be assigned to one *or more* sales regions. The bridging table would be designed as a factless fact table consisting of two columns: salesperson key and region key. Duplicate values can be stored in both columns.

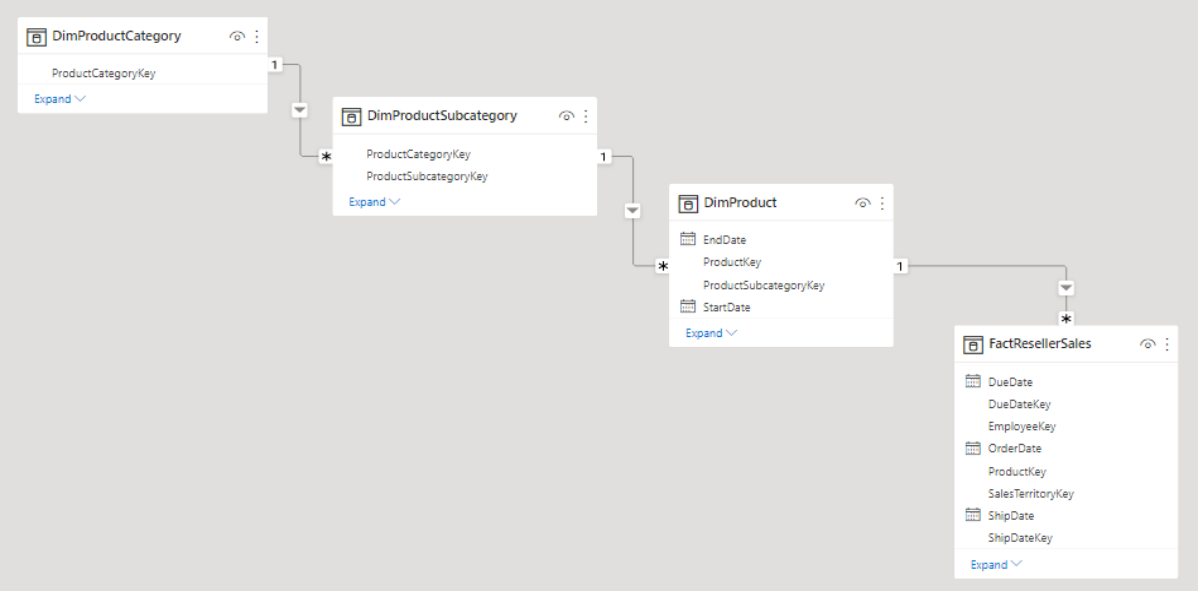


This many-to-many design approach is well documented, and it can be achieved without a bridging table. However, the bridging table approach is considered the best practice when relating two dimensions.

4.15 Snowflake Schema

A snowflake schema is a set of normalized tables for a single business entity. For example, Adventure Works classifies products by category and subcategory. Products are assigned to subcategories, and subcategories are in turn assigned to categories. In the Adventure Works relational data warehouse, the product dimension is normalized and stored in three related tables: Dim Product Category, Dim Product Subcategory, and Dim Product.

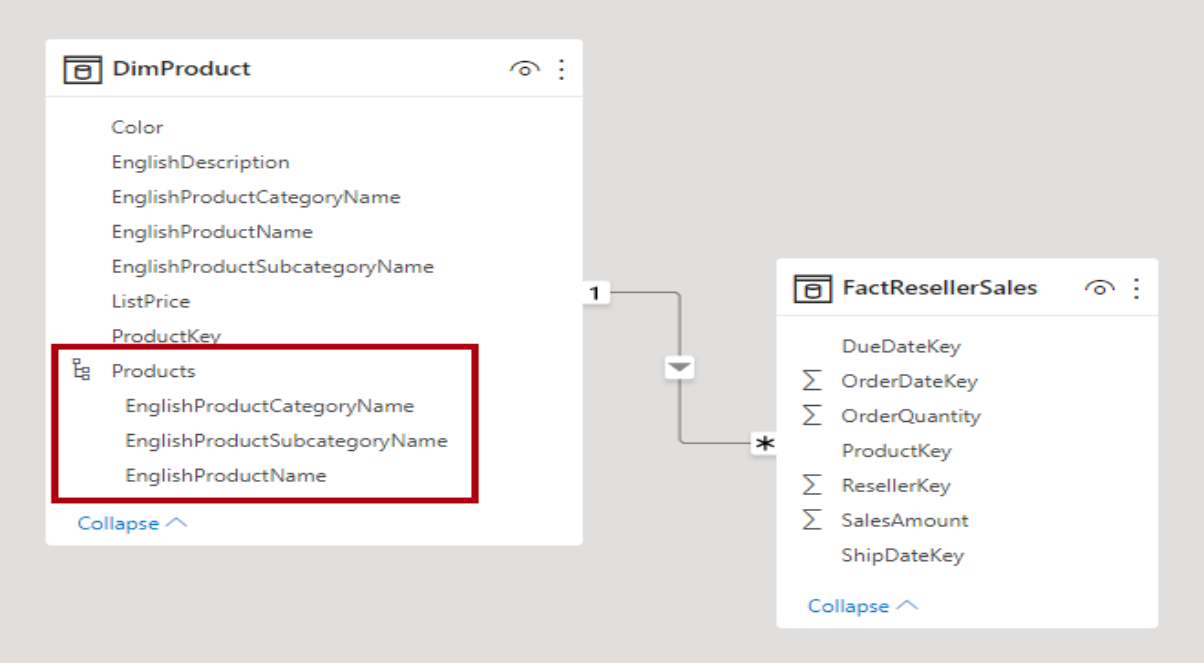




Brought to you by:

In Power BI Desktop, you can choose to mimic a snowflake dimension design (perhaps because your source data does) or integrate (denormalize) the source tables into a single model table. Generally, the benefits of a single model table outweigh the benefits of multiple model tables. The most optimal decision can depend on the volumes of data and the usability requirements for the model.

When you choose to integrate into a single model table, you can also define a hierarchy that encompasses the highest and lowest grain of the dimension. Possibly, the storage of redundant denormalized data can result in increased model storage size, particularly for very large dimension tables.



4.16 Star schema vs. snowflake schema

Star schema's dimension tables do not contain any foreign keys. That is, the dimension tables do not reference any other tables, nor do they have any "sub-dimension tables." They are generally denormalized because some information may be duplicated in the dimension tables. This allows star schema databases to be optimized for read and query performance along specific dimensions.

A snowflake schema database is similar to a star schema in that it has a single fact table and many dimension tables. However, for a snowflake schema, each dimension table might have foreign keys that relate to other dimension tables.

In a star schema, the record would contain information such as "male" and "Texas Office." This would be duplicated data also in other rows for employees with the same gender or branch office.

In a snowflake schema, the gender or branch office would contain a foreign key value to a gender dimension table and a branch office dimension table. These could contain information such as gender, name, salutation (Mr. or Ms.), branch office name, branch office address or branch manager.

4.17 Create and manage relationships

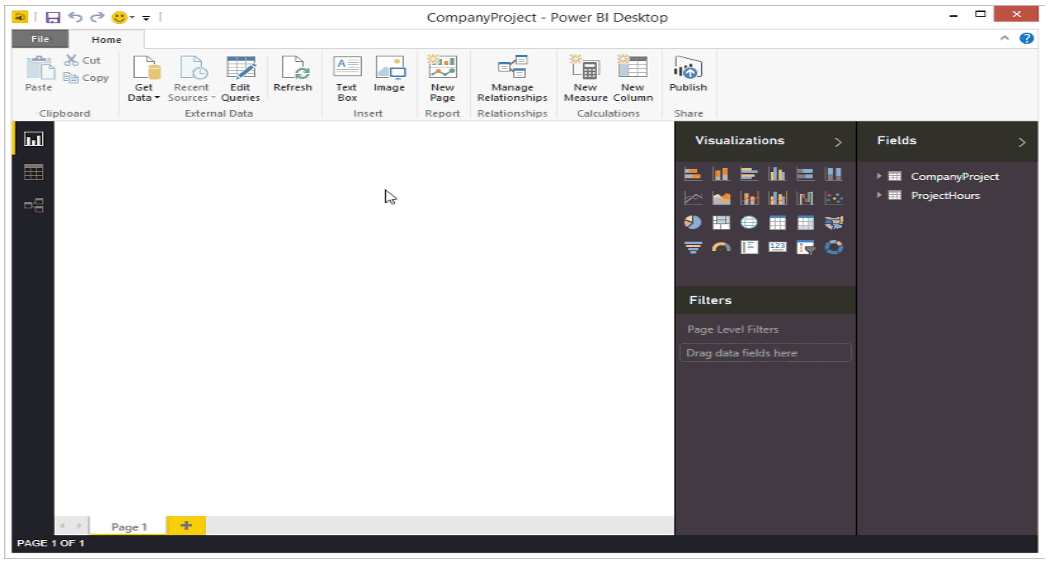
When you import multiple tables, chances are you'll do some analysis using data from all those tables. Relationships between those tables are necessary to accurately calculate results and display the correct information in your reports. Power BI Desktop makes creating those relationships easy. In fact, in most cases you won't have to do anything, the autodetect feature does it for you. However, sometimes you might have to create relationships yourself, or need to make changes to a relationship. Either way, it's important to understand relationships in Power BI Desktop and how to create and edit them.

Auto detect during load

If you query two or more tables at the same time, when the data is loaded, Power BI Desktop attempts to find and create relationships for you. The relationship options Cardinality, Cross filter direction, and Make this relationship active are automatically set. Power BI Desktop looks at column names in the tables you're querying to determine if there are any potential relationships. If there are, those relationships are created automatically. If Power BI Desktop can't determine with a high level of confidence there's a match, it doesn't create the relationship. However, you can still use the Manage relationships dialog box to manually create or edit relationships.

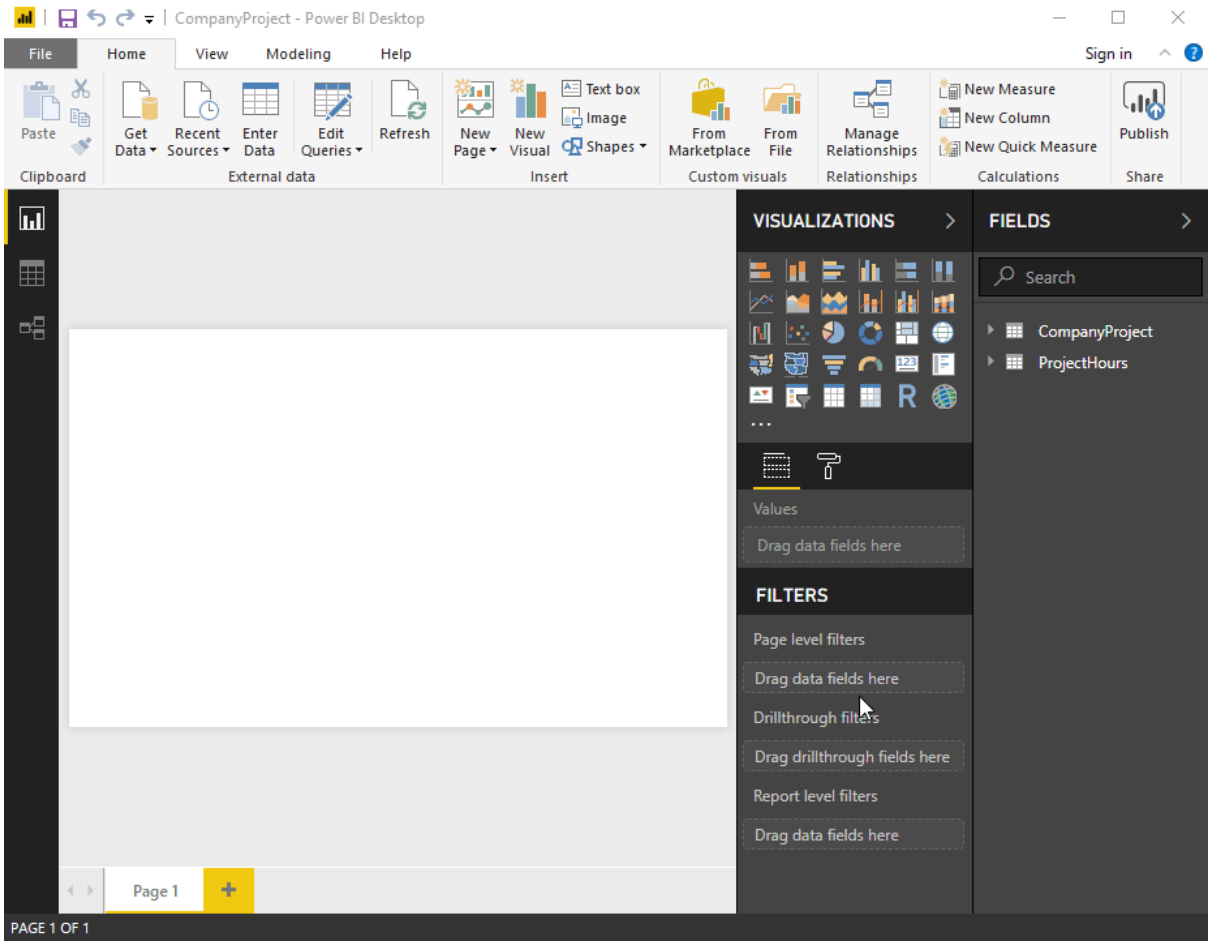
Create a relationship with auto detect:

On the Modelling tab, select Manage relationships > Auto detect.



Create a relationship manually:

- On the Modelling tab, select Manage relationships > New.
- In the Create relationship dialog box, in the first table drop-down list, select a table. Select the column you want to use in the relationship.
- In the second table drop-down list, select the other table you want in the relationship. Select the other column you want to use, and then select OK.

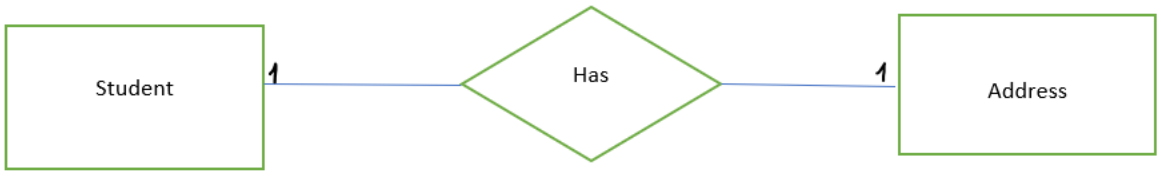


4.18 Types of relationships

One to one (1:1):

One to one relationship means one entity in a table is related to just one entity in another table. It should be rare in any relational database design.

For instance, each student has only one address and each address represents one student.



A one-to-one relationship can be created when both tables each contain a column of common and unique values.

There are two scenarios that involve one-to-one relationships:

- Degenerate dimensions: You can derive a degenerate dimension from a fact-type table.
- Row data spans across tables: A single business entity or subject is loaded as two (or more) model tables, possibly because their data is sourced from different data stores. This scenario can be common for dimension-type tables. For example, master product details are stored in an operational sales system, and supplementary product details are stored in a different source.

  It's unusual, however, that you'd relate two fact-type tables with a one-to-one relationship. It's because both fact-type tables would need to have the same dimensionality and granularity. Also, each fact-type table would need unique columns to allow the model relationship to be created.

Degenerate dimensions

When columns from a fact-type table are used for filtering or grouping, you can consider making them available in a separate table. This way, you separate columns used for filter or grouping, from those columns used to summarize fact rows.

This separation can:

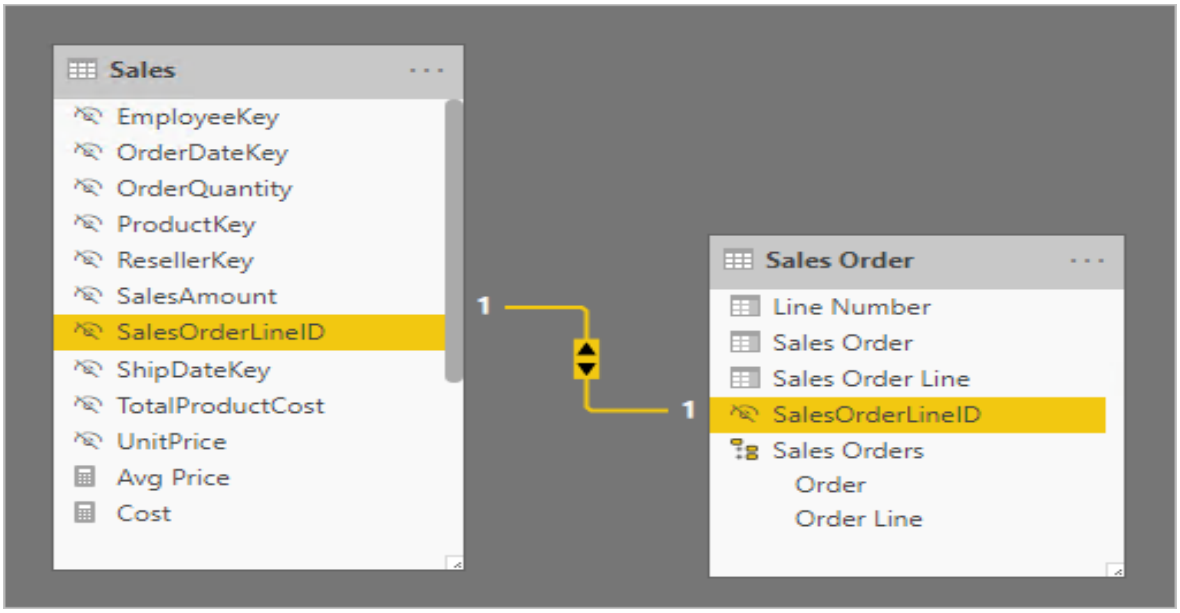
- Reduce storage space
- Simplify model calculations
- Contribute to improved query performance
- Deliver a more intuitive Fields pane experience to your report authors

Consider a source sales table that stores sales order details in two columns.

	OrderNumber	OrderLineNumber	OrderDate	DueDate	ShipDate	ProductID	CustomerID
1	43659	1	2018-12-29	2019-01-10	2019-01-05	349	676
2	43659	2	2018-12-29	2019-01-10	2019-01-05	350	676
3	43659	3	2018-12-29	2019-01-10	2019-01-05	351	676
4	43660	1	2018-12-29	2019-01-10	2019-01-05	326	117
5	43660	2	2018-12-29	2019-01-10	2019-01-05	319	117

The Order Number column stores the order number, and the Order Line Number column stores a sequence of lines within the order. In the following model diagram, notice that the

order number and order line number columns haven't been loaded to the Sales table. Instead, their values were used to create a surrogate key column named Sales Order Line ID. (The key value is calculated by multiplying the order number by 1000, and then adding the order line number.)



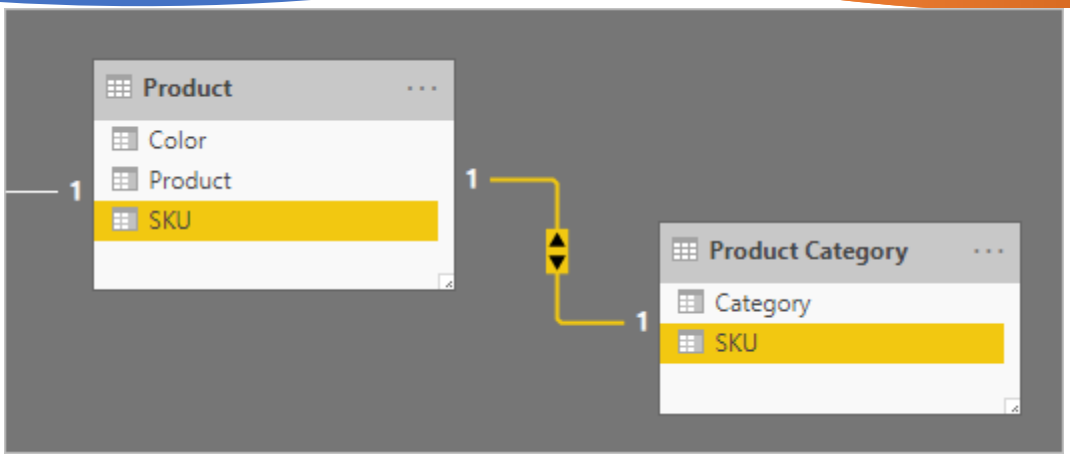
The Sales Order table provides a rich experience for report authors with three columns: Sales Order, Sales Order Line, and Line Number. It also includes a hierarchy. These table resources support report designs that need to filter, group by, or drill down through orders and order lines.

As the Sales Order table is derived from the sales data, there should be exactly the same number of rows in each table. Further, there should be matching values between each Sales Order Line ID column.

Row data spans across tables

Consider an example involving two one-to-one related dimension-type tables: Product, and Product Category. Each table represents imported data and has a SKU (Stock-Keeping Unit) column containing unique values.

Here's a partial model diagram of the two tables.



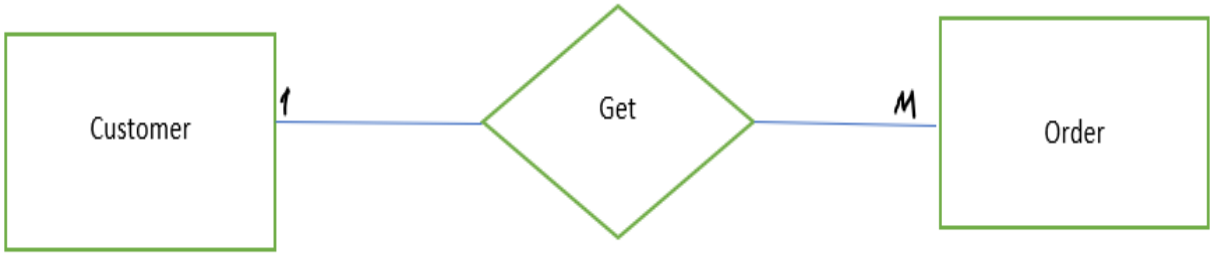
The first table is named Product, and it contains three columns: Color, Product, and SKU. The second table is named Product Category, and it contains two columns: Category, and SKU. A one-to-one relationship relates the two SKU columns. The relationship filters in both directions, which is always the case for one-to-one relationships.

To help describe how the relationship filter propagation works, the model diagram has been modified to reveal the table rows. All examples in this article are based on this data.

One to Many (1 : M) Relationship:

One to many relationship means one entity in a table is related to one or many entities in another table. A one to many (1 : M) relationship should be the norm in any relational database design and is found in all relational database environments.

For example, each customer has one or more orders.

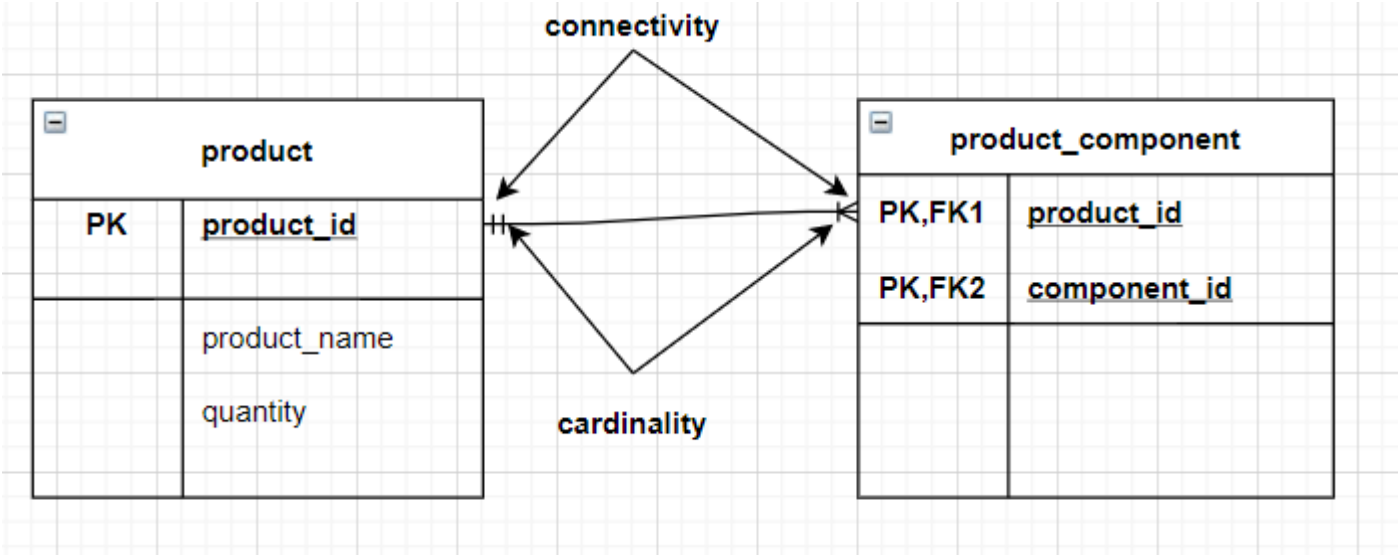


Before proceeding many to many relationships, let's look at the cardinality and connectivity. These are helpful to understand many to many relationships.

Cardinality and Connectivity

Cardinality describes the relationship between two data tables by expressing the minimum and maximum number of entity occurrences associated with one occurrence of a related entity. Either there is a relationship or not.

Connectivity is the relationship between two tables. It can be one to one or one to many.

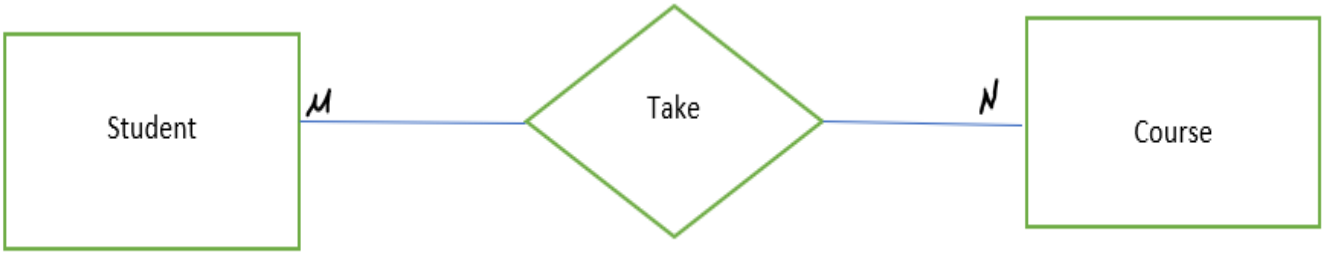


Many to one (*:1): A many-to-one relationship is the most common, default type of relationship. It means the column in a given table can have more than one instance of a value, and the other related table, often know as the lookup table, has only one instance of a value.

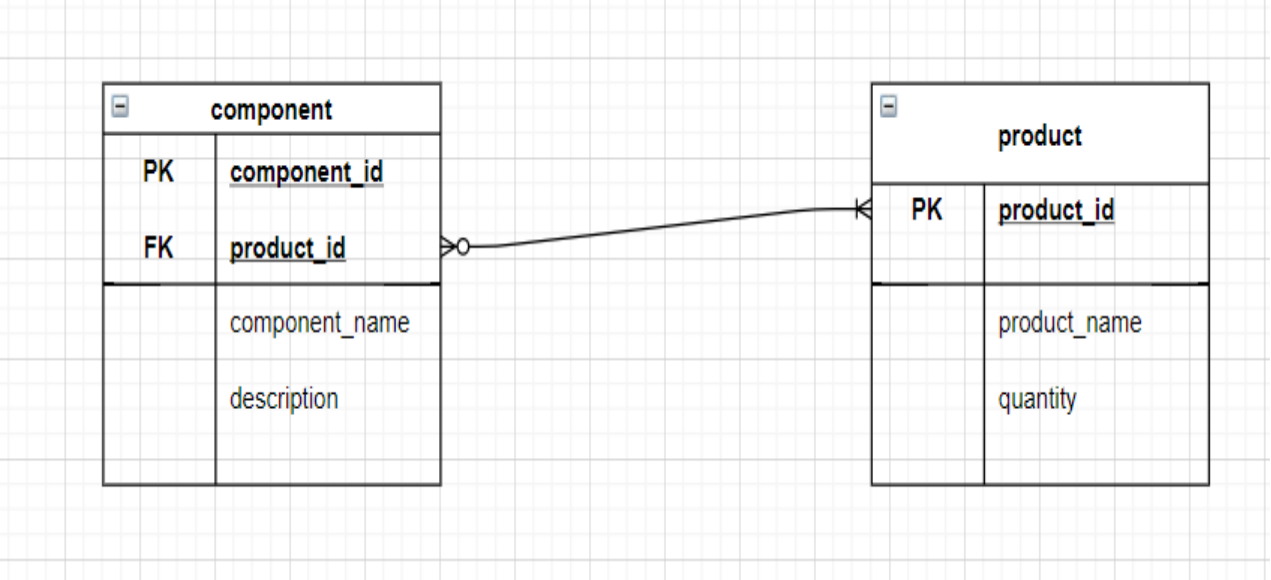
Many to Many (M : N) Relationship

Many to many relationship means multiple entities in a table can be associated with more than one entity in another table.

For instance, each student can attend many courses and, each course can consist of many students.



Many to many relationship causes duplications in database. Duplications cause false results from queries. For instance, a company produces products, which consist of several components. One product is made up of many components, and one component can be in more than one product.

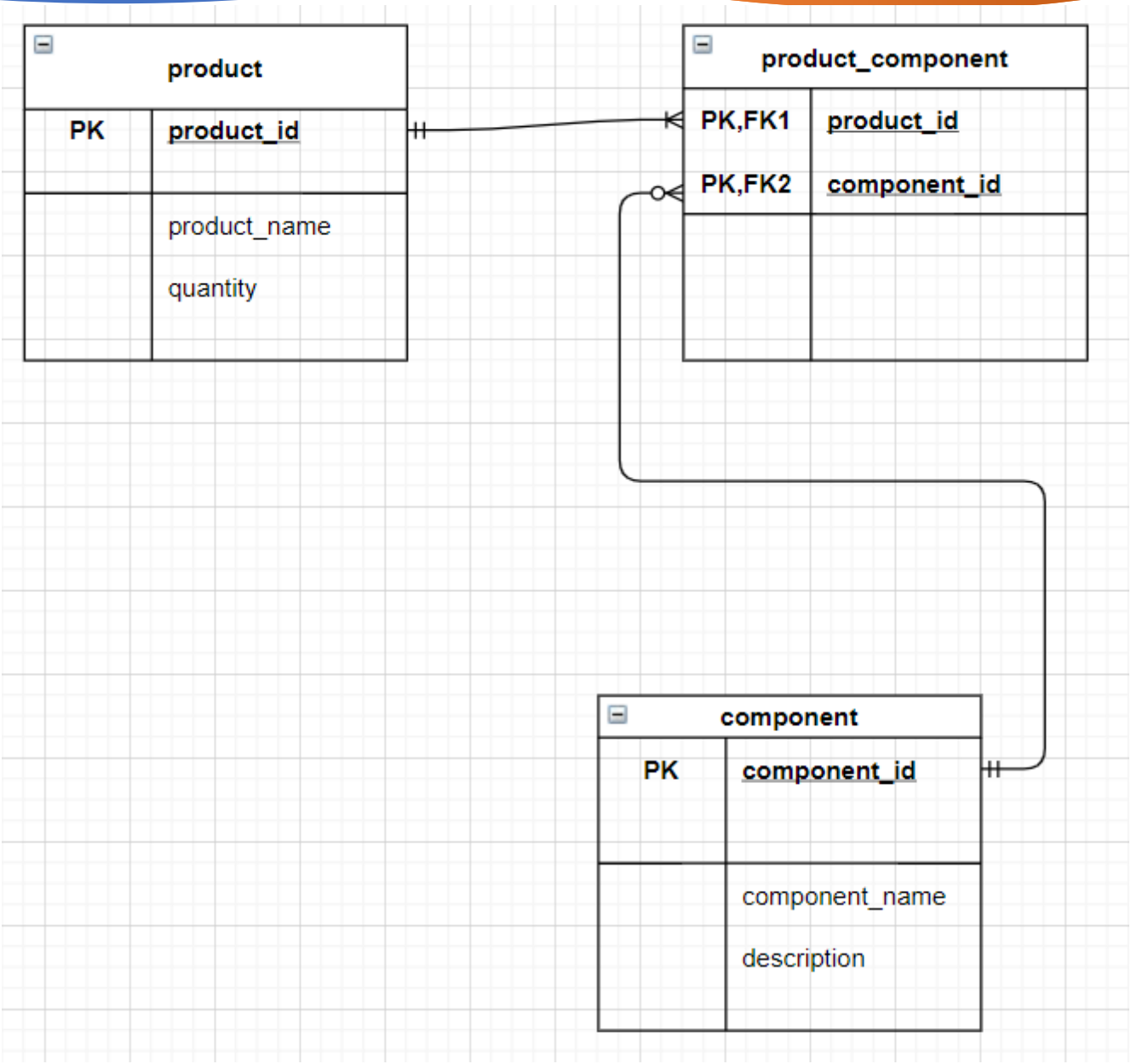


In the Entity-Relationship (ER) Diagram, "component_id" is specified as Primary Key in a component table, and Primary Key should be unique. Looking at the component table below, the primary key(component_id), component_name, and description are duplicated in order to record detailed information about each component.

Product		
product_id	product_name	quantity
1	x	2
2	y	3
3	z	2

Component			
component_id	product_id	component_name	description
1	1	a	b
1	2	a	b
1	3	a	b
2	1	d	e

Let's explain a way to prevent the duplication while setting many to many relationships. The solution is creating a totally new table. It's called as bridge table or join table.


product
PK
product_id
product_name
quantity
product_component
PK,FK1
product_id
PK,FK2
component_id
component
PK
component_id
component_name
description

Product		
product_id	product_name	quantity
1	x	2
2	y	3
3	z	2

Component		
component_id	component_name	description
1	a	b
2	d	e

Product_Component		
product_id	component_id	type
1	1	k
2	1	l
3	1	m
1	2	n

Bridge table consist of “product_id” , “component_id” from related tables’ Primary Keys. “product_id” and “component_id” are both Primary and Foreign Keys the bridge table. Also, new columns can be added that are not available in the product table and component table.
Entity-Relationship (ER) Diagram helps to make and understand Entity Relationship (ER) Data Model. This site which is the below can be used for Entity Relationship (ER) Data Modelling with Entity-Relationship (ER) Diagram. This website helps to create and visualize diagram. The visualization makes it easier to understand the relationship between the tables. Also, this website is free and speedy. It has to many options to create diagram.

To sum up, relational database has ability to create meaningful information by joining tables. Joining tables help to understand the relationship between the data and tables. Besides, relational databases eliminate data redundancy. Relational database has three relationships between tables. Many to many relationship is harder to understand than other relationships. The bridge table makes the database and relationship easy to understand, and it prevents data redundancy.

Assignment:

Create data model relationship using Power BI

Solution:

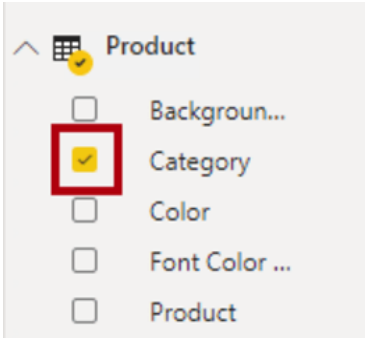
Creating Model Relationships

- Open the Power BI Desktop application and click on the “**File**” ribbon.

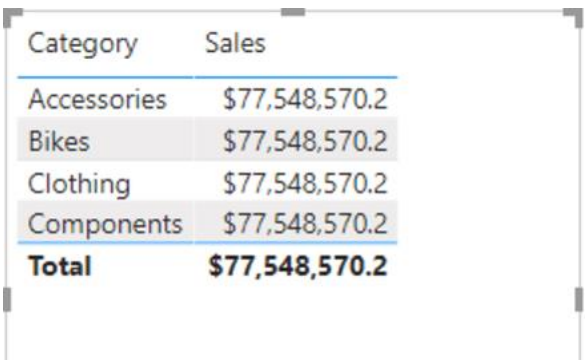
- Click on the "**Open**" option and import the data for making Power BI Data Model.
- For this tutorial, sample data from Sales Analysis will be used.
- Click on the "**Model View**" option from the left side of the Power BI, as shown in the image below.



- Here, you can view each table and relationship.
- In the "Fields" pane right-click on an empty area and click on the "Expand All" option to view all the table fields.
- Now, create a visual table by selecting the "Category" field inside the "Product" table, as shown in the image below.

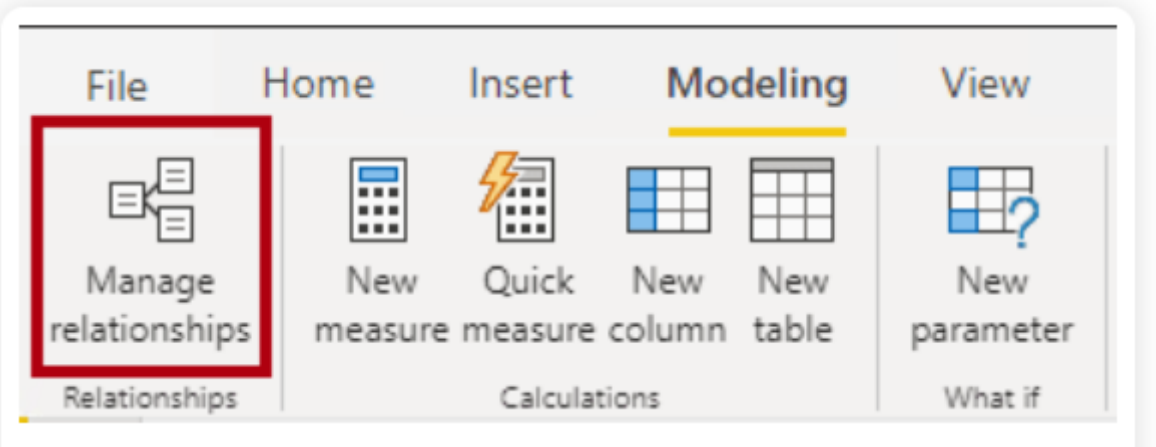


- For adding a column to the table, go to the "**Fields"** pane and select the "**Sales"** from the **"Sales"** table, as shown in the image below.

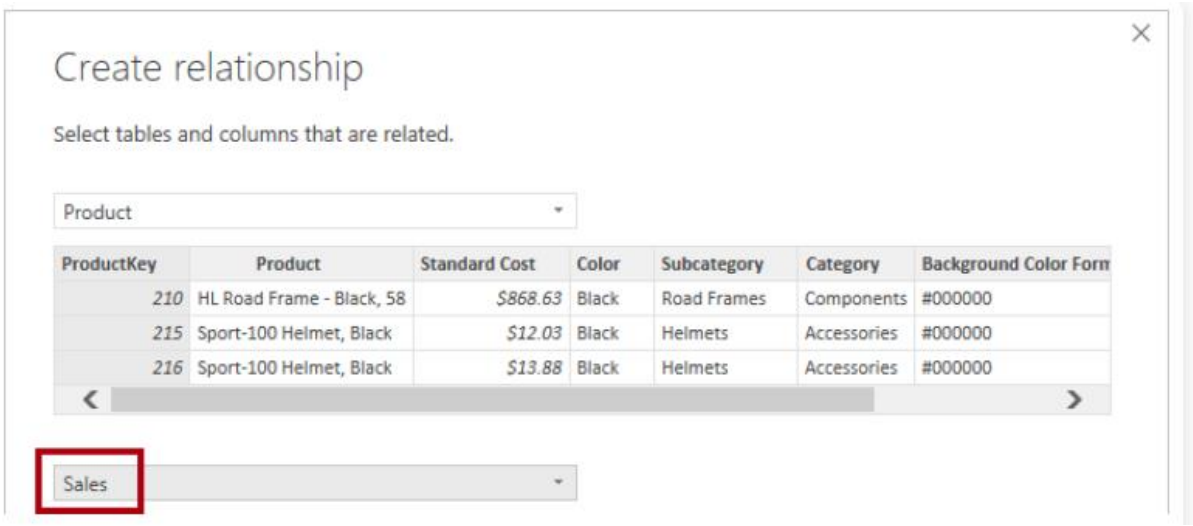


Category	Sales
Accessories	$77,548,570.2
Bikes	$77,548,570.2
Clothing	$77,548,570.2
Components	$77,548,570.2
Total	**$77,548,570.2**

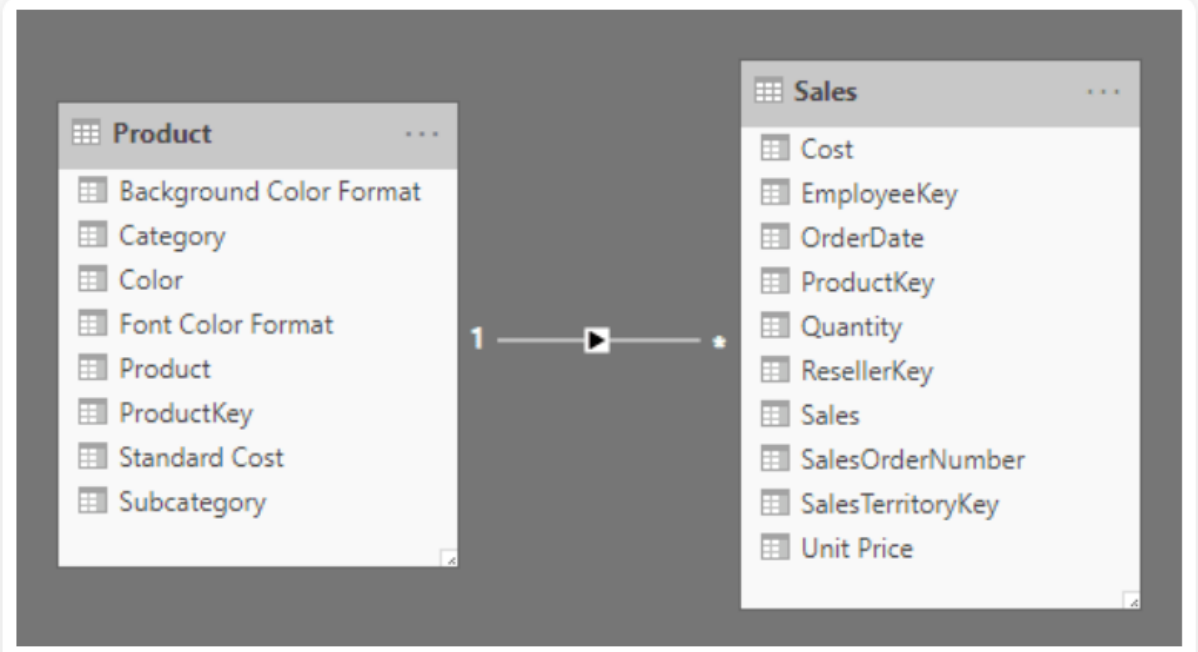
- There is no relationship between tables, so it will not filter the "**Sales**" table.
- Navigate to the "**Modeling**" ribbon, and from the "**Relationships**" group, select the "**Manage Relationships**" option, as shown in the image below.



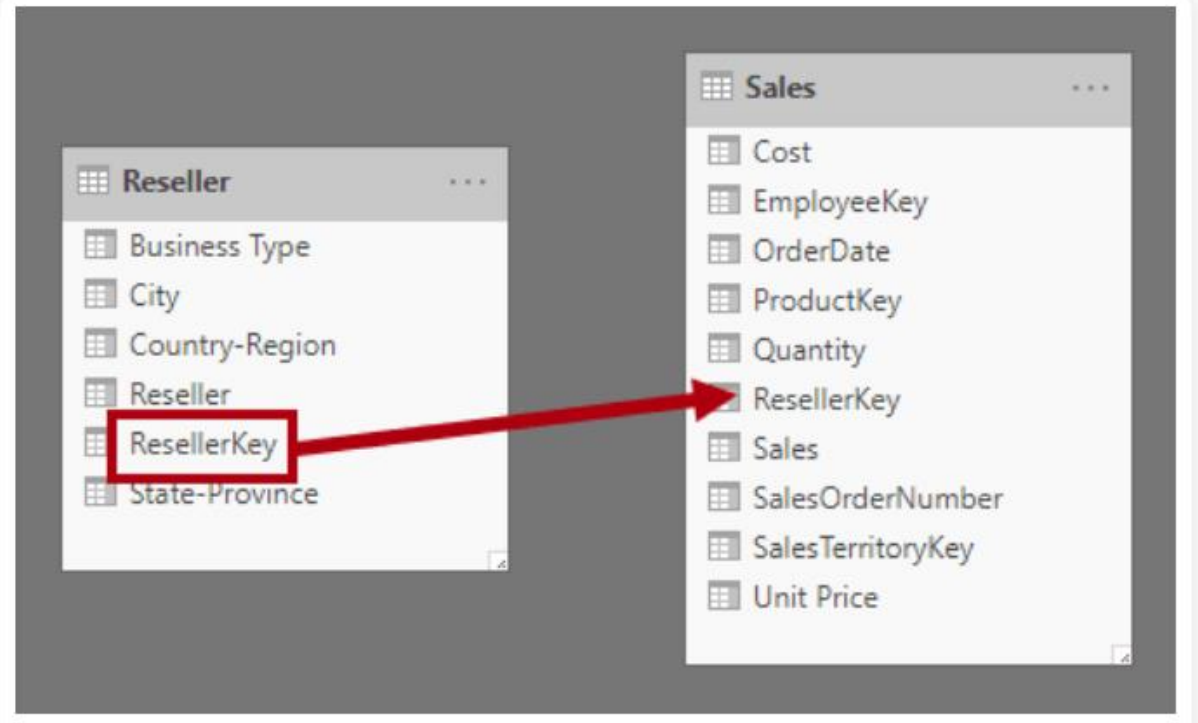
- Click on the "**New**" button to create a new relationship.
- It will open the "**Create Relationship**" window.
- Select the "**Product**" table from the first drop-down list, then select the "**Sales**" table from the second drop-down list, as shown in the image below.



- The "**ProductKey**" columns from both the tables are automatically selected as common columns because it has the same data type and name.
- Click on the "**Ok**" button.
- Now notice that the table visual has been updated to display values for each product category.
- If you open the "Model View", you will find a connector between the two tables that were not present before, as shown in the image below.



- Another way to create a relationship between tables is by dragging one column from one table to another.
- In this Model View, select the "**ResellerKey**" column from the "Reseller" table and drag it onto the "**Resellerkey**" column of the "**Sales**" table, as shown in the image below.



- Similarly, create the following relationships by dragging columns. listed below:
    - "**SalesTerritoryKey**" column from the "**Region**" table to "**SalesTerritoryKey**" column of "**Sales**" table.
    - "**EmployeeKey**" column from the "**Salesperson**" table to the "**EmployeeKey**" column of the "**Sales**" table.
- Now save the Power BI file.

Chapter: 5

Data Visualization

Scope:

- Overview of Power BI
- Advantages of using Power BI
- Getting Started (Power BI Desktop)
- Transforming Data
- Report Creation
- Chart Selection
- Different types of visualization charts
- Area Charts
- Line Charts
- Bar Charts
- Column Charts
- Combo Charts

Data visualization Overview

It is the representation of data through use of common graphics, such as charts, plots, infographics, and even animations. These visual displays of information communicate complex data relationships and data-driven insights in a way that is easy to understand.

Data visualization can be utilized for a variety of purposes, and it's important to note that is not only reserved for use by data teams. Management also leverages it to convey organizational structure and hierarchy while data analysts and data scientists use it to discover and explain patterns and trends. Harvard Business Review (link resides outside IBM) categorizes data visualization into four key purposes: idea generation, idea illustration, visual discovery, and everyday dataviz. We'll delve deeper into these below:

Data visualization is commonly used to spur idea generation across teams. They are frequently leveraged during brainstorming or Design Thinking sessions at the start of a project by supporting the collection of different perspectives and highlighting the common concerns of the collective. While these visualizations are usually unpolished and unrefined, they help set the foundation within the project to ensure that the team is aligned on the problem that they're looking to address for key stakeholders.

Types of data visualizations

The earliest form of data visualization can be traced back the Egyptians in the pre-17th century, largely used to assist in navigation. As time progressed, people leveraged data visualizations for broader applications, such as in economic, social, health disciplines. Perhaps most notably, Edward Tufte published The Visual Display of Quantitative Information (link resides outside IBM), which illustrated that individuals could utilize data visualization to present data in a more effective manner. His book continues to stand the test of time, especially as companies turn to dashboards to report their performance metrics in real-time. Dashboards are effective data visualization tools for tracking and visualizing data from multiple data sources, providing visibility into the effects of specific behaviors by a team or an adjacent one on performance. Dashboards include common visualization techniques, such as:

Tables: This consists of rows and columns used to compare variables. Tables can show a great deal of information in a structured way, but they can also overwhelm users that are simply looking for high-level trends.

Pie charts and stacked bar charts: These graphs are divided into sections that represent parts of a whole. They provide a simple way to organize data and compare the size of each component to one other.

Line graphs and area charts: These visuals show change in one or more quantities by plotting a series of data points over time. Line graphs utilize lines to demonstrate these changes while area charts connect data points with line segments, stacking variables on top of one another and using color to distinguish between variables.

Histograms: This graph plots a distribution of numbers using a bar chart (with no spaces between the bars), representing the quantity of data that falls within a particular range. This visual makes it easy for an end user to identify outliers within a given dataset.

Scatter plots: These visuals are beneficial in reveling the relationship between two variables, and they are commonly used within regression data analysis. However, these can sometimes be confused with bubble charts, which are used to visualize three variables via the x-axis, the y-axis, and the size of the bubble.

Heat maps: These graphical displays are helpful in visualizing behavioral data by location. This can be a location on a map, or even a webpage.

Tree maps: which display hierarchical data as a set of nested shapes, typically rectangles. Tree maps are great for comparing the proportions between categories via their area size.

Open-source visualization tools

Access to data visualization tools has never been easier. Open source libraries, such as D3.js, provide a way for analysts to present data in an interactive way, allowing them to engage a broader audience with new data. Some of the most popular open source visualization libraries include:

D3.js: It is a JavaScript library for producing dynamic, interactive data visualizations in web browsers. D3.js (link resides outside IBM) uses HTML, CSS, and SVG to create visual representations of data that can be viewed on any browser. It also provides features for interactions and animations.

ECharts: A powerful charting and visualization library that offers an easy way to add intuitive, interactive, and highly customizable charts to products, research papers, presentations, etc. Echarts (link resides outside IBM) is based in JavaScript and ZRender, a lightweight canvas library.

Vega: Vega (link resides outside IBM) defines itself as “visualization grammar,” providing support to customize visualizations across large datasets which are accessible from the web.

deck.gl: It is part of Uber's open source visualization framework suite. deck.gl (link resides outside IBM) is a framework, which is used for exploratory data analysis on big data. It helps build high-performance GPU-powered visualization on the web.

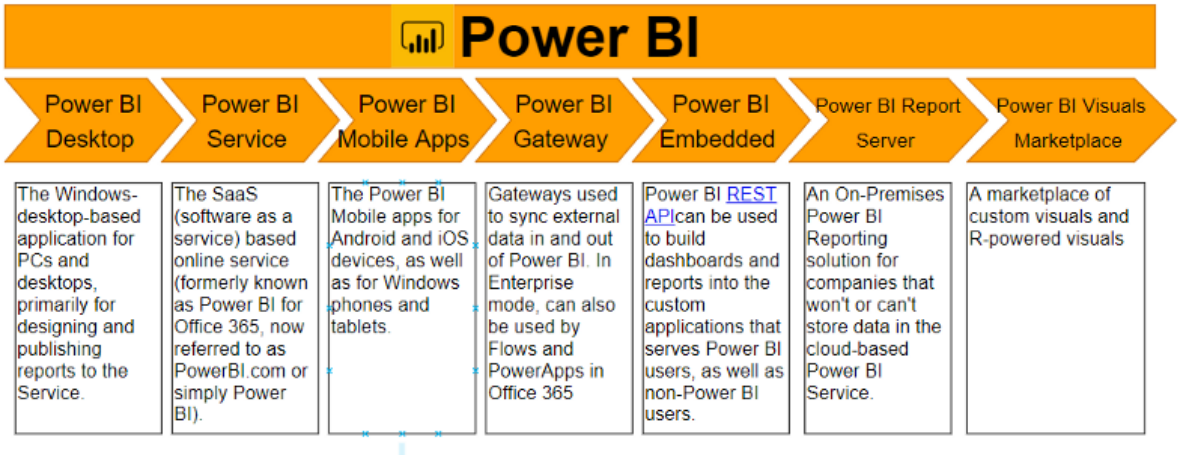
5.1 Overview of Power BI

Power BI gives the ability to analyze and explore data on-premise as well as in the cloud. Power BI provides the ability to collaborate and share customized dashboards and interactive reports across colleagues and organizations, easily and securely.



Power BI's Components:

Power BI consists of various components which are available in the market separately and can be used exclusively.



Power BI Desktop

The Windows desktop-based application for PCs and desktops, primarily for designing and publishing reports to the Service.

Power BI Service

The SaaS-based (software as a service) online service. This was formerly known as Power BI for Office 365, now referred to as PowerBI.com, or simply Power BI.

Power BI Mobile Apps

The Power BI Mobile apps for Android and iOS devices, as well as for Windows phones and tablets.

Power BI Gateway

Gateways are used to sync external data in and out of Power BI and are required for automated refreshes. In Enterprise mode, can also be used by Power Automate (previously called Flows) and PowerApps in Office 365.

Power BI Embedded

Power BI REST API can be used to build dashboards and reports into the custom applications that serves Power BI users, as well as non-Power BI users.

Power BI Report Server

An on-premises Power BI reporting solution for companies that won't or can't store data in the cloud-based Power BI Service.

Power BI Premium

Capacity-based offering that includes flexibility to publish reports broadly across an enterprise, without requiring recipients to be licensed individually per user. Greater scale and performance than shared capacity in the Power BI Service.

Power BI Visuals Marketplace

A marketplace of custom visuals and R-powered visuals.

Power BI Dataflow

A Power Query implementation in the cloud that can be used for data transformations to make a common Power BI Dataset that can be made available for several report developers through Microsoft's Common Data Service. It can be used as an alternative to for example doing transformations in SSAS and may ensure that several report developers use data that has been transformed in a similar way.

Power BI Dataset

A Power BI Dataset can work as a collection of data for use in Power BI reports, and can either be connected to or imported into a Power BI Report. A Dataset can be connected to and get its source data through one or more Data flows.

5.2 Advantages of using Power BI:

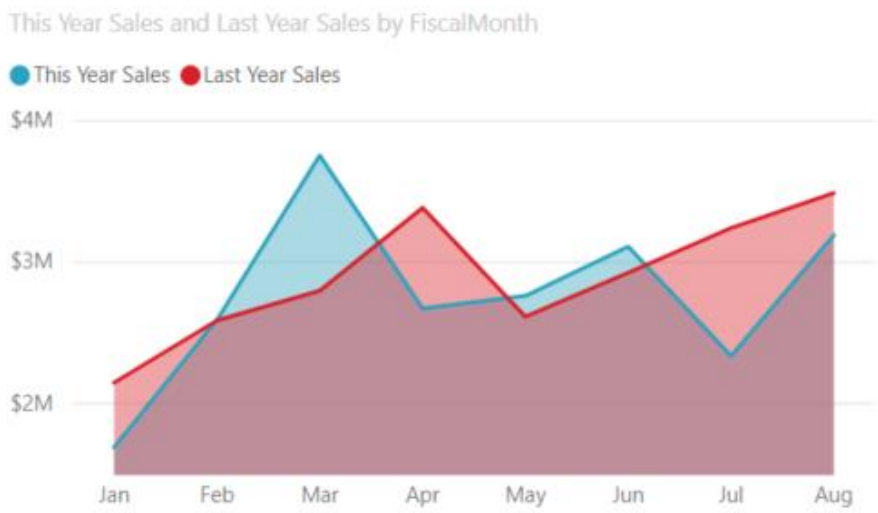
- **A quick start**. You'll be able to get insights quickly with an uncomplicated setup, no required training, and included dashboards for services such as Salesforce, Google Analytics, and Microsoft Dynamics.
- **Streamlined publication and distribution**. Instead of emailing large files or putting them on a shared drive, analysts upload reports and visualizations to the Power BI service, and their data is refreshed whenever the underlying dataset is updated.
- **Real-time information**. Dashboards update in real time, as data is pushed or streamed in, which gives viewers the ability to solve problems and identify opportunities quickly. Any report or dashboard can display and update real-time data and visuals. Sources of streaming data can be factory sensors, social media sources, or anything from which time-sensitive data can be collected or transmitted.
- **Ability to customize Power BI app navigation**. An "app navigation experiences" feature gives report developers the power to customize navigation to help viewers

find content quickly and understand the relationships between different reports and dashboards.

- **Ability to customize security features**. Report developers can set up row-level security (RLS) access filters to ensure that viewers see only data relevant to them, mitigating the risk of people seeing data they shouldn't.
- **Cortana integration**. Power BI works with Microsoft's digital assistant, Cortana. Users can verbally ask questions in natural language to access charts and graphs. This can be especially helpful for users with mobile devices.
- **Artificial Intelligence**. Power BI users can access image recognition and text analytics, create machine learning models, and integrate with Azure Machine Learning.

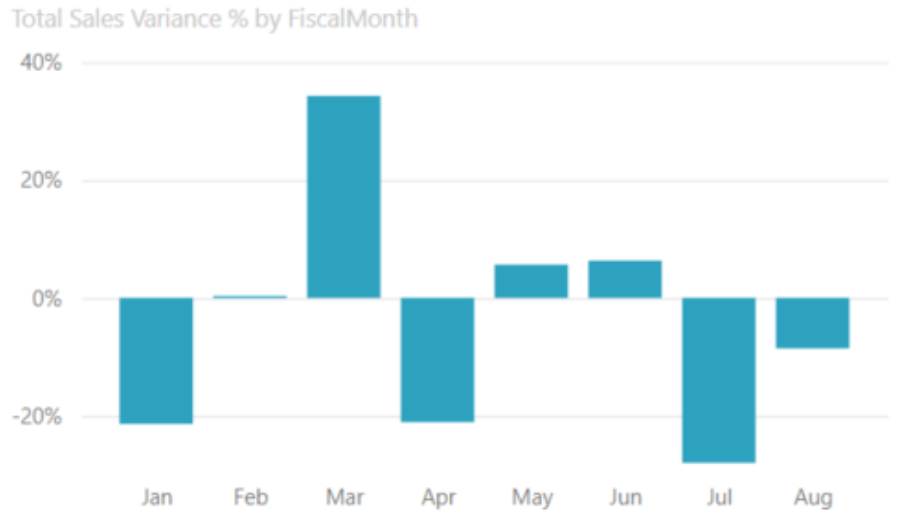
5.3 Chart Selection:

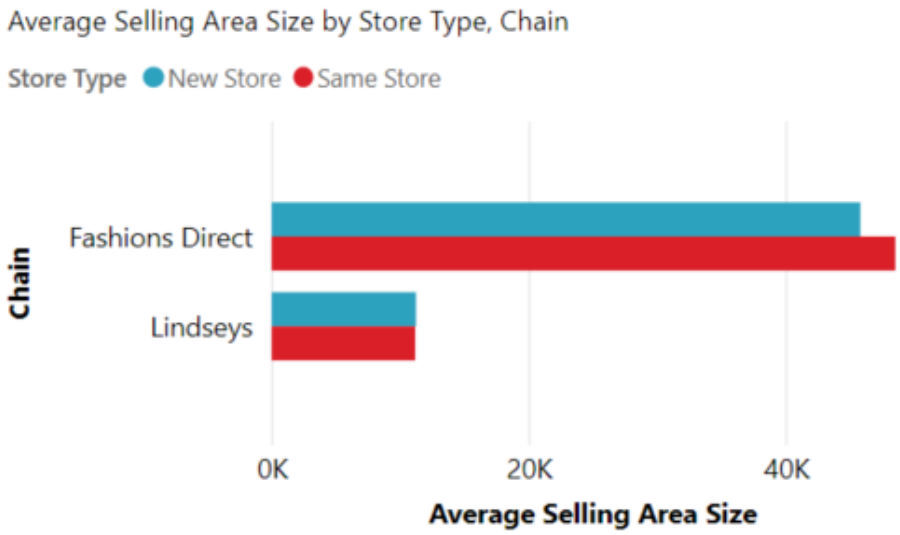
Area charts: Basic (Layered) and Stacked



The basic area chart is based on the line chart with the area between the axis and line filled in. Area charts emphasize the magnitude of change over time, and can be used to draw attention to the total value across a trend. For example, data that represents profit over time can be plotted in an area chart to emphasize the total profit.

Bar and column charts

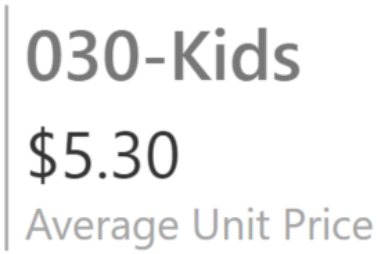




Bar charts are the standard for looking at a specific value across different categories.

Cards

Multi row



Multi row cards display one or more data points, one per row.

Single number



Single number cards display a single fact, a single data point. Sometimes a single number is the most important thing you want to track in your Power BI dashboard or report, such as total sales, market share year over year, or total opportunities.

Combo charts



A combo chart combines a column chart and a line chart. Combining the two charts into one lets you make a quicker comparison of the data. Combo charts can have one or two Y axes, so be sure to look closely.

Combo charts are a great choice:

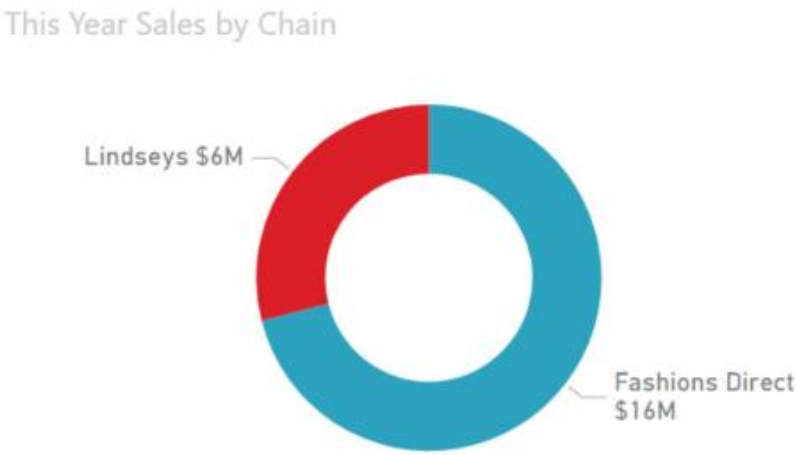
- When you have a line chart and a column chart with the same X axis.
- To compare multiple measures with different value ranges.
- To illustrate the correlation between two measures in one visual.
- To check whether one measure meets the target which is defined by another measure.
- To conserve canvas space.

Decomposition tree



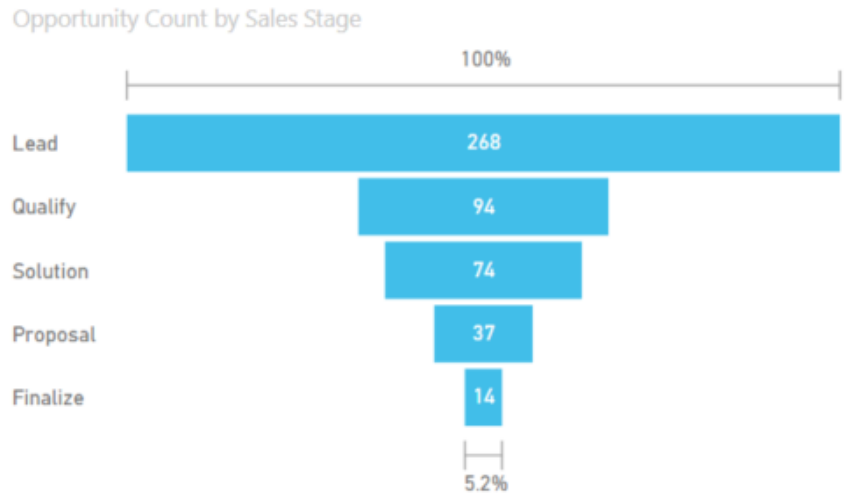
The decomposition tree visual lets you visualize data across multiple dimensions. It automatically aggregates data and enables drilling down into your dimensions in any order. It is also an artificial intelligence (AI) visualization, so you can ask it to find the next dimension to drill down into based on certain criteria. This makes it a valuable tool for ad hoc exploration and conducting root cause analysis.

Doughnut charts



Doughnut charts are similar to pie charts. They show the relationship of parts to a whole. The only difference is that the center is blank and allows space for a label or icon.

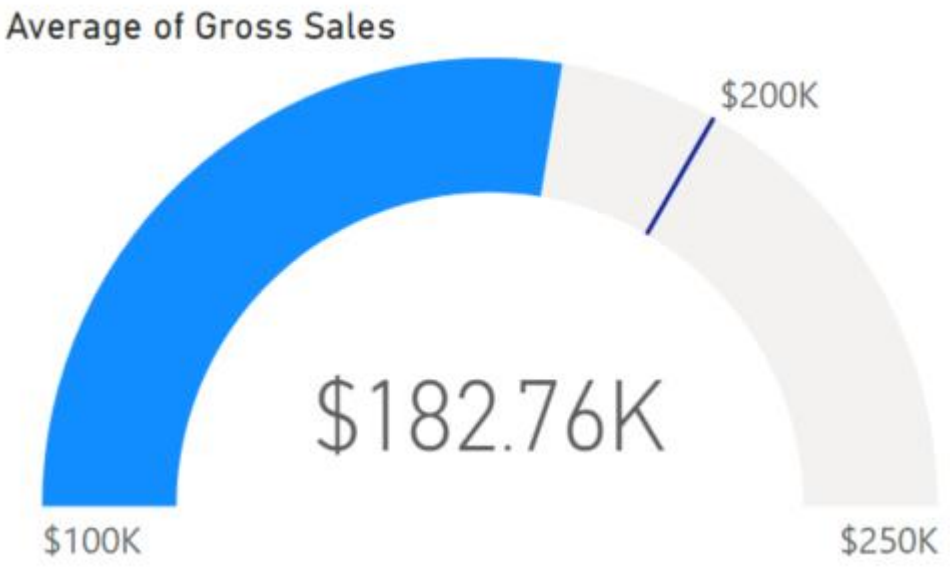
Funnel charts



Funnels help visualize a process that has stages, and items flow sequentially from one stage to the next. One example is a sales process that starts with leads and ends with purchase fulfillment.

For example, a sales funnel that tracks customers through stages: Lead > Qualified Lead > Prospect > Contract > Close. At a glance, the shape of the funnel conveys the health of the process you're tracking. Each funnel stage represents a percentage of the total. So, in most cases, a funnel chart is shaped like a funnel -- with the first stage being the largest, and each subsequent stage smaller than its predecessor. A pear-shaped funnel is also useful -- it can identify a problem in the process. But typically, the first stage, the "intake" stage, is the largest.

Gauge charts



A radial gauge chart has a circular arc and displays a single value that measures progress toward a goal. The goal, or target value, is represented by the line (needle). Progress toward

that goal is represented by the shading. And the value that represents that progress is shown in bold inside the arc. All possible values are spread evenly along the arc, from the minimum (left-most value) to the maximum (right-most value).

Radial gauges are a great choice to:

- Show progress toward a goal.
- Represent a percentile measure, like a KPI.
- Show the health of a single measure.
- Display information that can be quickly scanned and understood.

Key influencers chart



A key influencer chart displays the major contributors to a selected result or value.

Key influencers are a great choice to help you understand the factors that influence a key metric. For example, *what influences customers to place a second order* or *why were sales so high last June*.

KPIs

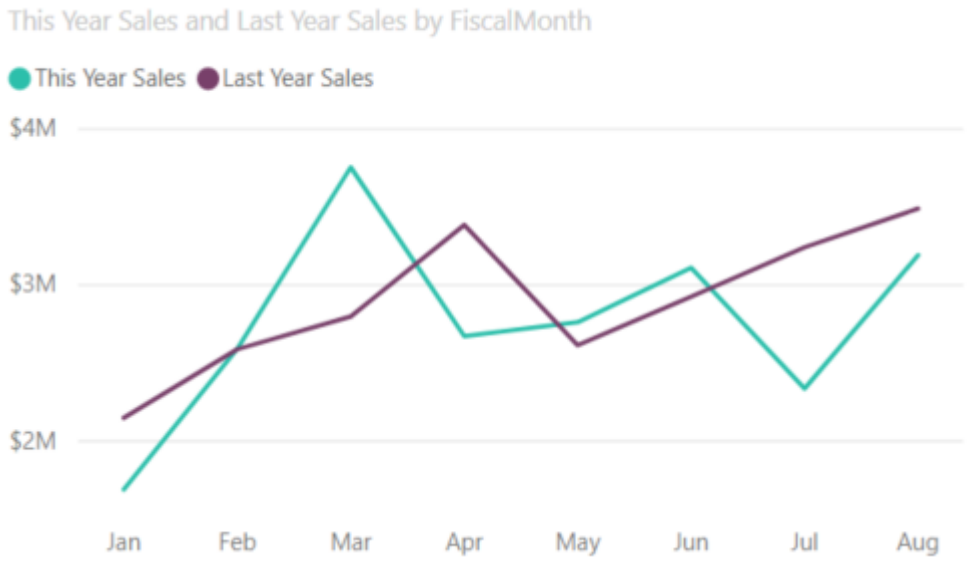


A Key Performance Indicator (KPI) is a visual cue that communicates the amount of progress made toward a measurable goal.

KPIs are a great choice:

- To measure progress (what am I ahead or behind on?).
- To measure distance to a metric (how far ahead or behind am I?).

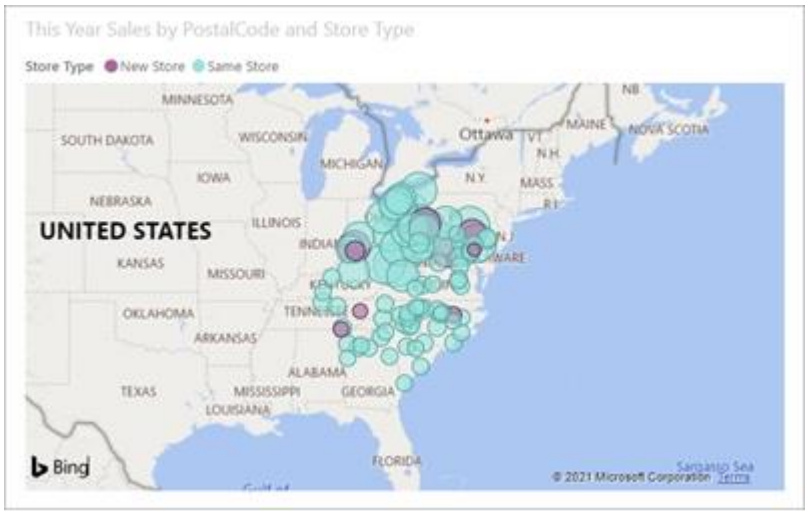
Line charts



Line charts emphasize the overall shape of an entire series of values, usually over time.

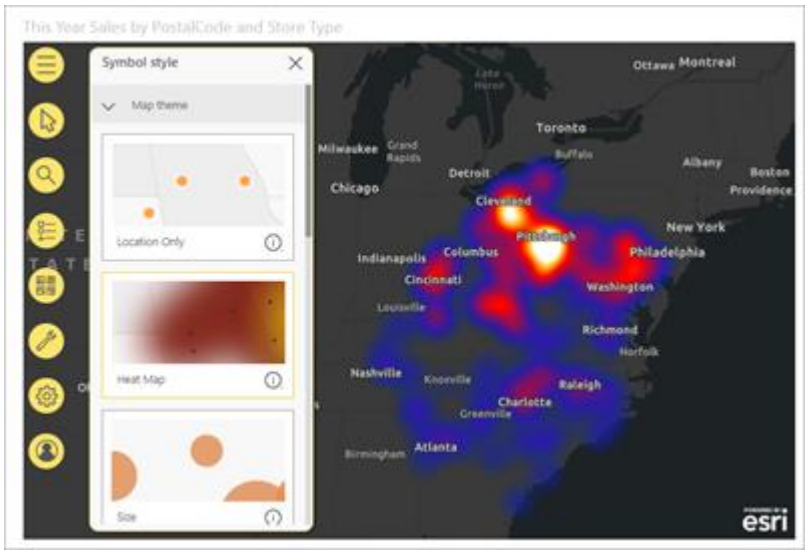
Maps

Basic map



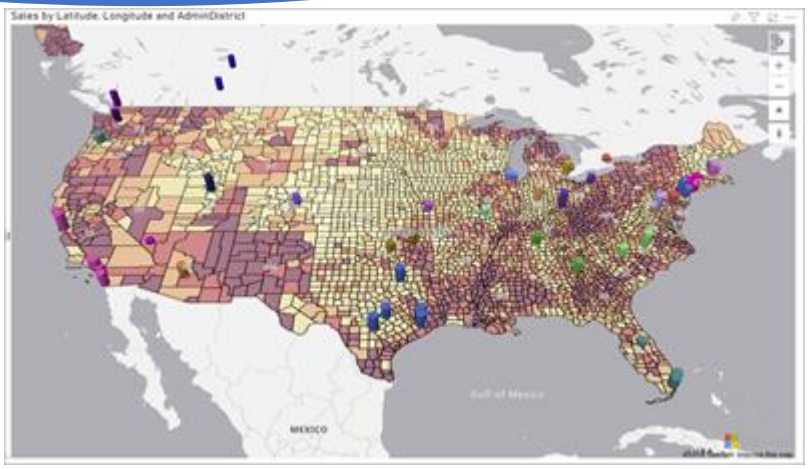
Use a basic map to associate both categorical and quantitative information with spatial locations.

ArcGIS map

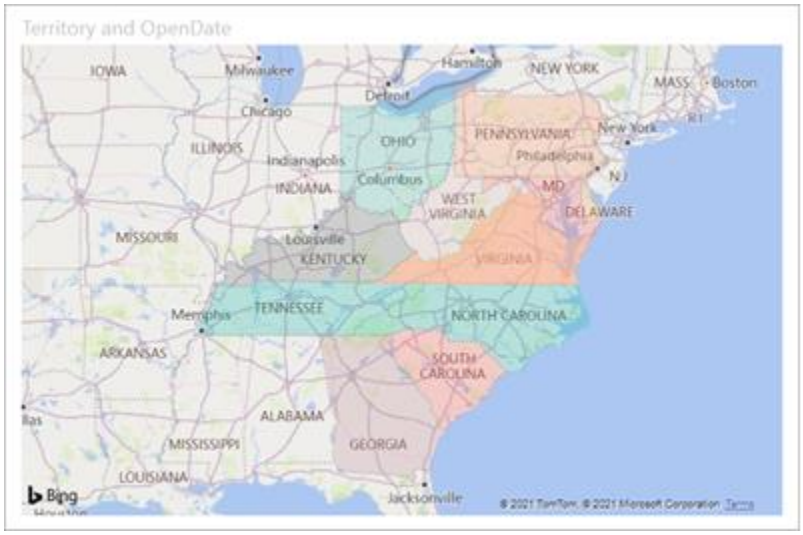


The combination of ArcGIS maps and Power BI takes mapping beyond the presentation of points on a map to a whole new level. The available options for base maps, location types, themes, symbol styles, and reference layers creates gorgeous informative map visuals. The combination of authoritative data layers (such as census data) on a map with spatial analysis conveys a deeper understanding of the data in your visual.

Azure map

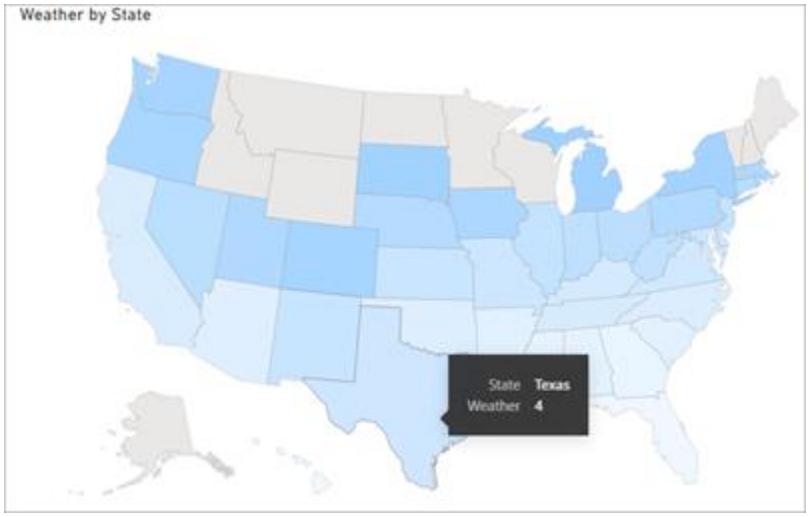


Filled map (Choropleth)



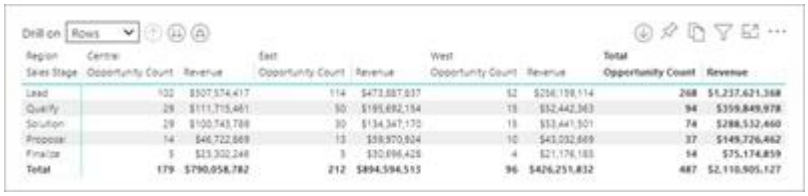
A filled map uses shading or tinting or patterns to display how a value differs in proportion across a geography or region. Quickly display these relative differences with shading that ranges from light (less-frequent/lower) to dark (more-frequent/more).

Shape map



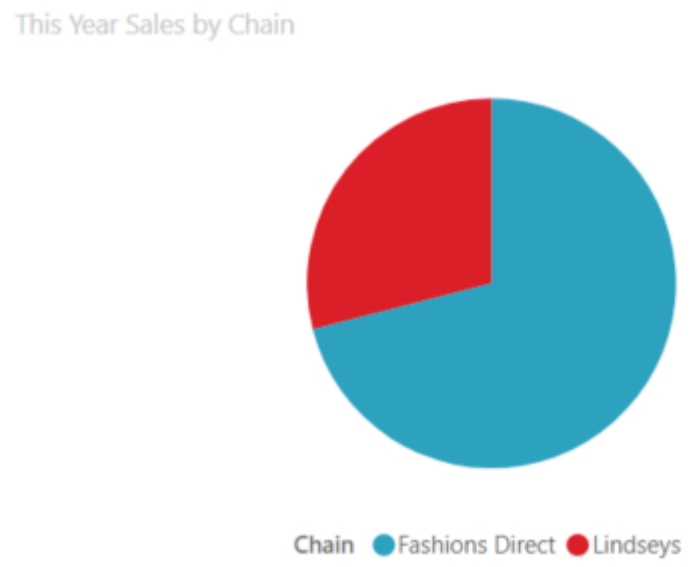
Shape maps compare regions on a map using color. A shape map can't show precise geographical locations of data points on a map. Instead, its main purpose is to show relative comparisons of regions on a map by coloring them differently.

Matrix



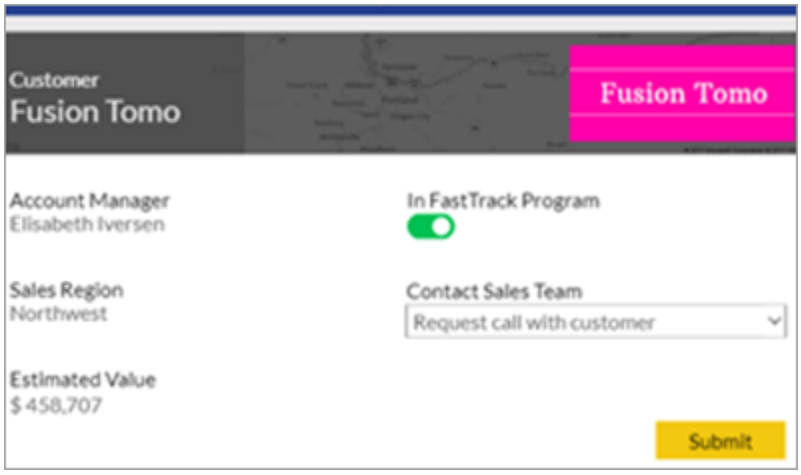
The matrix visual is a type of table visual that supports a stepped layout. A table supports two dimensions, but a matrix makes it easier to display data meaningfully across multiple dimensions. Often, report designers include matrixes in reports and dashboards to allow users to select one or more element (rows, columns, cells) in the matrix to cross-highlight other visuals on a report page.

Pie charts



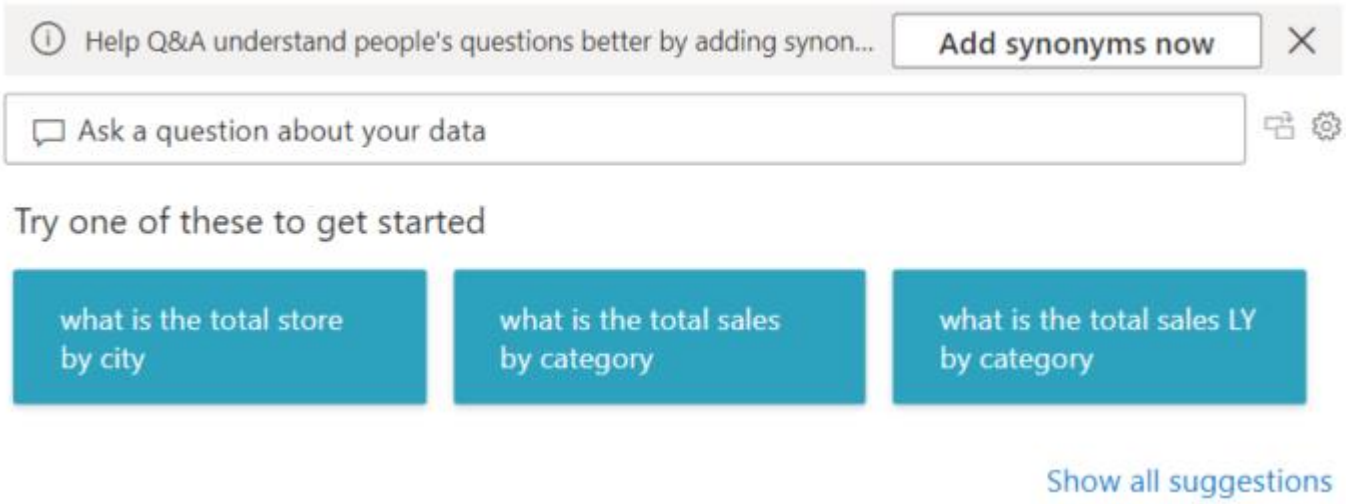
Pie charts show the relationship of parts to a whole.

Power Apps visual

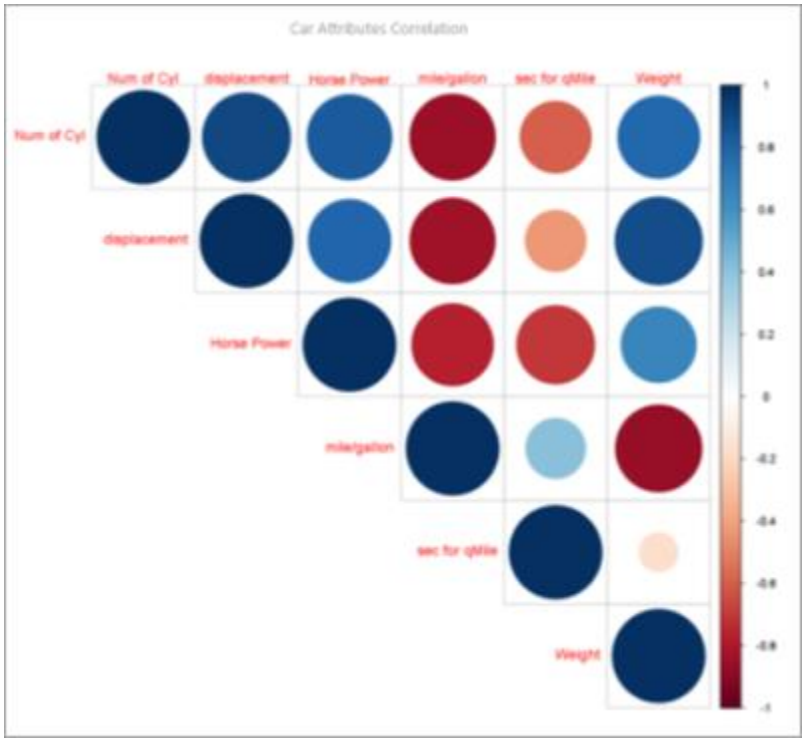


Report designers can create a Power App and embed it into a Power BI report as a visual. Consumers can interact with that visual within the Power BI report.

Q&A visual



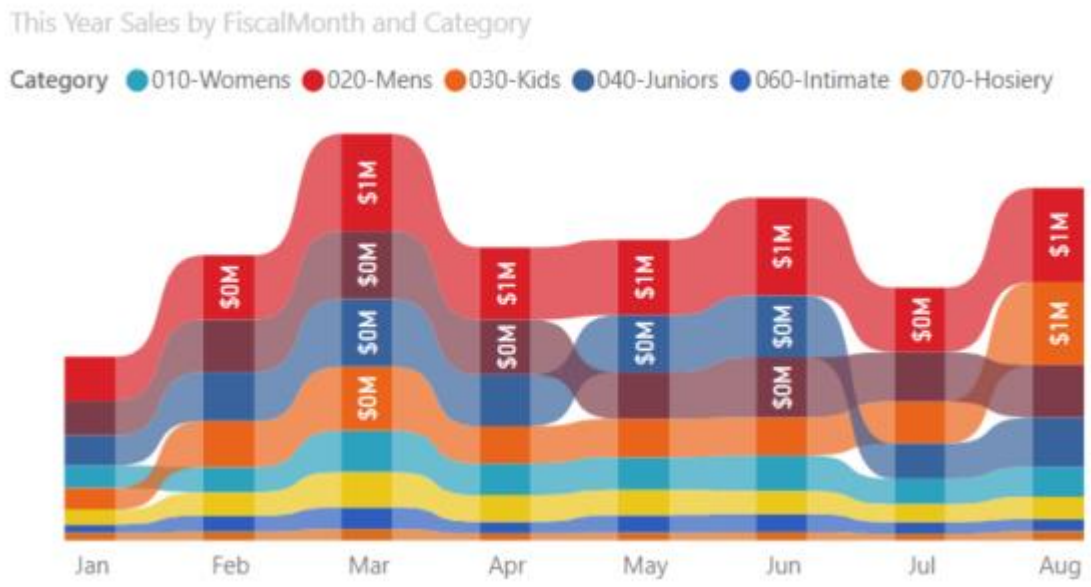
R script visuals



Tip

Visuals created with R scripts, commonly called *R visuals*, can present advanced data shaping and analytics such as forecasting, using the rich analytics and visualization power of R. R visuals can be created in Power BI Desktop and published to the Power BI service.

Ribbon chart

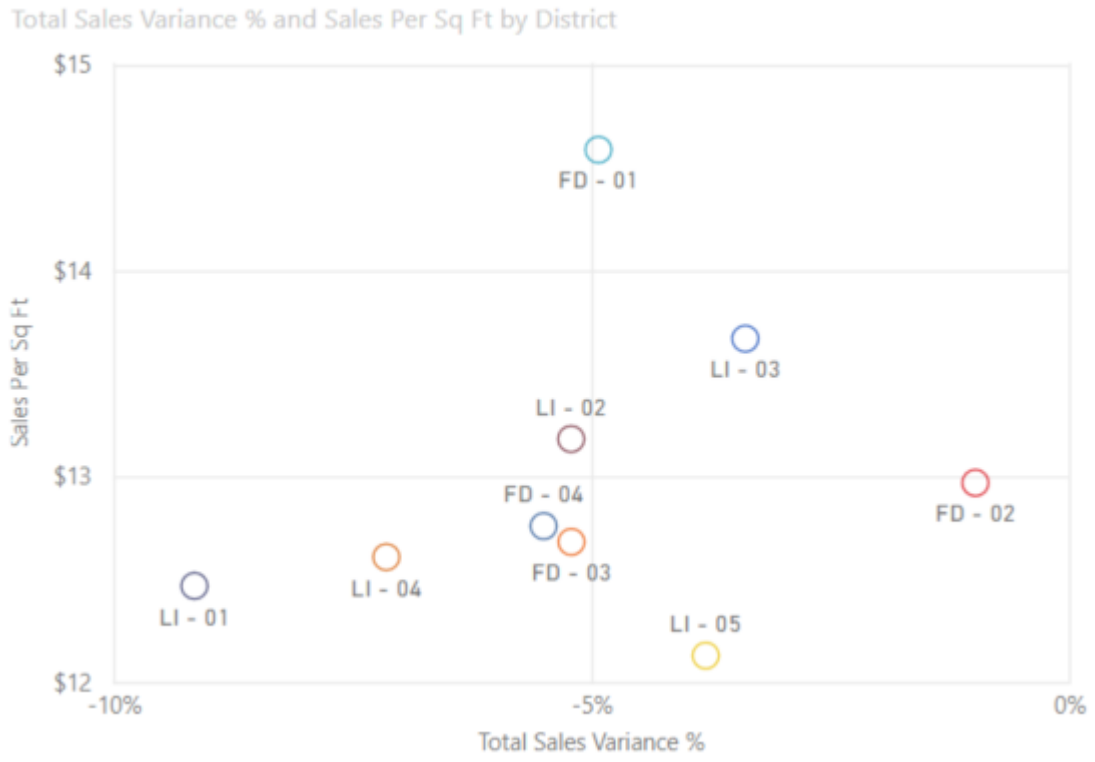


Ribbon charts show which data category has the highest rank (largest value). Ribbon charts are effective at showing rank change, with the highest range (value) always displayed on top for each time period.

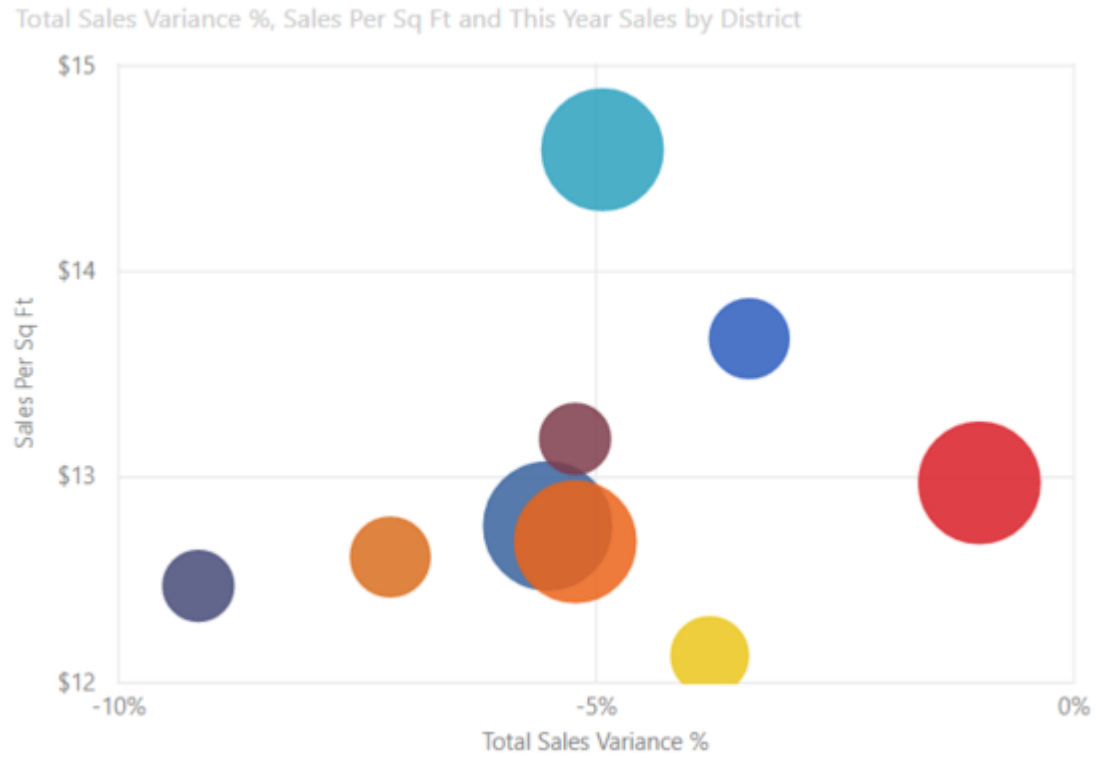
Scatter

Scatter, bubble, and dot plot chart

A scatter chart always has two value axes to show one set of numerical data along a horizontal axis and another set of numerical values along a vertical axis. The chart displays points at the intersection of an x and y numerical value, combining these values into single data points. These data points may be distributed evenly or unevenly across the horizontal axis, depending on the data.

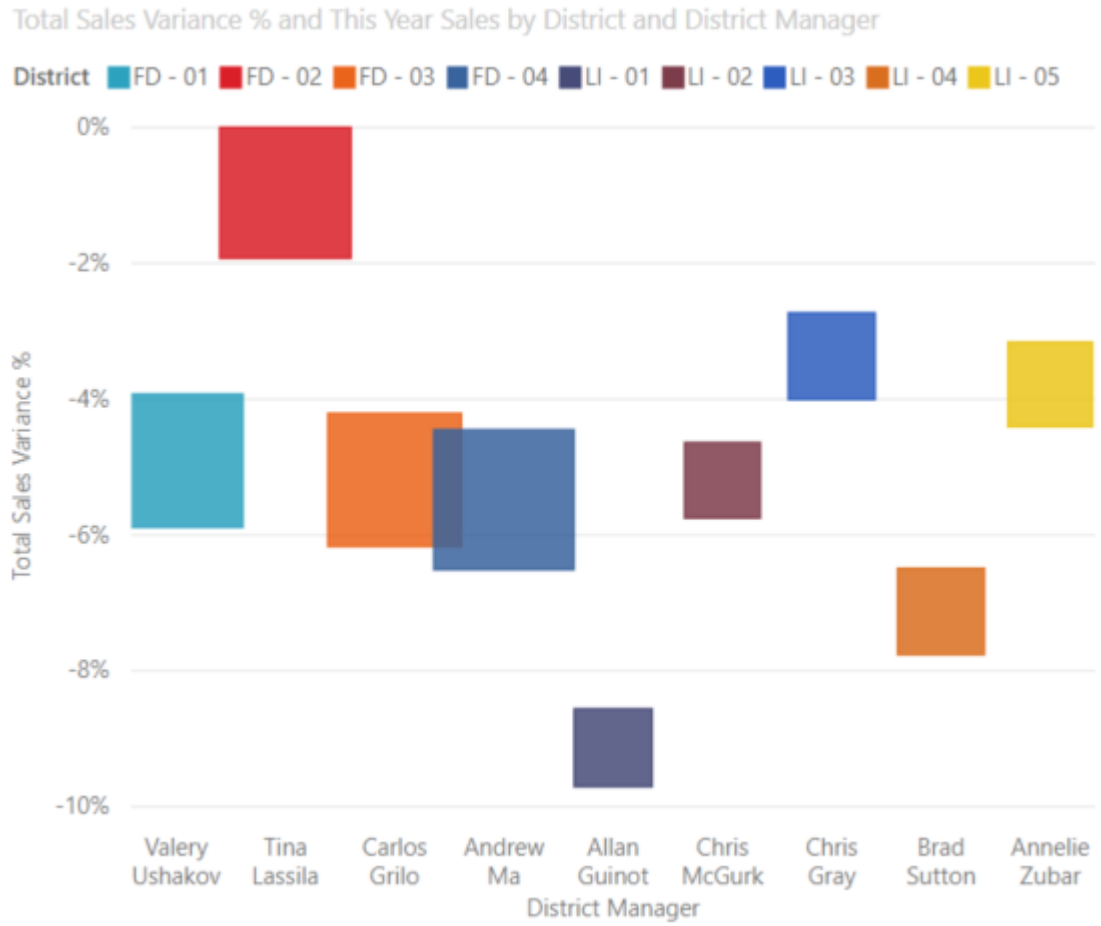


A bubble chart replaces data points with bubbles, with the bubble size representing an additional dimension of the data.

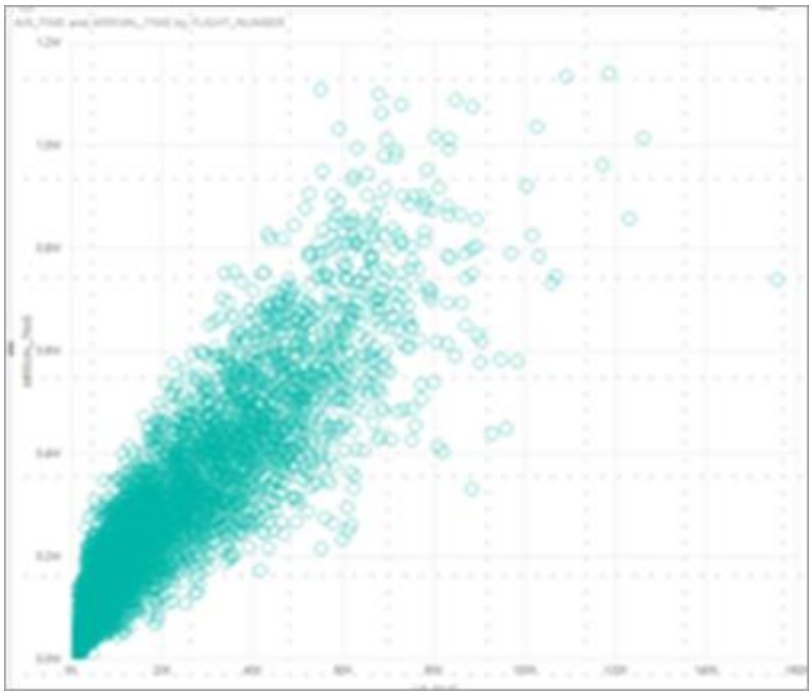


Both scatter and bubble charts can also have a play axis, which can show changes over time.

A dot plot chart is similar to a bubble chart and scatter chart except that it can plot numerical or categorical data along the X axis. This example happens to use squares instead of circles and plots sales along the X axis.



Scatter-high density

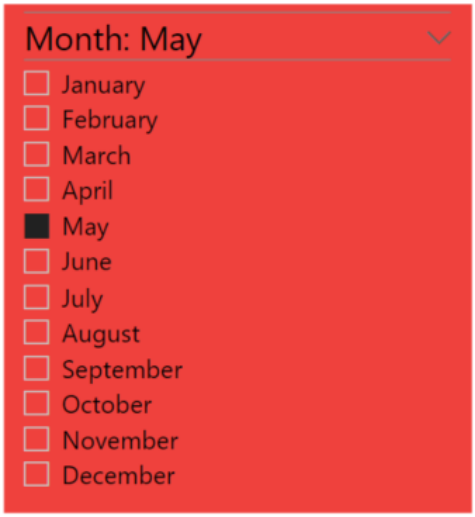


By definition, high-density data is sampled to create visuals reasonably quickly that are responsive to interactivity. High-density sampling uses an algorithm that eliminates

overlapping points, and ensures that all points in the data set are represented in the visual. It doesn't just plot a representative sample of the data.

This ensures the best combination of responsiveness, representation, and clear preservation of important points in the overall data set.

Slicers

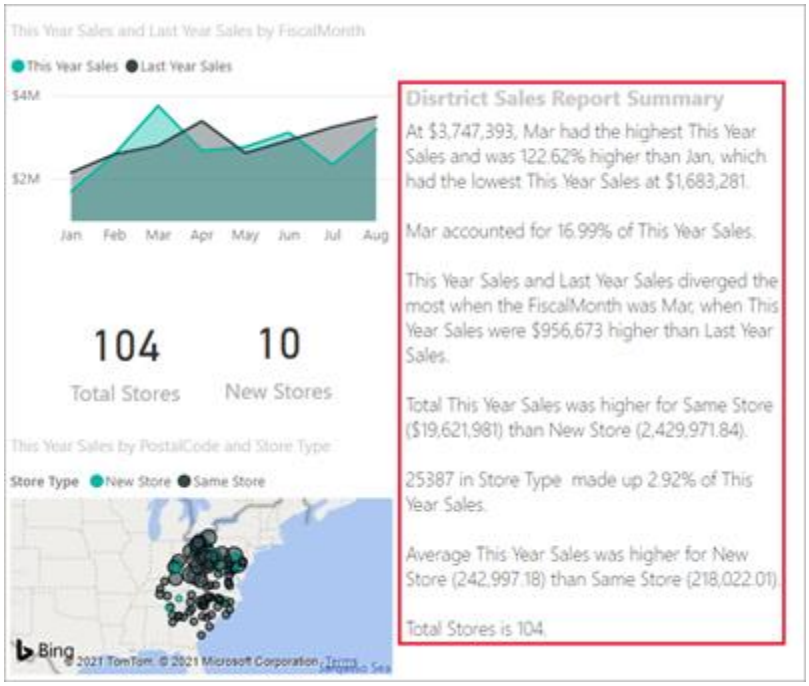


A slicer is a standalone chart that can be used to filter the other visuals on the page. Slicers come in many different formats (category, range, date, etc.) and can be formatted to allow selection of only one, many, or all of the available values.

Slicers are a great choice to:

- Display commonly used or important filters on the report canvas for easier access.
- Make it easier to see the current filtered state without having to open a drop-down list.
- Filter by columns that are unneeded and hidden in the data tables.
- Create more focused reports by putting slicers next to important visuals.

Smart narrative



The Smart narrative adds text to reports to point out trends, key takeaways, and add explanations and context. The text helps users to understand the data and identify the important findings quickly.

Standalone images



A standalone image is a graphic that has been added to a report or dashboard.

Tables

Category	This Year Sales Status	Average Unit Price	Last Year Sales	This Year Sales	This Year Sales Goal
080-Accessories		$4.84	$1,273,096	$1,379,259	$1,273,096
090-Home		$3.93	$2,913,647	$3,053,326	$2,913,647
100-Groceries		$1.47	$810,176	$829,776	$810,176
020-Mens		$7.12	$4,453,133	$4,452,421	$4,453,133
030-Kids		$5.30	$2,726,892	$2,705,490	$2,726,892
050-Shoes		$13.84	$3,640,471	$3,574,900	$3,640,471
010-Womens		$7.30	$2,680,662	$1,787,958	$2,680,662
040-Juniors		$7.00	$3,105,550	$2,930,385	$3,105,550
060-Intimate		$4.28	$955,370	$852,329	$955,370
070-Hosiery		$3.69	$573,604	$486,106	$573,604
Total		**$5.49**	**$23,132,601**	**$22,051,952**	**$23,132,601**

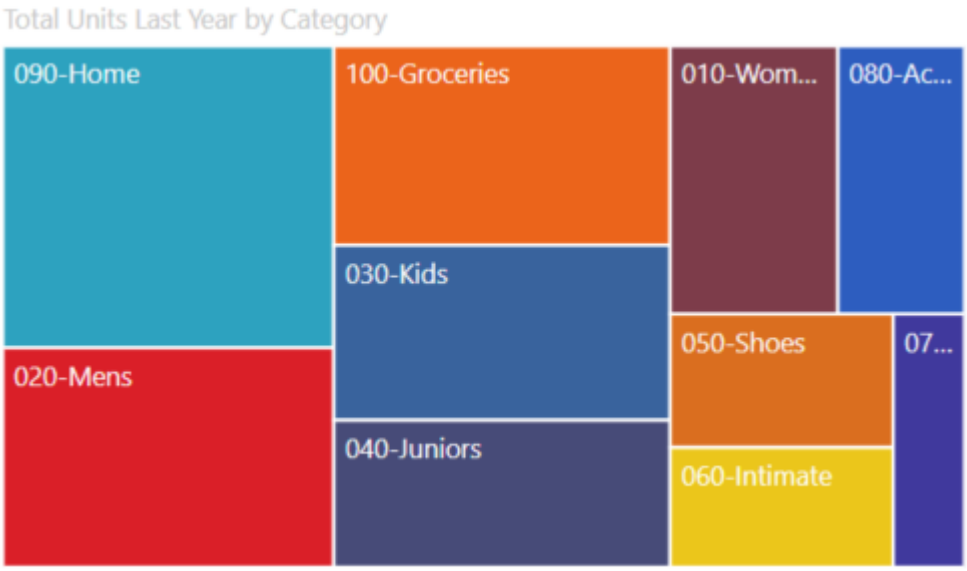
A table is a grid that contains related data in a logical series of rows and columns. It may also contain headers and a row for totals. Tables work well with quantitative comparisons where

you are looking at many values for a single category. For example, this table displays five different measures for Category.

Tables are a great choice:

- To see and compare detailed data and exact values (instead of visual representations).
- To display data in a tabular format.
- To display numerical data by categories.

Treemaps

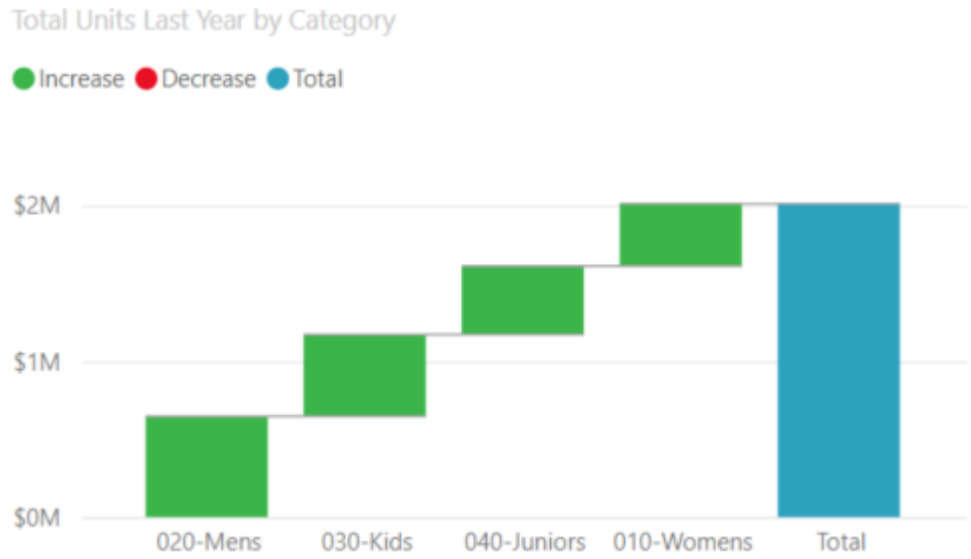


Treemaps are charts of colored rectangles, with size representing value. They can be hierarchical, with rectangles nested within the main rectangles. The space inside each rectangle is allocated based on the value being measured. And the rectangles are arranged in size from top left (largest) to bottom right (smallest).

Treemaps are a great choice:

- To display large amounts of hierarchical data.
- When a bar chart can't effectively handle the large number of values.
- To show the proportions between each part and the whole.
- To show the pattern of the distribution of the measure across each level of categories in the hierarchy.
- To show attributes using size and color coding.
- To spot patterns, outliers, most-important contributors, and exceptions.

Waterfall charts



A waterfall chart shows a running total as values are added or subtracted. It's useful for understanding how an initial value (for example, net income) is affected by a series of positive and negative changes.

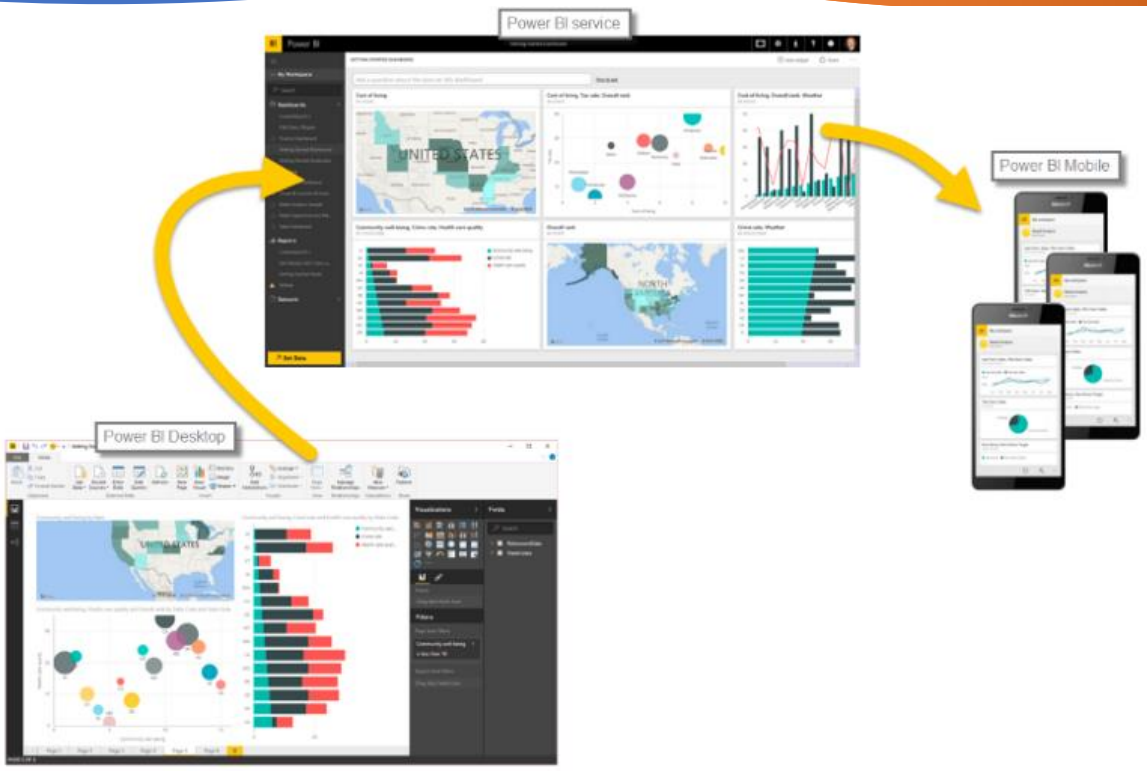
The columns are color coded so you can quickly tell increases and decreases. The initial and the final value columns often start on the horizontal axis, while the intermediate values are floating columns. Because of this "look", waterfall charts are also called bridge charts.

Waterfall charts are a great choice:

- When you have changes for the measure across time or across different categories.
- To audit the major changes contributing to the total value.
- To plot your company's annual profit by showing various sources of revenue and arrive at the total profit (or loss).
- To illustrate the beginning and the ending headcount for your company in a year.
- To visualize how much money you make and spend each month, and the running balance for your account.

5.4 Getting Started with Power BI Desktop:

Power BI is a free application that can be downloaded and installed on the system. It can be connected to multiple data sources. Usually, an analysis work begins in Power BI Desktop where report creation takes place. The report is then published to Power BI service from where it can be shared to the Power BI Mobile apps so that people can view the reports even on mobiles.



The most common uses for Power BI Desktop are as follows:

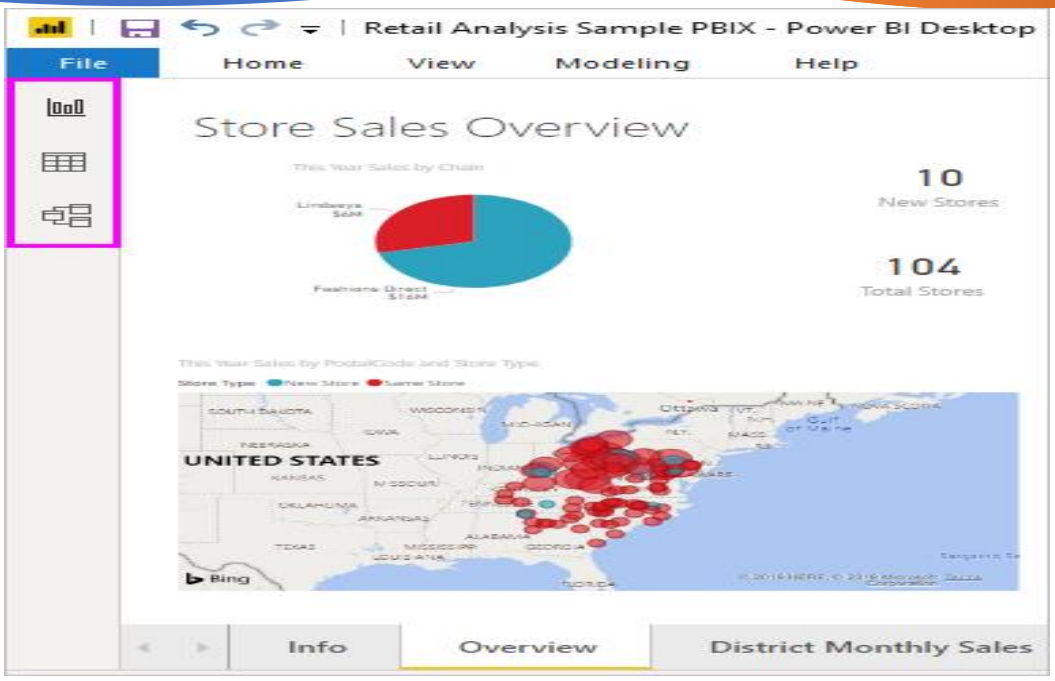
- Connect to data
- Transform and clean that data, to create a data model
- Create visuals, such as charts or graphs, that provide visual representations of the data
- Create reports that are collections of visuals, on one or more report pages
- Share reports with others by using the Power BI service

There are three views available in Power BI Desktop, which you select on the left side of the canvas.

The views, shown in the order they appear, are as follows:

- **Report**: In this view, you create reports and visuals, where most of your creation time is spent.
- **Data**: In this view, you see the tables, measures, and other data used in the data model associated with your report and transform the data for best use in the report's model.
- **Model**: In this view, you see and manage the relationships among tables in your data model.

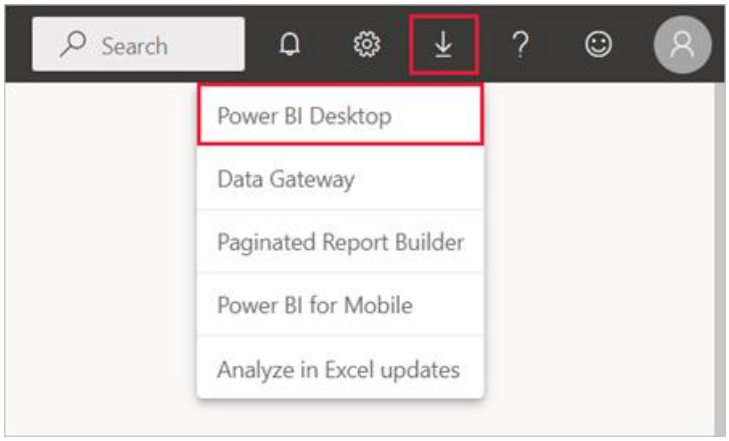
The following image shows the three views, as displayed along the left side of the canvas:



Install and run Power BI Desktop

To download Power BI Desktop, go to the Power BI Desktop download page and select Download Free. Or for download options, select See download or language options.

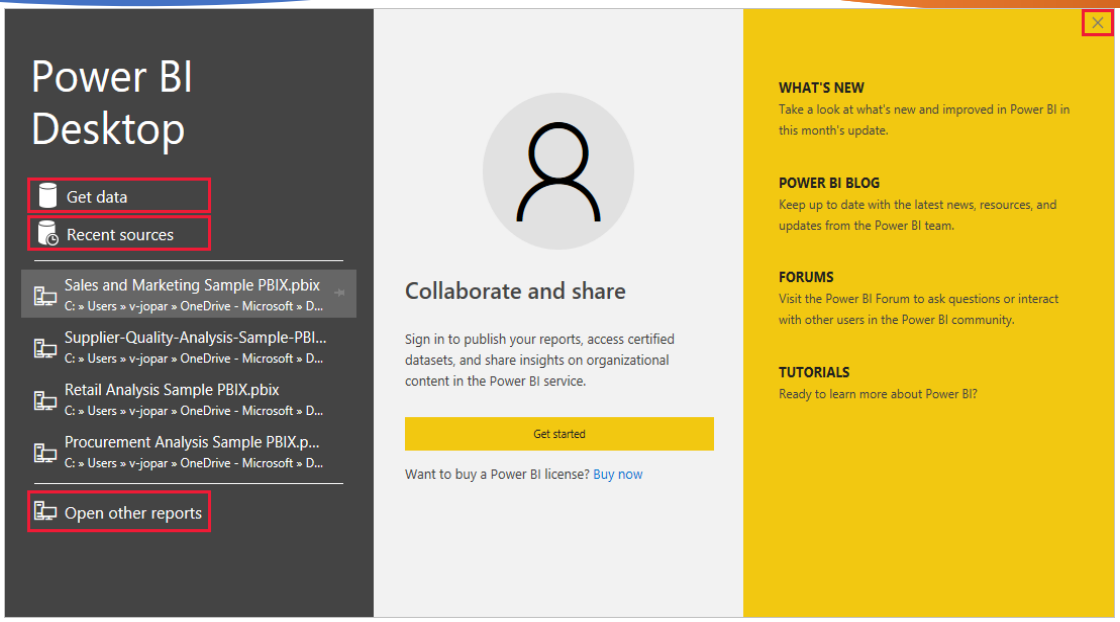
You can also download Power BI Desktop from the Power BI service. Select the Download icon in the top menu bar, and then select Power BI Desktop.



On the Microsoft Store page, select Get, and follow the prompts to install Power BI Desktop on your computer. Start Power BI Desktop from the Windows Start menu or from the icon in the Windows taskbar.

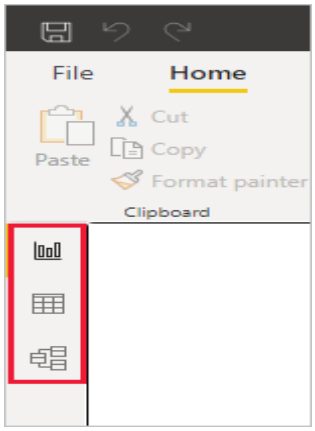
The first time Power BI Desktop starts, it displays the Welcome screen.

From the Welcome screen, you can Get data, see Recent sources, open recent reports, Open other reports, or select other links. Select the close icon to close the Welcome screen.

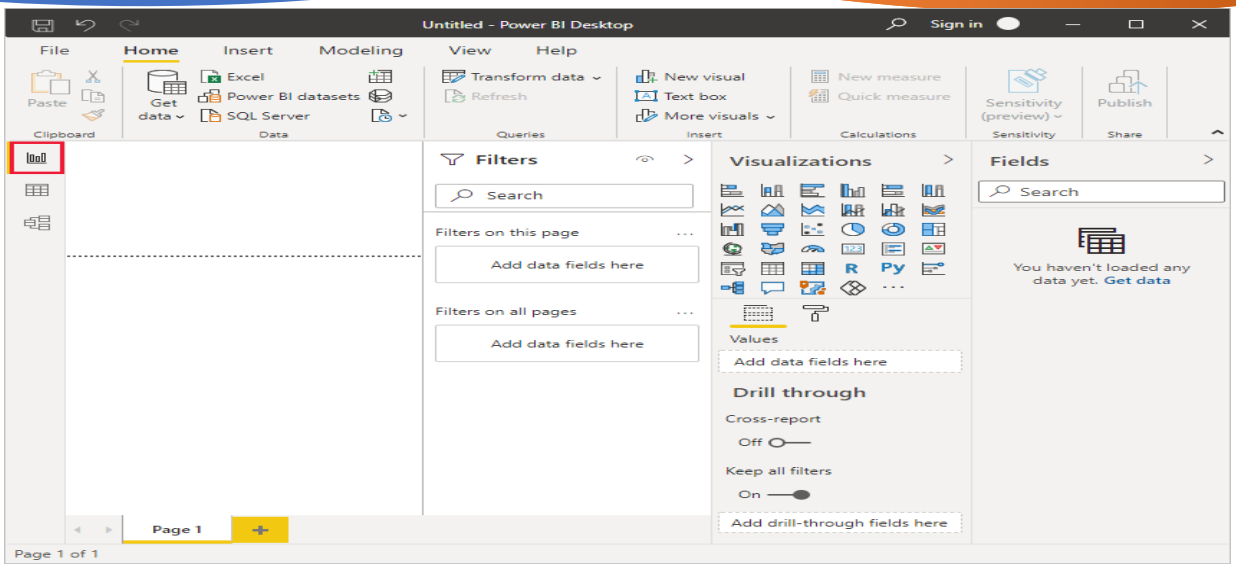


Along the left side of Power BI Desktop are icons for the three Power BI Desktop views: Report, Data, and Model, from top to bottom. The current view is indicated by the yellow bar along the left, and you can change views by selecting any of the icons.

If you are using keyboard navigation, press Ctrl + F6 to move focus to that section of buttons in the window.



Report view is the default view.



Power BI Desktop also includes the Power Query Editor, which opens in a separate window. In Power Query Editor, you can build queries and transform data, then load the refined data model into Power BI Desktop to create reports.

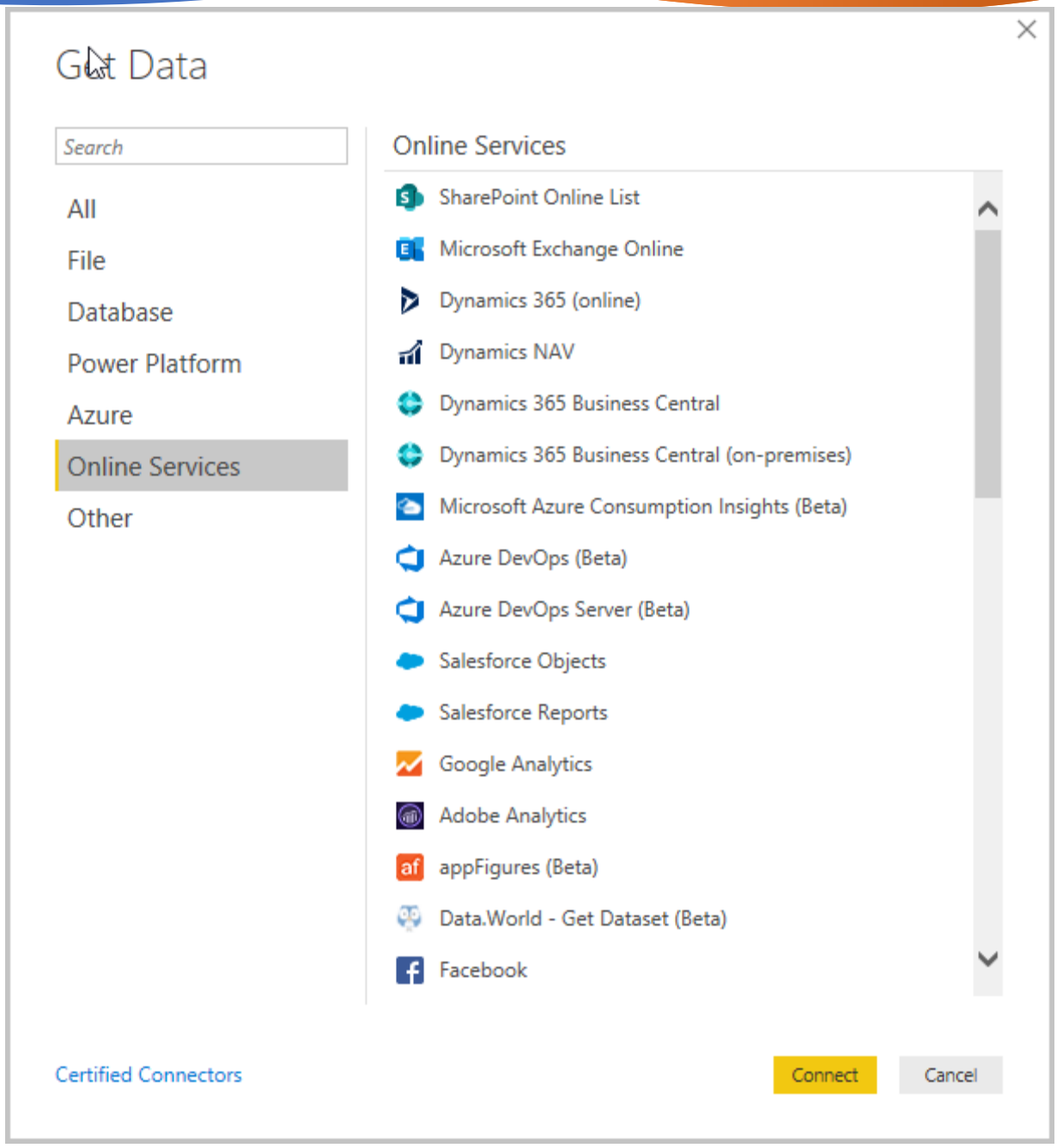
Connect to data

To get started with Power BI Desktop, the first step is to connect to data. There are many different data sources you can connect to from Power BI Desktop.

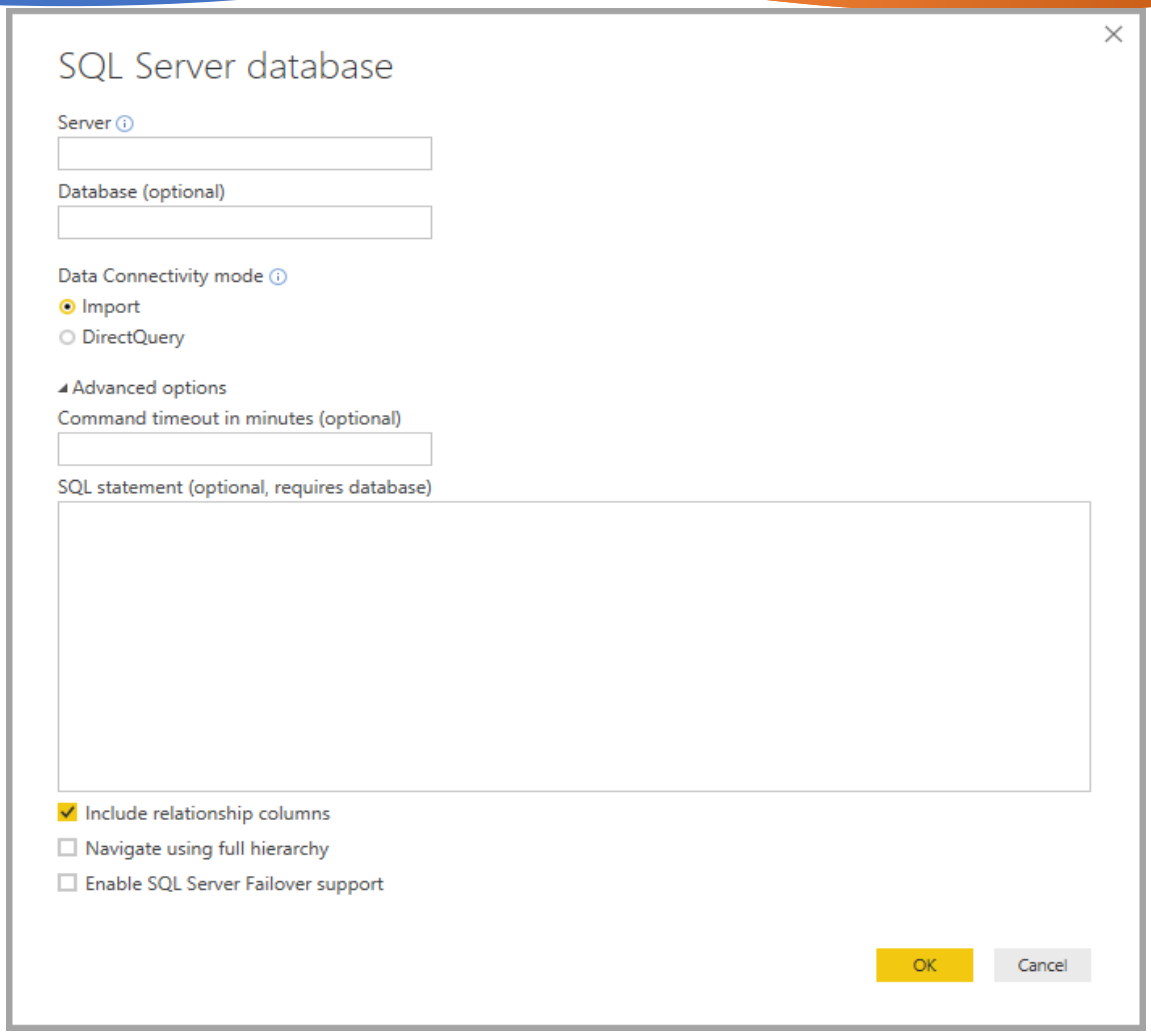
1. **To connect to data:**

From the Home ribbon, select Get Data > More.

The Get Data window appears, showing the many categories to which Power BI Desktop can connect.



2. When you select a data type, you're prompted for information, such as the URL and credentials, necessary for Power BI Desktop to connect to the data source on your behalf.



3. After you connect to one or more data sources, you may want to transform the data so it's useful for you.

5.5 Transform Data:

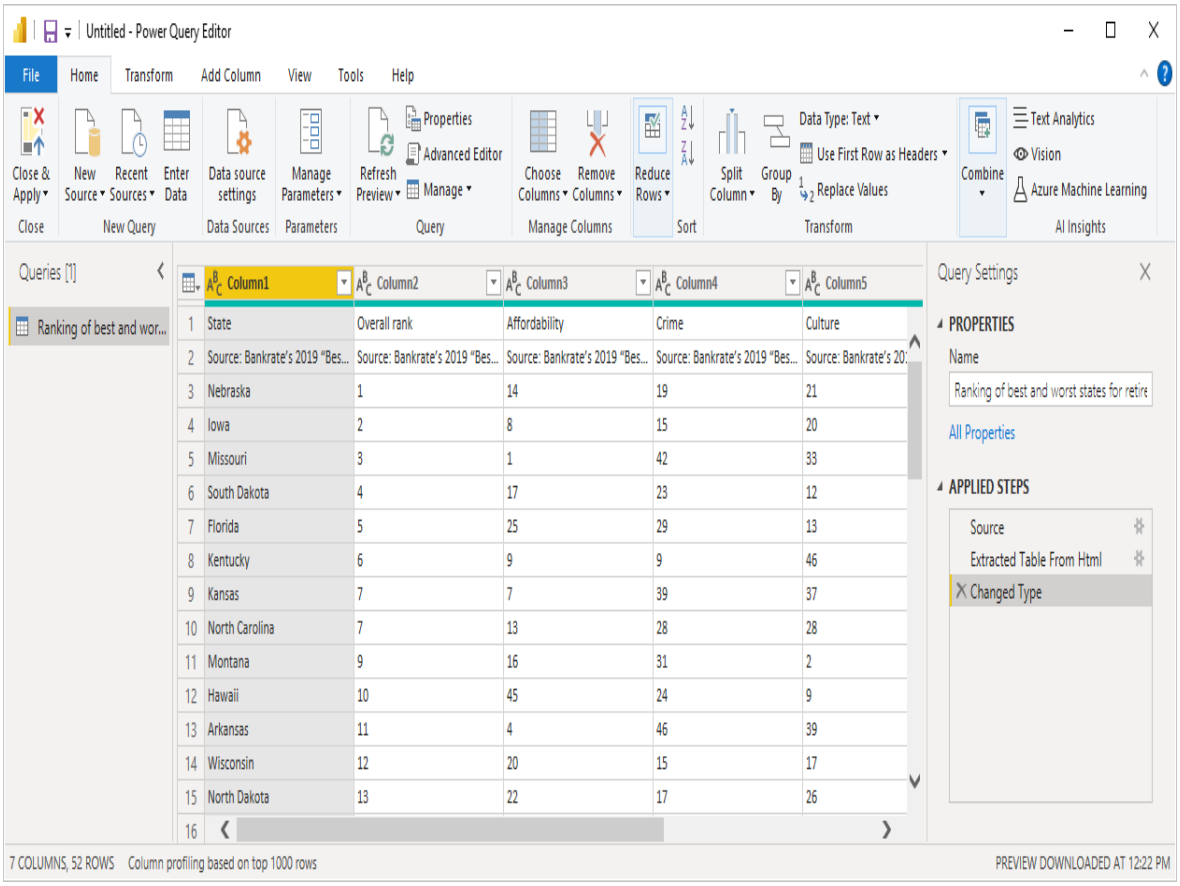
Transform and clean data, create a model

In Power BI Desktop, you can clean and transform data using the built-in Power Query Editor. With Power Query Editor, you make changes to your data, such as changing a data type, removing columns, or combining data from multiple sources. It's like sculpting: you start with a large block of clay (or data), then shave off pieces or add others as needed, until the shape of the data is how you want it.

To start Power Query Editor:

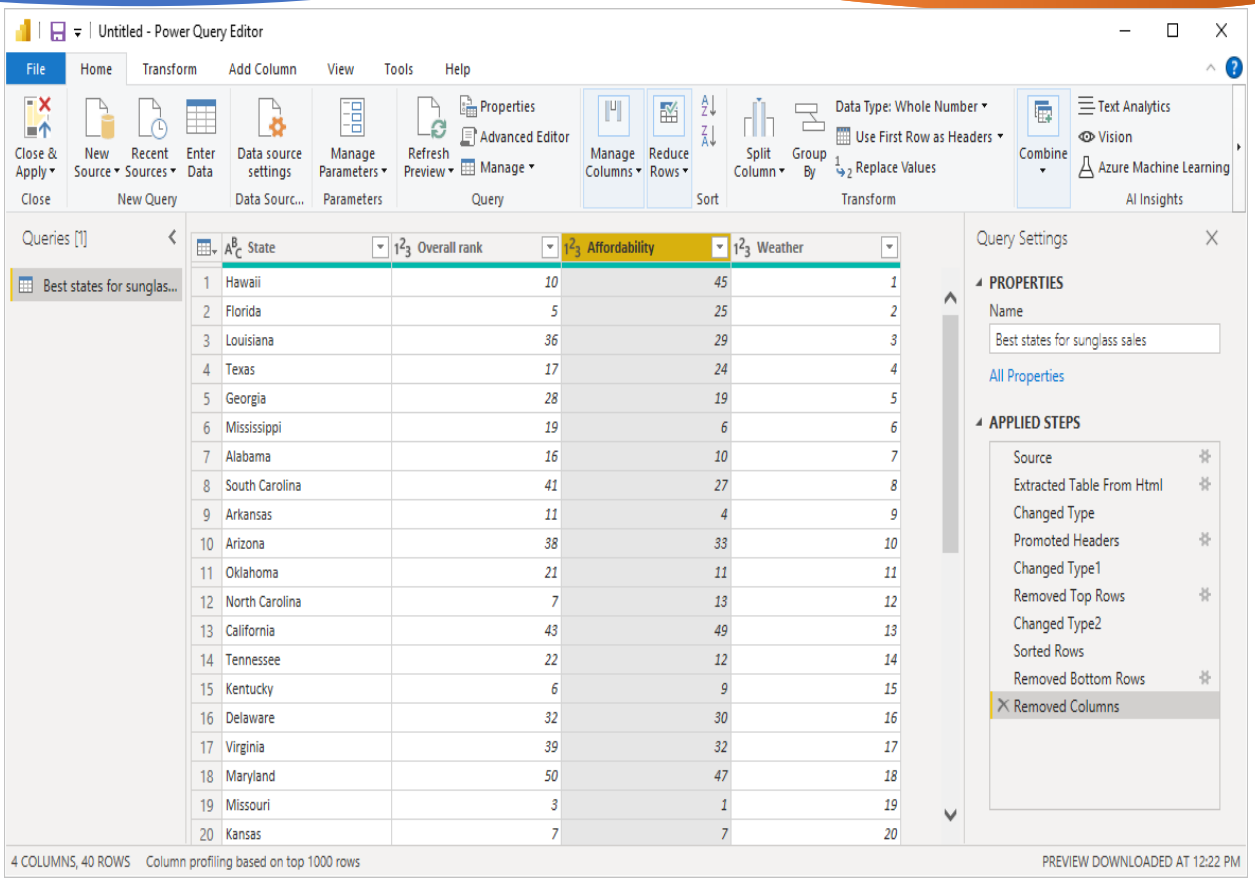
On the Home ribbon, in the Queries section, select Transform data.

The Power Query Editor window appears.



Each step you take in transforming data (such as renaming a table, transforming a data type, or deleting a column) is recorded by Power Query Editor. Every time this query connects to the data source, those steps are carried out so that the data is always shaped the way you specify.

The following image shows the **Power Query Editor** window for a query that has been shaped, and turned into a model.

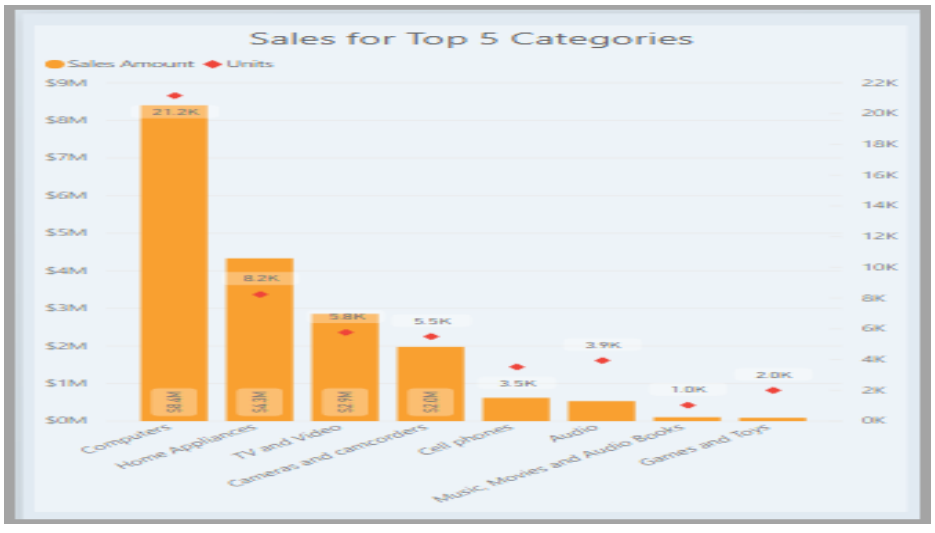


	State	Overall rank	Affordability	Weather
1	Hawaii	10	45	1
2	Florida	5	25	2
3	Louisiana	36	29	3
4	Texas	17	24	4
5	Georgia	28	19	5
6	Mississippi	19	6	6
7	Alabama	16	10	7
8	South Carolina	41	27	8
9	Arkansas	11	4	9
10	Arizona	38	33	10
11	Oklahoma	21	11	11
12	North Carolina	7	13	12
13	California	43	49	13
14	Tennessee	22	12	14
15	Kentucky	6	9	15
16	Delaware	32	30	16
17	Virginia	39	32	17
18	Maryland	50	47	18
19	Missouri	3	1	19
20	Kansas	7	7	20

Once your data is how you want it, you can create visuals.

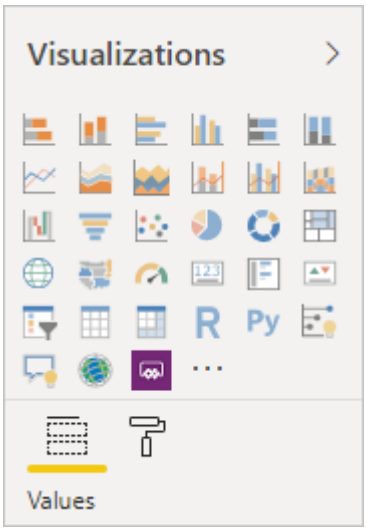
Create visuals

After you have a data model, you can drag fields onto the report canvas to create visuals. A visual is a graphic representation of the data in your model. There are many different types of visuals to choose from in Power BI Desktop. The following visual shows a simple column chart.



To create or change a visual:

- From the **Visualizations** pane, select the visual icon.



If you already have a visual selected on the report canvas, the selected visual changes to the type you selected.

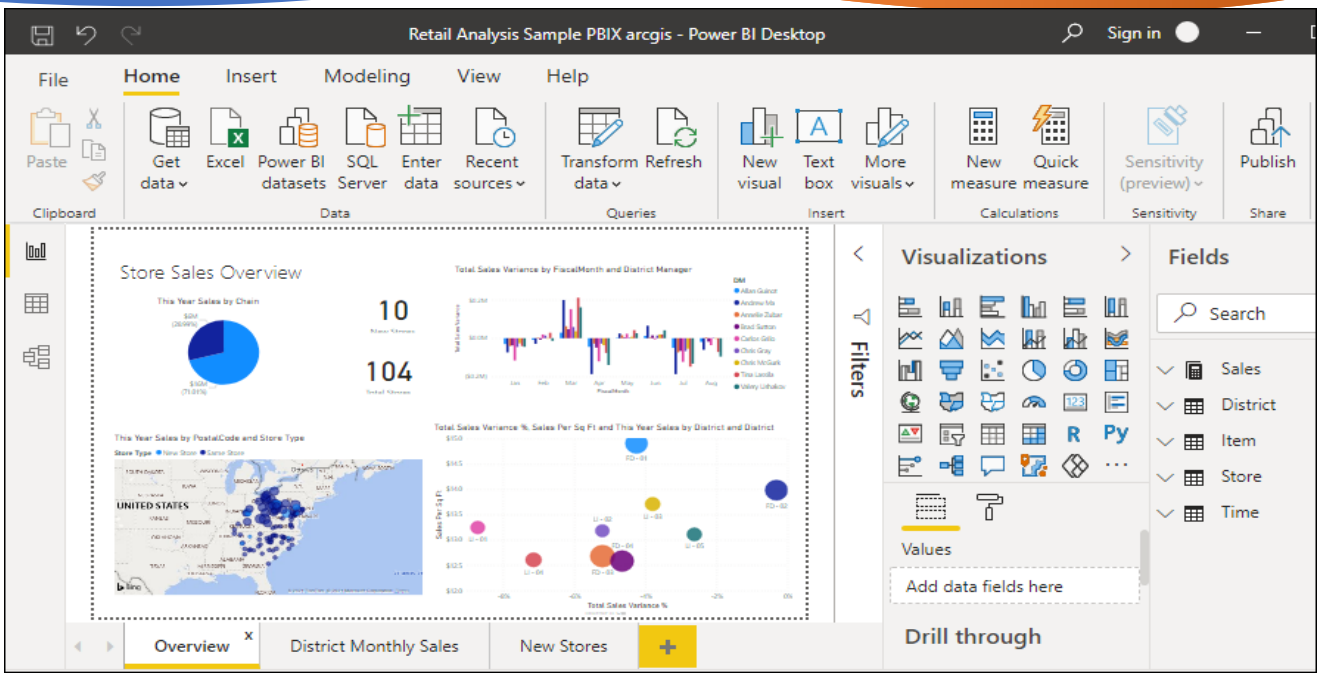
If no visual is selected on the canvas, a new visual is created based on your selection.

5.6 Report Creation:

More often, you'll want to create a collection of visuals that show various aspects of the data you've used to create your model in Power BI Desktop. A collection of visuals, in one Power BI Desktop file, is called a *report*. A report can have one or more pages, just like an Excel file can have one or more worksheets.

With Power BI Desktop you can create complex and visually rich reports, using data from multiple sources, all in one report that you can share with others in your organization.

In the following image, you see the first page of a Power BI Desktop report, named Overview, as seen on the tab near the bottom of the image.

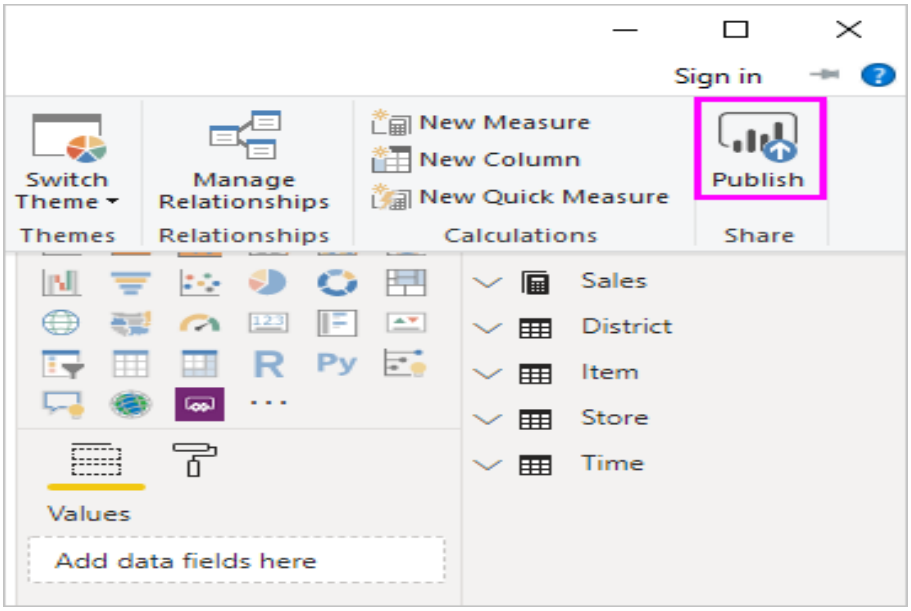


Share reports

After a report is ready to share with others, you can *publish* the report to the Power BI service, and make it available to anyone in your organization who has a Power BI license.

To publish a Power BI Desktop report:

1. Select **Publish** from the **Home** ribbon.



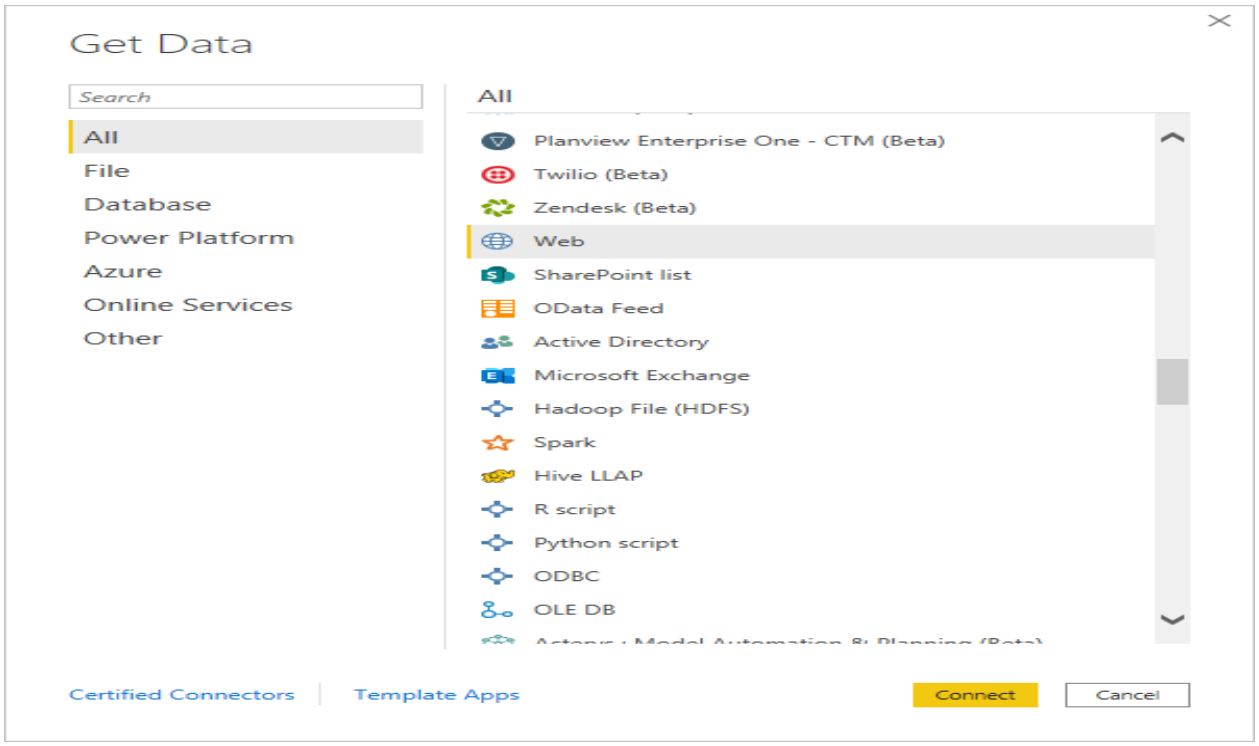
Power BI Desktop connects you to the Power BI service with your Power BI account.

2. Power BI prompts you to select where in the Power BI service you'd like to share the report, such as your workspace, a team workspace, or some other location in the Power BI service.

   You must have a Power BI license to share reports to the Power BI service.

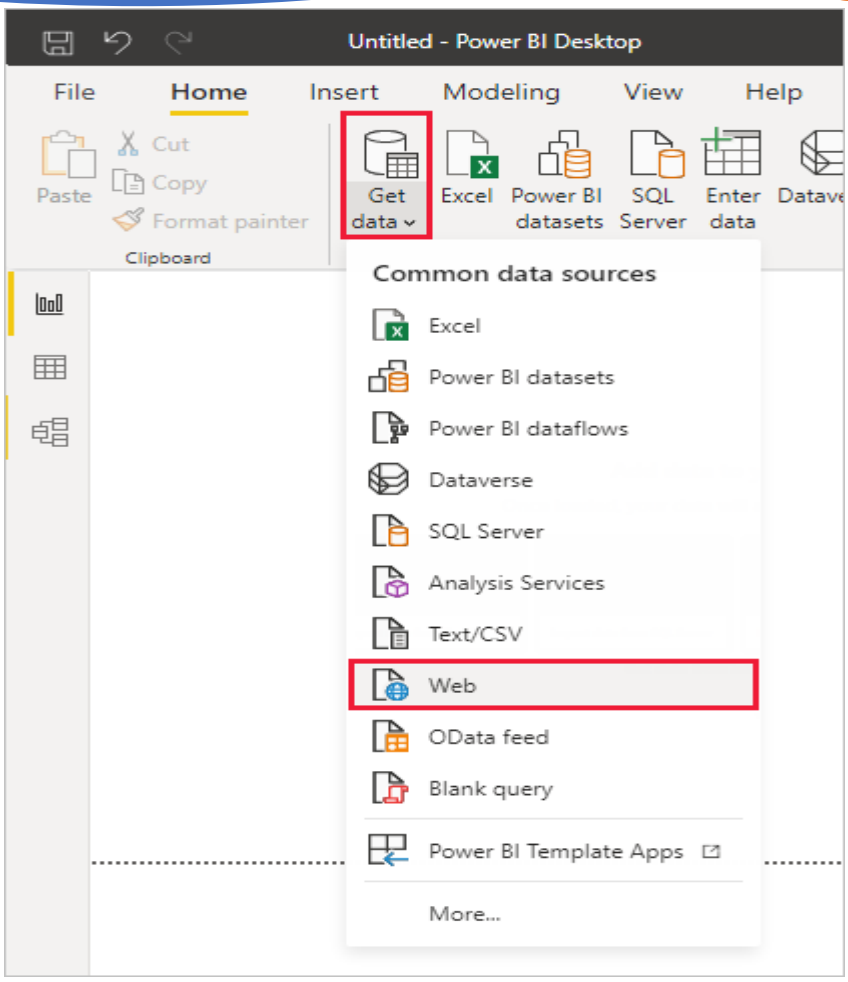
Connect to data

With Power BI Desktop installed, you're ready to connect to the ever-expanding world of data. To see the many types of data sources available, select Get Data > More in the Power BI Desktop Home tab, and in the Get Data window, scroll through the list of All data sources. In this quick tour, you connect to a couple of different Web data sources.

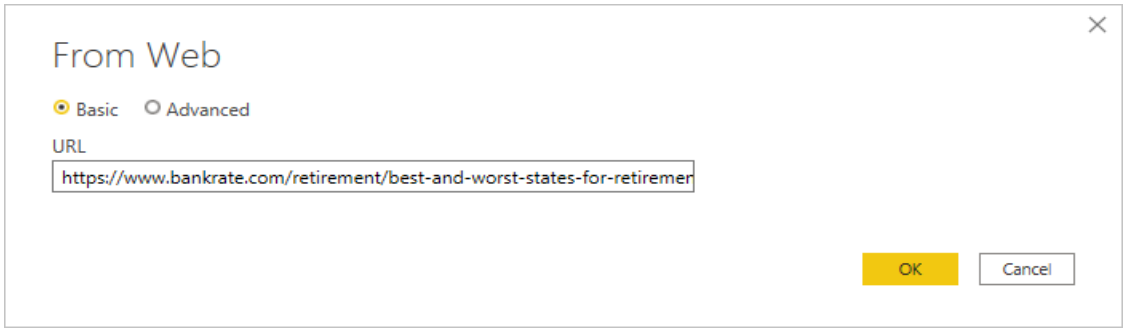


Imagine you're a data analyst working for a sunglasses retailer. You want to help your client target sunglasses sales where the sun shines most frequently. The Bankrate.com Best and worst states for retirement page has interesting data on this subject.

On the Power BI Desktop Home tab, select Get Data > Web to connect to a web data source.



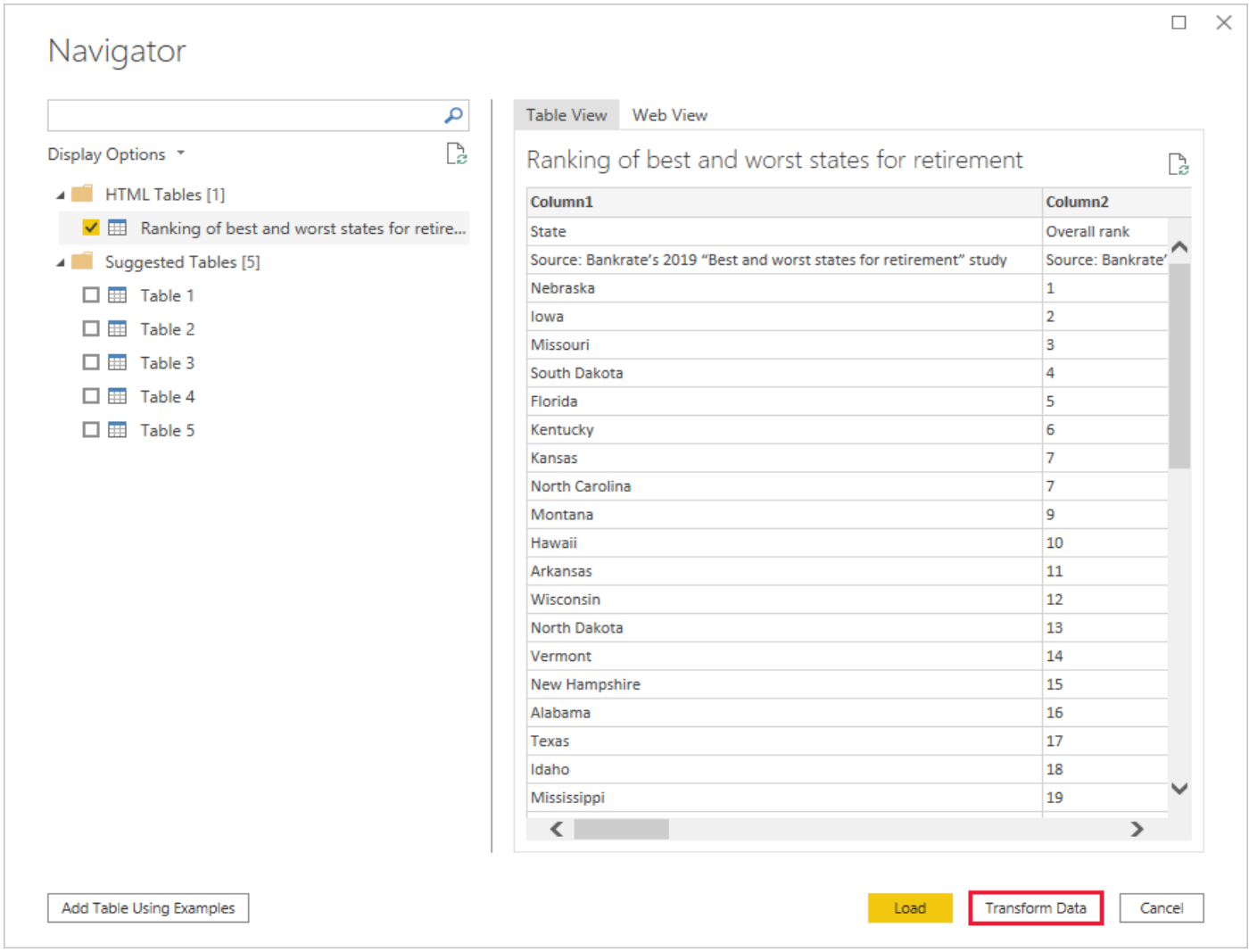
In the From Web dialog box, paste the address https://www.bankrate.com/retirement/best-and-worst-states-for-retirement/ into the URL field, and select OK.



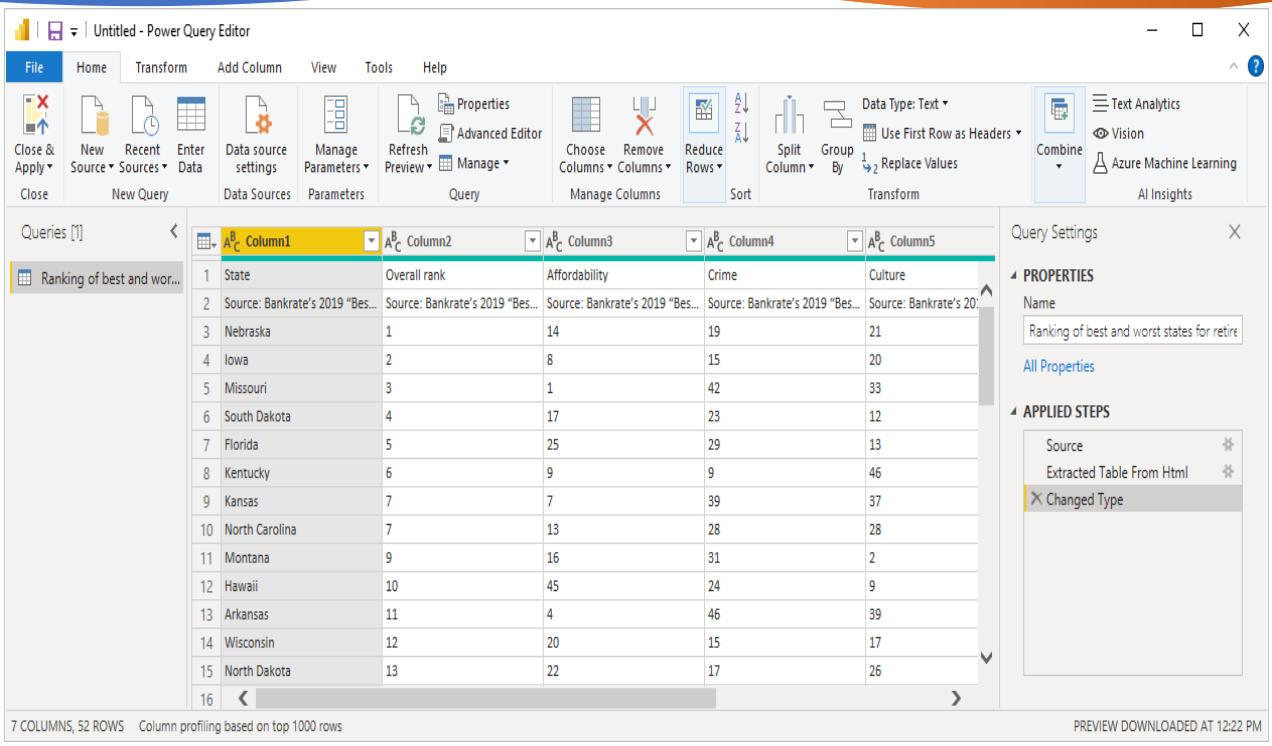
If prompted, on the Access Web Content screen, select Connect to use anonymous access.

The query functionality of Power BI Desktop goes to work and contacts the web resource. The Navigator window returns what it found on the web page, in this case an HTML table called Ranking of best and worst states for retirement, and five other suggested tables. You're interested in the HTML table, so select it to see a preview.

At this point you can select Load to load the table, or Transform data to make changes in the table before you load it.

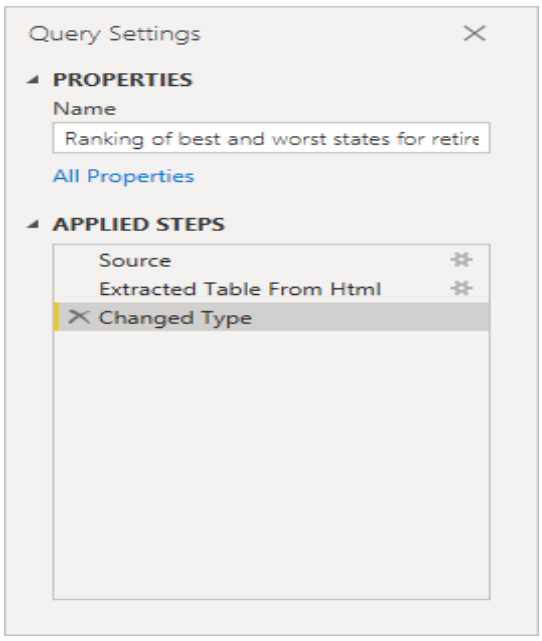


When you select Transform data, Power Query Editor launches, with a representative view of the table. The Query Settings pane is on the right, or you can always show it by selecting Query Settings on the View tab of Power Query Editor.

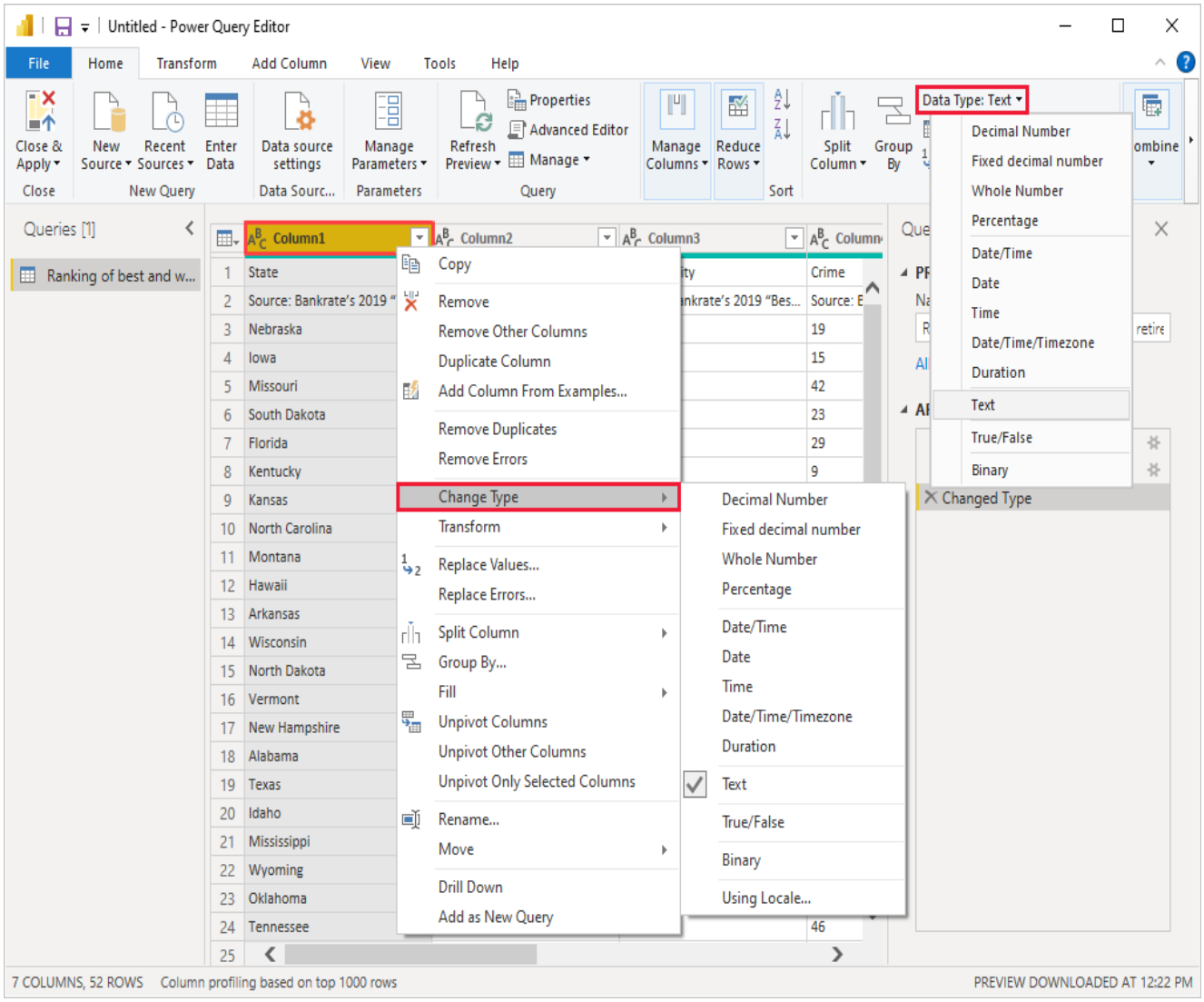


Shape data

Shaping can mean transforming the data, such as renaming columns or tables, removing rows or columns, or changing data types. Power Query Editor captures these steps sequentially under Applied Steps in the Query Settings pane. Each time this query connects to the data source, those steps are carried out, so the data is always shaped the way you specify. This process occurs when you use the query in Power BI Desktop, or when anyone uses your shared query, such as in the Power BI service.

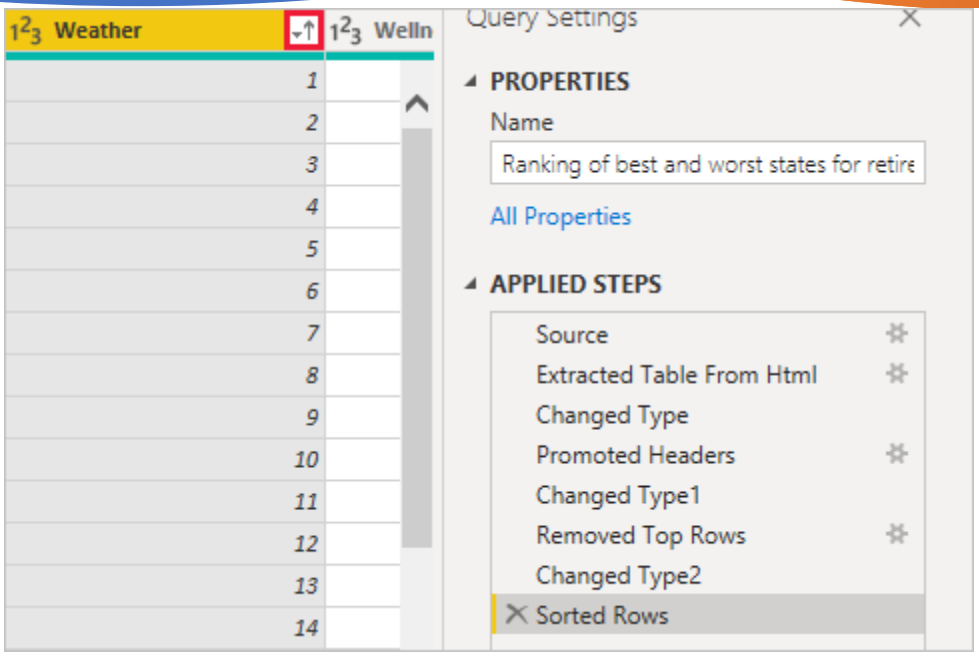


If you need to change a data type, select the column or columns to change. Hold down the Shift key to select several adjacent columns, or Ctrl to select non-adjacent columns. Either right-click a column header, select Change Type, and choose a new data type from the menu, or drop down the list next to Data Type in the Transform group of the Home tab, and select a new data type.

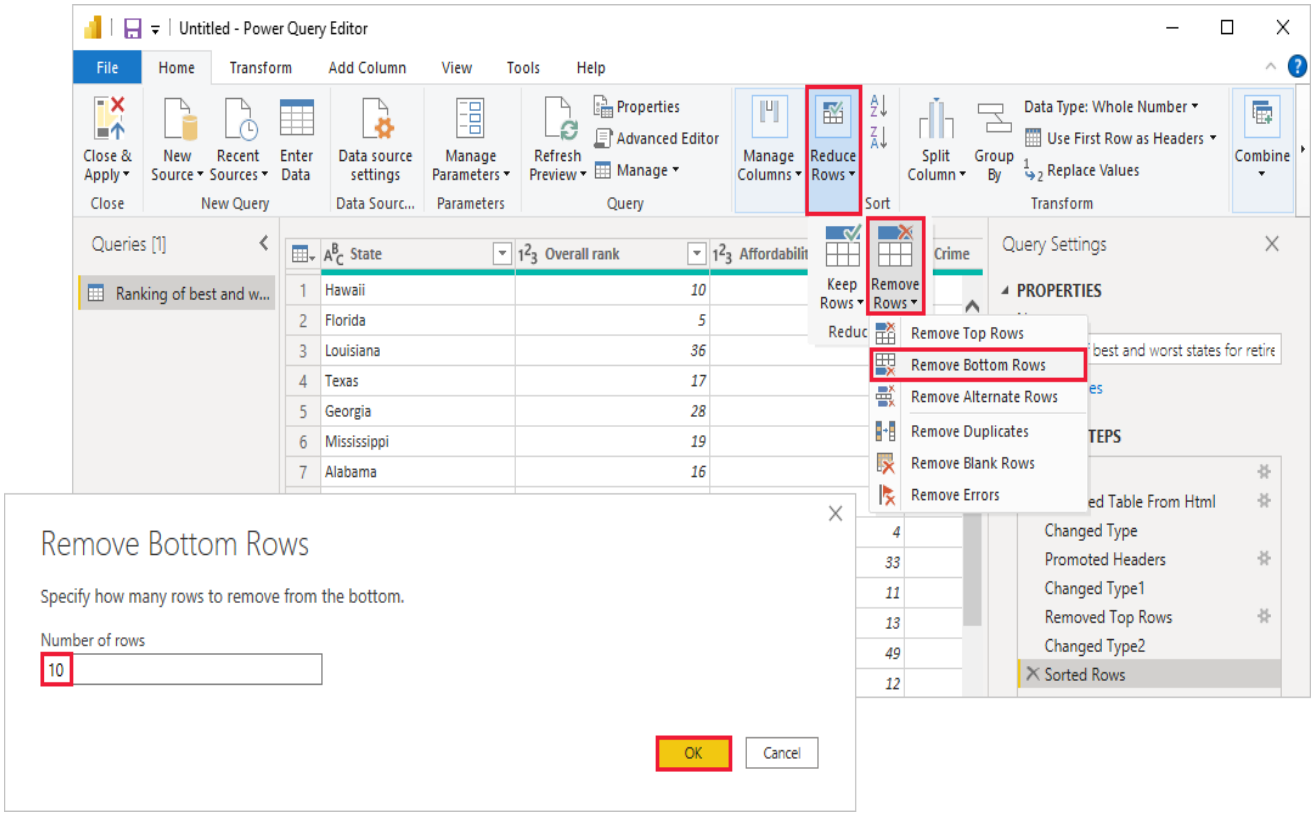


The Power Query Editor in Power BI Desktop uses the ribbon or the right-click menus for available tasks. Most of the tasks you can select on the Home or Transform tabs of the ribbon are also available by right-clicking an item and choosing from the menu that appears.

For example, for sunglasses sales you're most interested in the weather ranking, so you decide to sort the table by the Weather column instead of by Overall rank. Drop down the arrow next to the Weather header, and select Sort ascending. The data now appears sorted by weather ranking, and the step Sorted Rows appears in Applied Steps.



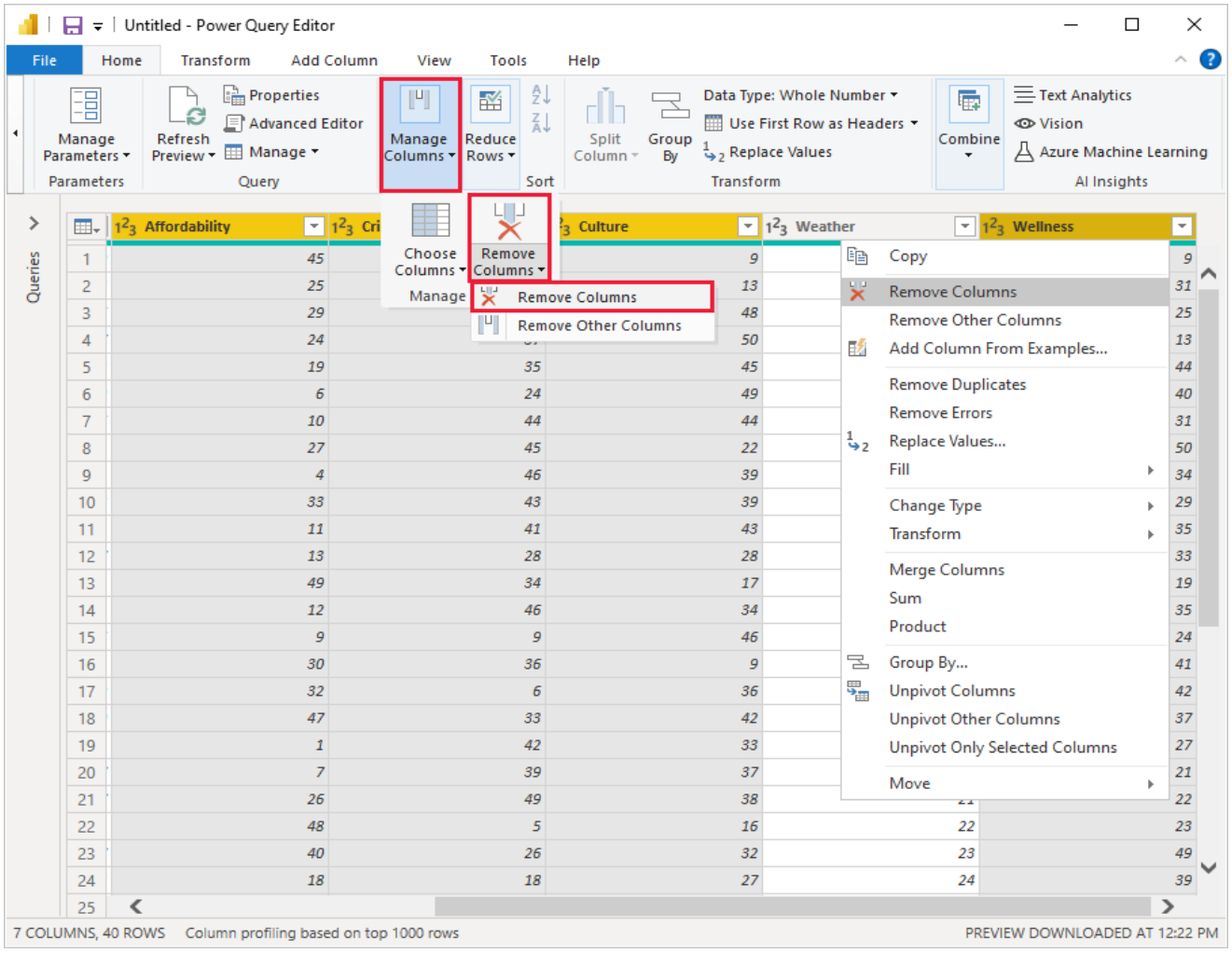
You're not very interested in selling sunglasses to the worst weather states, so you decide to remove them from the table. From the the Home tab, select Reduce Rows > Remove Rows > Remove Bottom Rows. In the Remove Bottom Rows dialog box, enter 10, and then select OK.



The bottom 10 worst weather rows are removed from the table, and the step Removed Bottom Rows appears in Applied Steps.

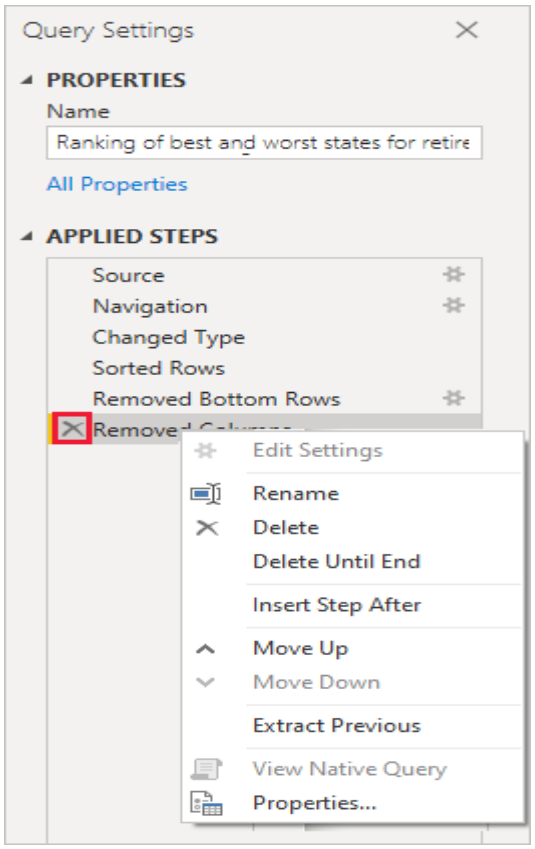
You decide the table has too much extra information for your needs, and to remove the Affordability, Crime, Culture, and Wellness columns. Select the header of each column that you want to remove. Hold down the Shift key to select several adjacent columns, or Ctrl to select non-adjacent columns.

Then, from the Manage Columns group of the Home tab, select Remove Columns. You can also right-click one of the selected column headers and select Remove Columns from the menu. The selected columns are removed, and the step Removed Columns appears in Applied Steps.

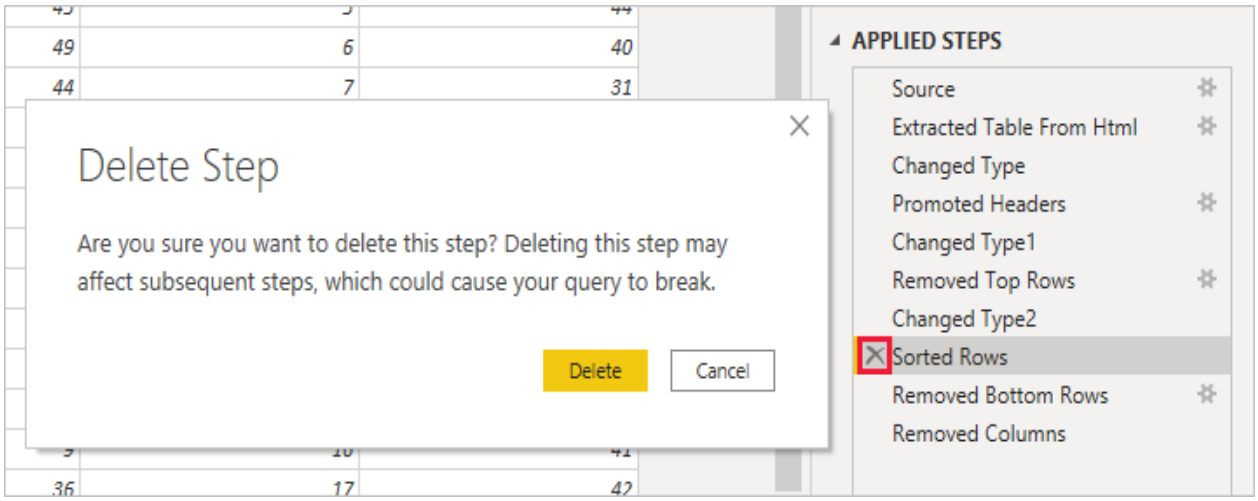


On second thought, Affordability might be relevant to sunglasses sales after all. You'd like to get that column back. You can easily undo the last step in the Applied Steps pane by selecting the X delete icon next to the step. Now redo the step, selecting only the columns you want to delete. For more flexibility, you could delete each column as a separate step.

You can right-click any step in the Applied Steps pane and choose to delete it, rename it, move it up or down in the sequence, or add or delete steps after it. For intermediate steps, Power BI Desktop will warn you if the change could affect later steps and break your query.

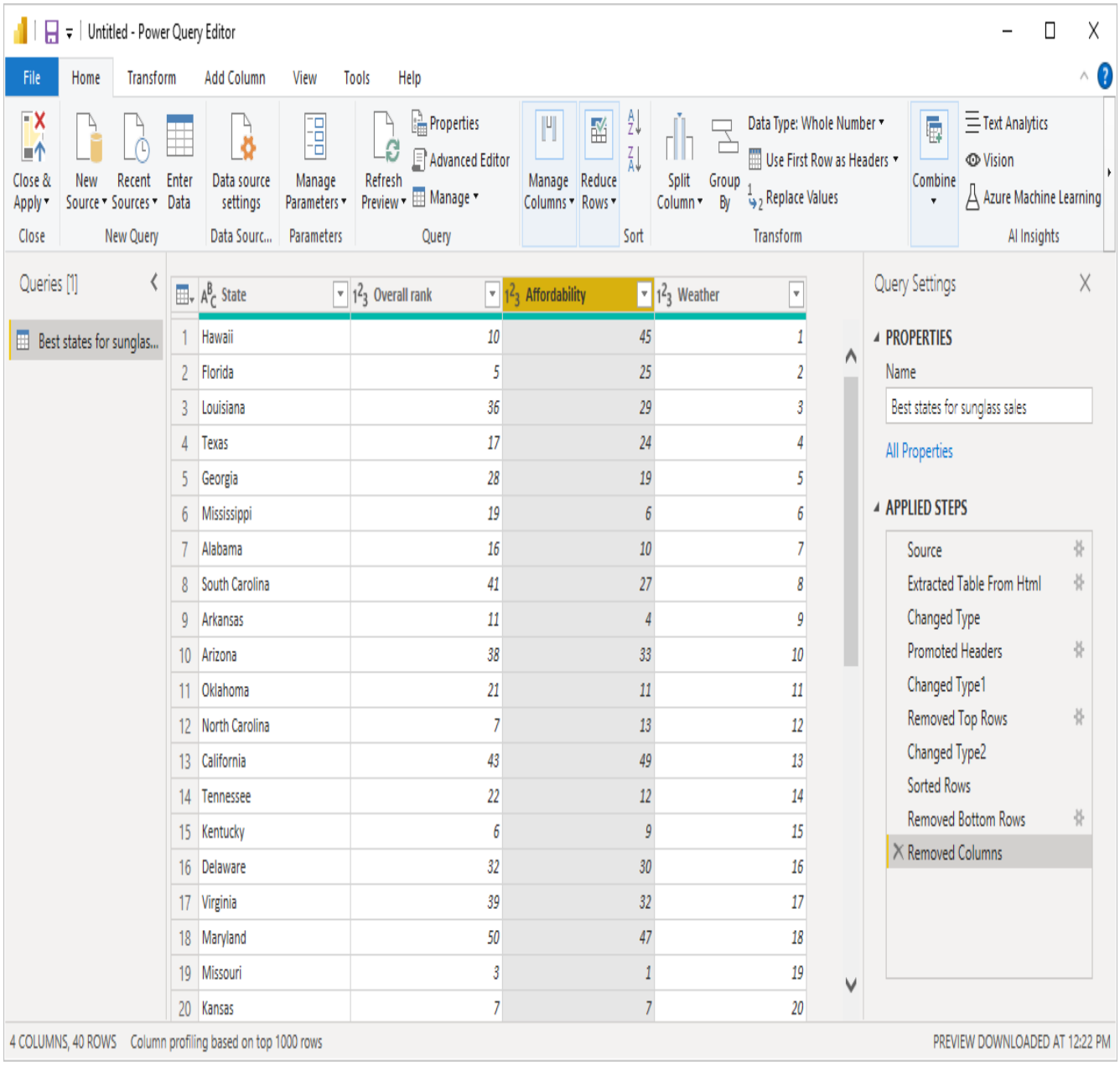


For example, if you no longer wanted to sort the table by **Weather**, you might try to delete the **Sorted Rows** step. Power BI Desktop warns you that deleting this step could cause your query to break. You removed the bottom 10 rows after you sorted by weather, so if you remove the sort, different rows will be removed. You also get a warning if you select the **Sorted Rows** step and try to add a new intermediate step at that point.



Finally, you change the table title to be about sunglass sales instead of retirement. Under Properties in the Query Settings pane, replace the old title with Best states for sunglass sales.

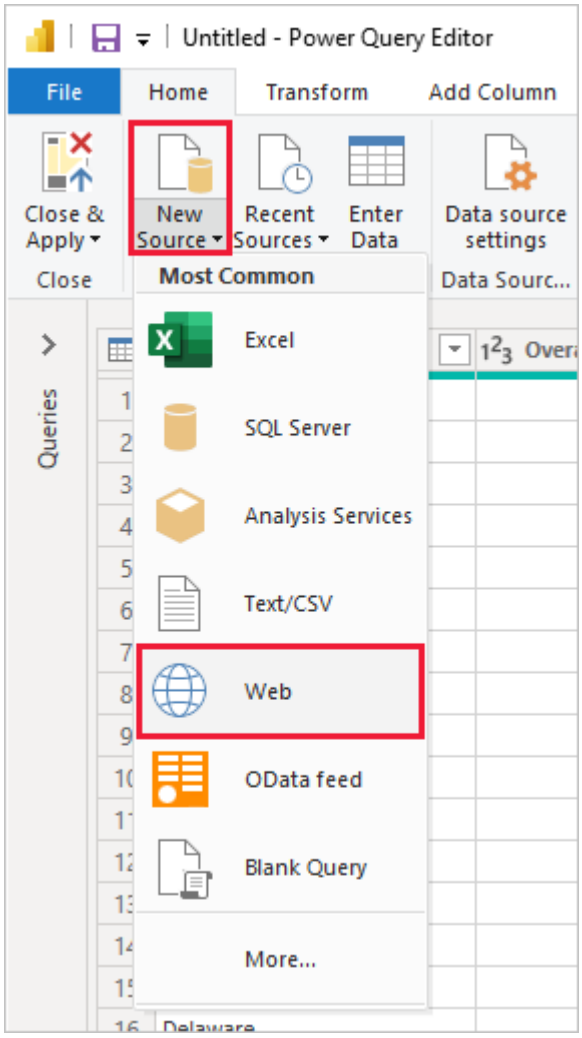
The finished query for your shaped data looks like this:



Combine data

The data about various states is interesting, and will be useful for building additional analysis efforts and queries. But there's one problem: most data out there uses two-letter abbreviations for state codes, not the full names of the states. To use that data, you need some way to associate your state names with their abbreviations.

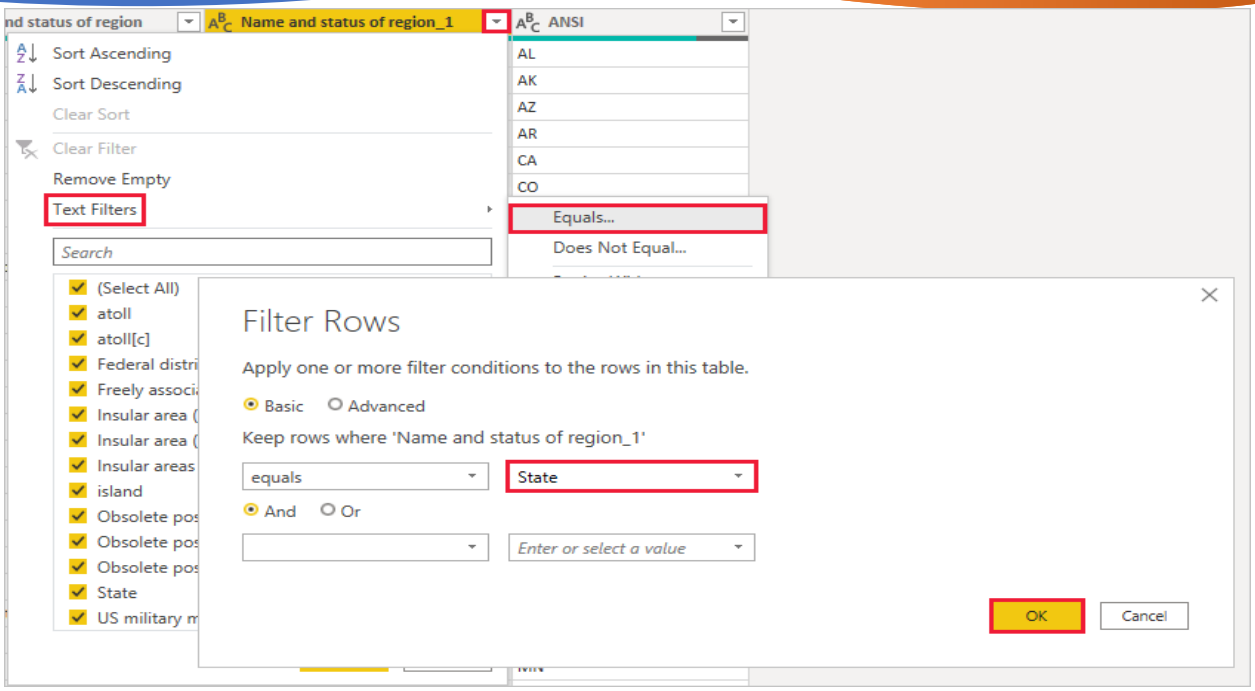
To import the state abbreviations data into Power Query Editor, select New Source > Web from the New Query group on the Home tab of the ribbon.



In the Navigator window, select the table Codes and abbreviations for U.S. states, federal district, territories, and other regions, and then select OK. The table opens in Power Query Editor.

Remove all columns except for Name and status of region, Name and status of region, and ANSI. To keep only these columns, hold down Ctrl and select the columns. Then, either right-click one of the column headers and select Remove Other Columns, or, from the Manage Columns group of the Home tab, select Remove Other Columns.

Drop down the arrow next to the Name and status of region_1 column header, and select Filters > Equals. In the Filter Rows dialog box, drop down the Enter or select a value field next to equals and select State. Select OK.



With extra values like Federal district and island removed, you now have a list of the 50 states and their official two-letter abbreviations. You can rename the columns to make more sense, for example State name, Status, and Abbreviation, by right-clicking the column headers and selecting Rename.

Note that all of these steps are recorded under Applied Steps in the Query Settings pane.

Your shaped table now looks like this:

	State name	Status	Abbreviation
1	Alabama	State	AL
2	Alaska	State	AK
3	Arizona	State	AZ
4	Arkansas	State	AR
5	California	State	CA
6	Colorado	State	CO
7	Connecticut	State	CT
8	Delaware	State	DE
9	Florida	State	FL
10	Georgia	State	GA
11	Hawaii	State	HI
12	Idaho	State	ID
13	Illinois	State	IL
14	Indiana	State	IN
15	Iowa	State	IA
16	Kansas	State	KS
17	Kentucky	State	KY

Retitle the table to State codes in the Properties field of Query Settings.

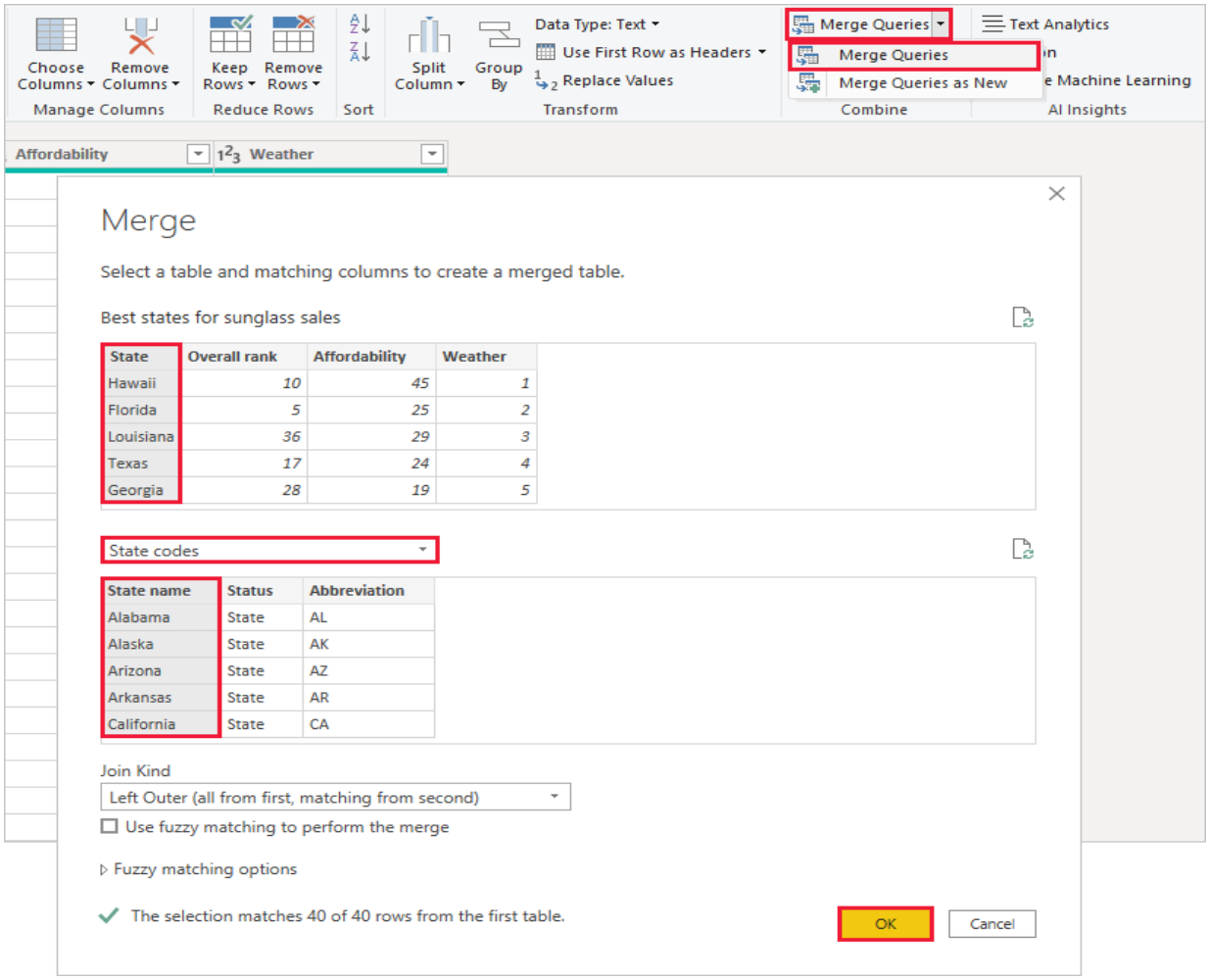
With the State codes table shaped, you can combine these two tables into one. Since the tables you now have are a result of queries you applied to the data, they're also called queries. There are two primary ways of combining queries: merge and append.

When you have one or more columns you'd like to add to another query, you merge the queries. When you have additional rows of data you'd like to add to an existing query, you append the query.

In this case, you want to merge the State codes query into the Best states for sunglasses query. To merge the queries, switch to the Best states for sunglasses query by selecting it from the Queries pane on the left side of Power Query Editor. Then select Merge Queries from the Combine group in the Home tab of the ribbon.

In the Merge window, drop down the field to select State codes from the other queries available. Select the column to match from each table, in this case State from the Best states for sunglasses query and State name from the State codes query.

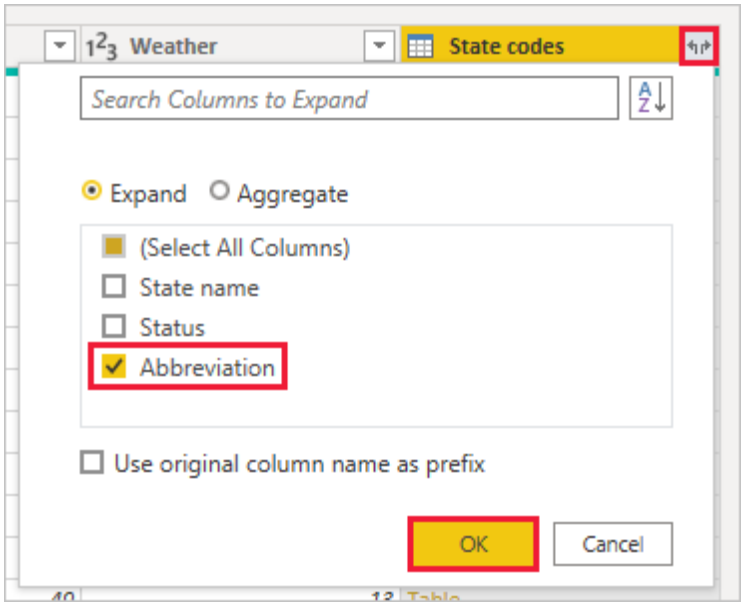
If you get a Privacy levels dialog, select Ignore privacy levels checks for this file and then select Save. Select OK.



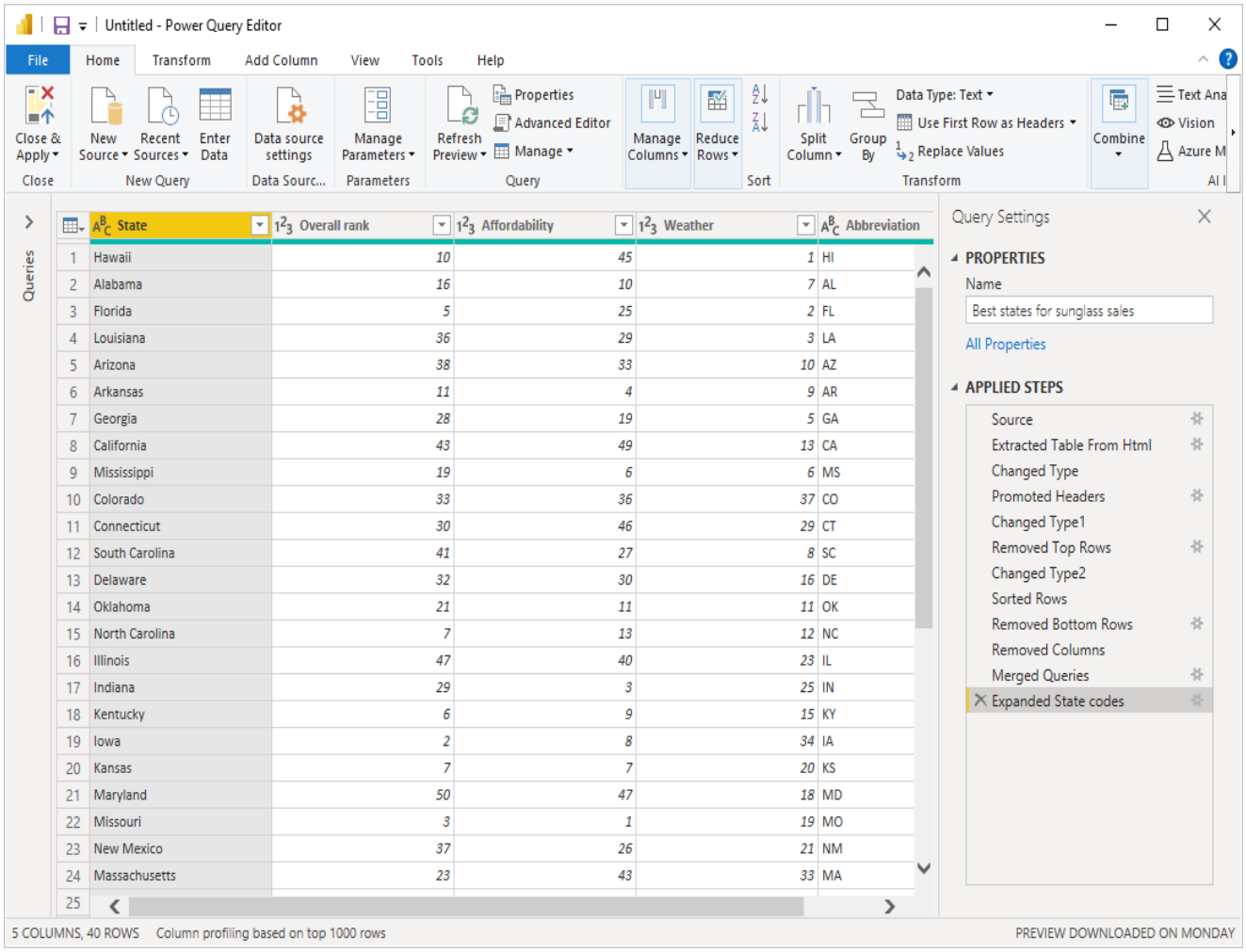
A new column called State codes appears on the right of the Best states for sunglass sales table. It contains the state code query that you merged with the best states for sunglass sales query. All the columns from the merged table are condensed into the State codes column. You can expand the merged table and include only the columns you want.

	State	Overall rank	Affordability	Weather	State codes
1	Hawaii	10	45	1	Table
2	Florida	5	25	2	Table
3	Louisiana	36	29	3	Table
4	Texas	17	24	4	Table
5	Georgia	28	19	5	Table
6	Mississippi	19	6	6	Table
7	Alabama	16	10	7	Table
8	South Carolina	41	27	8	Table
9	Arkansas	11	4	9	Table
10	Arizona	38	33	10	Table
11	Oklahoma	21	11	11	Table
12	North Carolina	7	13	12	Table
13	California	43	49	13	Table
14	Tennessee	22	12	14	Table
15	Kentucky	6	9	15	Table

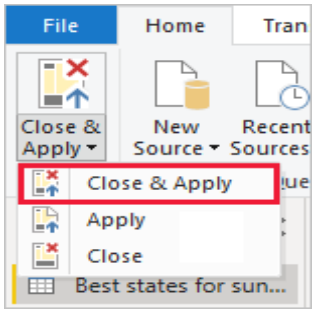
To expand the merged table and select which columns to include, select the Expand icon in the column header. In the Expand dialog box, select only the Abbreviation column. Deselect Use original column name as prefix, and then select OK.



You now have a single query table that combines two data sources, each of which has been shaped to meet your needs. This query can serve as a basis for lots of additional, interesting data connections, such as demographics, wealth levels, or recreational opportunities in the states.



For now, you have enough data to create an interesting report in Power BI Desktop. Since this is a milestone, apply the changes in Power Query Editor and load them into Power BI Desktop by selecting Close & Apply from the Home tab of the ribbon. You can also select just Apply to keep the query open in Power Query Editor while you work in Power BI Desktop.

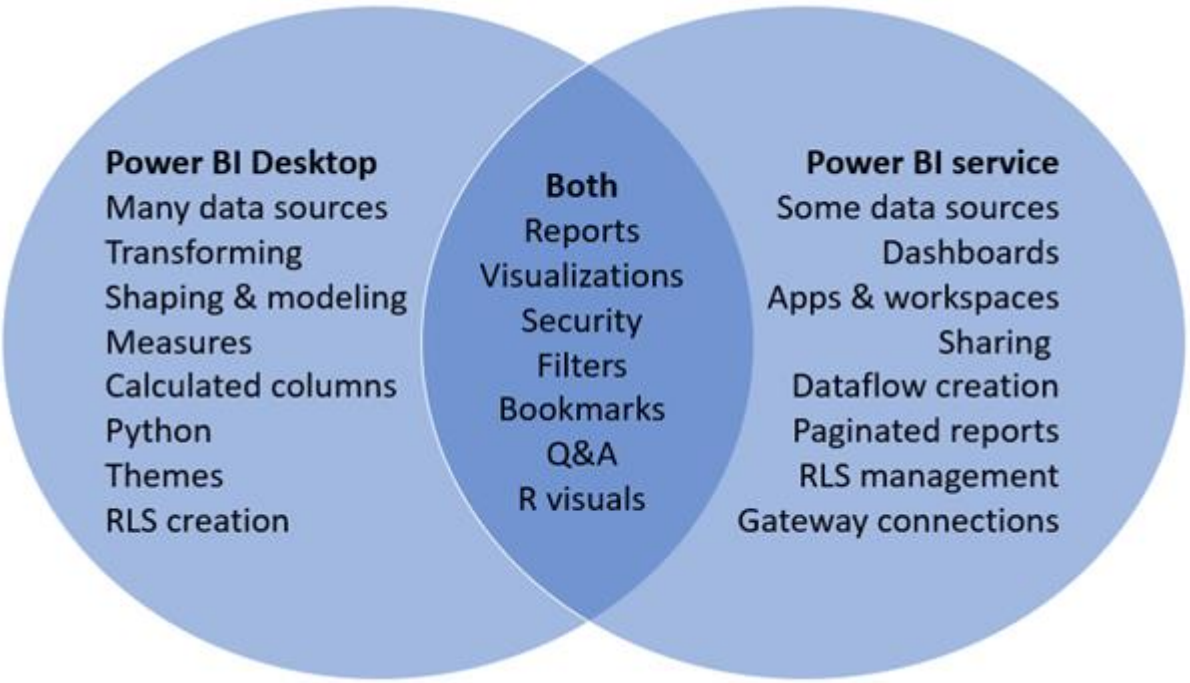


Comparing Power BI Desktop and the Power BI service

Power BI Desktop is an application that you download and install for free on your local computer. Desktop is a complete data analysis and report creation tool that is used to connect to, transform, visualize, and analyze your data. It includes the Query Editor, in which you can connect to many different sources of data, and combine them (often called Modelling) into a data model. Then you design a report based on that data model. Reports can be shared with others directly or by publishing to the Power BI service. The Power BI Desktop getting started guide walks through the process.

The Power BI service is a cloud-based service, or software as a service (SaaS). It supports report editing and collaboration for teams and organizations. You can connect to data sources in the Power BI service, too, but Modelling is limited. The Power BI service is used to do things such as creating dashboards, creating and sharing apps, analyzing and exploring your data to uncover business insights, and much more. What is the Power BI service details many of the capabilities of the Power BI service.

In a Venn diagram comparing Power BI Desktop and the Power BI service, the area in the middle shows some of the areas where the two overlap. Some tasks you can do in either Power BI Desktop or the service. The two outer sides of the Venn diagram show the features that are unique to either the Desktop application or to the Power BI service.



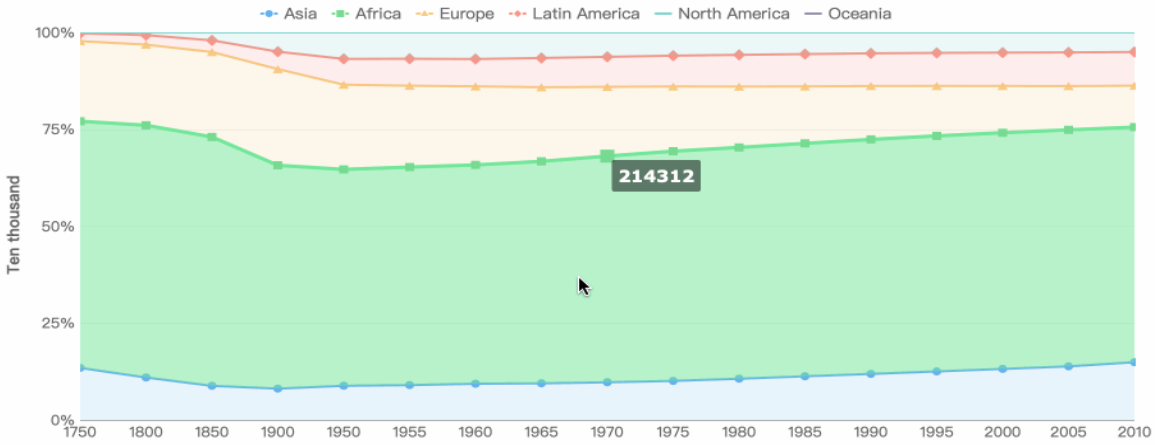
5.7 Different types of visualization charts

1. Column Chart
2. Bar Chart
3. Line Chart
4. Area Chart
5. Pie Chart
6. Scatter Plot
7. Bubble Chart
8. Gauge
9. Radar Chart
10. Frame Diagram
11. Rectangular Tree Diagram
12. Funnel Chart
13. Word Cloud Chart
14. Gantt Chart

5.8 Area Chart

The area chart is formed on the basis of the line chart. It fills the area between the polyline and the axis in the line chart with color. The filling of the color can better highlight the trend information.

The fill color of the area chart should have a certain transparency. The transparency can help the user to observe the overlapping relationship between different series. The area without transparency will cause the different series to cover each other.

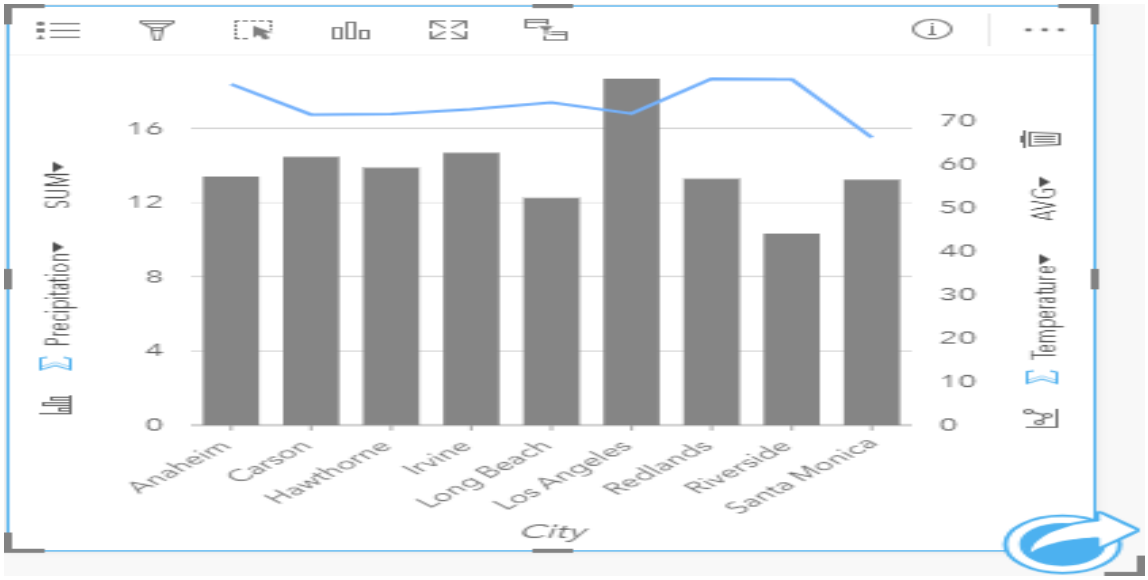


Combo Charts: A combo chart is a combination of two column charts, two line graphs, or a column chart and a line graph.

Example:

An environmental organization is tracking the drought conditions in Southern California and wants to compare temperatures and precipitation to determine which cities are most

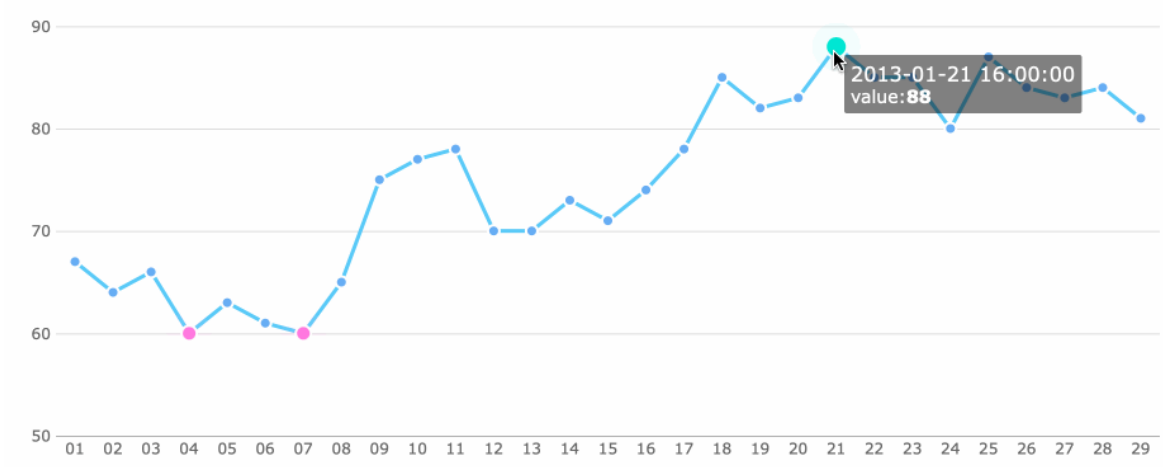
vulnerable. The organization uses a combo chart to show both the total precipitation and the average temperature for each city in one chart.



5.9 Line Chart

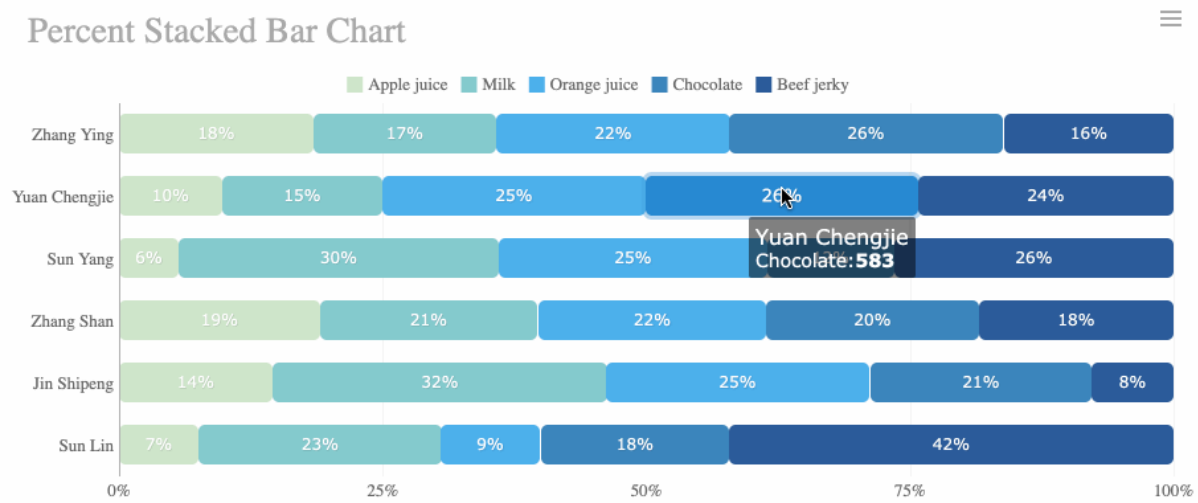
A line chart is used to show the change of data over a continuous time interval or time span. It is characterized by a tendency to reflect things as they change over time or ordered categories.

It should be noted that the number of data records of the line graph should be greater than 2, which can be used for trend comparison of large data volume. And it is better not to exceed 5 polylines on the same graph.



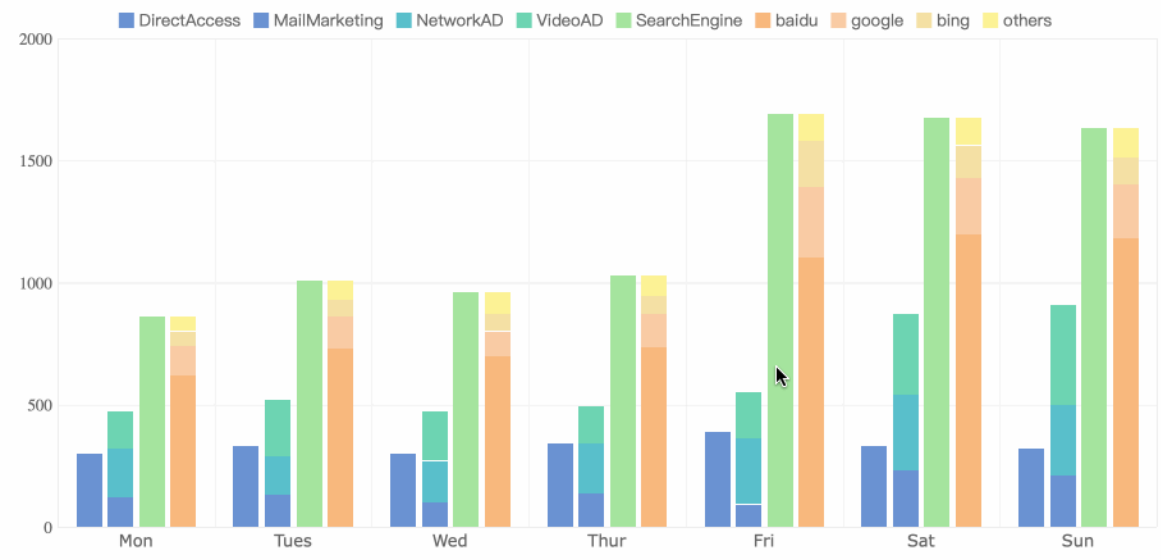
5.10 Bar Chart

Bar charts are similar to column charts, but the number of bars can be relatively large. Compared with the column chart, the positions of its two axes are changed.



5.11 Column Chart

Column charts use vertical columns to show numerical comparisons between categories, and the number of columns should not be too large (the labels of the axis may appear incomplete if there are too many columns).



5.12 Combo Chart

In Power BI, a combo chart is a single visualization that combines a line chart and a column chart. Combining the two charts into one lets you make a quicker comparison of the data.

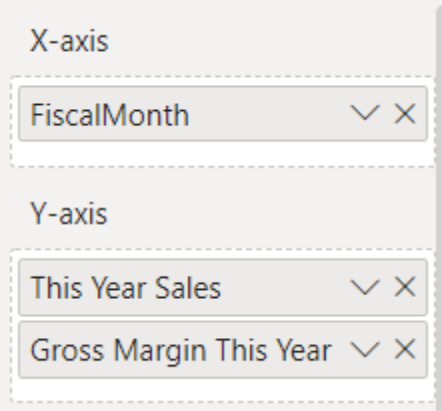
Create a basic single-axis combo chart

1. Start on a blank report page and create a column chart that displays this year's sales and gross margin by month.

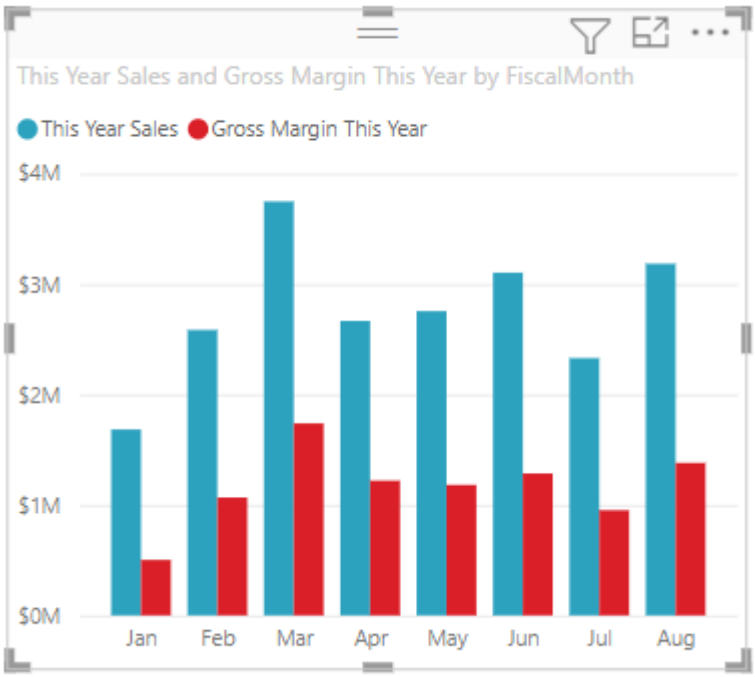
   a. From the Fields pane, select **Sales** > **This Year Sales** > **Value**.

   b. Select **Sales** > **Gross Margin This Year** and drag it to the **Y-axis** well.

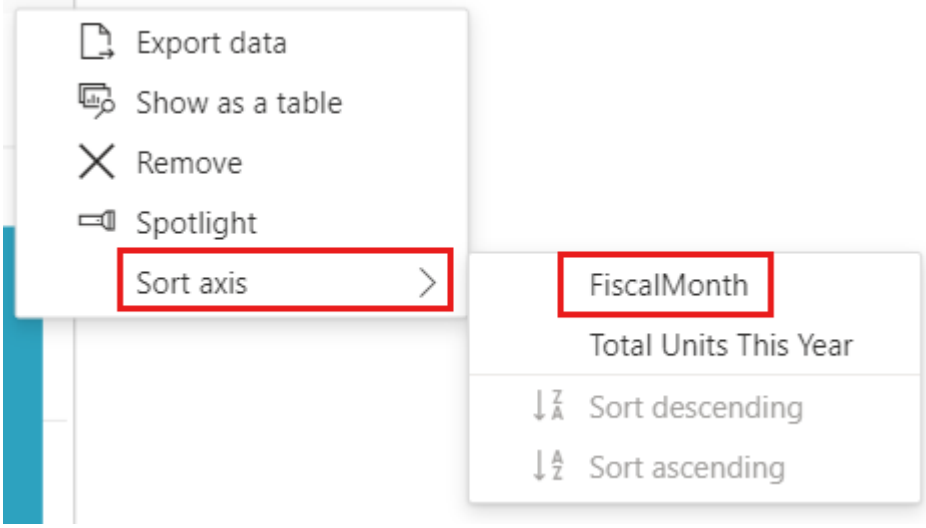
   c. Select **Time** > **FiscalMonth** and drag it to the **X-axis** well.



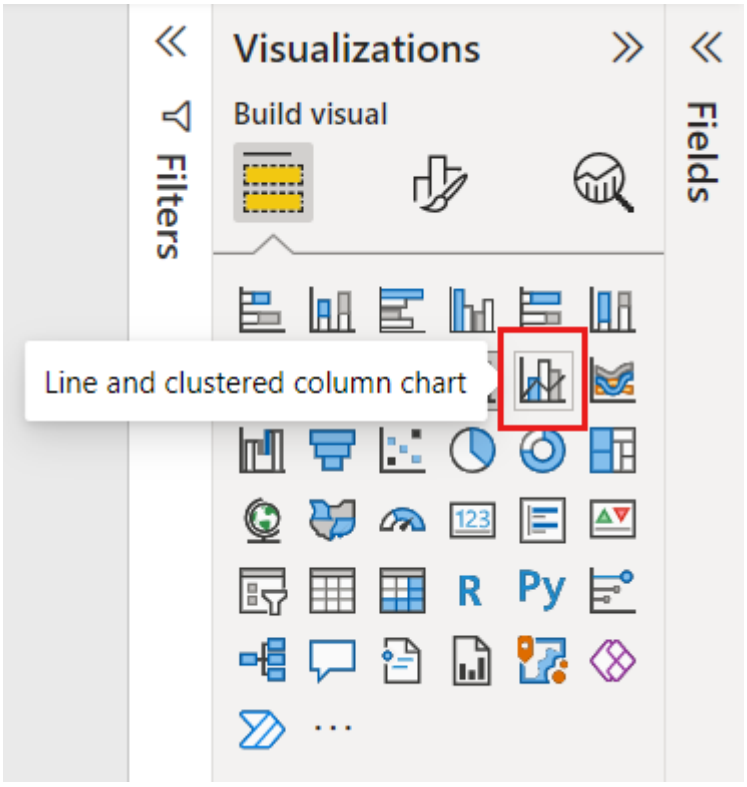
2. The visualization will be similar to this one.



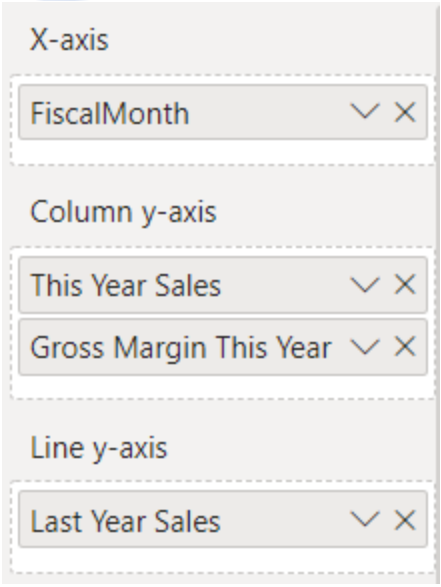
3. In the upper-right corner of the visual, select the **More options** ellipsis (...) and select **Sort axis** > **FiscalMonth**.



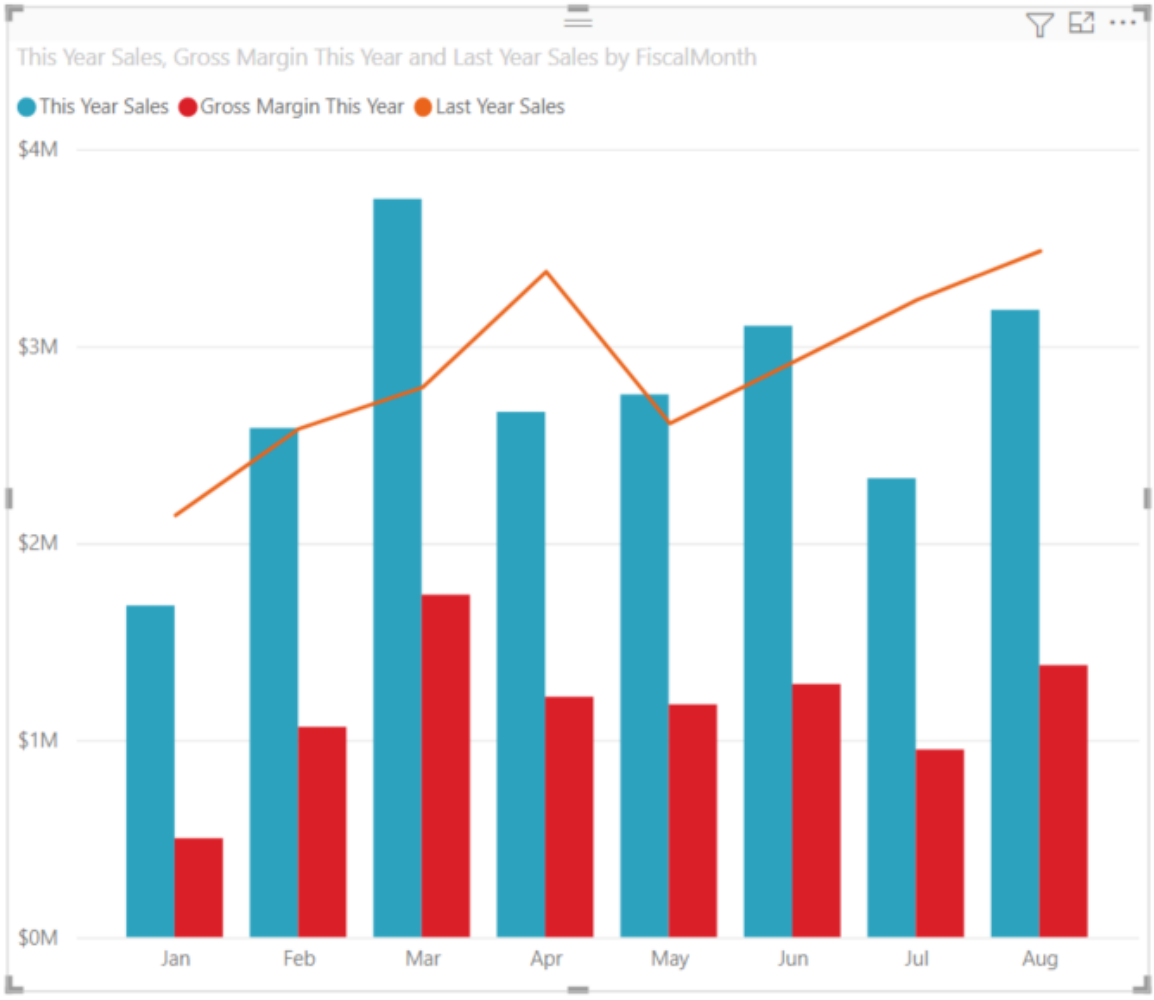
4. Select the ellipsis again and choose **Sort axis** > **Sort ascending**.
5. Convert the column chart to a combo chart. There are two combo charts available: **Line and stacked column** and **Line and clustered column**. With the column chart selected, from the **Visualizations** pane select the **Line and clustered column chart**.



6. From the **Fields** pane, drag **Sales** > **Last Year Sales** to the **Line y-axis** bucket.



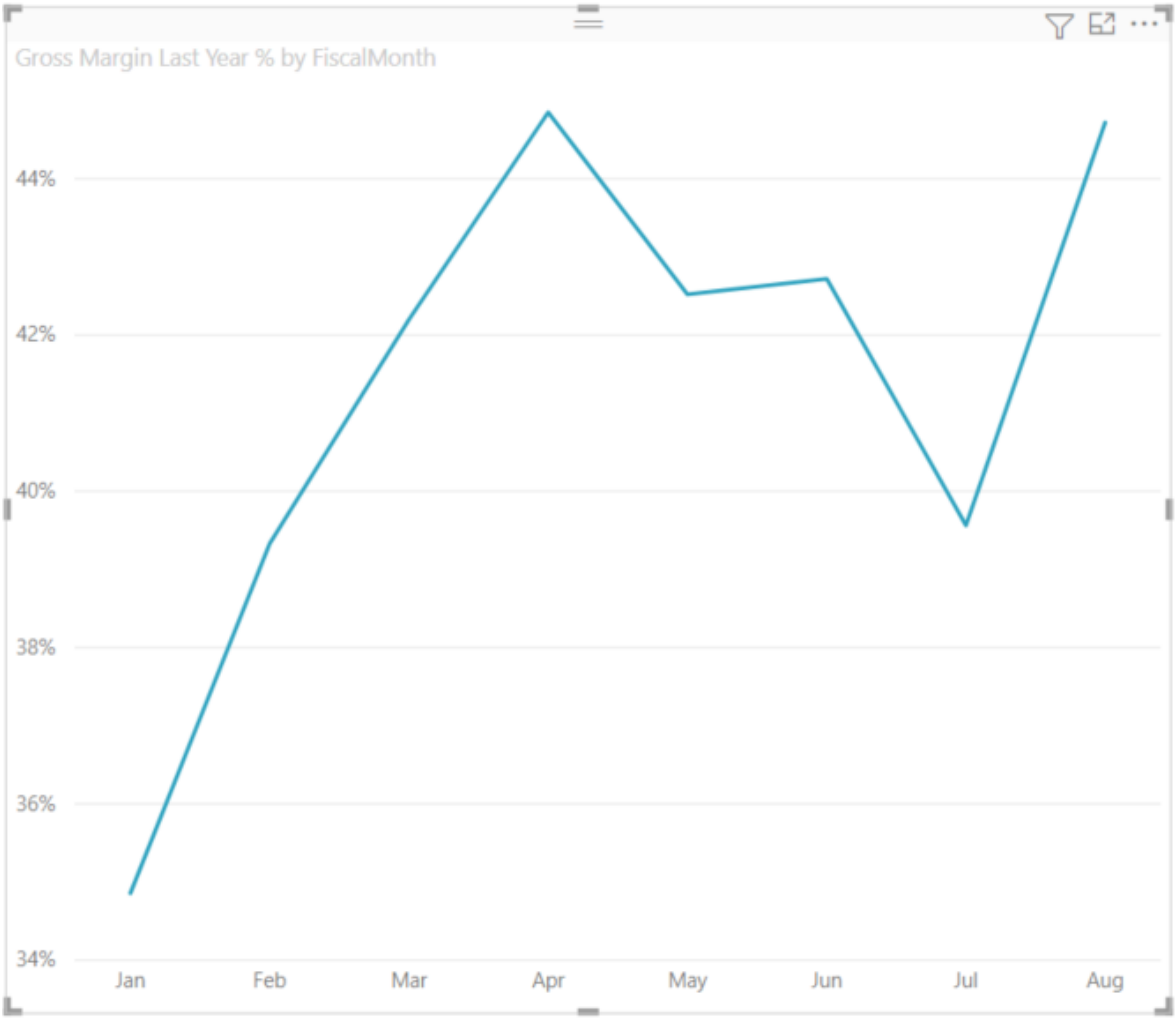
Your combo chart should look something like this:



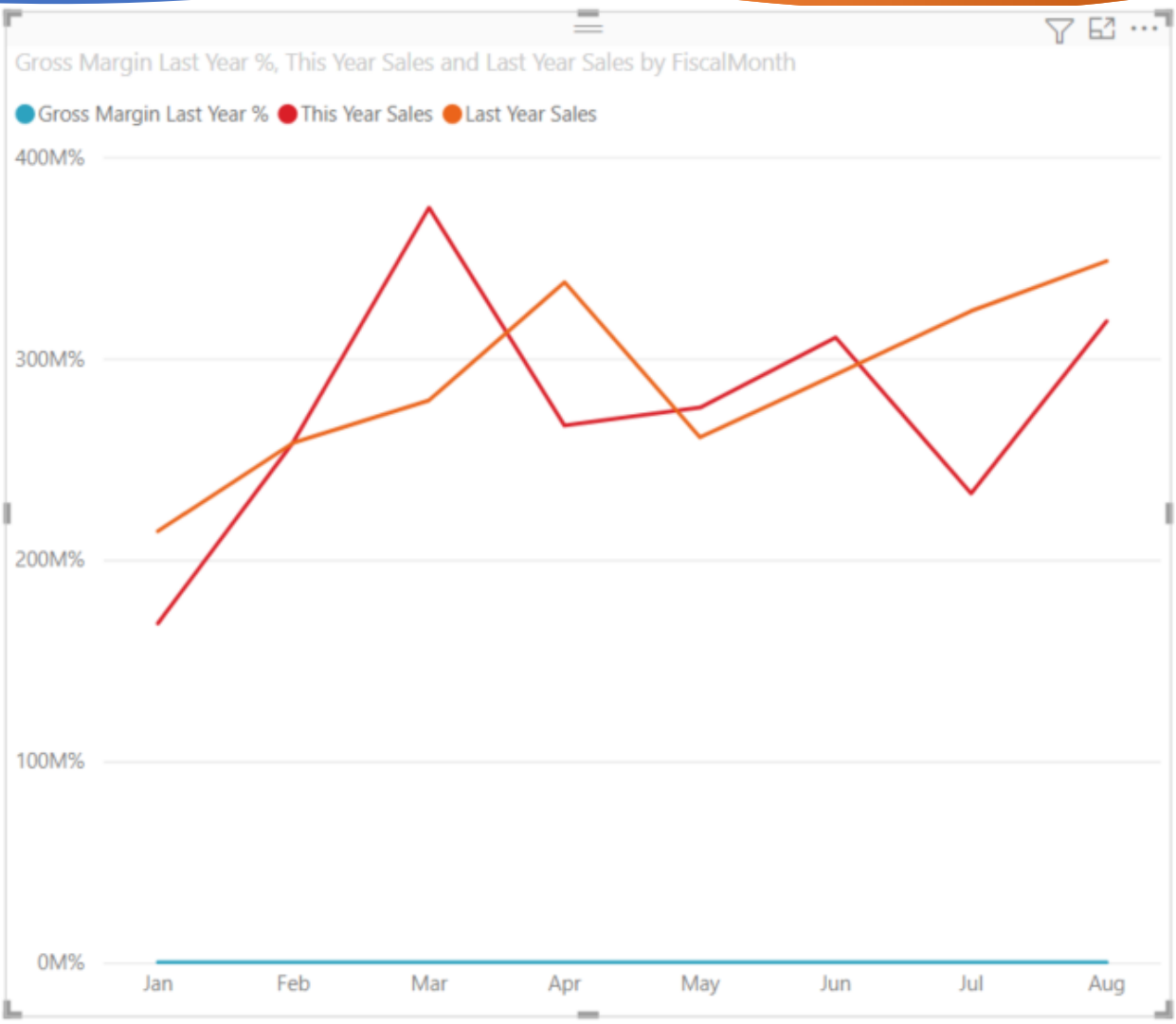
Create a combo chart with two axes

In this task, we'll compare gross margin and sales.

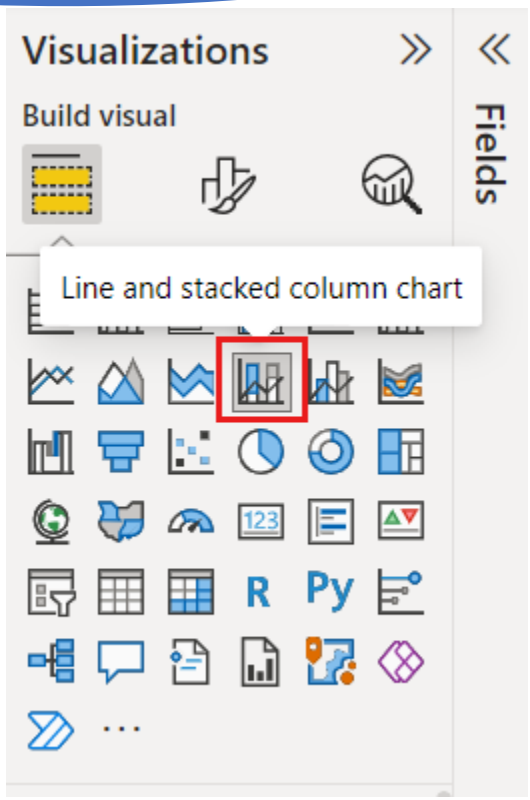
1. Create a new line chart that tracks **Gross Margin Last Year %** by **FiscalMonth**.
2. Select the ellipsis to sort it by **FiscalMonth**, then select the ellipsis again and choose **Sort axis** > **Sort ascending**.
3. In January GM% was 35%, peaked at 45% in April, dropped in July and peaked again in August. Will we see a similar pattern in sales last year and this year?



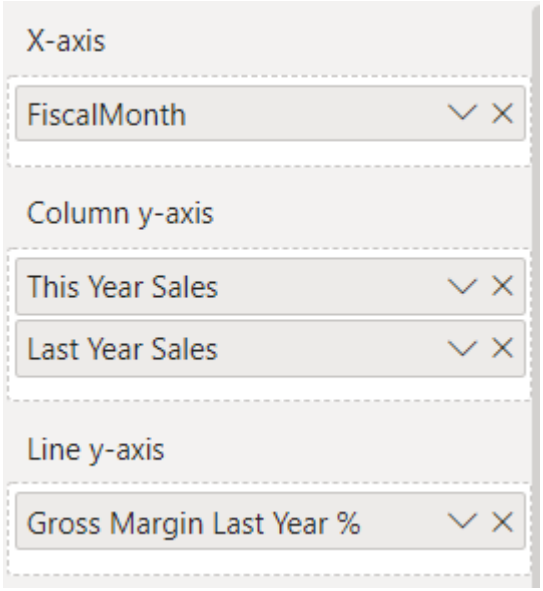
4. Add **This Year Sales** > **Value** and **Last Year Sales** to the line chart. The scale of **Gross Margin Last Year %** is much smaller than the scale of **Sales** which makes it difficult to compare.



5. To make the visual easier to read and interpret, convert the line chart to a **Line and stacked column chart**.



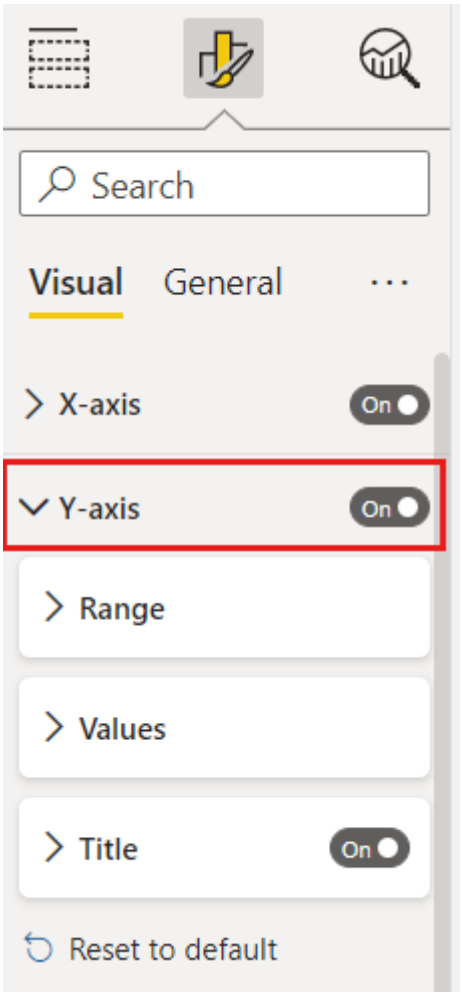
6. Drag **Gross Margin Last Year %** from **Column y-axis** into **Line y-axis**.



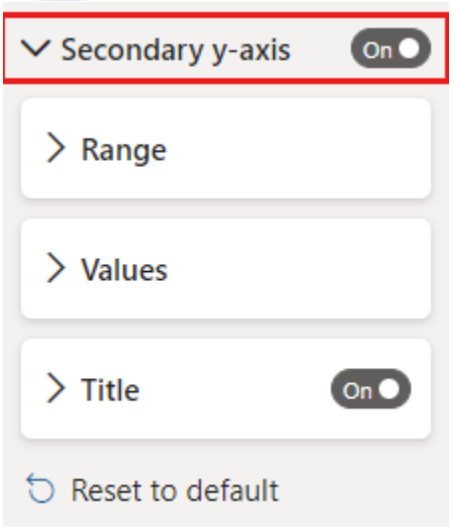
7. Power BI creates two axes, thus allowing the datasets to be scaled differently; the left measures sales dollars and the right measures percentage. And we see the answer to our question: yes, we do see a similar pattern.

Add titles to the axes

1. Select the paintbrush icon  to open the **Formatting** pane.
2. Set **Y-axis** to **On**, then select the down arrow to expand the **Y-axis** options.
3. Set **Y-axis** > **Values** > **Display units** to **Millions**.
4. Set **Y-axis** > **Title** to **On**, then set **Style** to **Show title only**

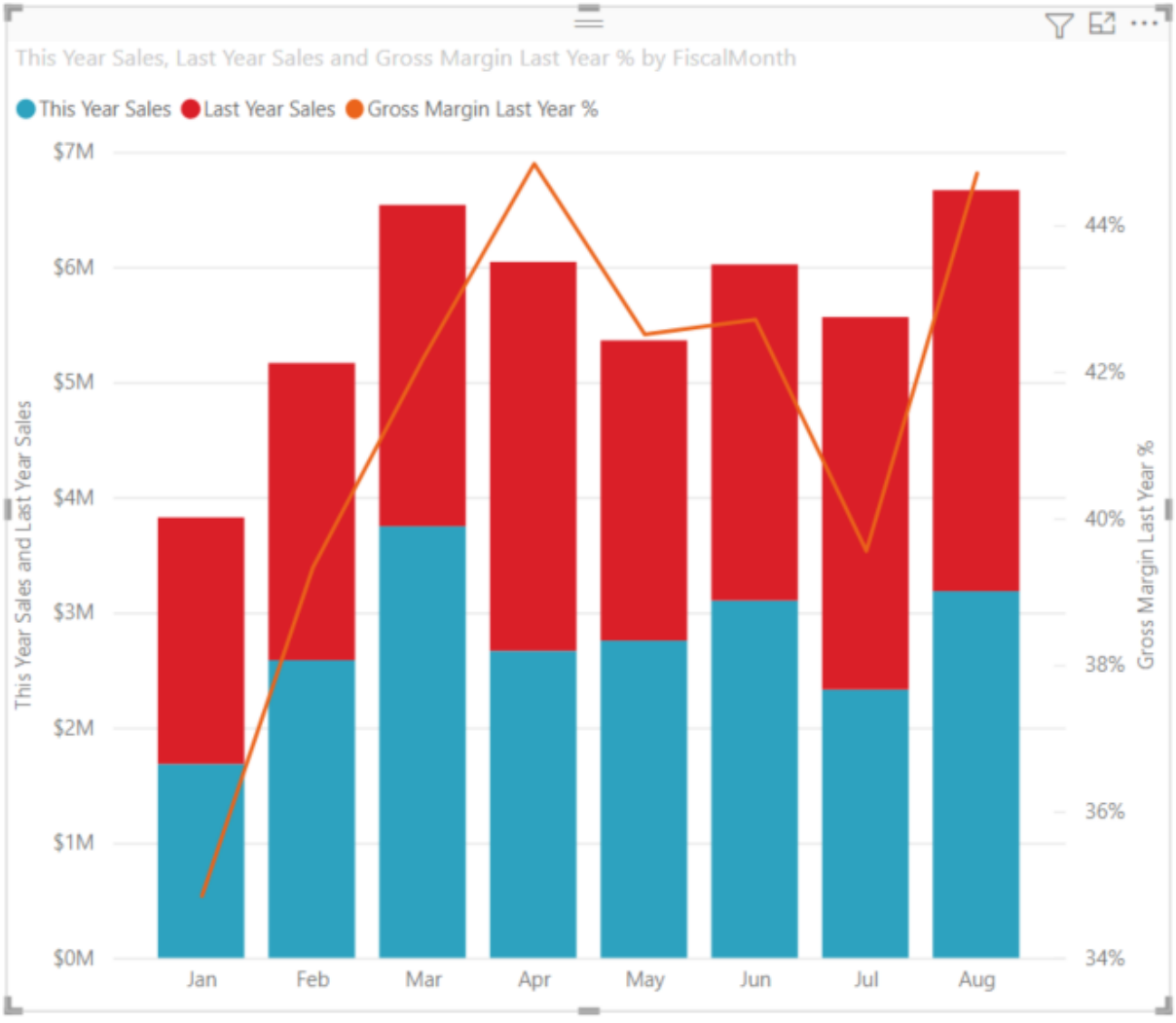


5. Set **Secondary y-axis** to **On** to display options for formatting the line chart portion of the combo chart.



6. Under **Secondary y-axis**, set **Title** to **On**.

   Your combo chart now displays dual axes, both with titles.

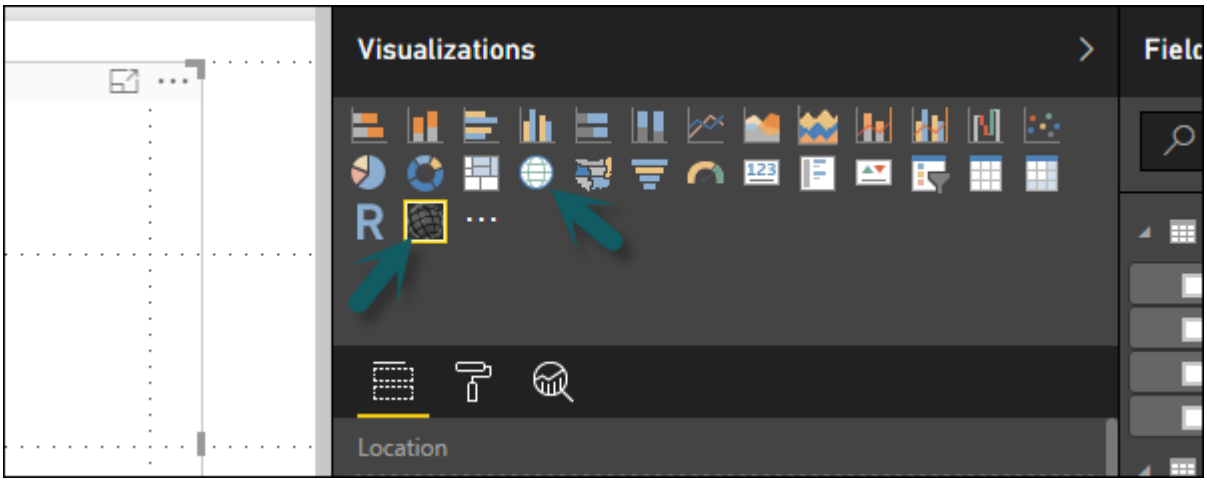


7. Optionally, modify the text font, size, and color and set other formatting options to improve the display and readability of the chart.

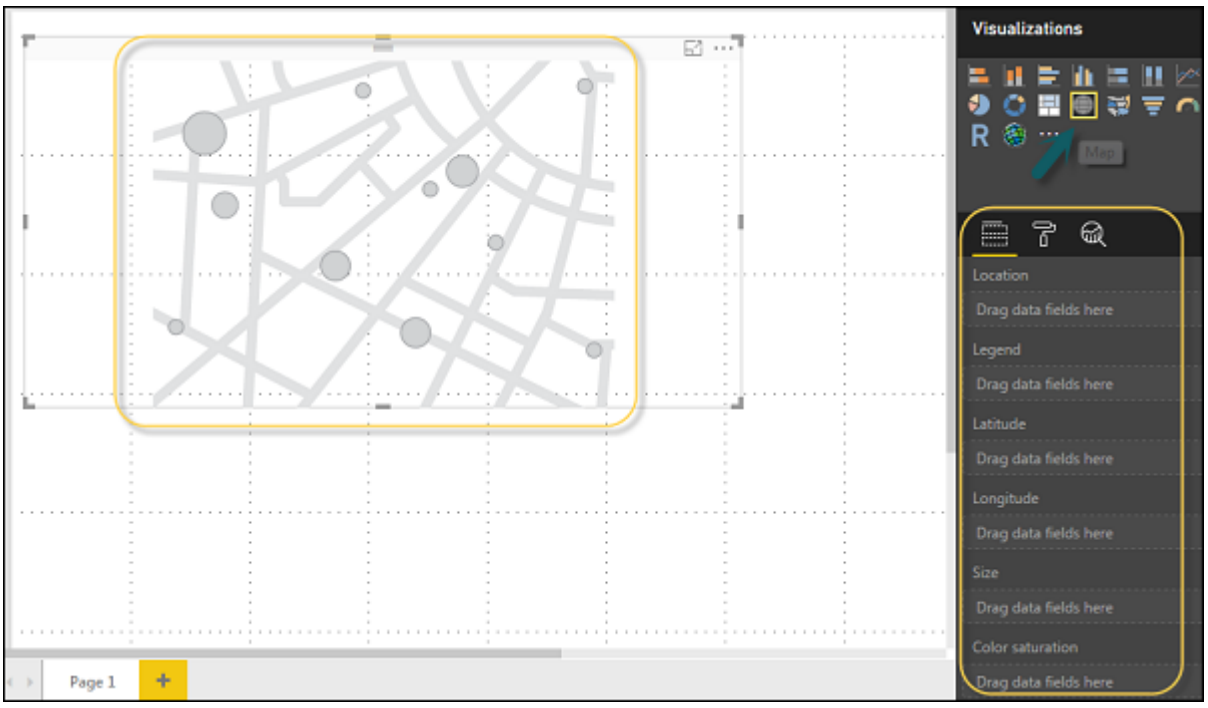
Assignment: Create map visualization using Power BI

Solution:

In Power BI, we have two types of map visualization - bubble maps and shape maps. If you want to create a bubble map, select the map option from the visualization pane.

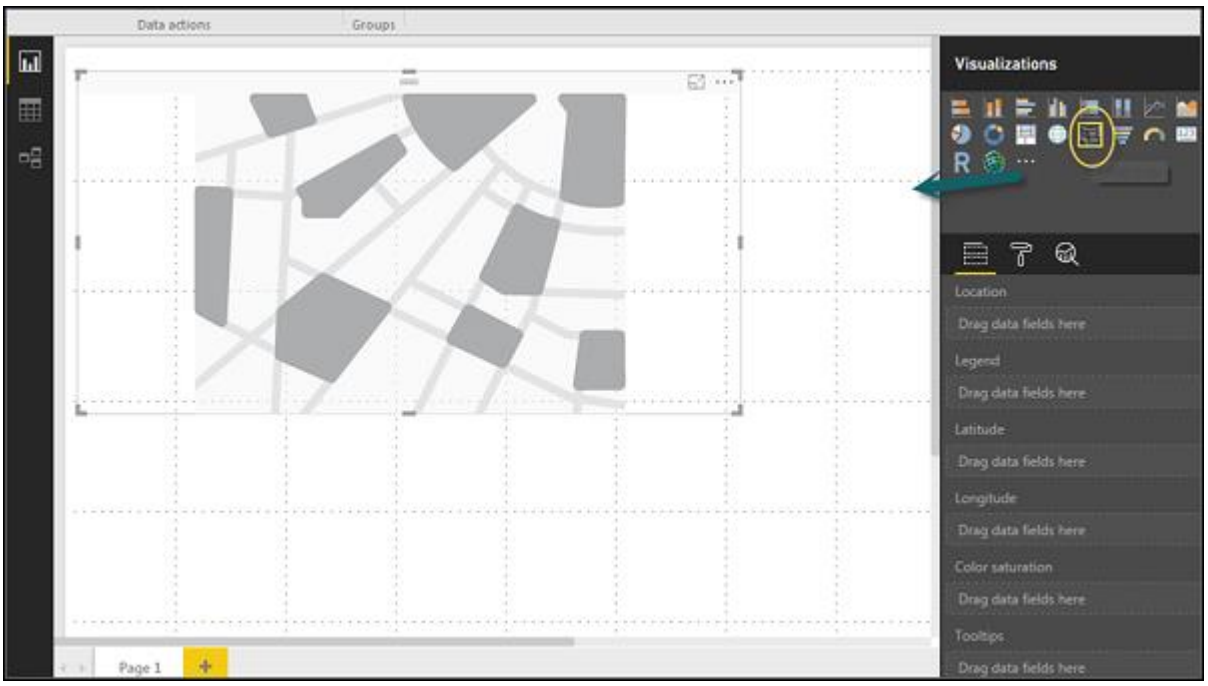


To use a bubble map, drag the map from Visualizations to the Report Canvas. To display values, you have to add any location object to the axis.



In the value fields, you can see that it accepts values axis such as City and State and or you can also add longitude and latitude values. To change the bubble size, you need to add a field to the value axis.

You can also use a filled map in data visualization, just by dragging the filled map to the Report Canvas.



Note – If you see a warning symbol on top of your map visualization, it means that you need to add more locations to your map chart.

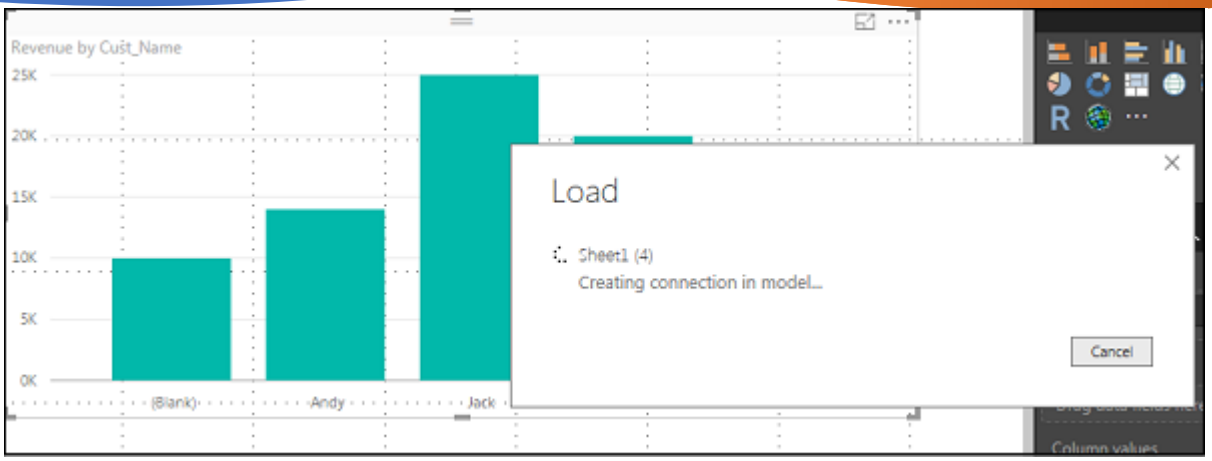
Practice task:

Create combination chart using Power BI

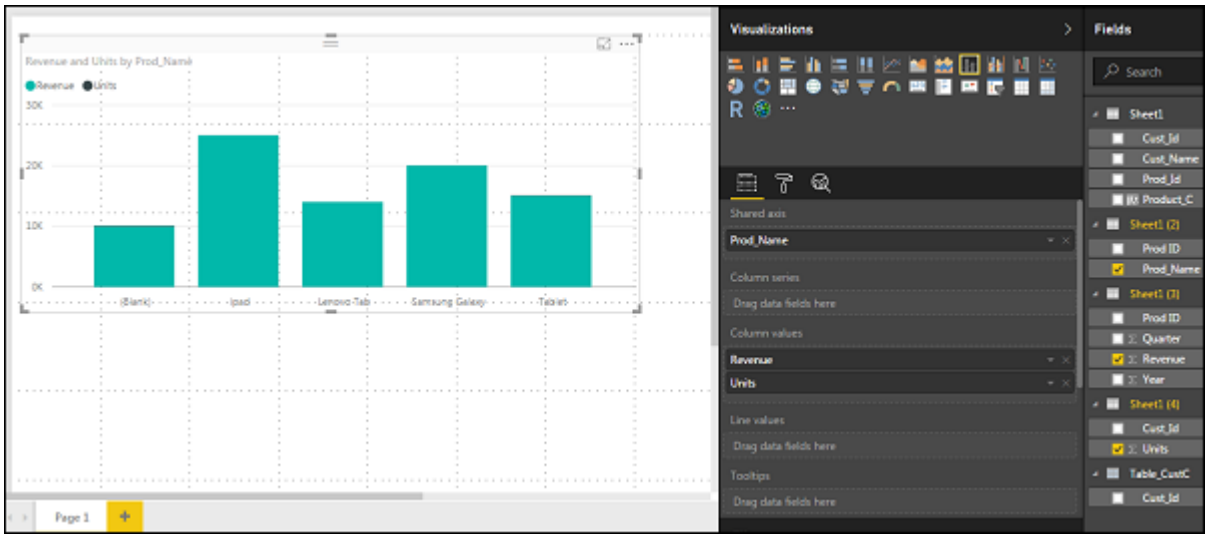
Solution:

In data visualization, it is also required to plot multiple measures in a single chart. Power BI supports various combination chart types to plot measure values. Let us say you want to plot revenue and unit_solds in one chart. Combination charts are the most suitable option for these kind of requirement.

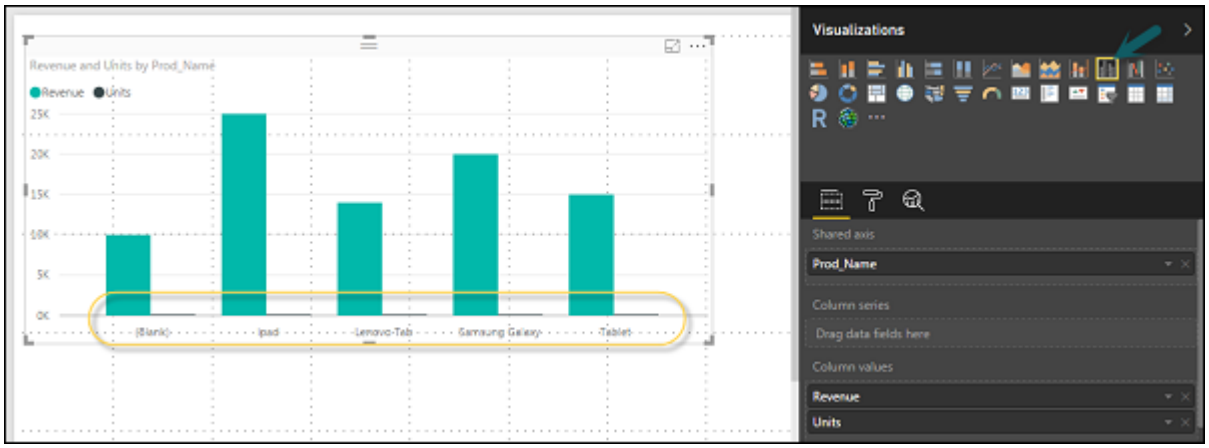
One of the most common Combination chart in Power BI is Line and Stacked column charts. Let us say we have a revenue field and we have added a new data source that contains customer-wise unit quantity and we want to plot this in our visualization.



Once you add a data source, it will be added to the list of fields on the right side. You can add units to the column axis as shown in the following screenshot.



You have other type of combine chart that you can use in Power BI - Line and Clustered Column.



Chapter: 6

Data Regression

Scope:

- Types of Regression Models
- Simple Linear Regression Model
- Simple Linear Regression Equation
- Equation for the best straight line
- Graph of the best straight line
- Interpreting the Results
- Measures of Variation

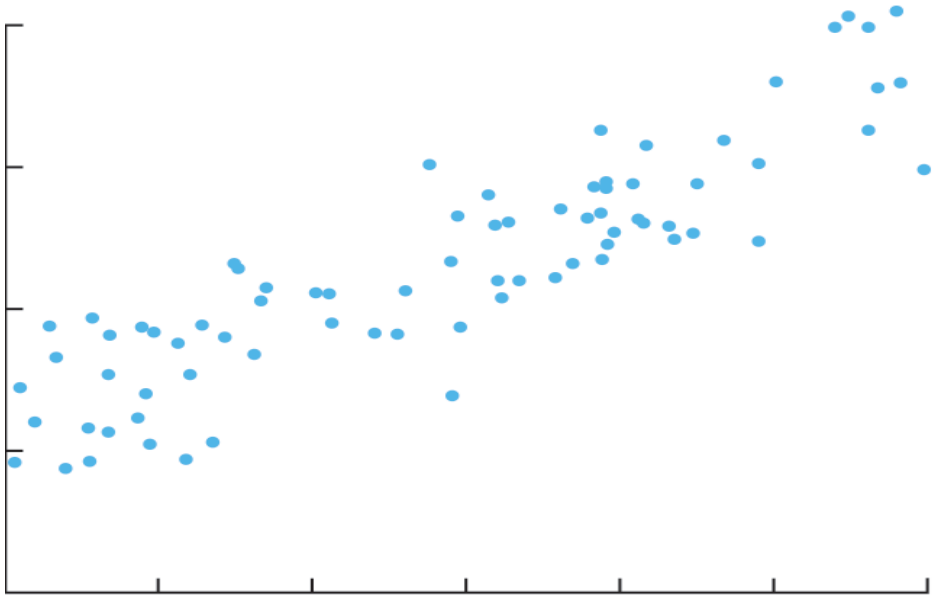
Regression Models:

Regression analysis is a predictive modelling technique that analyzes the relation between the target or dependent variable and independent variable in a dataset. The different types of regression analysis techniques get used when the target and independent variables show a linear or non-linear relationship between each other, and the target variable contains continuous values. The regression technique gets used mainly to determine the predictor strength, forecast trend, time series, and in case of cause & effect relation.
Regression analysis is the primary technique to solve the regression problems in machine learning using data modelling. It involves determining the best fit line, which is a line that passes through all the data points in such a way that distance of the line from each data point is minimized.

How does it work?

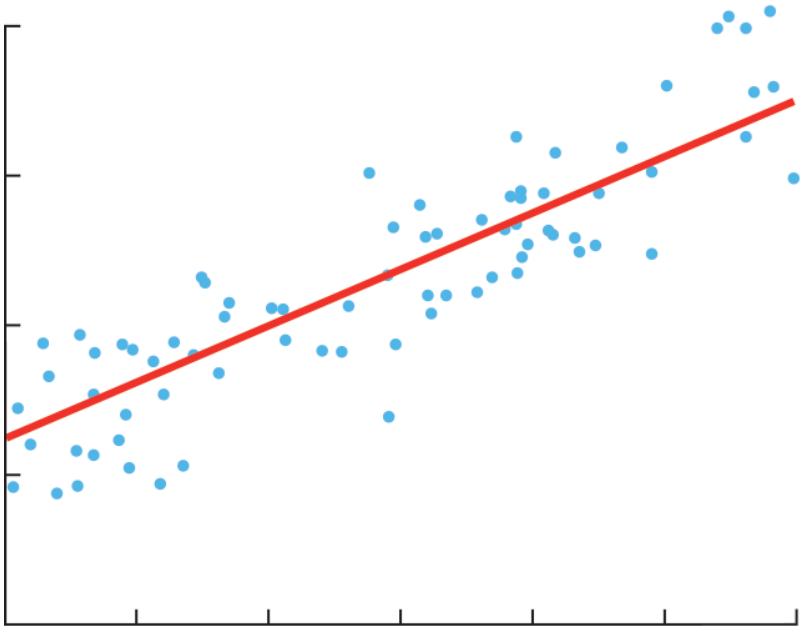
In order to conduct a regression analysis, you gather the data on the variables in question. (Reminder: you likely don't have to do this yourself, but it's helpful for you to understand the process your data analyst colleague uses.) You take all of your monthly sales numbers for, say, the past three years and any data on the independent variables you're interested in. So, in this case, let's say you find out the average monthly rainfall for the past three years as well.

Then you plot all of that information on a chart that looks like this:



The y-axis is the amount of sales (the dependent variable, the thing you're interested in, is always on the y-axis) and the x-axis is the total rainfall. Each blue dot represents one month's data—how much it rained that month and how many sales you made that same month.

Building a Regression model



This is called the regression line and it's drawn (using a statistics program like SPSS or STATA or even Excel) to show the line that best fits the data. In other words, explains Redman, "The red line is the best explanation of the relationship between the independent variable and dependent variable."

In addition to drawing the line, your statistics program also outputs a formula that explains the slope of the line and looks something like this:

$$Y = 200 + 5X + \text{error term}$$

Ignore the error term for now. It refers to the fact that regression isn't perfectly precise. Just focus on the model:

$$Y = 200 + 5X$$

What this formula is telling you is that if there is no "x" then Y = 200. So, historically, when it didn't rain at all, you made an average of 200 sales and you can expect to do the same going forward assuming other variables stay the same. And in the past, for every additional inch of rain, you made an average of five more sales. "For every increment that x goes up one, y goes up by five," says Redman.

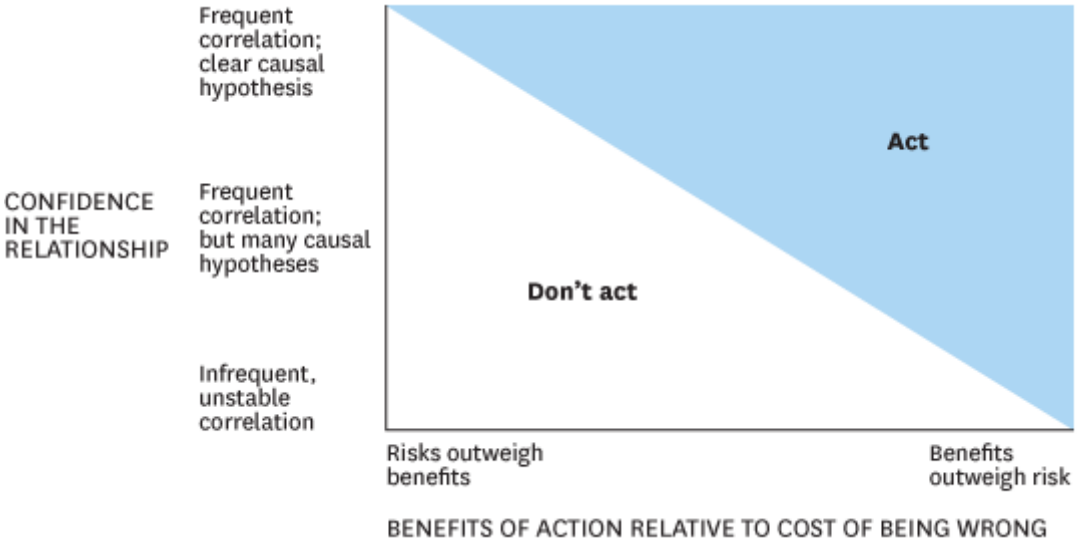
Now let's return to the error term. You might be tempted to say that rain has a big impact on sales if for every inch you get five more sales, but whether this variable is worth your attention will depend on the error term. A regression line always has an error term because, in real life, independent variables are never perfect predictors of the dependent variables. Rather the line is an estimate based on the available data. So the error term tells you how certain you can be about the formula. The larger it is, the less certain the regression line.

The above example uses only one variable to predict the factor of interest — in this case rain to predict sales. Typically you start a regression analysis wanting to understand the impact of several independent variables. So you might include not just rain but also data about a competitor's promotion. "You keep doing this until the error term is very small," says Redman. "You're trying to get the line that fits best with your data." While there can be dangers to trying to include too many variables in a regression analysis, skilled analysts can minimize those risks. And considering the impact of multiple variables at once is one of the biggest advantages of regression.

What mistakes do people make when working with regression analysis?

First, don't tell your data analyst to go out and figure out what is affecting sales. "The way most analyses go haywire is the manager hasn't narrowed the focus on what he or she is looking for," says Redman. It's your job to identify the factors that you suspect are having an impact and ask your analyst to look at those. "If you tell a data scientist to go on a fishing expedition, or to tell you something you don't know, then you deserve what you get, which is bad analysis," he says. In other words, don't ask your analysts to look at every variable they can possibly get their hands on all at once. If you do, you're likely to find relationships that don't really exist. It's the same principle as flipping a coin: do it enough times, you'll eventually think you see something interesting, like a bunch of heads all in a row.

Second, "analyses are very sensitive to bad data" so be careful about the data you collect and how you collect it, and know whether you can trust it. "All the data doesn't have to be correct or perfect," explains Redman but consider what you will be doing with the analysis. If the decisions you'll make as a result don't have a huge impact on your business, then it's OK if the data is "kind of leaky." But "if you're trying to decide whether to build 8 or 10 of something and each one costs $1 million to build, then it's a bigger deal," he says. The chart below explains how to think about whether to act on the data.



6.1 Types of Regression Models:

There are many types of regression analysis techniques, and the use of each method depends upon the number of factors. These factors include the type of target variable, shape of the regression line, and the number of independent variables.
Below are the different regression techniques:

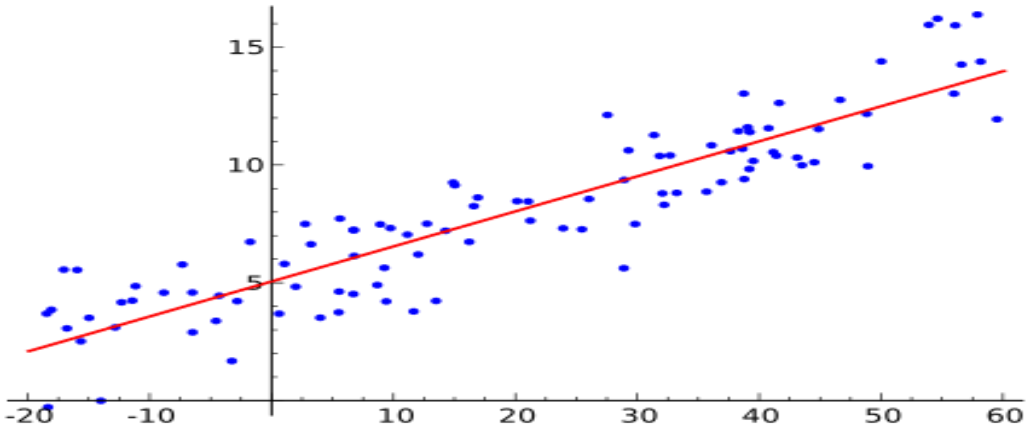
1. Linear Regression
2. Logistic Regression
3. Ridge Regression
4. Lasso Regression
5. Polynomial Regression
6. Bayesian Linear Regression

Linear Regression:

Linear regression is one of the most basic types of regression in machine learning. The linear regression model consists of a predictor variable and a dependent variable related linearly to each other. In case the data involves more than one independent variable, then linear regression is called multiple linear regression models.

The below-given equation is used to denote the linear regression model:
$y=mx+c+e$

where m is the slope of the line, c is an intercept, and e represents the error in the model.



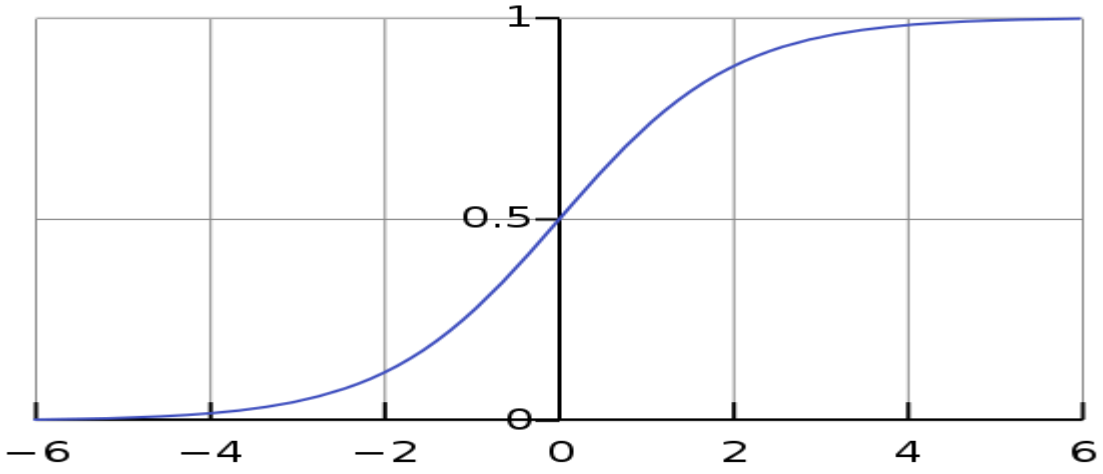
The best fit line is determined by varying the values of m and c. The predictor error is the difference between the observed values and the predicted value. The values of m and c get selected in such a way that it gives the minimum predictor error. It is important to note that a simple linear regression model is susceptible to outliers. Therefore, it should not be used in case of big size data.

Logistic Regression:

Logistic regression is one of the types of regression analysis technique, which gets used when the dependent variable is discrete. Example: 0 or 1, true or false, etc. This means the target variable can have only two values, and a sigmoid curve denotes the relation between the target variable and the independent variable.
Logit function is used in Logistic Regression to measure the relationship between the target variable and independent variables. Below is the equation that denotes the logistic regression.

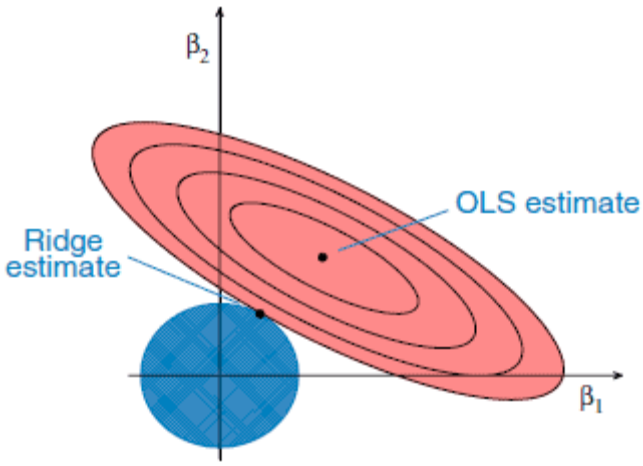
$$\text{logit}(p) = \ln(p/(1-p)) = b0+b1X1+b2X2+b3X3....+bkXk$$

where p is the probability of occurrence of the feature.



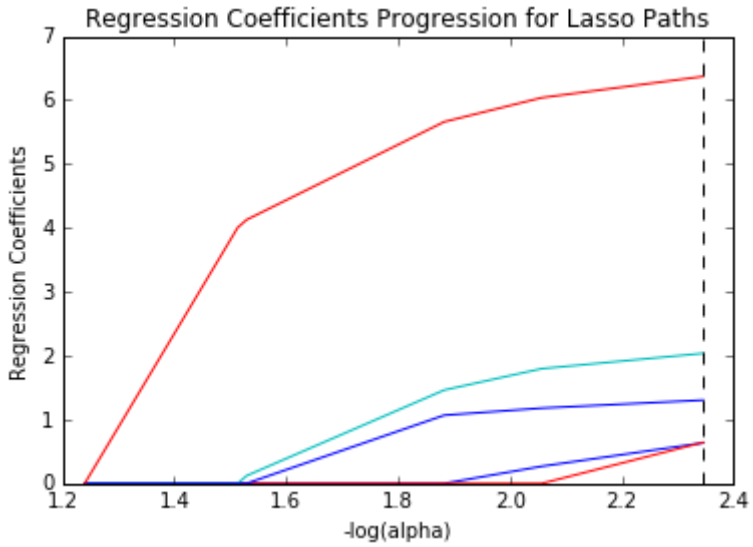
For selecting logistic regression, as the regression analyst technique, it should be noted, the size of data is large with the almost equal occurrence of values to come in target variables. Also, there should be no multicollinearity, which means that there should be no correlation between independent variables in the dataset.

Ridge Regression:



This is another one of the types of regression in machine learning which is usually used when there is a high correlation between the independent variables. This is because, in the case of multi collinear data, the least square estimates give unbiased values. But, in case the collinearity is very high, there can be some bias value. Therefore, a bias matrix is introduced in the equation of Ridge Regression. This is a powerful regression method where the model is less susceptible to overfitting.
Below is the equation used to denote the Ridge Regression, where the introduction of λ (lambda) solves the problem of multicollinearity:
β = (X^{T}X + λ*I)^{-1}X^{T}y

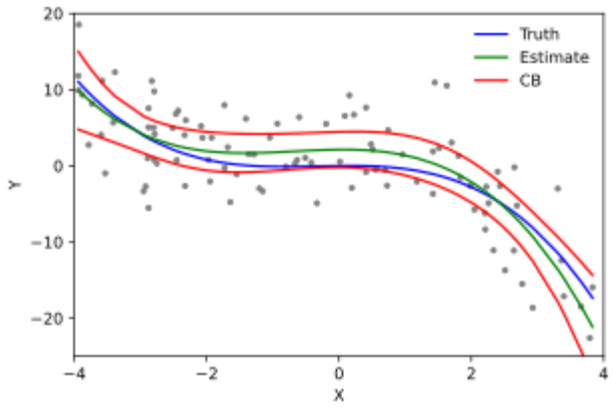
Lasso Regression:
Lasso Regression is one of the types of regression in machine learning that performs regularization along with feature selection. It prohibits the absolute size of the regression coefficient. As a result, the coefficient value gets nearer to zero, which does not happen in the case of Ridge Regression.
Due to this, feature selection gets used in Lasso Regression, which allows selecting a set of features from the dataset to build the model. In the case of Lasso Regression, only the required features are used, and the other ones are made zero. This helps in avoiding the over fitting in the model. In case the independent variables are highly collinear, then Lasso regression picks only one variable and makes other variables to shrink to zero.



Below is the equation that represents the Lasso Regression method:
N^{-1}Σ^{N}_{i=1}f(x_{i}, y_{I}, α, β)

Polynomial Regression:

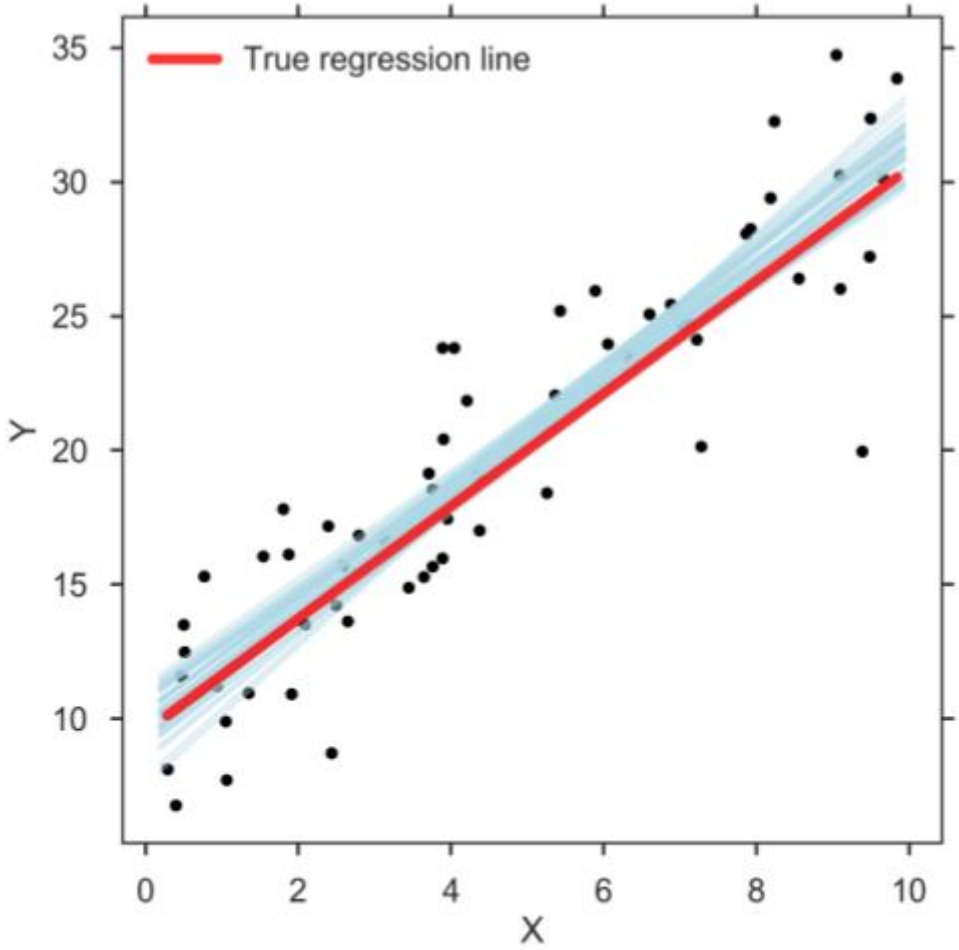
Polynomial Regression is another one of the **types of regression analysis** techniques in machine learning, which is the same as Multiple Linear Regression with a little modification. In Polynomial Regression, the relationship between independent and dependent variables, that is X and Y, is denoted by the n-th degree.
It is a linear model as an estimator. Least Mean Squared Method is used in Polynomial Regression also. The best fit line in Polynomial Regression that passes through all the data points is not a straight line, but a curved line, which depends upon the power of X or value of n.



While trying to reduce the Mean Squared Error to a minimum and to get the best fit line, the model can be prone to overfitting. It is recommended to analyze the curve towards the end as the higher Polynomials can give strange results on extrapolation.

Below equation represents the Polynomial Regression:
$l = \beta 0 + \beta 0x1 + \varepsilon$

Bayesian Linear Regression:
Bayesian Regression is one of the types of regression in machine learning that uses the Bayes theorem to find out the value of regression coefficients. In this method of regression, the posterior distribution of the features is determined instead of finding the least-squares. Bayesian Linear Regression is like both Linear Regression and Ridge Regression but is more stable than the simple Linear Regression.



Regression analysis is used to find equations that fit data. Once we have the regression equation, we can use the model to make predictions. One type of regression analysis is linear analysis. When a correlation coefficient shows that data is likely to be able to predict future outcomes and a scatter plot of the data appears to form a straight line, you can use simple linear regression to find a predictive function. If you recall from elementary algebra, the equation for a line is $y = mx + b$. This article shows you how to take data, calculate linear regression, and find the equation $y' = a + bx$. Note: If you're taking AP statistics, you may see the equation written as $b_0 + b_1x$, which is the same thing.

6.2 Simple Linear Regression Model

Regression models describe the relationship between variables by fitting a line to the observed data. Linear regression models use a straight line, while logistic and nonlinear regression models use a curved line. Regression allows you to estimate how a dependent variable changes as the independent variable(s) change.Simple linear regression is used to estimate the relationship between two quantitative variables.

Assumptions of simple linear regression:

Simple linear regression is a parametric test, meaning that it makes certain assumptions about the data. These assumptions are: Homogeneity of variance (homoscedasticity): the size of the error in our prediction doesn't change significantly across the values of the independent variable.

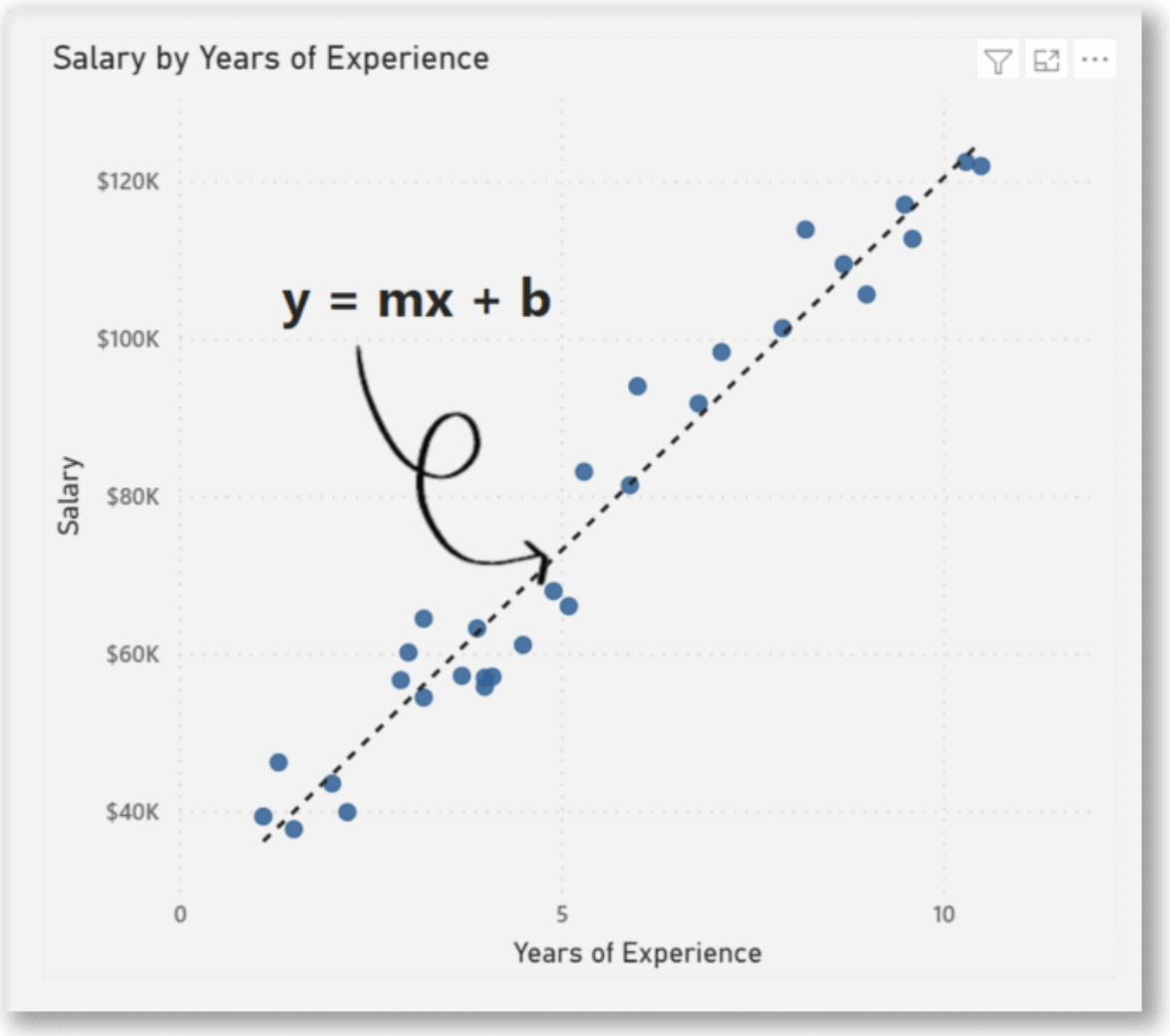
Independence of observations: the observations in the dataset were collected using statistically valid sampling methods, and there are no hidden relationships among observations.

Normality: The data follows a normal distribution.

Linear regression makes one additional assumption: The relationship between the independent and dependent variable is linear: the line of best fit through the data points is a straight line (rather than a curve or some sort of grouping factor).

Example:

Consider this Scatterplot visual. Illustrated is the relationship between Years of Experience and Salary at a fictional company. By fitting a trend line to the Scatterplot, we can see that the more years of experience an employee has, the more they will get paid. Our y data points represent Salary in thousands, and the x data points represent Years of Experience.



According to the data above, if an employee has 15 years of experience, then we have to find out how much will they be paid? What about 20 years of experience? 30 years?

Steps we need to take:

Create calculated columns and measures for the components of the equation of a line

Create a What-if parameter for the x-values

Create the measure of the equation of a line and insert a card visual with the measure

you can practice using the Salary_data dataset from Kaggle.com.

Step 1: Create Calculated Columns and Measures

The formula for the equation of a line is y = mx + b.

Where:

y = How far up the y axis

m = Slope (Change in y divided by change in x) =

$$\frac{n(\sum xy)-(\sum x)(\sum y)}{n(\sum x^2)-(\sum x)^2}$$

x = How far along the x axis

b = b-intercept (The value of y when x = 0) =

$$= \frac{(\sum y)(\sum x^2)-(\sum x)(\sum xy)}{n(\sum x^2)-(\sum x)^2}$$

After finding out m and b with some calculations, we can *input any data point for x and the output will be y.*

We are going to create this formula using DAX calculated columns and measures.

These are the Calculated Columns we need to make and their DAX syntax:

Math Formula	DAX Formula
x^2	xsq = Salary_Data[Years of Experience]^2
xy	xy = Salary_Data[Years of Experience]*Salary_Data[Salary]

These are the Measures we need to make and their DAX syntax:

Math Formula	DAX Formula
n	n = COUNTROWS(Salary_Data)
∑xy	xysum = SUM(Salary_Data[xy])
∑x	xsum = SUM(Salary_Data[Years of Experience])
∑y	ysum = SUM(Salary_Data[Salary])
$\sum x^2$	xsqrsum = SUM(Salary_Data[xsq])

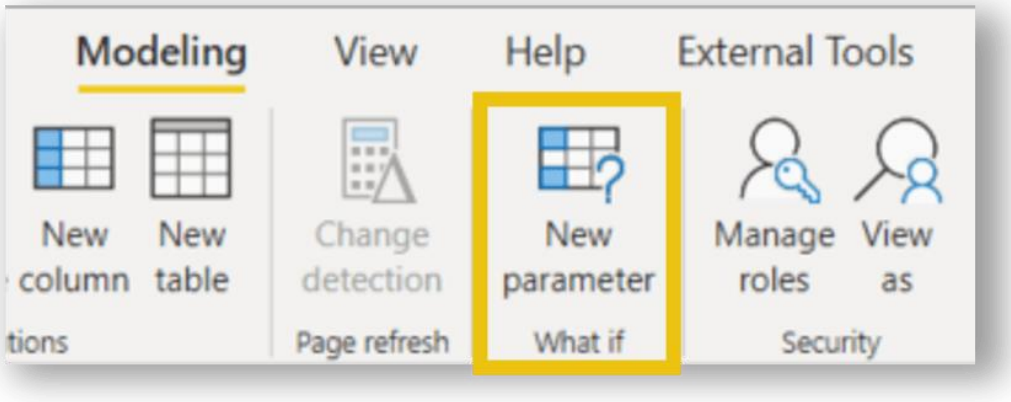
(n(∑xy)-(∑x)(∑y))/(n(∑x²)-(∑x)²)	m (Slope) = DIVIDE([n]*[xysum]-[Xsum]*[Ysum], [n]*[xsqrsum]-[Xsum]^2, 0)
((∑y)(∑x^2)-(∑x)(∑xy))/(n(∑x²)-(∑x)²)	b (Intercept) = DIVIDE([ysum]*[xsqrsum]-[Xsum]*[xysum], [n]*[xsqrsum]-[Xsum]^2, 0)

Step 2: Setting up a What-if parameter

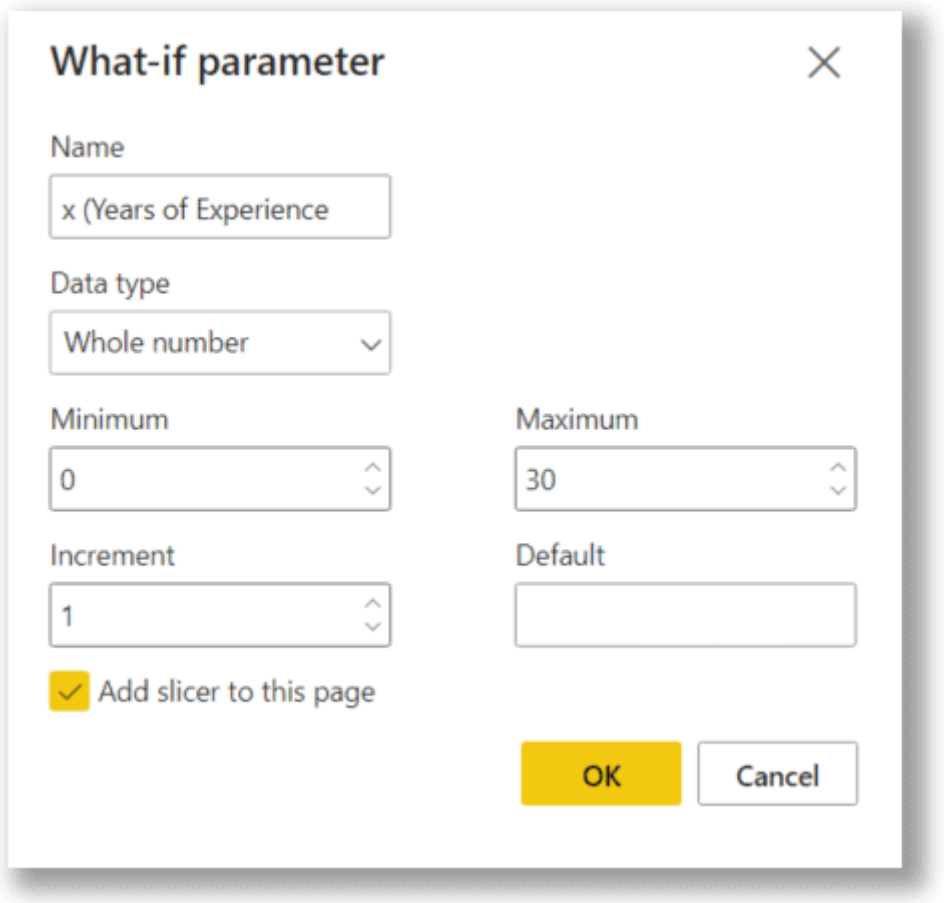
Now that we have performed our calculations, for any number we insert into x, the formula will predict y.

To demonstrate this, we will be using a **What-if** parameter with a slicer.

A **What-if** parameter allows you to create an interactive slicer with a variable to visualize and quantify another value in your report.

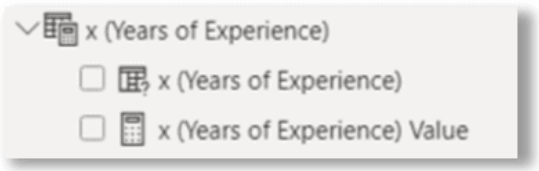


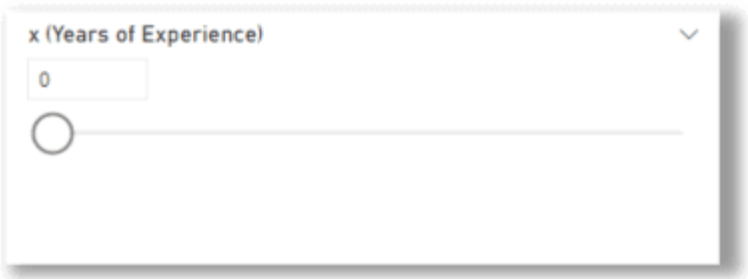
After clicking the **What-if** parameter on the modeling ribbon, we can input these values:



Name: x (Years of Experience)
Data type: Whole Number
Minimum: 0
Maximum: 30
Increment: 1
Add slicer to this page: Yes

Once we press 'OK', the **What-if** parameter will have generated a series of x values and a slicer to use for our **What-if** scenarios.



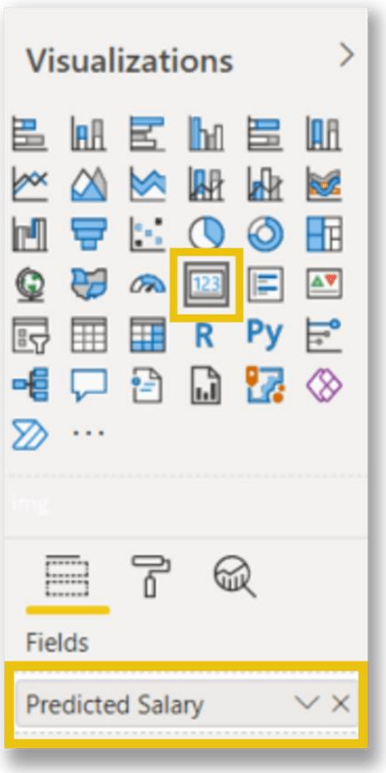


Step 3: Complete the measure for the equation of a line and visualize

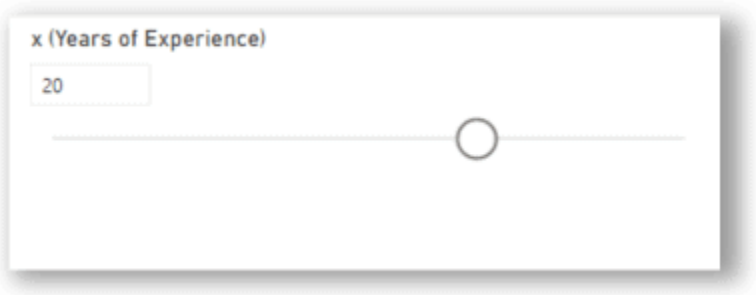
Finally, we create the **Predicted Salary** measure that contains the complete formula for the regression equation of a line **y = mx + b**, using the components we have created previously.

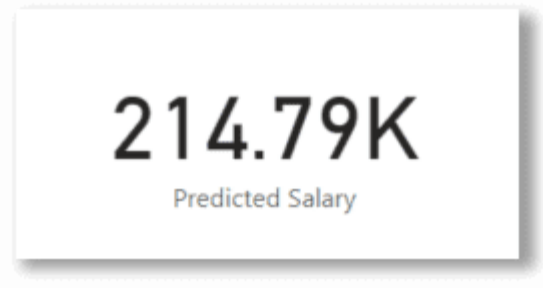
Math Formula	DAX Formula
y = mx + b	Predicted Salary = ([m (Slope)]* 'x (Years of Experience)'[x (Years of Experience)Value]+ [b (Intercept)])

We can now insert a card visual in the report and select the measure **Predicted Salary** for the field.



We can test the calculation by toggling the slicer left and right. This action will interact with the Card visual and represent the predicted salary for the selected **Years of Experience**.





Create awesome visualizations!

There are many ways to visualize the prediction that we have set up using Linear Regression. A toggle and a card provide enough information, but we can also integrate that calculation into a chart to visualize the existing data and the predicted value.



6.3 Simple Linear Regression Equation:

Linear regression is a way to model the relationship between two variables. You might also recognize the equation as the slope formula. The equation has the form Y= a + bX, where Y is the dependent variable (that's the variable that goes on the Y axis), X is the independent variable (i.e. it is plotted on the X axis), b is the slope of the line and a is the y-intercept.

$$a = \frac{(\Sigma y)(\Sigma x^2) - (\Sigma x)(\Sigma xy)}{n(\Sigma x^2) - (\Sigma x)^2}$$

$$b = \frac{n(\Sigma xy) - (\Sigma x)(\Sigma y)}{n(\Sigma x^2) - (\Sigma x)^2}$$

The first step in finding a linear regression equation is to determine if there is a relationship between the two variables. This is often a judgment call for the researcher. You'll also need a list of your data in x-y format (i.e. two columns of data—independent and dependent variables).

How to Find a Linear Regression Equation:

Step 1: Make a chart of your data, filling in the columns in the same way as you would fill in the chart if you were finding the *Pearson's Correlation Coefficient.*

SUBJECT	AGE X	GLUCOSE LEVEL Y	XY	X^2	Y^2
1	43	99	4257	1849	9801
2	21	65	1365	441	4225
3	25	79	1975	625	6241
4	42	75	3150	1764	5625
5	57	87	4959	3249	7569
6	59	81	4779	3481	6561
Σ	247	486	20485	11409	40022

From the above table, Σx = 247, Σy = 486, Σxy = 20485, Σx2 = 11409, Σy2 = 40022. n is the sample size (6, in our case).

Step 2: Use the following equations to find a and b.

$$a = \frac{(\Sigma y)(\Sigma x^2) - (\Sigma x)(\Sigma xy)}{n(\Sigma x^2) - (\Sigma x)^2}$$

$$b = \frac{n(\Sigma xy) - (\Sigma x)(\Sigma y)}{n(\Sigma x^2) - (\Sigma x)^2}$$

a = 65.1416
b = .385225

step-by-step instructions for solving this formula.

Find a:

- ((486 × 11,409) – ((247 × 20,485)) / 6 (11,409) – 247^2)
- 484979 / 7445
- =65.14

Find b:

- (6(20,485) – (247 × 486)) / (6 (11409) – 247^2)
- (122,910 – 120,042) / 68,454 – 247^2
- 2,868 / 7,445
- = .385225

Step 3: *Insert the values into the equation*.
y' = a + bx
y' = 65.14 + .385225x

Least Square Regression Line:

The most popular method to fit a regression line in the XY plot is the method of least-squares. This process determines the best-fitting line for the noted data by reducing the sum of the squares of the vertical deviations from each data point to the line. If a point rests on the fitted line accurately, then its perpendicular deviation is 0. Because the variations are first squared, then added, their positive and negative values will not be cancelled.



Linear regression determines the straight line, called the least-squares regression line or LSRL, that best expresses observations in a bivariate analysis of data set.

Suppose Y is a dependent variable, and X is an independent variable, then the population regression line is given by;

Y = B0+B1X , Where

B0 is a constant

B1 is the regression coefficient

If a random sample of observations is given, then the regression line is expressed by;

$\hat{y} = b0 + b1x$

where b0 is a constant, b1 is the regression coefficient, x is the independent variable, and $\hat{y}$ is the predicted value of the dependent variable.

In the linear regression line, we have seen the equation is given by;

Y = B0+B1X , Where

B0 is a constant

B1 is the regression coefficient

Now, let us see the formula to find the value of the regression coefficient.

$B1 = b1 = \Sigma [(xi – x)(yi – y)] / \Sigma [(xi – x)2]$

Where xi and yi are the observed data sets.

And x and y are the mean value.

№ i/o	X	Y	$\hat{y}_x$	$y-\hat{y}_x$
1	1	4,24	4,28	-0,04
2	2	5,0	5,01	-0,01
3	3	5,9	5,74	0,16
4	4	6,49	6,47	0,02
5	5	7,09	7,2	-0,11
Total	15	28,72	28,7	0,02
Average	3	5,74	5,74	0,004

Here and below: X – set number, Y – parameters. number of errors, $\hat{y}_X$, $y-\hat{y}_X$ – additional values for finding linear regression

$\hat{y}1 = 3,55+0,73*1=4,28;$
$\hat{y}_2=3,55+0,73\cdot2=5,01;$
$\hat{y}_3=3,55+0,73\cdot3=5,74;$
$\hat{y}_4=3,55+0,73\cdot4=6,47;$
$\hat{y}_5=3,55+0,73\cdot5=7,2.$

6.4 EQUATION FOR THE BEST STRAIGHT LINE

A line of best fit is a straight line that is the best approximation of the given set of data.

It is used to study the nature of the relation between two variables. (We're only considering the two-dimensional case, here.)

A line of best fit can be roughly determined using an eyeball method by drawing a straight line on a scatter plot so that the number of points above the line and below the line is about equal (and the line passes through as many points as possible).

A more accurate way of finding the line of best fit is the least square method .

Use the following steps to find the equation of line of best fit for a set of ordered pairs (x1,y1),(x2,y2),...(xn,yn)(x1,y1),(x2,y2),...(xn,yn) .
Step 1: Calculate the mean of the xx -values and the mean of the yy -values.
$X^{---}=\sum i=1nxinY^{---}=\sum i=1nyinX^{-}=\sum i=1nxinY^{-}=\sum i=1nyin$
Step 2: The following formula gives the slope of the line of best fit:

$$m=\sum i=1n(xi-X^{---})(yi-Y^{---})\sum i=1n(xi-X^{---})2m=\sum i=1n(xi-X^{-})(yi-Y^{-})\sum i=1n(xi-X^{-})2$$

Step 3: Compute the yy -intercept of the line by using the formula:
$b=Y^{---}-mX^{---}b=Y^{-}-mX^{-}$

Step 4: Use the slope mm and the yy -intercept bb to form the equation of the line.
Example:

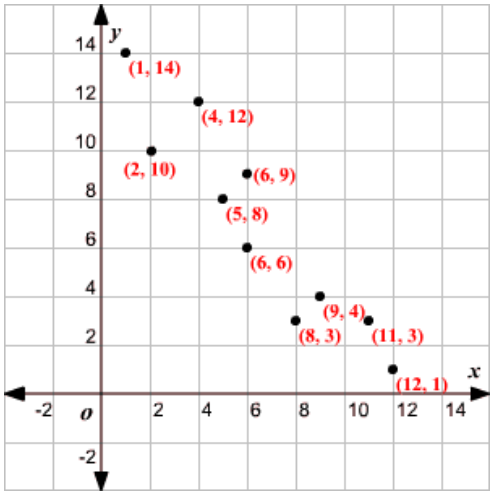
Use the least square method to determine the equation of line of best fit for the data. Then plot the line.

xx	88	22	1111	66	55	44	1212	99	66	11

yy	33	1010	33	66	88	1212	11	44	99	1414

Solution:

Plot the points on a coordinate plane .



Calculate the means of the xx -values and the yy -values.
$X^{---}=8+2+11+6+5+4+12+9+6+110=6.4Y^{---}=3+10+3+6+8+12+1+4+9+1410=7X^{-}=8+2+11+6+5+4+12+9+6+110=6.4Y^{-}=3+10+3+6+8+12+1+4+9+1410=7$
Now calculate $xi-X^{---}xi-X^{-}$, $yi-Y^{---}yi-Y^{-}$, $(xi-X^{---})(yi-Y^{---})(xi-X^{-})(yi-Y^{-})$, and $(xi-X^{---})2(xi-X^{-})2$ for each ii .

ii	xixi	yiyi	$xi-X^{---}xi-X^{-}$	$yi-Y^{---}yi-Y^{-}$	$(xi-X^{---})(yi-Y^{---})(xi-X^{-})(yi-Y^{-})$	$(xi-X^{---})2(xi-X^{-})2$
11	88	33	1.61.6	−4−4	−6.4−6.4	2.562.56
22	22	1010	−4.4−4.4	33	−13.2−13.2	19.3619.36
33	1111	33	4.64.6	−4−4	−18.4−18.4	21.1621.16
44	66	66	−0.4−0.4	−1−1	0.40.4	0.160.16
55	55	88	−1.4−1.4	11	−1.4−1.4	1.961.96

66	44	1212	−2.4−2.4	55	−12−12	5.765.76
77	1212	11	5.65.6	−6−6	−33.6−33.6	31.3631.36
88	99	44	2.62.6	−3−3	−7.8−7.8	6.766.76
99	66	99	−0.4−0.4	22	−0.8−0.8	0.160.16
10 10	11	1414	−5.4−5.4	77	−37.8−37.8	29.1629.16
					$\sum i=1n(xi-X^{---})(yi-Y^{---})=-131\sum i=1n(xi-X^{-})(yi-Y^{-})=-131$	$\sum i=1n(xi-X^{---})2=118.4\sum i=1n(xi-X^{-})2=118.4$

Calculate the slope.

$m=\sum i=1n(xi-X^{---})(yi-Y^{---})\sum i=1n(xi-X^{---})2=-131118.4\approx-1.1m=\sum i=1n(xi-X^{-})(yi-Y^{-})\sum i=1n(xi-X^{-})2=-131118.4\approx-1.1$
Calculate the yy -intercept.

Use the formula to compute the yy -intercept.
$b=Y^{---}-mX^{---}=7-(-1.1\times6.4)=7+7.04\approx14.0b=Y^{-}-mX^{-}=7-(-1.1\times6.4)=7+7.04\approx14.0$
Use the slope and yy -intercept to form the equation of the line of best fit.
The slope of the line is $-1.1-1.1$ and the yy -intercept is 14.014.0 .
Therefore, the equation is $y=-1.1x+14.0y=-1.1x+14.0$.
Draw the line on the scatter plot.

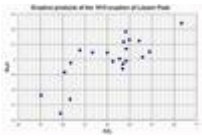


6.5 GRAPH OF THE BEST STRAIGHT LINE

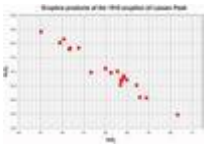
A best-fit line is meant to mimic the trend of the data. In many cases, the line may not pass through very many of the plotted points. Instead, the idea is to get a line that has equal numbers of points on either side. Most people start by eye-balling the data.

1. Take a look at the data and as yourself these questions

Does the data look like a line? or a big blob? Try to approximate the general trend of the data with your mind (even if it's just a blob)



Does the trend of the points look positively



correlated (like they rise up to the right; click on image at right) or negatively correlated (like they start high near the x-axis and get lower as they approach the y-axis; see image

to the left)? Your trendline (when you're finished with the next steps) should mimic those correlations.

If you blur your eyes, can you see a thick line trending in one direction or another? This is another way to visualize the trend of the data.

2. Now that you have an idea of the general trend of the data, there are two possible ways to construct a best-fit line by eye. You may use either of them; both are correct and

relatively easy ways to get a pretty accurate representation of a best-fit line. Pick the one that makes the most sense to you. The first method involves enclosing the data in an area:

The second method involves dividing data into two equal groups, approximating the center of each group and constructing a line between the two centers.

6.6 Interpreting the Results

Introduction

Once you've recorded the diagnostics you want to use, the next step is being able to understand what they say.

It's helpful to have a good understanding of what exactly each column in the query diagnostics schema means, which we're not going to repeat in this short tutorial.

In general, when building visualizations, it's better to use the full detailed table. Because regardless of how many rows it is, what you're probably looking at is some kind of depiction of how the time spent in different resources adds up, or what the native query emitted was.

As mentioned in our article on recording the diagnostics, I'm working with the OData and SQL traces for the same table (or nearly so)—the Customers table from Northwind. In particular, I'm going to focus on common ask from our customers, and one of the easier to interpret sets of traces: full refresh of the data model.

Building the visualizations

When you're going through traces, there are many ways you can evaluate them. In this article, we're going to focus on a two visualization split--one to show the details you care about, and the other to easily look at time contributions of various factors. For the first visualization, a table is used. You can pick any fields you like, but the ones recommended for an easy, high level look at what's going on are:

- Id
- Start Time
- Query
- Step
- Data Source Query
- Exclusive Duration (%)
- Row Count
- Category
- Is User Query
- Path

For the second visualization, one choice is to use a Stacked Column Chart. In the 'Axis' parameter, you might want to use 'Id' or 'Step'. If we're looking at the Refresh, because it doesn't have anything to do with steps in the Editor itself, we probably just want to look at 'Id'. For the 'Legend' parameter, you should set 'Category' or 'Operation' (depending on the granularity you want). For the 'Value', set 'Exclusive

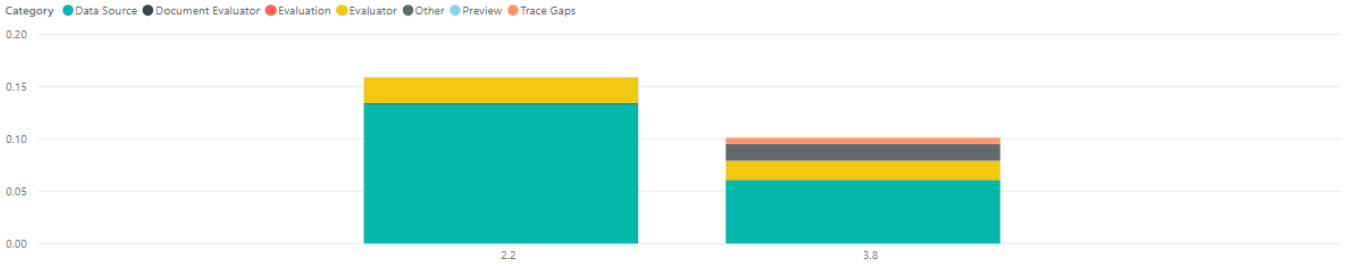
Duration' (and make sure it's not the %, so that you get the raw duration value). Finally, for the Tooltip, set 'Earliest Start Time'.

Once your visualization is built, make sure you sort by 'Earliest Start Time' ascending so you can see the order things happen in.

Detailed Traces Table

Id	Start Time	Query	Step	Exclusive Duration (%)	Row Count	Category	Is User Query	Path	Data Source Query
2.2	10/15/2019 12:28:13 PM	Customers (2)	Grouped Rows	0.01		Document Evaluator	False	0/1	
2.2	10/15/2019 12:28:13 PM	Customers (2)	Grouped Rows	0.00		Evaluation	False	0	
2.2	10/15/2019 12:28:13 PM	Customers (2)	Grouped Rows	0.08		Evaluator	False	0/2	
2.2	10/15/2019 12:28:13 PM	Customers (2)	Grouped Rows	0.00		Evaluator	False	0/3	
2.2	10/15/2019 12:28:13 PM	Customers (2)	Grouped Rows	0.01		Evaluator	False	0/3/4	
2.2	10/15/2019 12:28:13 PM	Customers (2)	Grouped Rows	0.00		Evaluator	False	0/3/5	
2.2	10/15/2019 12:28:13 PM	Customers (2)	Grouped Rows	0.03		Evaluator	False	0/3/5/6	
2.2	10/15/2019 12:28:13 PM	Customers (2)	Grouped Rows	0.03		Evaluator	False	0/3/5/7	
2.2	10/15/2019 12:28:13 PM	Customers (2)	Grouped Rows	0.00		Data Source	False	0/3/5/7/8/9/10	

Exclusive Duration and Earliest Start Time by Id and Category



While your exact needs might differ, this combination of charts is a good place to start for looking at numerous diagnostics files and for a number of purposes.

Interpreting the visualizations

As mentioned above, there's many questions you can try to answer with query diagnostics, but the two that we see the most often are asking how time is spent, and asking what the query sent to the source is.

Asking how the time is spent is easy, and will be similar for most connectors. A warning with query diagnostics, as mentioned elsewhere, is that you'll see drastically different capabilities depending on the connector. For example, many ODBC based connectors won't have an accurate recording of what query is sent to the actual back-end system, as Power Query only sees what it sends to the ODBC driver.

If we want to see how the time is spent, we can just look at the visualizations we built above.

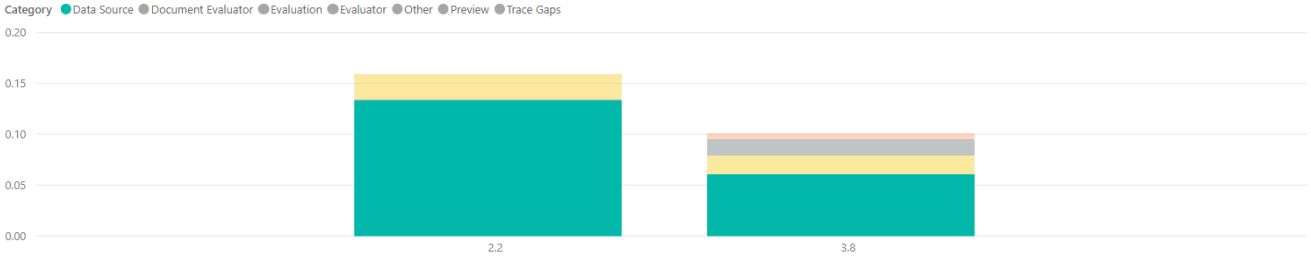
Now, because the time values for the sample queries we're using here are so small, if we want to work with how Power BI reports time it's better if we convert the Exclusive Duration column to 'Seconds' in the Power Query editor. Once we do this this conversion, we can look at our chart and get a decent idea of where time is spent.

For my OData results, I see in the image that the vast majority of the time was spent retrieving the data from source—if I select the 'Data Source' item on the legend, it shows me all of the different operations related to sending a query to the Data Source.

Detailed Traces Table

Exclusive Duration (%)	Path	Data Source Query
0.40	0/3/13/14/16	Request: GET https://services.odata.org/V4/Northwind/Northwind.svc/Customers?$filter=ContactTitle eq 'Sales Representative'&$select=CustomerID%2 Response: https://services.odata.org/V4/Northwind/Northwind.svc/Customers?$filter=ContactTitle eq 'Sales Representative'&$select=CustomerID%2CCo HTTP/1.1 200 OK
0.35	0/3/5/7/8/9	Request: https://services.odata.org/V4/Northwind/Northwind.svc/ HTTP/1.1 Content-Type: application/json;odata.metadata=minimal;q=1.0,application/json;odata=minimalmetadata;q=0.9,application/atomsvc+xml;q=0.8,application/a

Exclusive Duration and Earliest Start Time by Id and Category

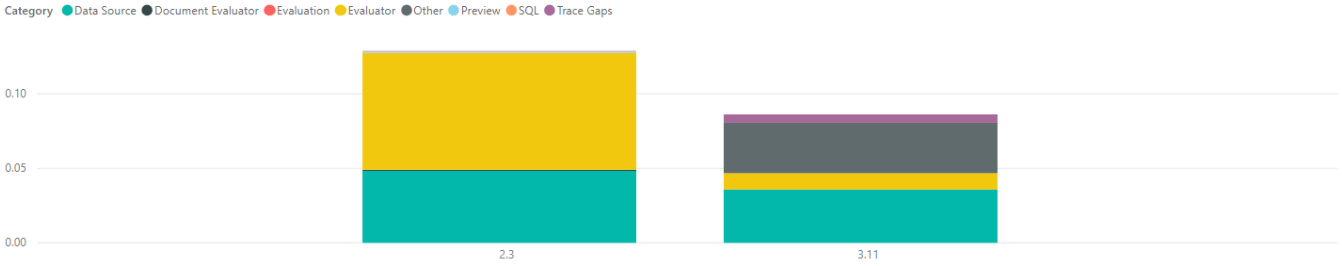


If we perform all the same operations and build similar visualizations, but with the SQL traces instead of the ODATA ones, we can see how the two data sources compare!

Detailed Traces Table

Id	Start Time	Query	Step	Exclusive Duration (%)	Row Count	Category	Is User Query	Path	Data Source Query
2.3	10/15/2019 12:44:37 PM	Customers (3)	Grouped Rows	0.01		Document Evaluator	False	0/1	
2.3	10/15/2019 12:44:37 PM	Customers (3)	Grouped Rows	0.00		Evaluation	False	0	
2.3	10/15/2019 12:44:37 PM	Customers (3)	Grouped Rows	0.09		Evaluator	False	0/2	
2.3	10/15/2019 12:44:37 PM	Customers (3)	Grouped Rows	0.00		Evaluator	False	0/3	
2.3	10/15/2019 12:44:37 PM	Customers (3)	Grouped Rows	0.00		Evaluator	False	0/3/4	
2.3	10/15/2019 12:44:37 PM	Customers (3)	Grouped Rows	0.00		Evaluator	False	0/3/5	
2.3	10/15/2019 12:44:37 PM	Customers (3)	Grouped Rows	0.06		Evaluator	False	0/3/5/6	
2.3	10/15/2019 12:44:37 PM	Customers (3)	Grouped Rows	0.44		Evaluator	False	0/3/5/7	
2.3	10/15/2019 12:44:37 PM	Customers (3)	Grouped Rows	0.00		Data Source	False	0/3/5/7/8	

Exclusive Duration and Earliest Start Time by Id and Category



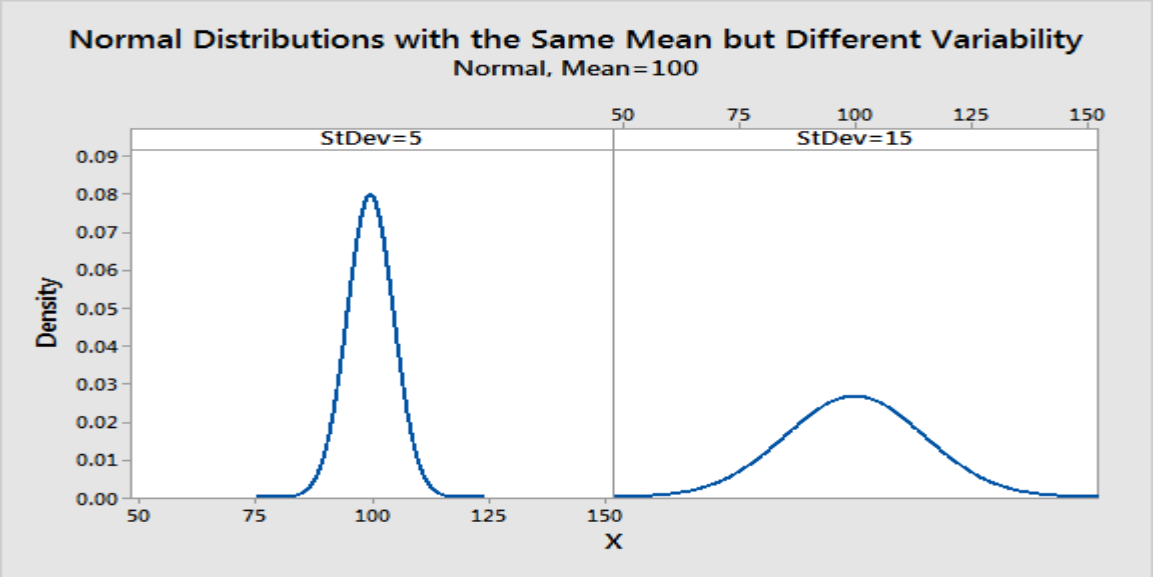
If we select the Data Source table, like with the ODATA diagnostics we can see the first evaluation (2.3 in this image) emits metadata queries, with the second evaluation actually retrieving the data we care about. Because we're retrieving small amounts of data in this case, the data pulled back takes a small amount of time (less than a tenth of a second for the entire second evaluation to happen, with less than a twentieth of a second for data retrieval itself), but that won't be true in all cases.

6.7 MEASURES OF VARIATION

A measure of variability is a summary statistic that represents the amount of dispersion in a dataset. How spread out are the values? While a measure of central tendency describes the typical value, measures of variability define how far away the data points tend to fall from the center. We talk about variability in the context of a distribution of values. A low dispersion indicates that the data points tend to be clustered tightly around the center. High dispersion signifies that they tend to fall further away.

In statistics, variability, dispersion, and spread are synonyms that denote the width of the distribution. Just as there are multiple measures of central tendency, there are several measures of variability. In this blog post, you'll learn why understanding the variability of your data is critical. Then, I explore the most common measures of variability—the range, interquartile range, variance, and standard deviation. I'll help you determine which one is best for your data.

The two plots below show the difference graphically for distributions with the same mean but more and less dispersion. The panel on the left shows a distribution that is tightly clustered around the average, while the distribution in the right panel is more spread out.

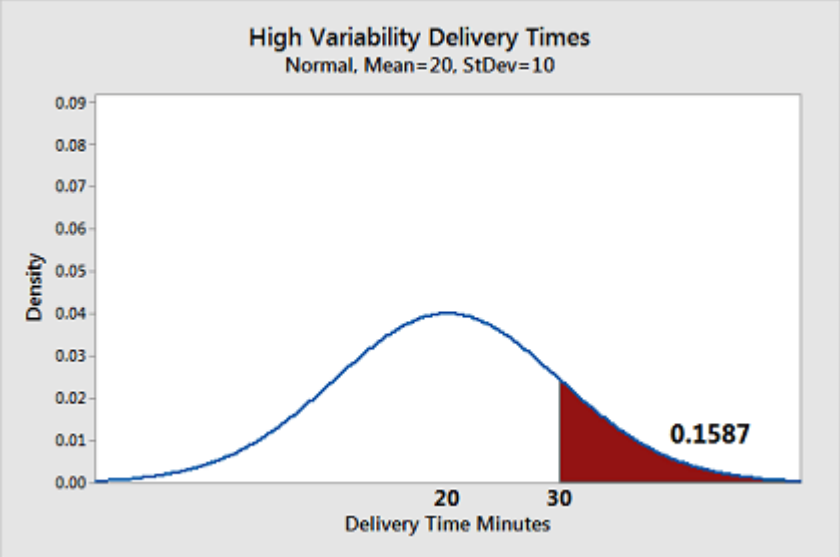


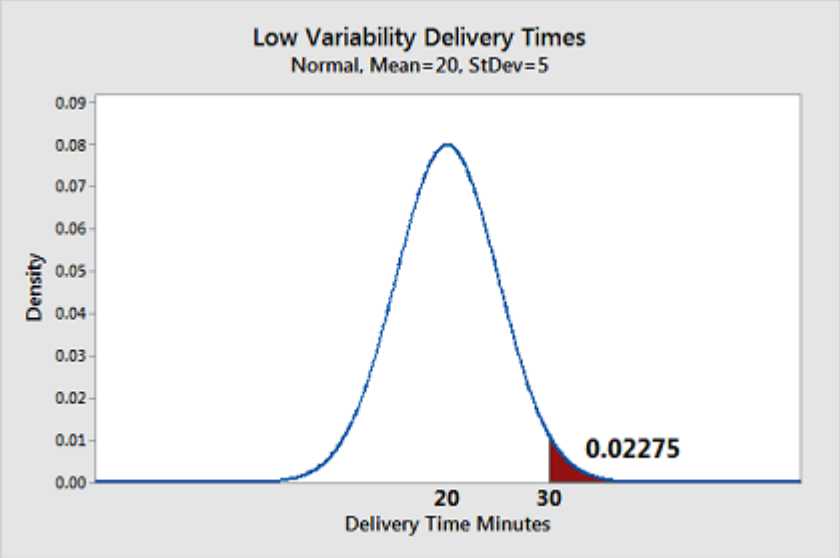
Example of Different Amounts of Variability

Let's take a look at two hypothetical pizza restaurants. They both advertise a mean delivery time of 20 minutes. When we're ravenous, they both sound equally good! However, this equivalence can be deceptive! To determine the restaurant that you should order from when you're hungry, we need to analyze their variability.

Suppose we study their delivery times, calculate the variability for each place, and determine that their variabilities are different. We've computed the standard deviations for both restaurants—which is a measure that we'll come back to later in this post. How significant is this difference in getting pizza to their customers promptly?

The graphs below display the distribution of delivery times and provide the answer. The restaurant with more variable delivery times has the broader distribution curve. I've used the same scales in both graphs so you can visually compare the two distributions.





In these graphs, we consider a 30-minute wait or longer to be unacceptable. We're hungry after all! The shaded area in each chart represents the proportion of delivery times that surpass 30 minutes. Nearly 16% of the deliveries for the high variability restaurant exceed 30 minutes. On the other hand, only 2% of the deliveries take too long with the low variability restaurant. They both have an average delivery time of 20 minutes, but I know where I'd place my order when I'm hungry!

As this example shows, the central tendency doesn't provide complete information. We also need to understand the variability around the middle of the distribution to get the full picture. Now, let's move on to the different ways of measuring variability!

Range

Let's start with the range because it is the most straightforward measure of variability to calculate and the simplest to understand. The range of a dataset is the difference between

the largest and smallest values in that dataset. For example, in the two datasets below, dataset 1 has a range of 20 – 38 = 18 while dataset 2 has a range of 11 – 52 = 41. Dataset 2 has a broader range and, hence, more variability than dataset 1.

Dataset 1	Dataset 2
20	11
21	16
22	19
25	23
26	25
29	32
33	39
34	46
38	52

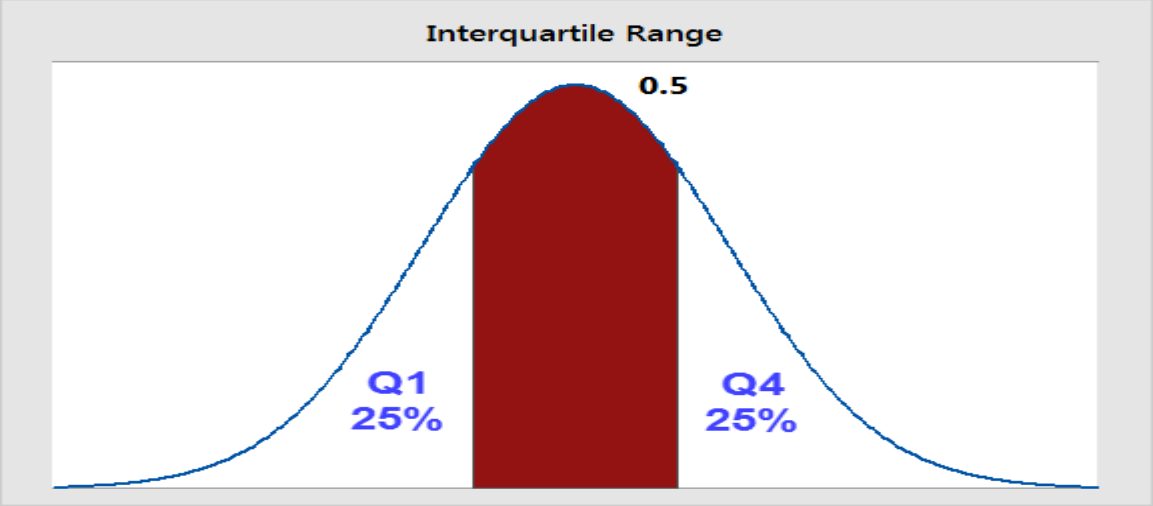
While the range is easy to understand, it is based on only the two most extreme values in the dataset, which makes it very susceptible to outliers. If one of those numbers is unusually high or low, it affects the entire range even if it is atypical.

Additionally, the size of the dataset affects the range. In general, you are less likely to observe extreme values. However, as you increase the sample size, you have more opportunities to obtain these extreme values. Consequently, when you draw random samples from the same population, the range tends to increase as the sample size increases. Consequently, use the range to compare variability only when the sample sizes are similar.

The Interquartile Range (IQR) . . . and other Percentiles

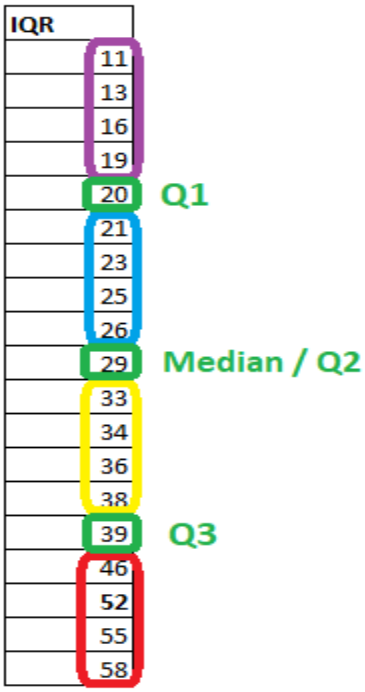
The interquartile range is the middle half of the data. To visualize it, think about the median value that splits the dataset in half. Similarly, you can divide the data into quarters. Statisticians refer to these quarters as quartiles and denote them from low to high as Q1, Q2, and Q3. The lowest quartile (Q1) contains the quarter of the dataset with the smallest values. The upper quartile (Q4) contains the quarter of the dataset with the highest values. The interquartile range is the middle half of the data that is in between the upper and lower

quartiles. In other words, the interquartile range includes the 50% of data points that fall between Q1 and Q3. The IQR is the red area in the graph below.



The interquartile range is a robust measure of variability in a similar manner that the median is a robust measure of central tendency. Neither measure is influenced dramatically by outliers because they don't depend on every value. Additionally, the interquartile range is excellent for skewed distributions, just like the median. As you'll learn, when you have a normal distribution, the standard deviation tells you the percentage of observations that fall specific distances from the mean. However, this doesn't work for skewed distributions, and the IQR is a great alternative.

I've divided the dataset below into quartiles. The interquartile range (IQR) extends from the low end of Q2 to the upper limit of Q3. For this dataset, the range is 39 – 20 = 19.



Using other percentiles

When you have a skewed distribution, I find that reporting the median with the interquartile range is a particularly good combination. The interquartile range is equivalent to the region

between the 75th and 25th percentile (75 – 25 = 50% of the data). You can also use other percentiles to determine the spread of different proportions. For example, the range between the 97.5th percentile and the 2.5th percentile covers 95% of the data. The broader these ranges, the higher the variability in your dataset.

Variance

Variance is the average squared difference of the values from the mean. Unlike the previous measures of variability, the variance includes all values in the calculation by comparing each value to the mean. To calculate this statistic, you calculate a set of squared differences between the data points and the mean, sum them, and then divide by the number of observations. Hence, it's the average squared difference.

There are two formulas for the variance depending on whether you are calculating the variance for an entire population or using a sample to estimate the population variance. The equations are below, and then I work through an example in a table to help bring it to life.

Population variance

The formula for the variance of an entire population is the following:

$$\sigma^2 = \frac{\sum(X-\mu)^2}{N}$$

In the equation, σ2 is the population parameter for the variance, μ is the parameter for the population mean, and N is the number of data points, which should include the entire population.

Sample variance

To use a sample to estimate the variance for a population, use the following formula. Using the previous equation with sample data tends to underestimate the variability. Because it's usually impossible to measure an entire population, statisticians use the equation for sample variances much more frequently.

$$s^2 = \frac{\sum(X-M)^2}{N-1}$$

In the equation, s2 is the sample variance, and M is the sample mean. N-1 in the denominator corrects for the tendency of a sample to underestimate the population variance.

Standard Deviation

The standard deviation is the standard or typical difference between each data point and the mean. When the values in a dataset are grouped closer together, you have a smaller standard deviation. On the other hand, when the values are spread out more, the standard deviation is larger because the standard distance is greater.

Conveniently, the standard deviation uses the original units of the data, which makes interpretation easier. Consequently, the standard deviation is the most widely used measure of variability. For example, in the pizza delivery example, a standard deviation of 5 indicates that the typical delivery time is plus or minus 5 minutes from the mean. It's often reported along with the mean: 20 minutes (s.d. 5).

The standard deviation is just the square root of the variance. Recall that the variance is in squared units. Hence, the square root returns the value to the natural units. The symbol for the standard deviation as a population parameter is σ while s represents it as a sample estimate. To calculate the standard deviation, calculate the variance as shown above, and then take the square root of it. Voila! You have the standard deviation!

In the variance section, we calculated a variance of 201 in the table.

Therefore, the standard deviation for that dataset is 14.177.

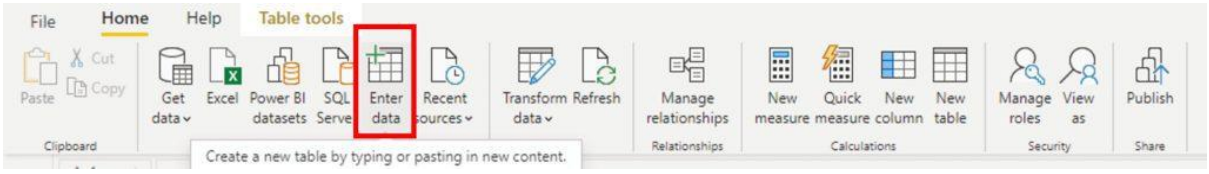
The standard deviation is similar to the mean absolute deviation. Both use the original data units and they compare the data values to mean to assess variability. However, there are differences.

Assignment:

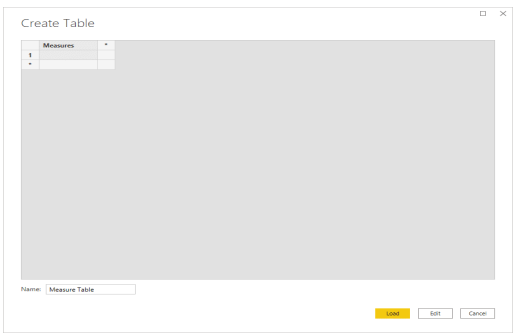
Create Measure Table using Power BI.

Solution:

In the Home tab of the Desktop ribbon, click **Enter Data**. This will pop up a screen that allows you to manually enter data. You can actually paste in data that you copied from Excel to this screen, but we're not going to use that functionality right now.

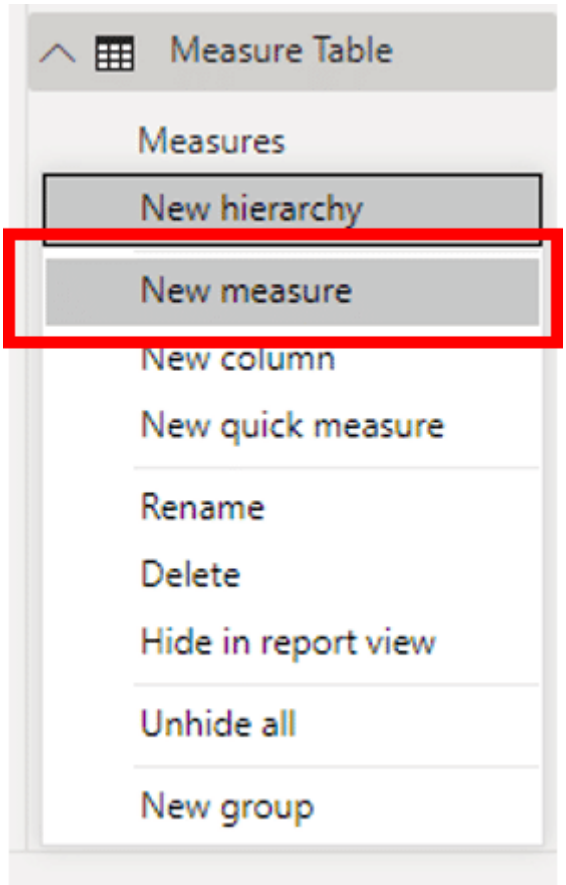


For now, we are going to rename the column to Measures and then rename the table name at the bottom to Measure Table.

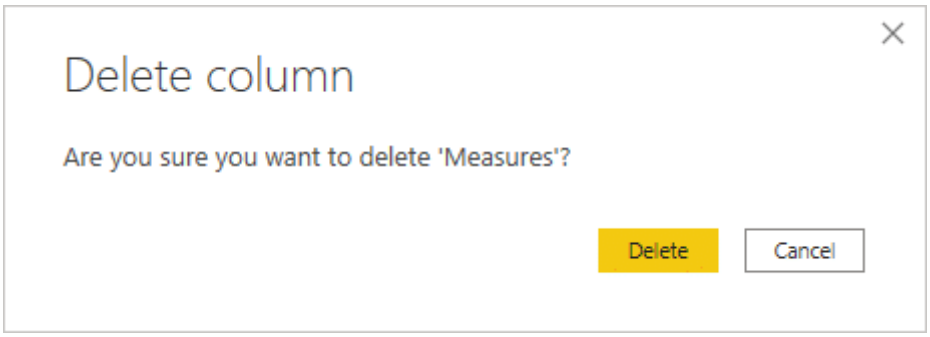


Now you have a new table with one blank measure in it named Measures. Right click in your new table and click **New measure**. Enter in the calculation that you want to add to your report.

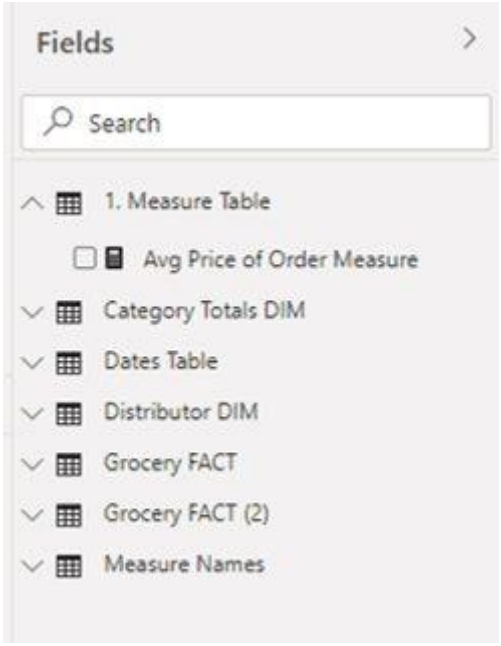
All measures that you create under this Measure Table can reference other tables in your data model. Since measures aren't actually adding data to your model, only the measure itself lives here allowing you to separate out your measures from your other tables.



we named the column Measures, so go ahead and delete that Measures column from the Measure Table.



now you have a table specifically for storing your measures separate from the other tables in your data model.



We do it because it adds simplicity to our interactions with Power BI Desktop. This way, you don't have to scroll through seven tables and hundreds of columns to find measures. While we could easily use the Search field, this approach makes our model more organized.

On the flip side, we understand how people would like the measures to be in the tables that they refer to. This would help people understand what tables measures were referring to without looking through the DAX.

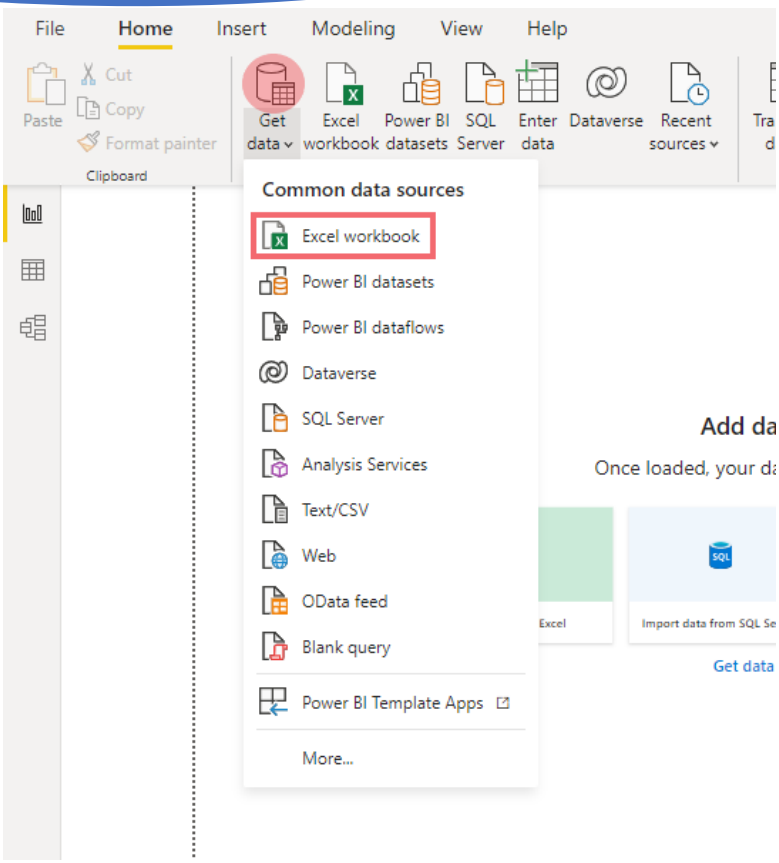
Practice task:

How to find Linear regression in Power BI?

Step1:

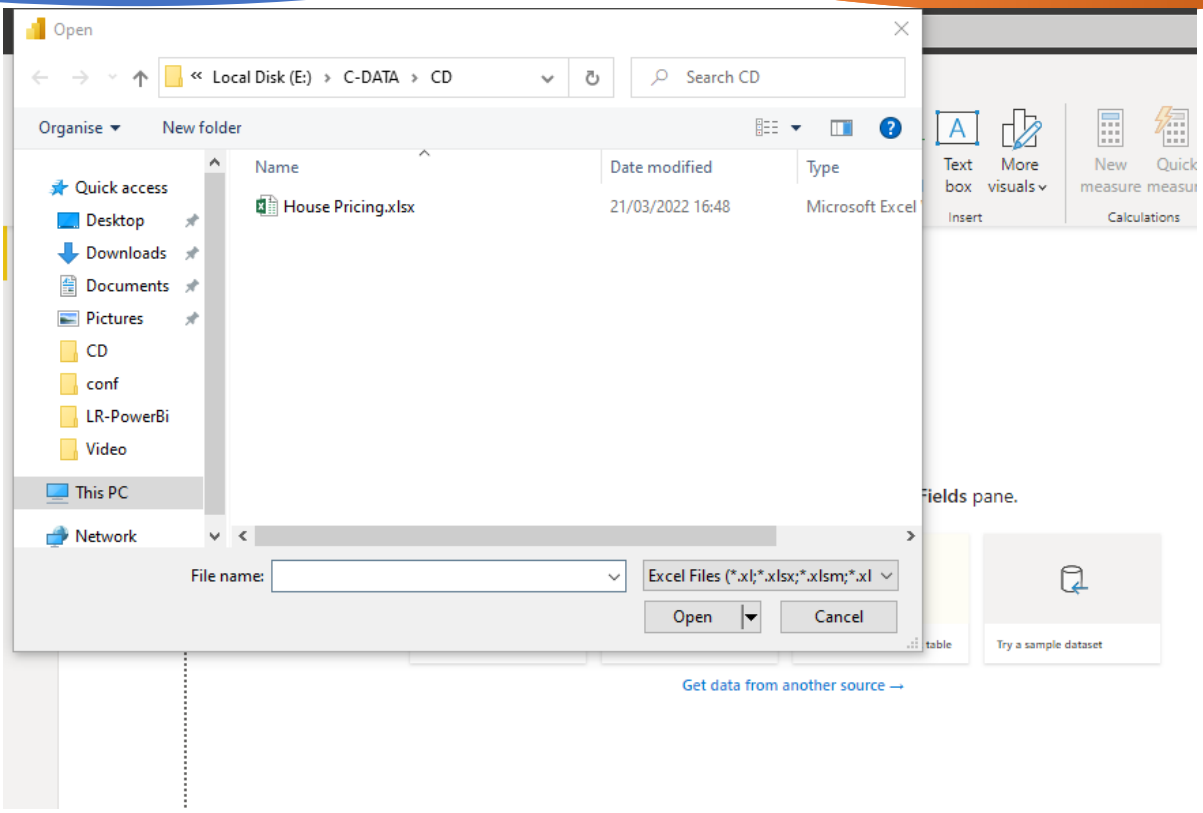
The first step is to **get your dataset** into Power BI.

Dataset can be on SQL Server, Oracle, Power BI datasets, Power BI dataflows, MySQL database, Text/CSV, PDF, Access, XML, JSON, or on any other source. Our is in Excel. So, referring to that, go and click on the **Get Data** tab and **select Excel workbook**.



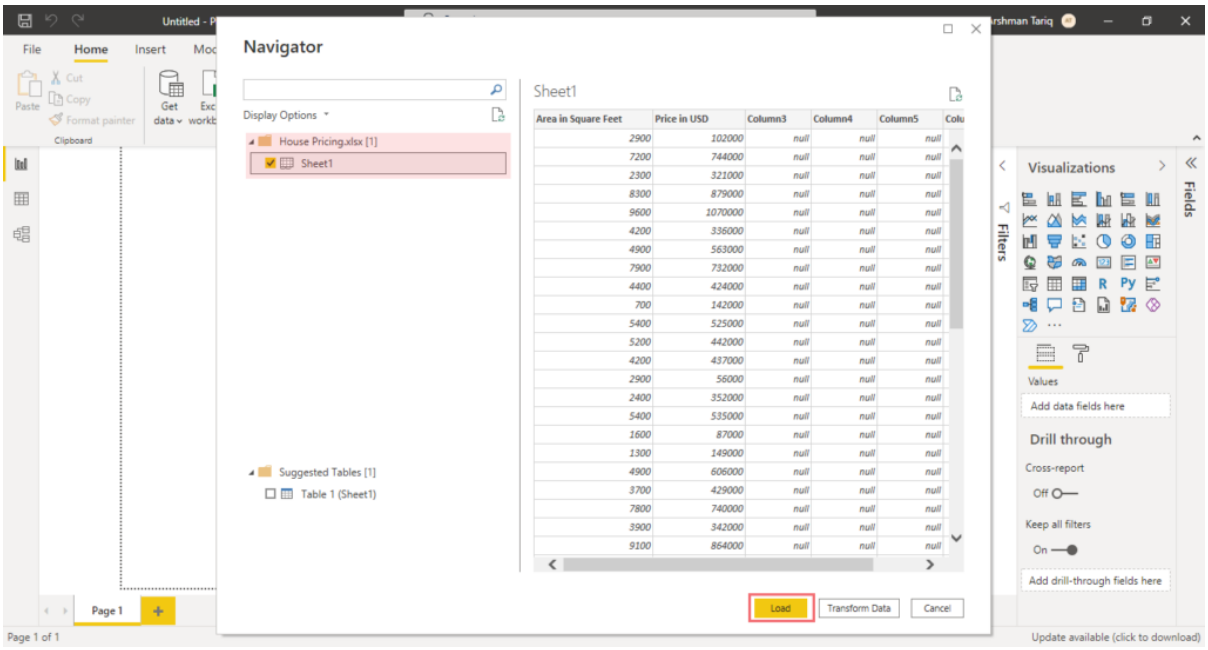
Dataset Selection

A dialogue box will open. Reach your destination, **select, and open your Excel file** which in this example is “House Pricing.xlsx”.



Dataset Selection in Power BI

Now **select a sheet** in your workbook having your dataset and load the dataset into Power BI.

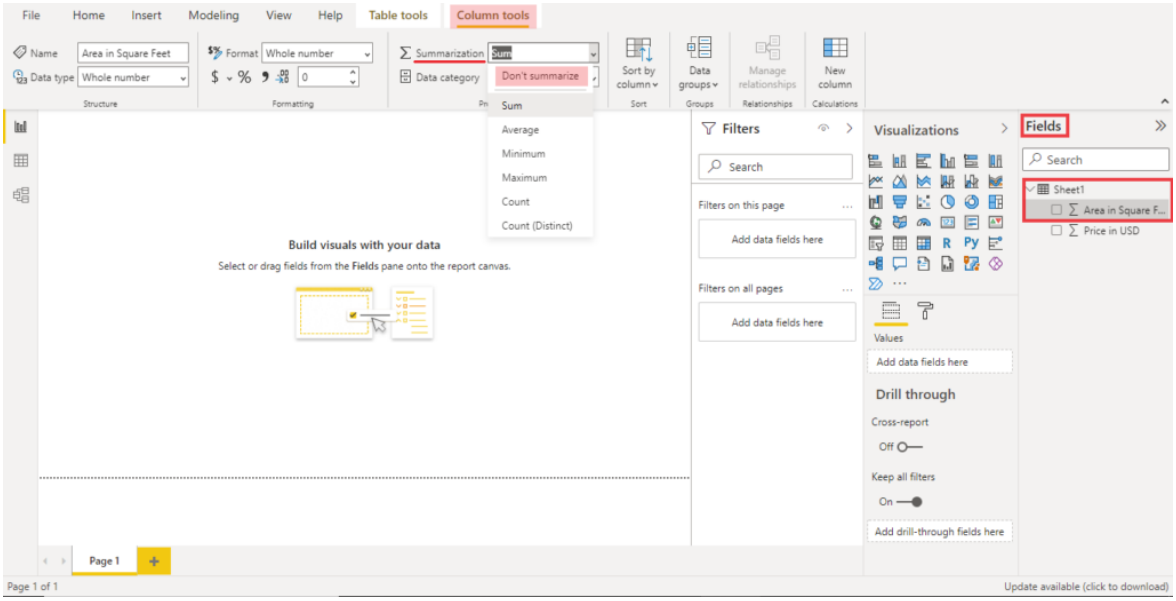


Sheet selection

Step 2: Creating Scatter Plot for Linear Regression

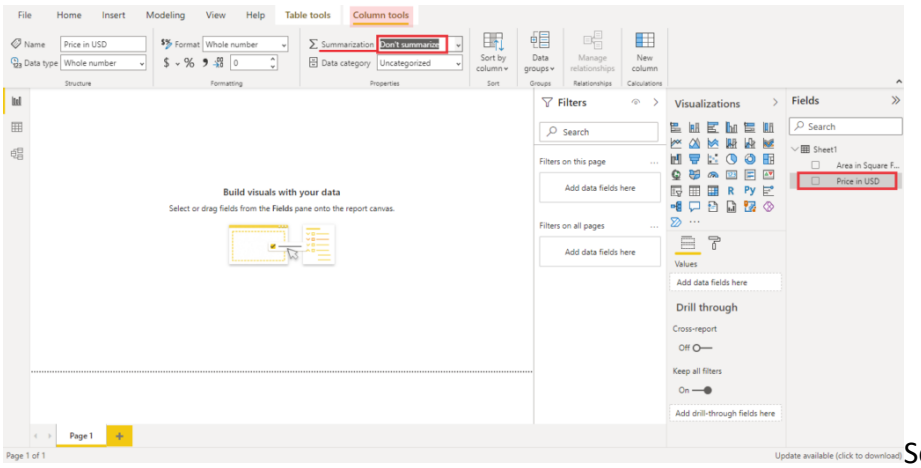
Now dataset has been loaded into Power BI and the second step would be to design a scatter plot for linear regression analysis. For this,
Go into **Fields**, click on your **sheet**, and then on a **column** to select it and set a measure. In

this example, we first selected the column “Area in Square Feet”.
After selecting a column, go to the **“Column tools”** tab and then to **“Properties”** in the ribbon. After that, set the measure for **“Summarization”** from **“sum”** to **“Don’t Summerize”**.



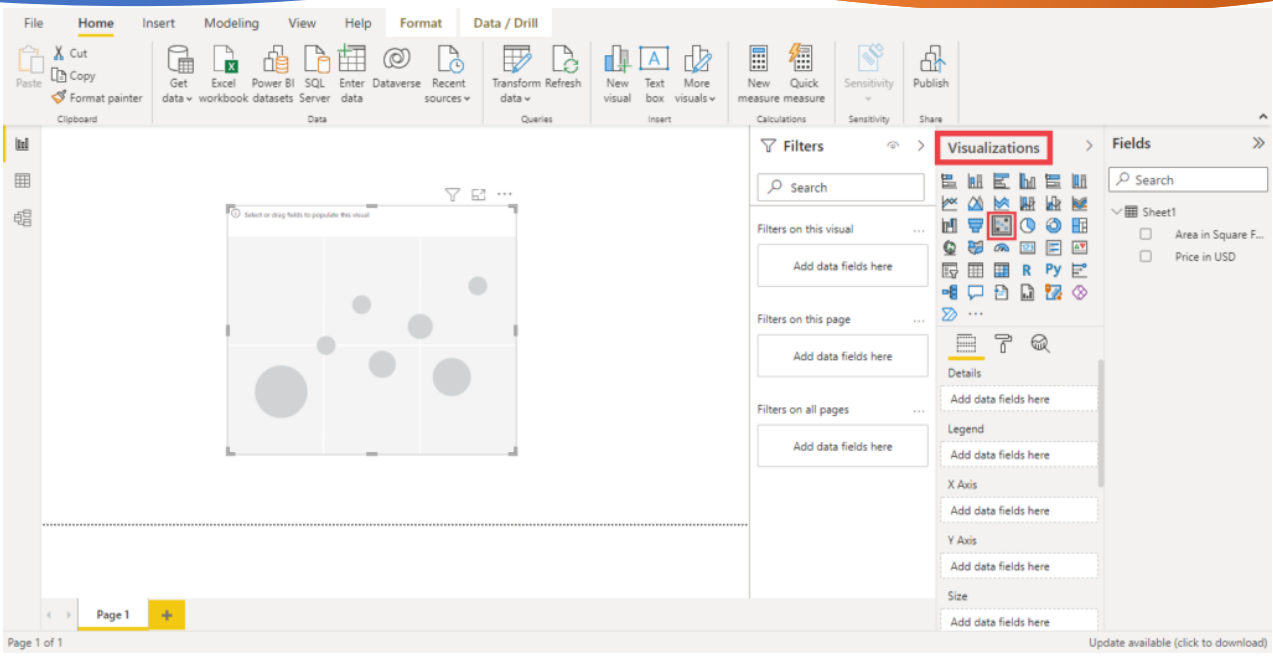
First column selection and setting its measure

Repeat this step for your other variable. In this case, the other column i.e., “Price in USD” represents the other variable. We selected that column and set the measure to “Don’t Summarize” for this too.



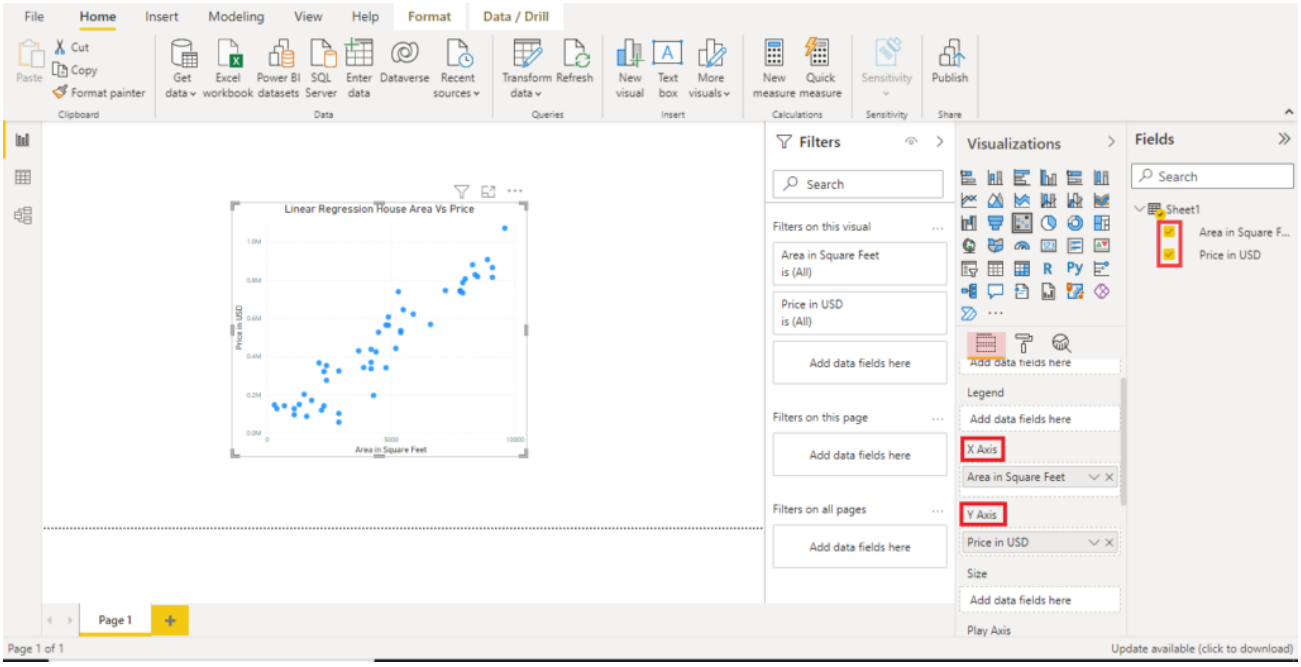
Second column selection and setting its measure

Now, we have selected our columns of variables and now we have to select for **SCATTER PLOT**. For this, go to the **“Visualizations”** and select **“SCATTER PLOT”**



Selecting scatter plot from visualizations

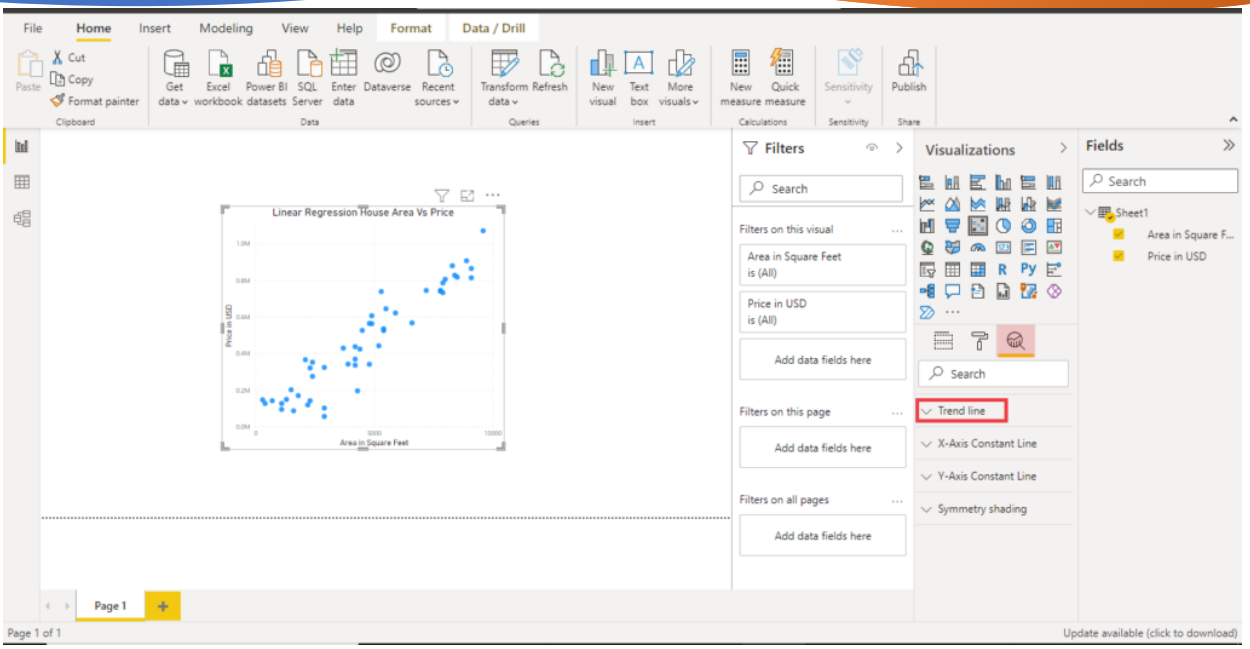
Now select your independent and dependent variable for your x and y-axis (in the plot) either from "**fields**" or from the **"Details"** section below the **"Visualizations"**. In this example, the factor **"Area"** is an independent variable and is set at the x-axis while the factor "Price" being the dependent variable is set on the y-axis.



Specifying variables on the X and Y-axis

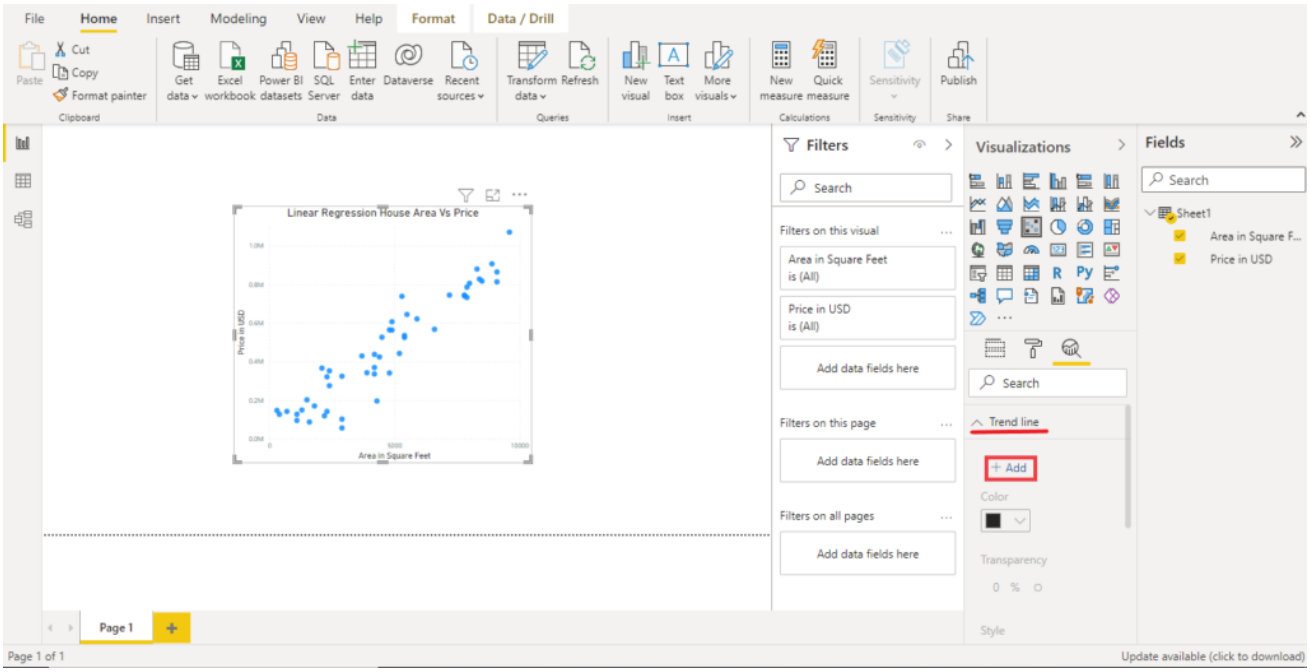
Step 3: Adding Trend Line in Scatter Plot for linear regression

After specifying, the x and y-axis, the next step is to add a trend line. Now, we are going to add a trend line to our plot. For this, click on **"Analytics"** and then on **"Trend Line"**.



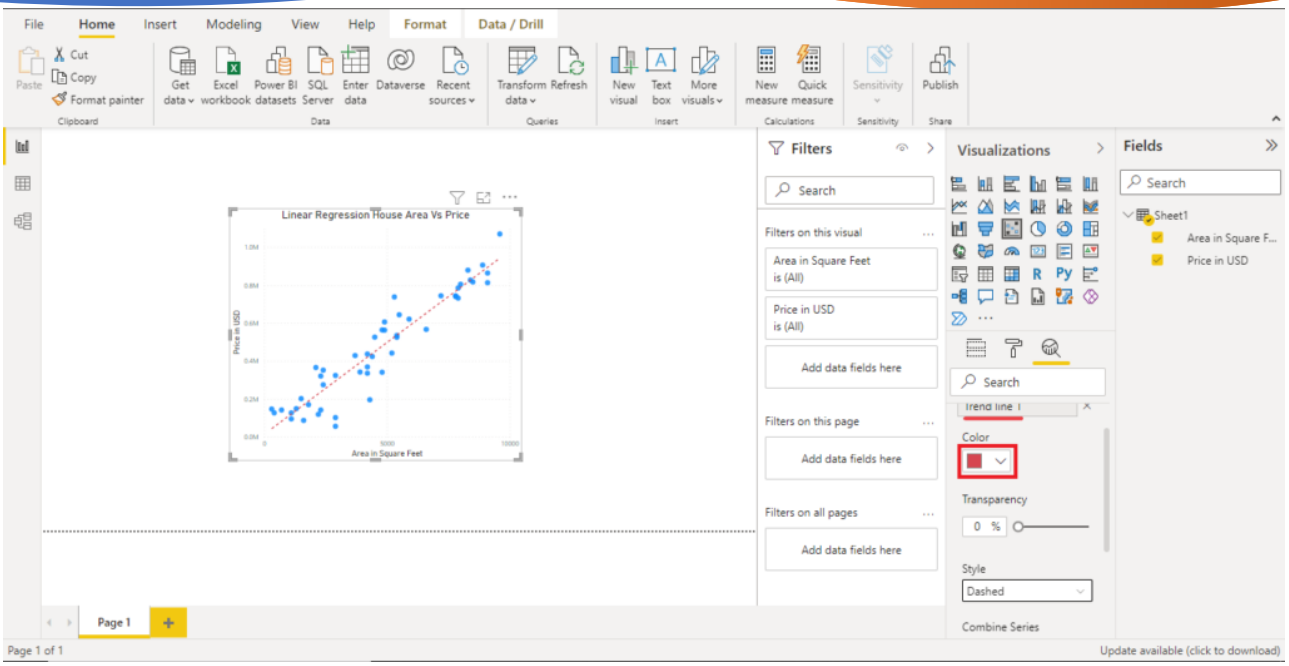
Adding trend line in scatter plot

In the **"Trend Line"** click on **"ADD"**



Setting trend line colour

And select the colour of your choice. In this example, we set the colour of the trend line to red.



Linear regression of House Area (in sq. ft) Vs Price (in USD)

This trend line is showing the relation i.e., the correlation between two values which is positive and implies as the independent variable i.e., Area of House (in ft^2) increases, the dependent variable i.e., Price (in USD), also increases.

Course Assignment:

Create a report from Excel file using Power BI desktop

Solution:

1st, download the sample:

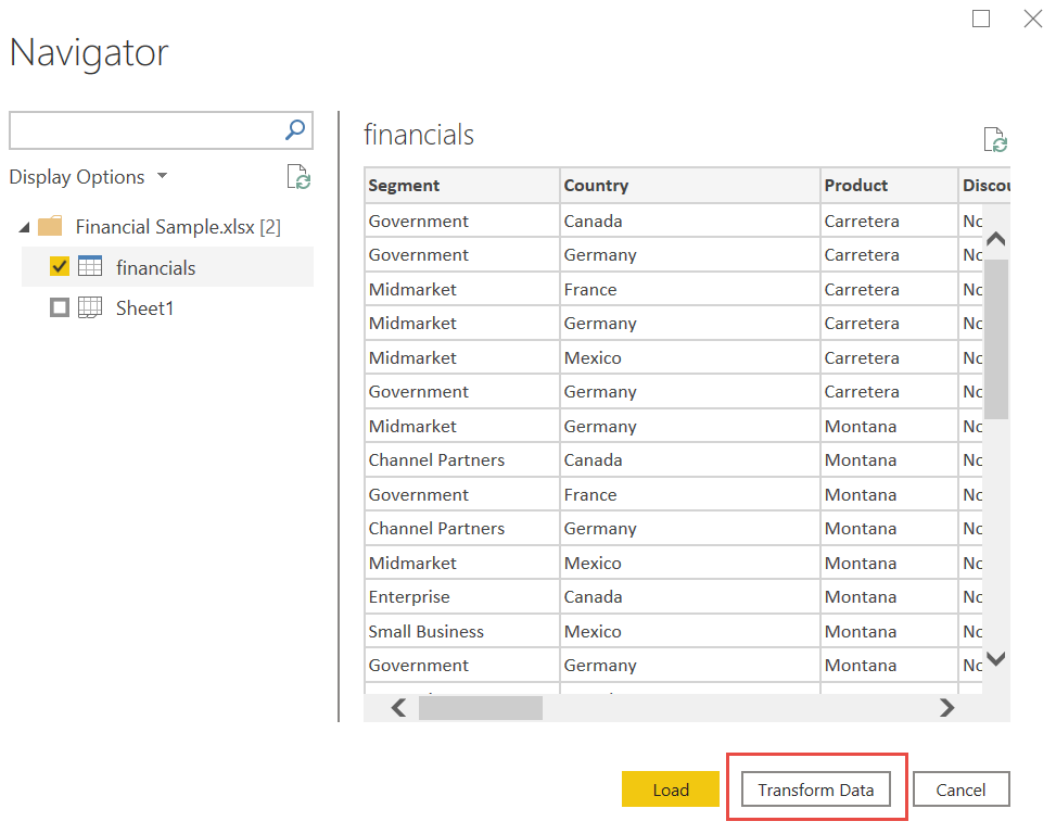
You can download the sample workbook directly.

1. Download sample file from this link: Financial sample Excel workbook.
2. Open Power BI Desktop.
3. In the **Data** section of the **Home** ribbon, select **Excel**.
4. Navigate to where you saved the sample workbook, and select **Open**.

Next, Prepare your data:

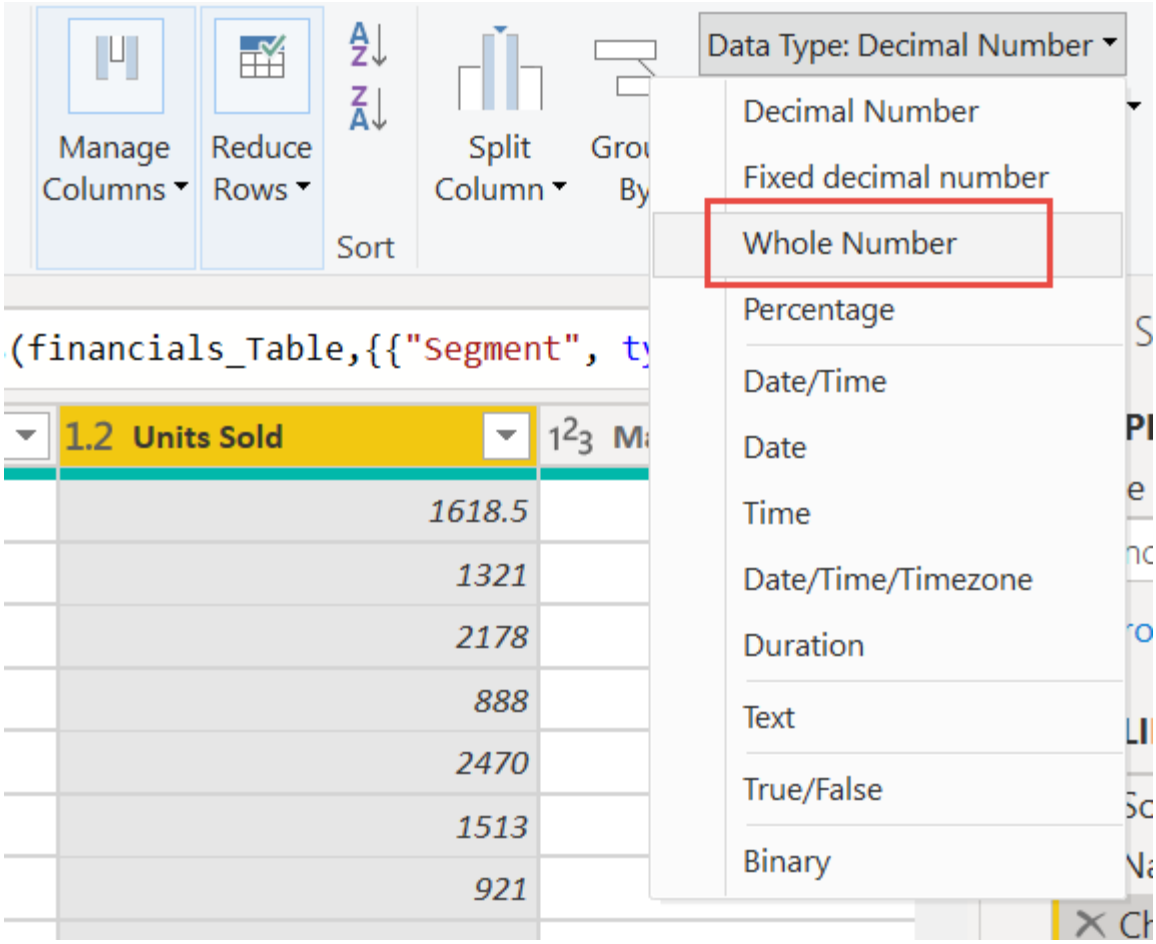
In **Navigator**, you have the option to *transform* or *load* the data. The Navigator provides a preview of your data so you can verify that you have the correct range of data. Numeric data types are italicized. If you need to make changes, transform your data before loading. To make the visualizations easier to read later, we do want to transform the data now. As you do each transformation, you see it added to the list under **Query Settings** in **Applied Steps**

1. Select the **Financials** table, and choose **Transform Data**.

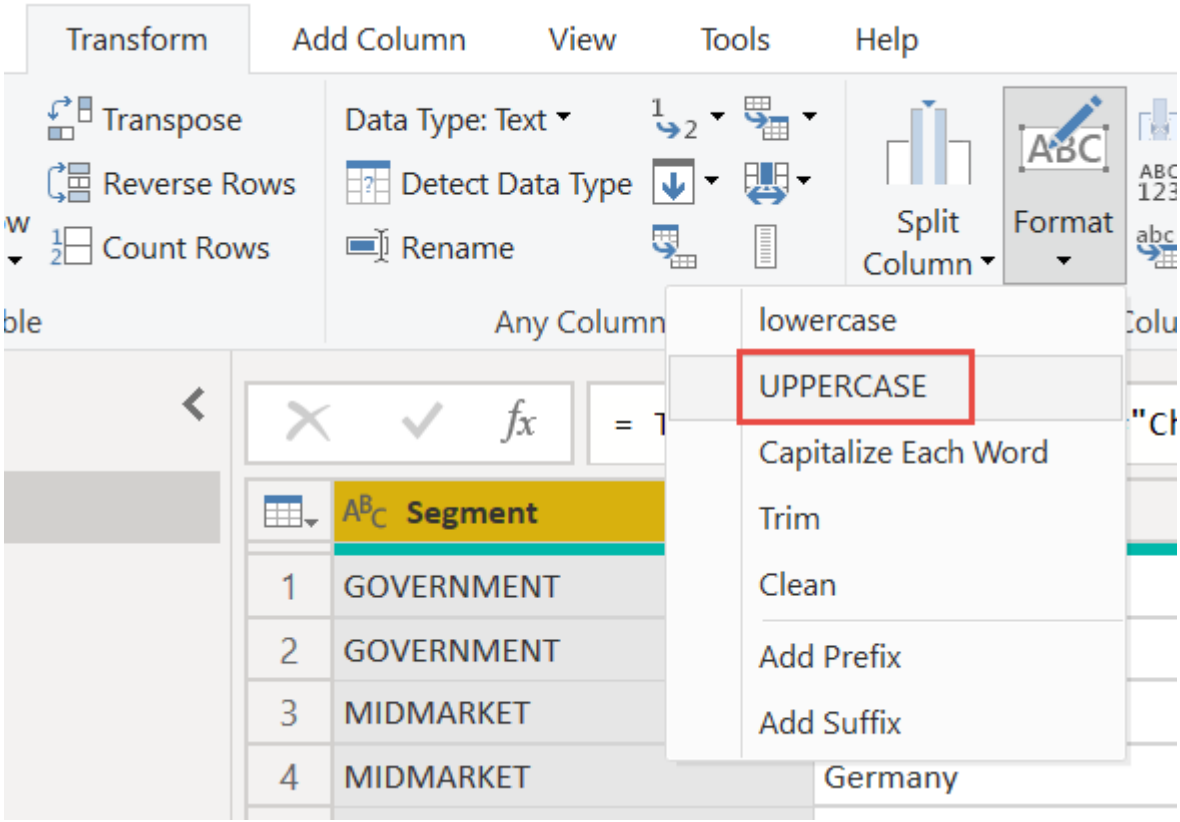


2. Select the **Units Sold** column. On the **Transform** tab, select **Data Type**, then select **Whole Number**. Choose **Replace current** to change the column type.

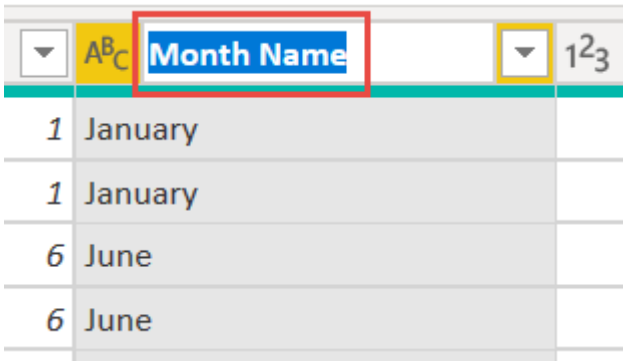
The top data cleaning step users do most often is changing data types. In this case, the units sold are in decimal form. It doesn't make sense to have 0.2 or 0.5 of a unit sold, does it? So let's change that to whole number.



3. Select the **Segment** column. We want to make the segments easier to see in the chart later, so let's format the Segment column. On the **Transform** tab, select **Format**, then select **UPPERCASE**.

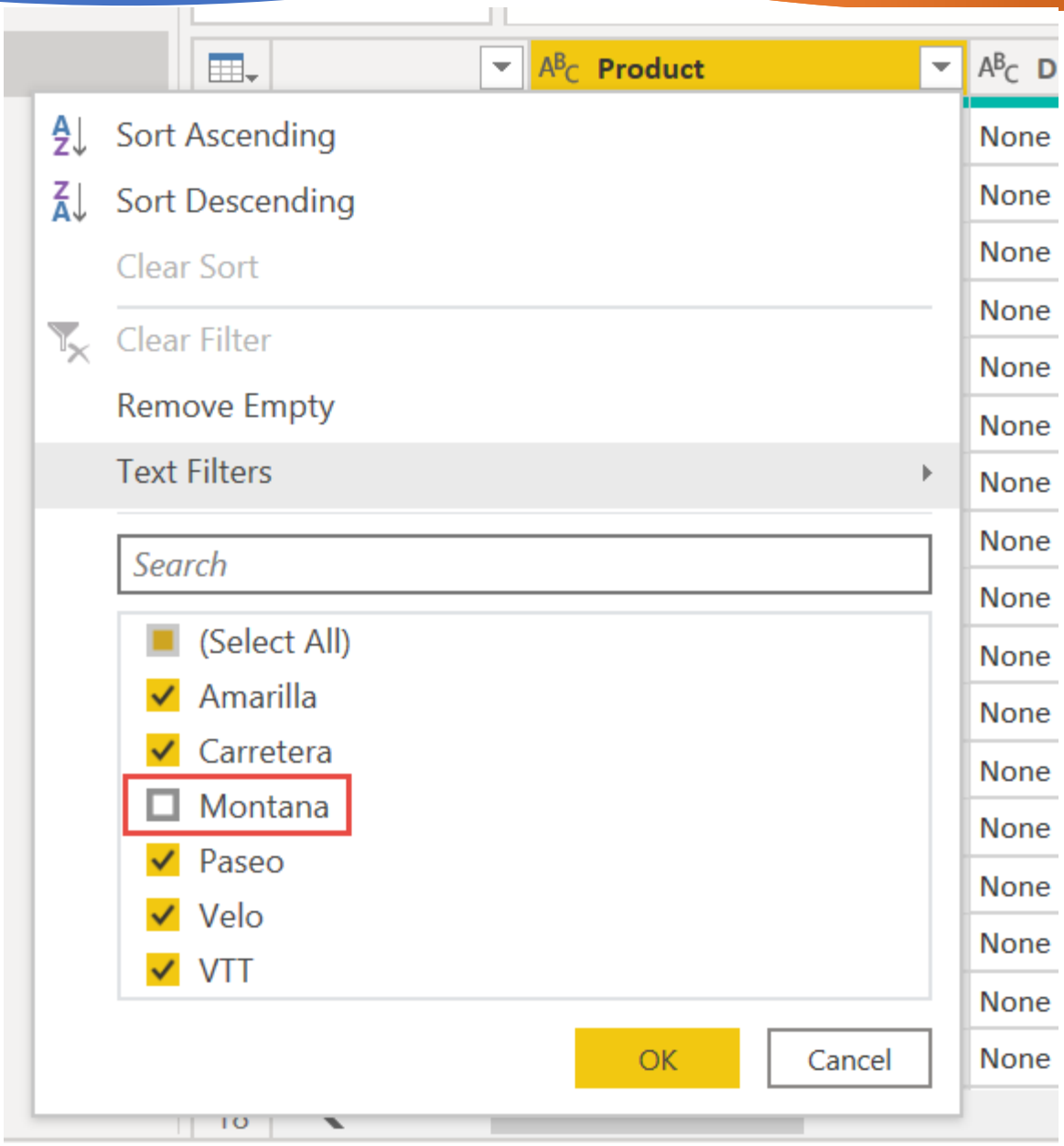


4. Let's shorten the column name from **Month Name** to just **Month**. Double-click the **Month Name** column, and rename to just **Month**.

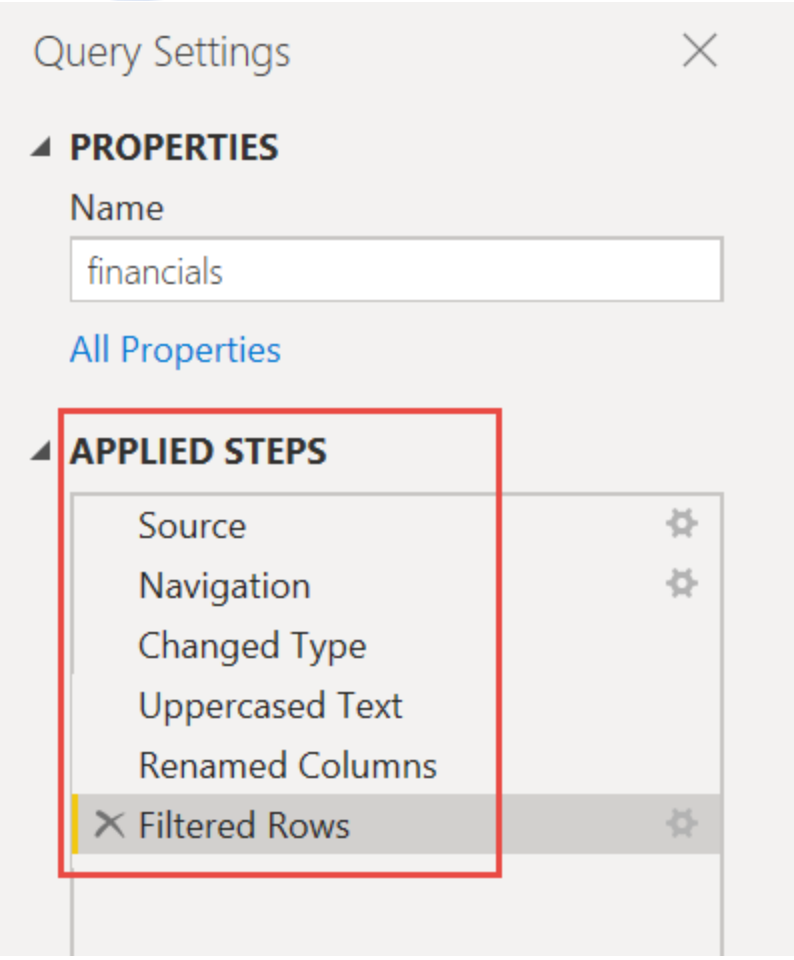


5. In the **Product** column, select the dropdown and clear the box next to **Montana**.

We know the Montana product was discontinued last month, so we want to filter this data from our report to avoid confusion.

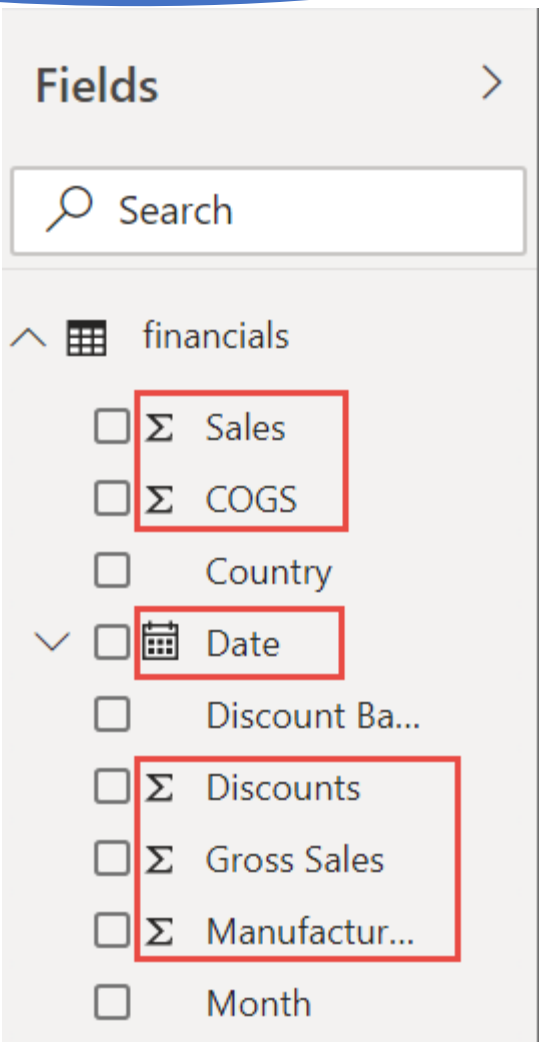


6. You see that each transformation has been added to the list under **Query Settings** in **Applied Steps**.



7. Back on the **Home** tab, select **Close & Apply**. Our data is almost ready for building a report.

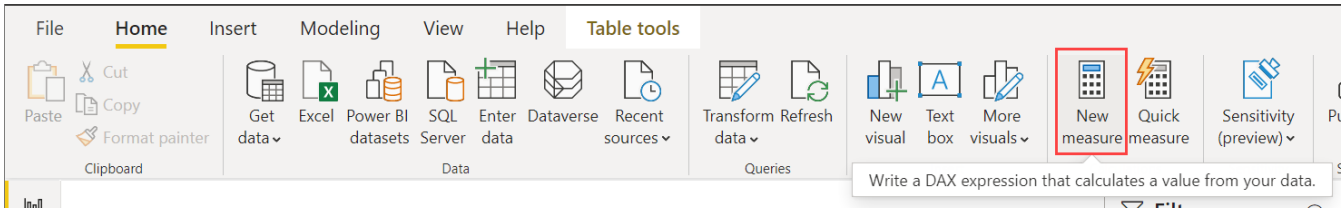
You see the Sigma symbol in the Fields list? Power BI has detected that those fields are numeric. Power BI also indicates the date field with a calendar symbol.



Extra credit: Write an expression in DAX

Writing *measures* and creating *tables* in the *DAX* formula language is super powerful for data modeling. There's lots to learn about DAX in the Power BI documentation. For now, let's write a basic expression and join two tables.

1. On the **Home** ribbon, select **New measure**.

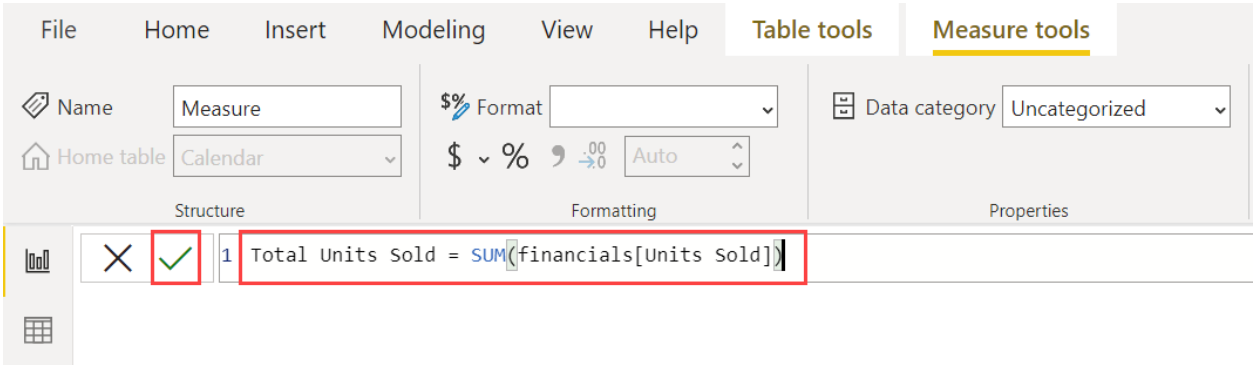


2. Type this expression to add all the numbers in the Units Sold column.

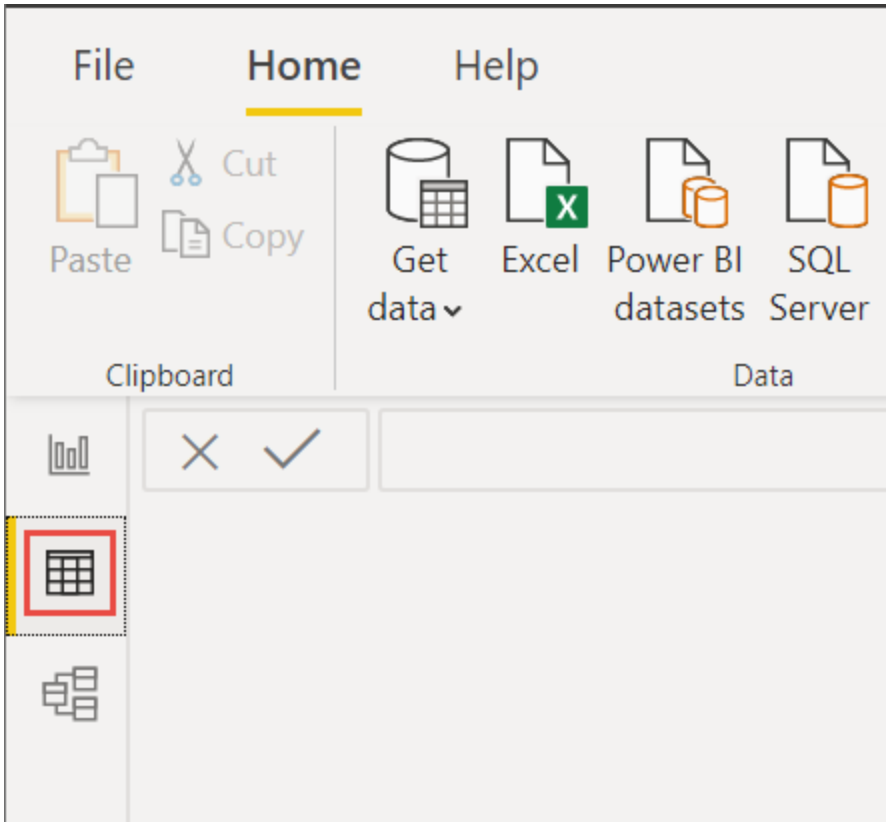
DAX:

Total Units Sold = SUM(financials[Units Sold])

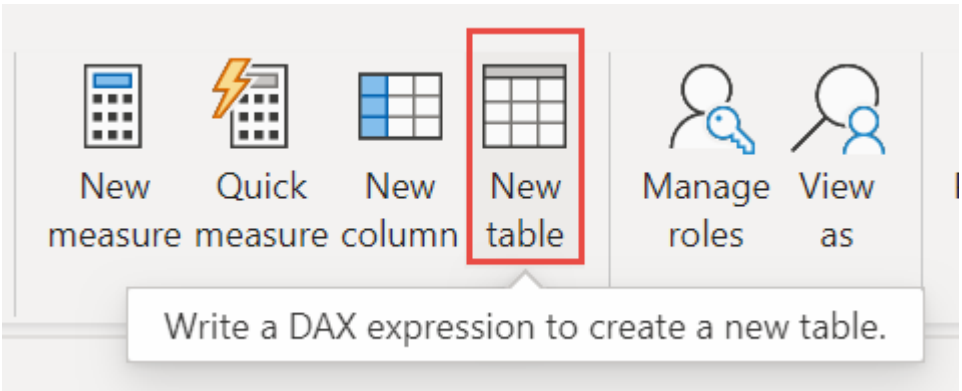
3. Select the check mark to commit.



4. Now select the **Data** view on the left.



5. On the **Home** ribbon, select **New table**.

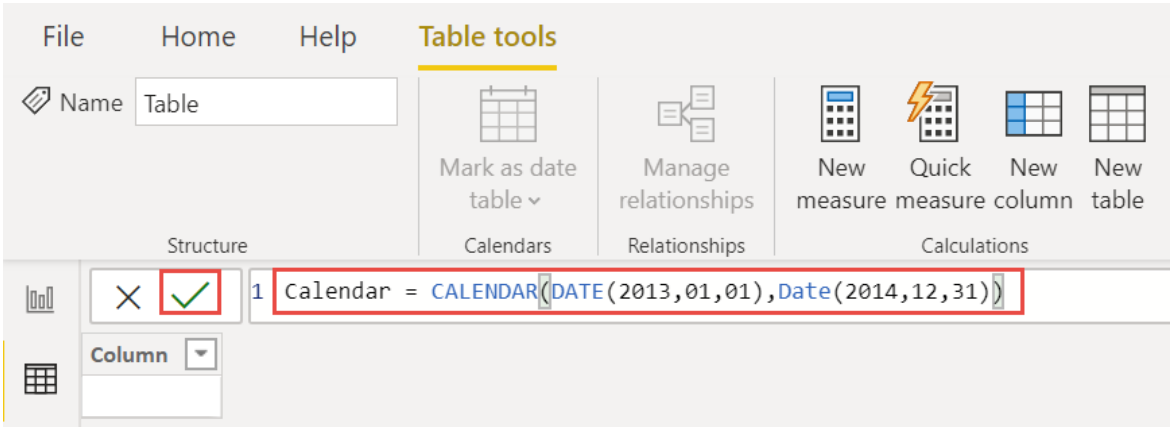


6. Type this expression to generate a Calendar table of all dates between January 1, 2013, and December 31, 2014.

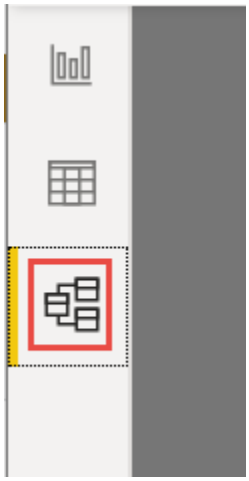
DAX:

Calendar = CALENDAR(DATE(2013,01,01),Date(2014,12,31))

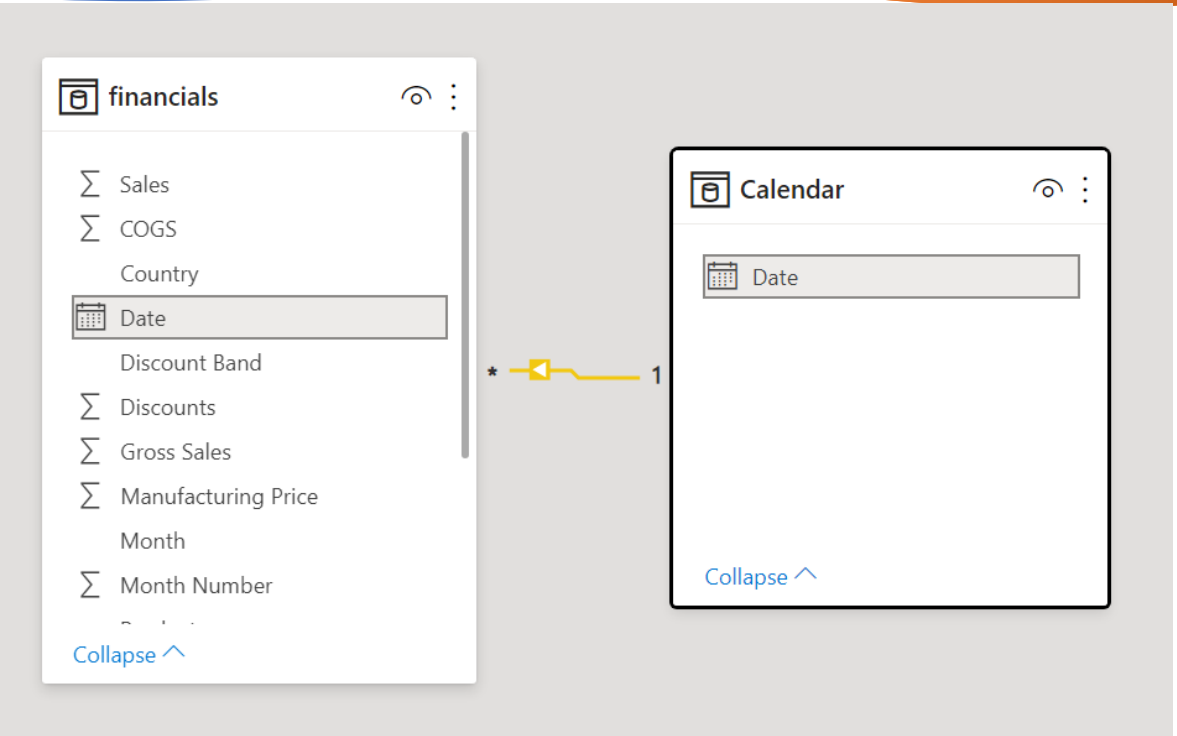
7. Select the check mark to commit.



8. Now select **Model** view on the left.



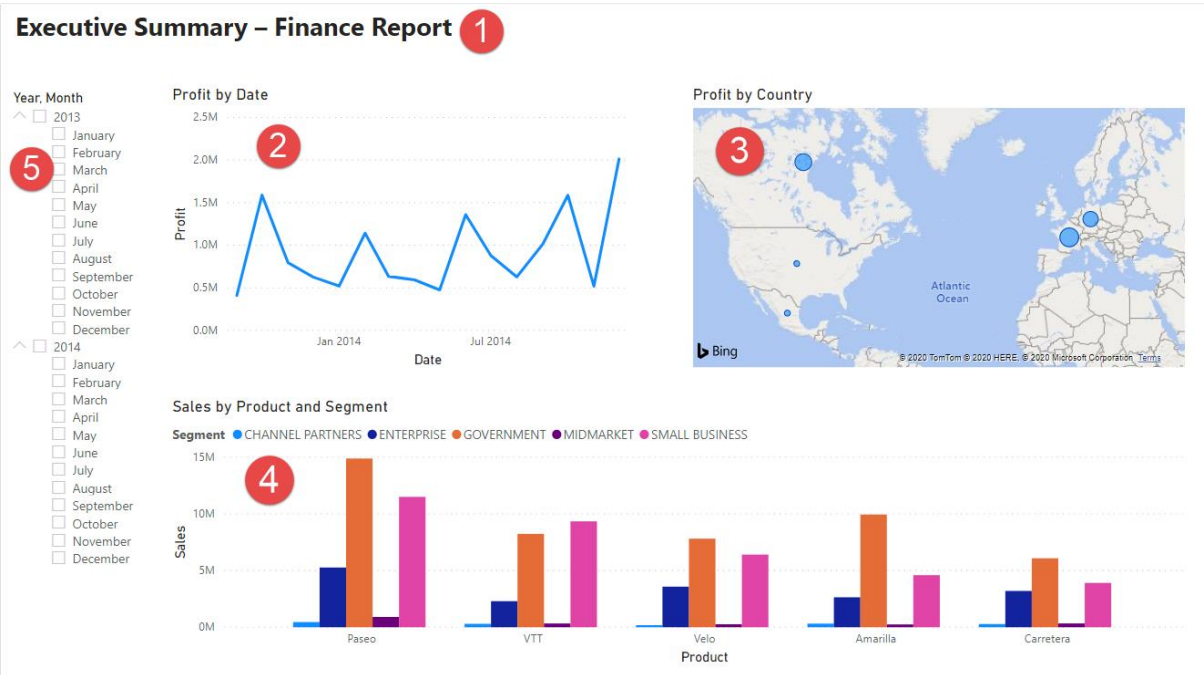
9. Drag the **Date** field from the financials table to the **Date** field in the Calendar table to join the tables, and create a *relationship* between them.



Now the main part, **Build your report**

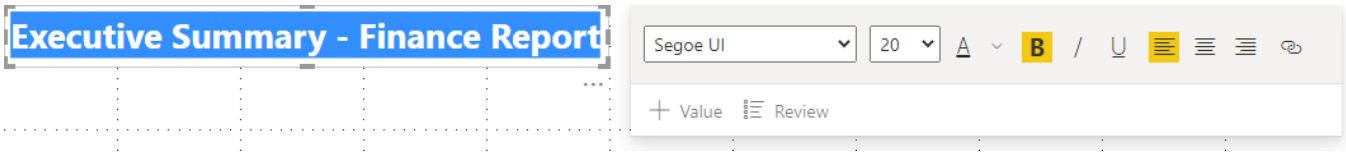
Now that you've transformed and loaded your data, it's time to create your report. In the Fields pane on the right, you see the fields in the data model you created.

The final report, one visual at a time will look like this following image



Visual 1: Add a title

1. On the **Insert** ribbon, select **Text Box**. Type "Executive Summary – Finance Report".
2. Select the text you typed. Set the **Font Size** to 20 and **Bold**.



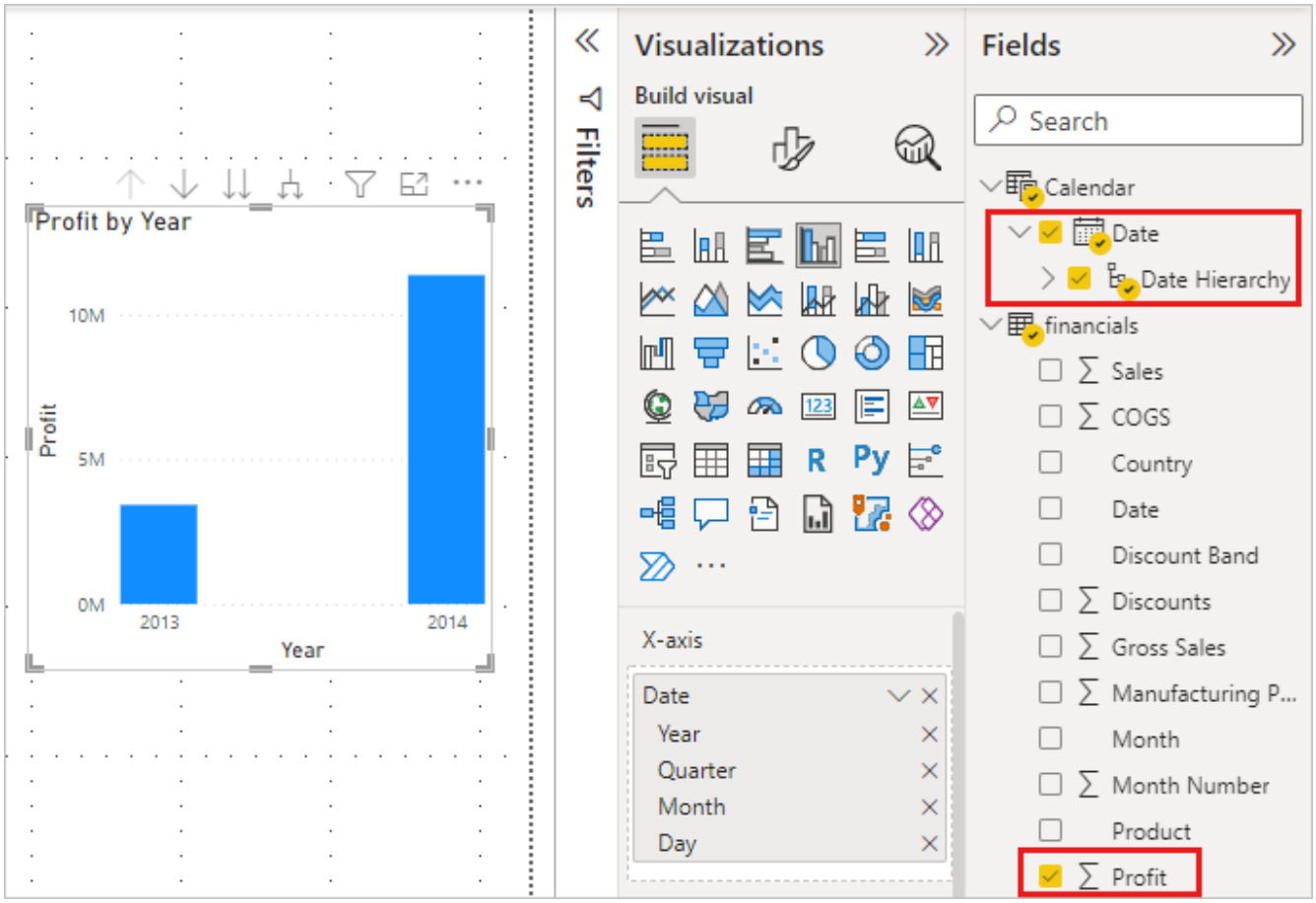
3. Resize the box to fit on one line.

Visual 2: Profit by Date

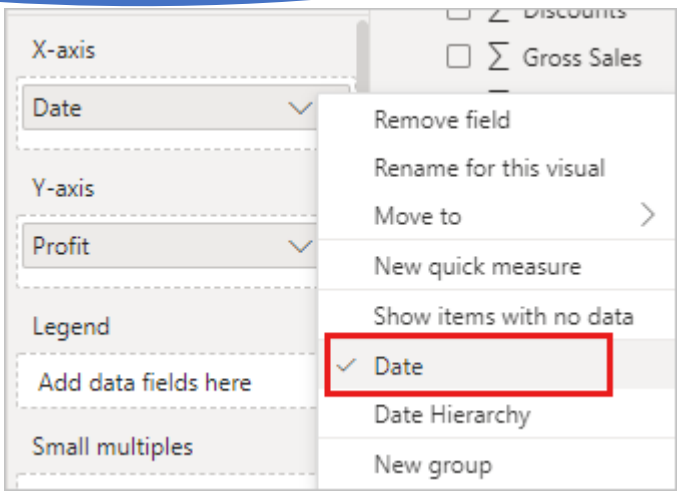
Now, you create a line chart to see which month and year had the highest profit.

1. From the Fields pane, drag the **Profit** field to a blank area on the report canvas. By default, Power BI displays a column chart with one column, Profit.
2. Drag the **Date** field to the same visual.

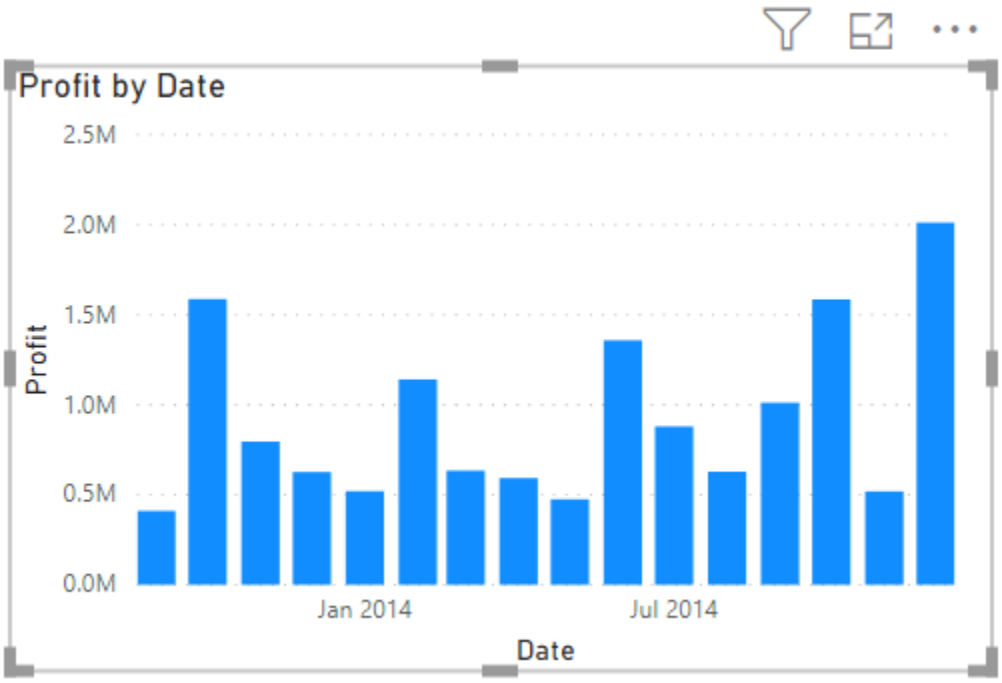
Power BI updates the column chart to show profit by the two years.



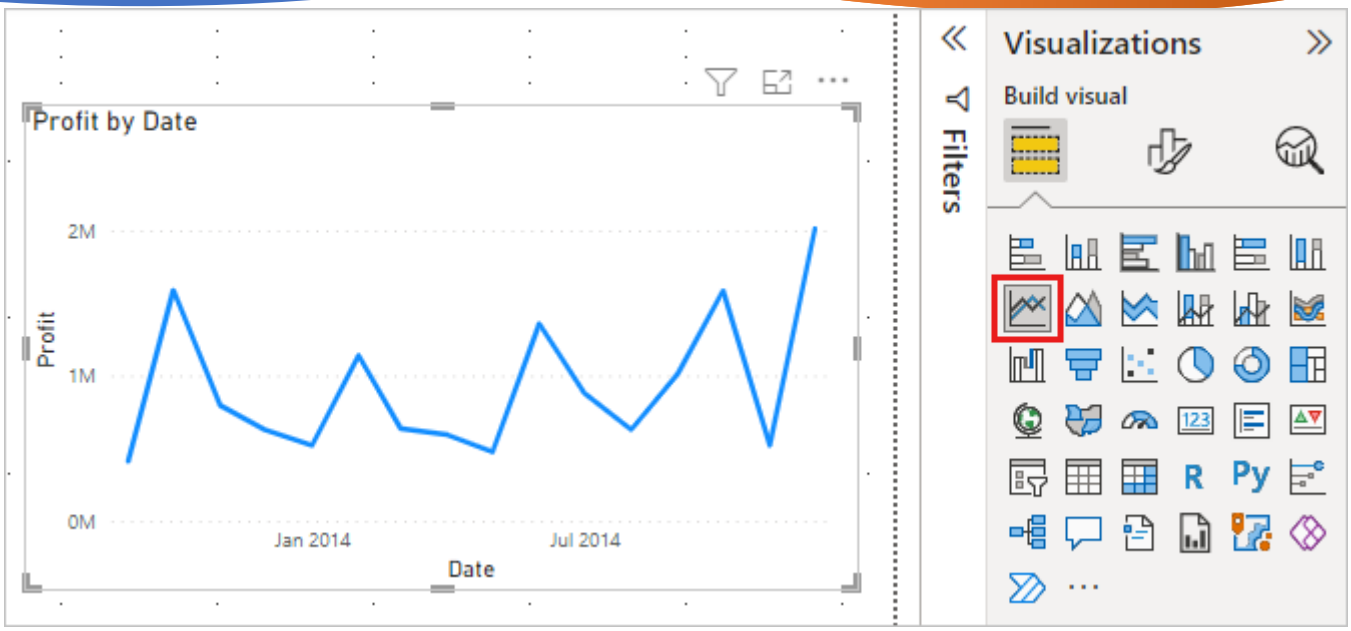
3. In the **Fields** section of the Visualizations pane, select the drop-down in the **X-axis** value. Change **Date** from **Date Hierarchy** to **Date**.



Power BI updates the column chart to show profit for each month.



4. In the Visualizations pane, change the visualization type to **Line chart**.



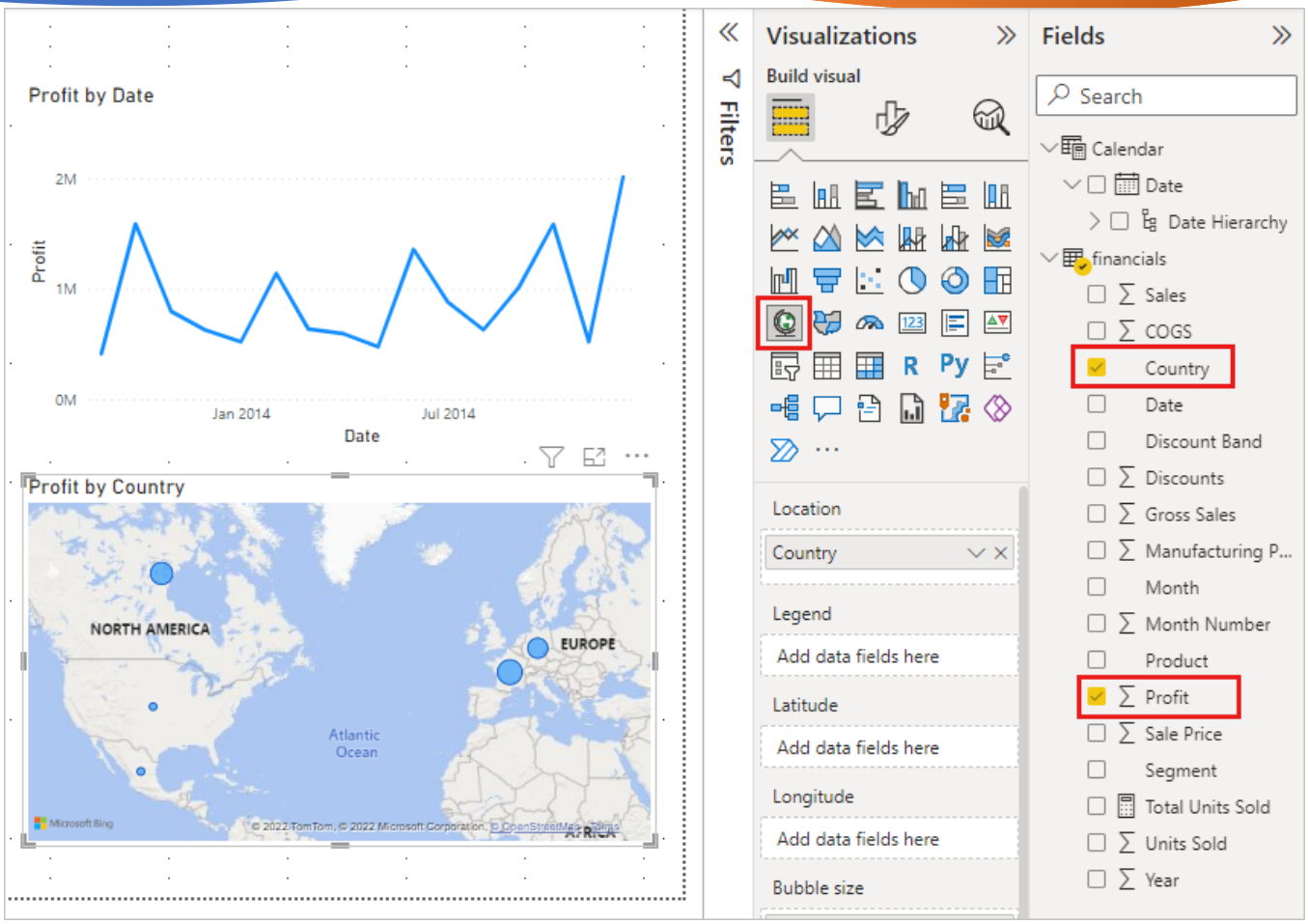
Now you can easily see that December 2014 had the most profit.

Visual 3: Profit by Country/Region

Create a map to see which country/region had the highest profits.

1. From the Fields pane, drag the **Country** field to a blank area on your report canvas to create a map.
2. Drag the **Profit** field to the map.

Power BI creates a map visual with bubbles representing the relative profit of each location.



Europe seems to be performing better than North America.

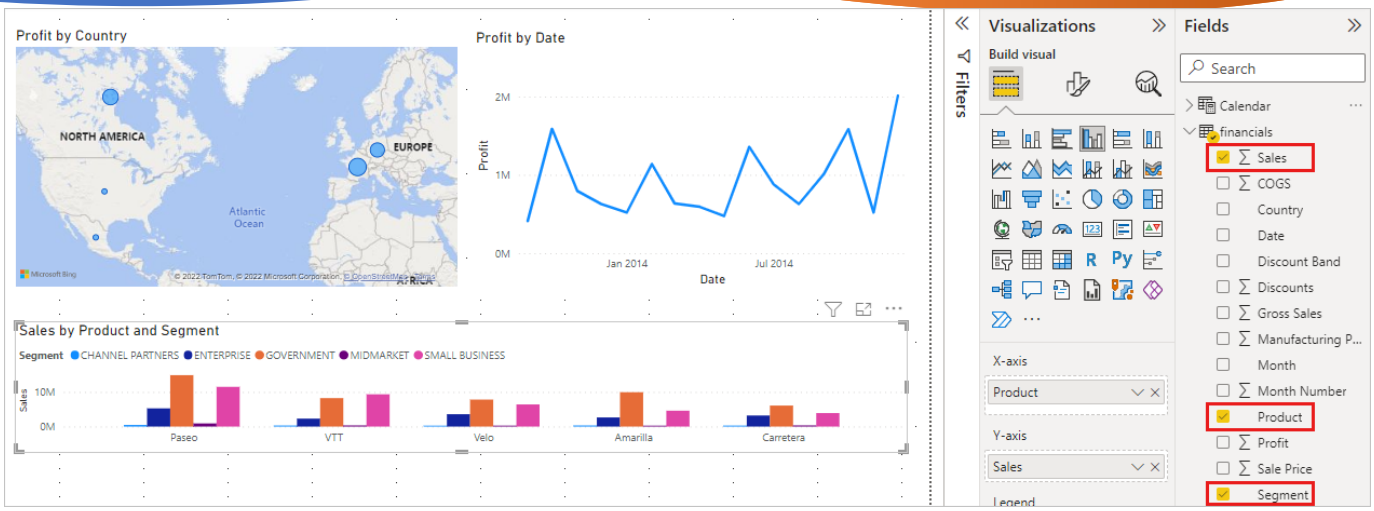
Visual 4: Sales by Product and Segment

Create a bar chart to determine which companies and segments to invest in.

1. Drag the two charts you've created to be side by side in the top half of the canvas. Save some room on the left side of the canvas.
2. Select a blank area in the lower half of your report canvas.
3. In the Fields pane, select the **Sales**, **Product**, and **Segment** fields.

Power BI automatically creates a clustered column chart.

4. Drag the chart so it's wide enough to fill the space under the two upper charts.



Looks like the company should continue to invest in the Paseo product and target the Small Business and Government segments.

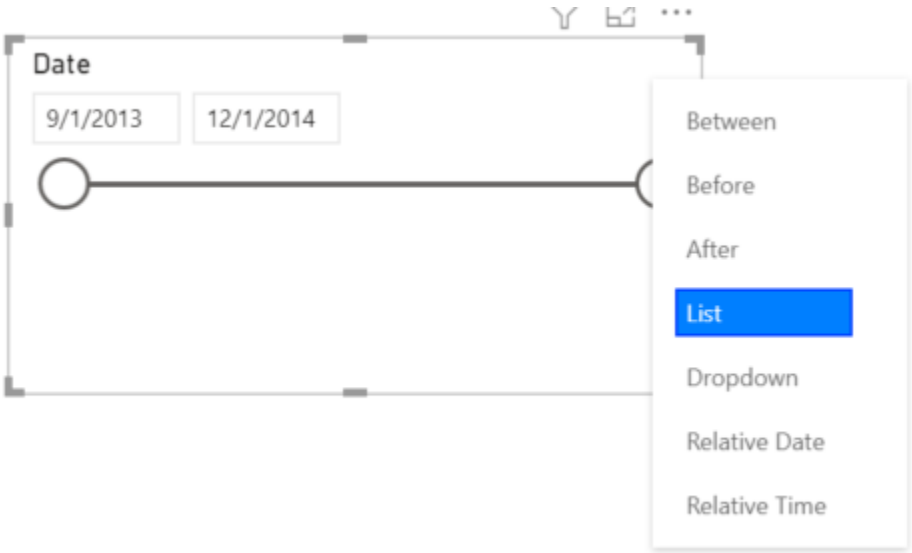
Visual 5: Year slicer

Slicers are a valuable tool for filtering the visuals on a report page to a specific selection. In this case, we can create two different slicers to narrow in on performance for each month and year. In which one slicer uses the date field in the original table.

Date slicer using the original table

1. In the Fields pane, select the **Date** field in the Financials table. Drag it to the blank area on the left of the canvas.
2. In the Visualizations pane, choose **Slicer**.

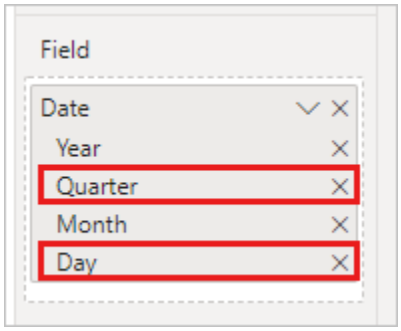
Power BI automatically creates a numeric range slicer.



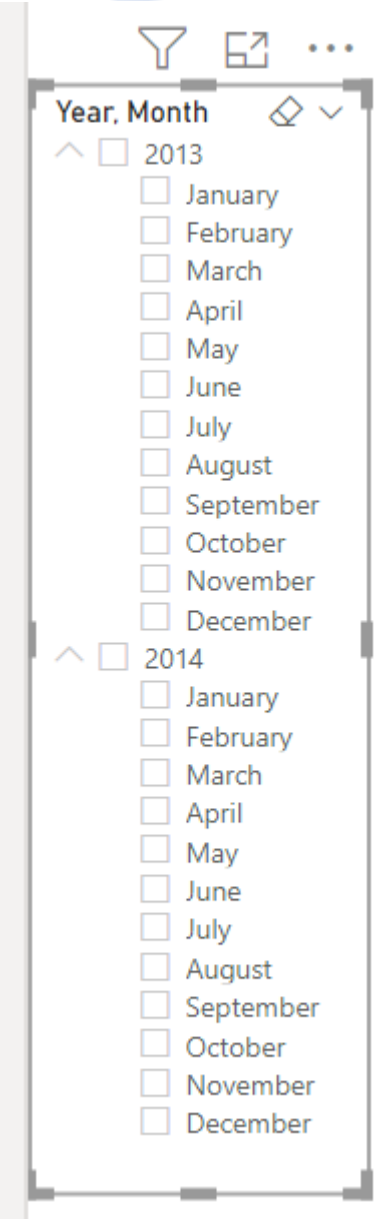
3. You can drag the ends to filter, or select the arrow in the upper-right corner and change it to a different type of slicer.

Date slicer using the DAX table

1. In the Fields pane, select the **Date** field in the Calendar table. Drag it to the blank area on the left of the canvas.
2. In the Visualizations pane, choose **Slicer**.
3. In the Fields section of the Visualizations pane, select the drop-down in **Fields**. Remove Quarter and Day so only Year and Month are left.



4. Expand each year and resize the visual, so all months are visible.



We'll use this slicer in the finished report.

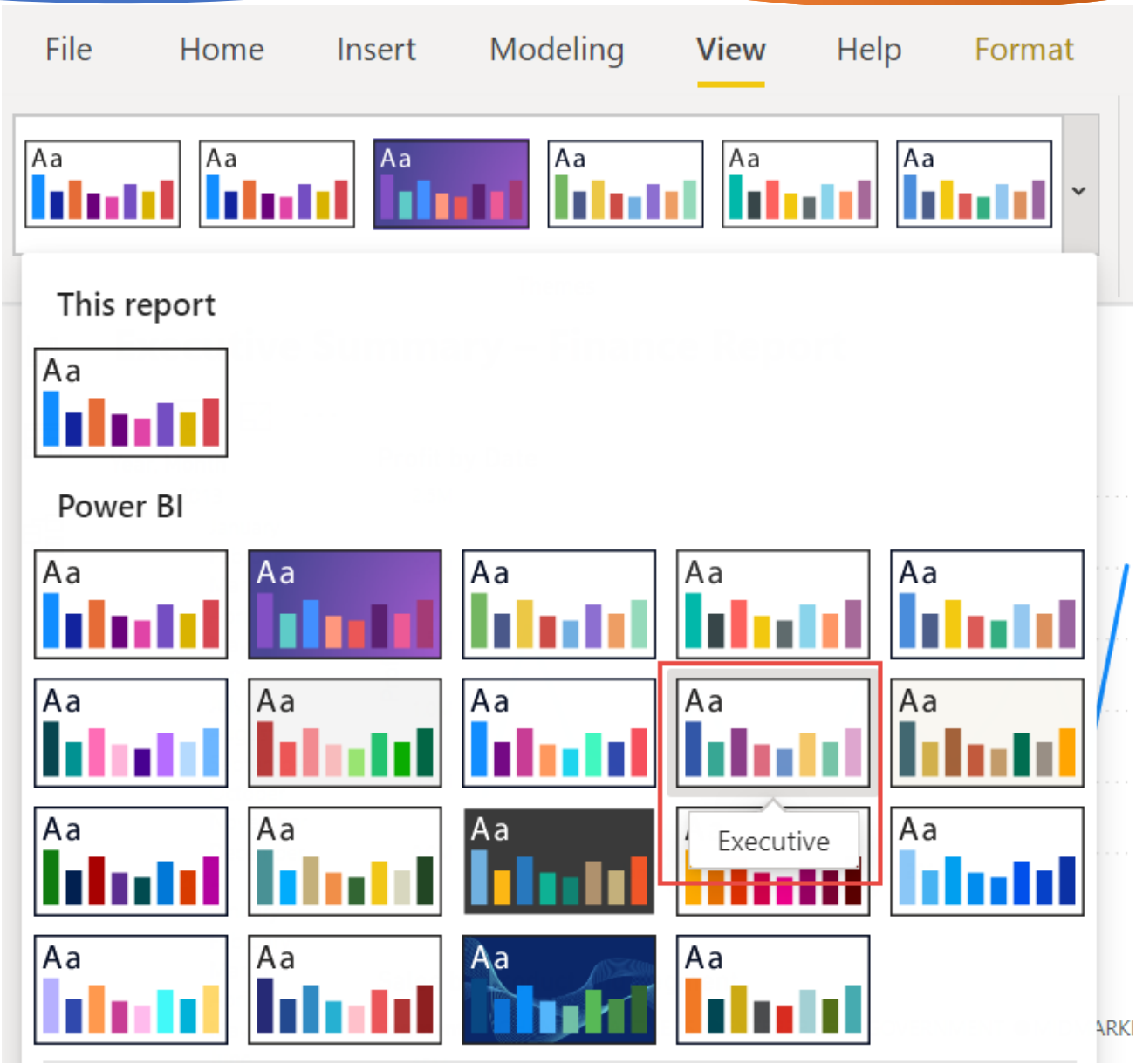
Now if your manager asks to see just 2013 data, you can use either slicer to select years, or specific months of each year.

Extra credit: Format the report

If you want to do some light formatting on this report to add more polish, here are a few easy steps.

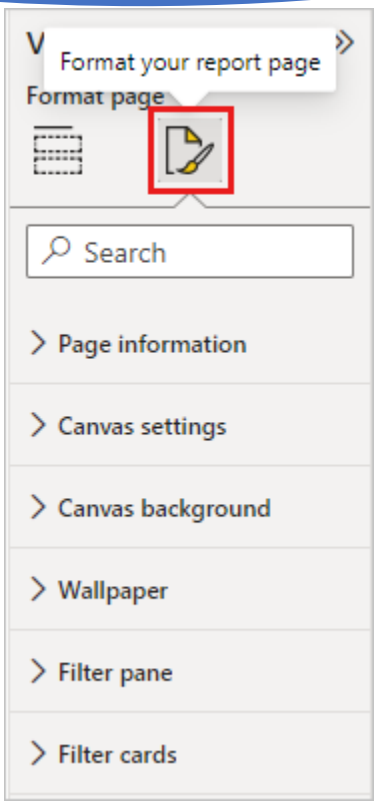
Theme

- On the **View** ribbon, change the theme to **Executive**.



Spruce up the visuals

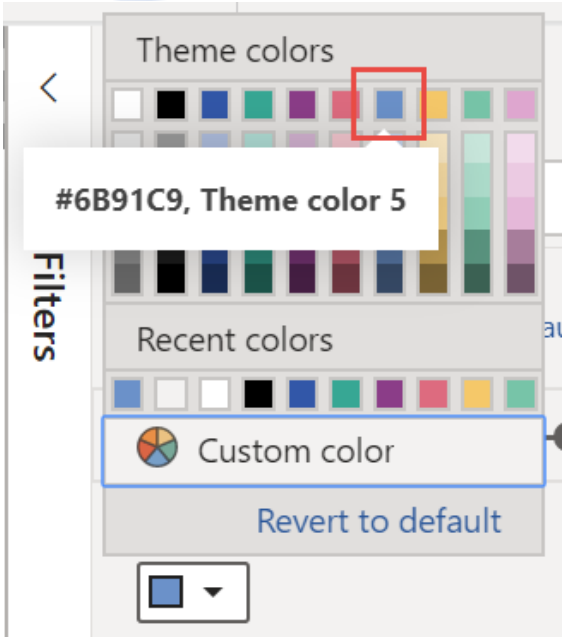
Make the following changes on the **Format** tab in the Visualizations pane.



1. Select Visual 2. In the **Title** section, change **Title text** to “Profit by Month and Year” and **Text size** to **16 pt**. Toggle **Shadow** to **On**.
2. Select Visual 3. In the **Map styles** section, change **Theme** to **Grayscale**. In the **Title** section, change title **Text size** to **16 pt**. Toggle **Shadow** to **On**.
3. Select Visual 4. In the **Title** section, change title **Text size** to **16 pt**. Toggle **Shadow** to **On**.
4. Select Visual 5. In the **Selection controls** section, toggle **Show "Select all" option** to **On**. In the **Slicer header** section, increase **Text size** to **16 pt**.

Add a background shape for the title

1. On the **Insert** ribbon, select **Shapes** > **Rectangle**. Place it at the top of the page, and stretch it to be the width of the page and height of the title.
2. In the **Format shape** pane, in the **Outline** section, change **Transparency** to **100%**.
3. In the **Fill** section, change **Fill color** to **Theme color 5 #6B91C9** (blue).



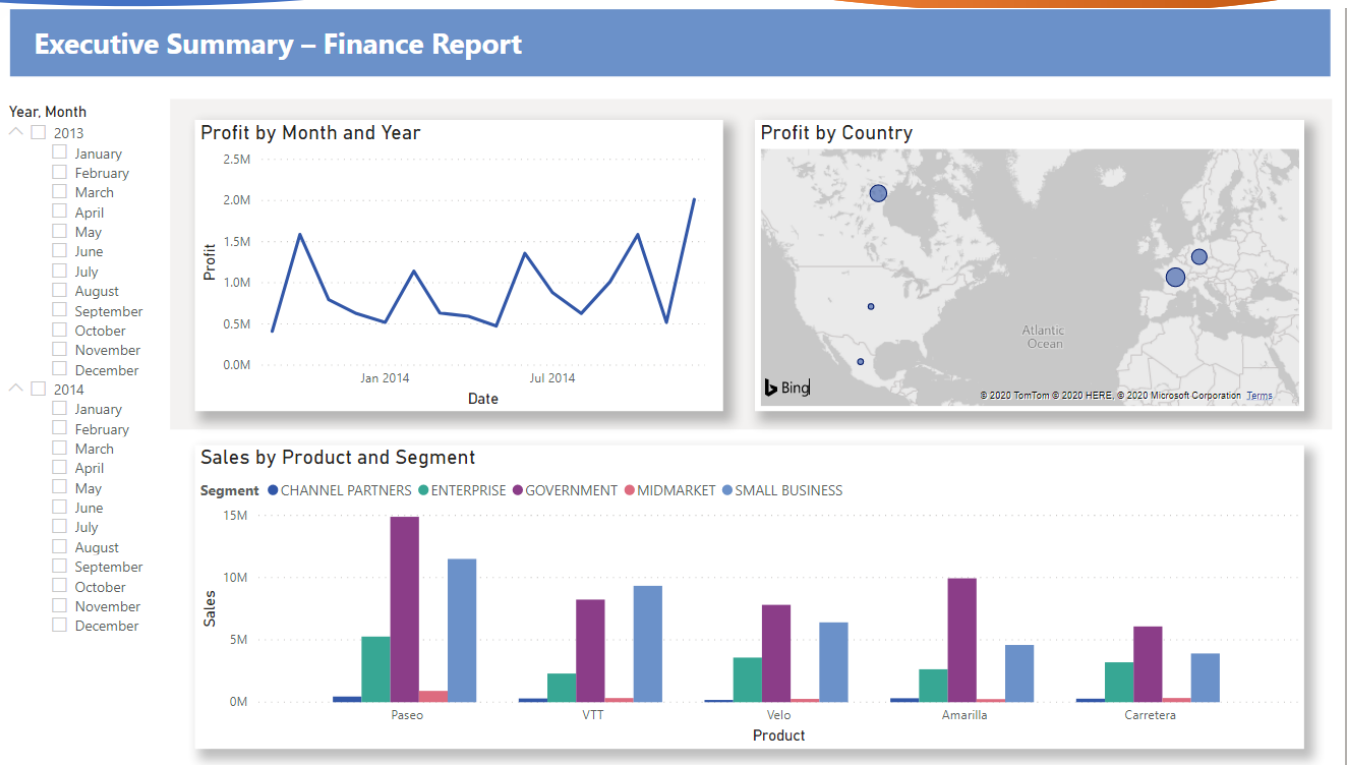
4. On the **Format** tab, select **Send backward** > **Send to back**.
5. Select the text in Visual 1, the title, and change the font color to **White**.

Add a background shape for visuals 2 and 3

1. On the **Insert** ribbon, select **Shapes** > **Rectangle**, and stretch it to be the width and height of Visuals 2 and 3.
2. In the **Format shape** pane, in the **Outline** section, change **Transparency** to **100%**.
3. In the **Fill** section, set the color to **White, 10% darker**.
4. On the **Format** tab, select **Send backward** > **Send to back**.

Finished report

Here's how your final polished report will look:



In summary, this report answers these top questions:

- Which month and year had the most profit?

  December 2014

- Which country/region is the company seeing the most success in?

  In Europe, specifically France and Germany.

- Which product and segment should the company continue to invest in?

  The company should continue to invest in the Paseo product and target the Small Business and Government segments.

Save your report

- On the **File** menu, select **Save**.